

RTL Design Sherpa

RTL AMBA Monitor Subsystem Module Reference — Rev 0.90

October 2026

Table of Contents

1 apb_monitor_addr_check....41
1.1 Overview....41
1.2 Parameters....42
1.3 Ports....43
1.3.1 Clock and Reset....43
1.3.2 Timestamp....43
1.3.3 Snooped APB Command Stream....43
1.3.4 Range Configuration....44
1.3.5 Outgoing MonBus Packet....44
1.4 Functional Description....45
1.4.1 Block diagram....45
1.4.2 Combinational Range Hits....45
1.4.3 Pending Mask and Latched Snapshot....45
1.4.4 Packet Encoding....46
1.4.5 The is_read Carve-Out....46
1.4.6 Timestamp Side-Band....46
1.5 Timing Characteristics....46
1.6 Usage Examples....47
1.7 Design Notes....48
1.7.1 Mirror of the AXI Variant....48
1.7.2 Backpressure Safety....49
1.7.3 Address Field Width....49

1.7.4 Exact-Match Watchpoints....49
1.8 Related Modules....49
1.9 Testing....49
1.10 References....50
1.11 Navigation....51
2 arbiter_monbus_common....52
2.1 Overview....52
2.2 Parameters....53
2.2.1 Derived Parameters (do not override)....55
2.3 Ports....56
2.4 Functional Description....59
2.4.1 Block diagram....59
2.5 Timing Characteristics....59
2.6 Usage Examples....60
2.7 Design Notes....61
2.8 Related Modules....61
2.9 Testing....61
2.10 References....62
2.11 Navigation....62
3 arbiter_rr_pwm_monbus....63
3.1 Overview....63
3.2 Parameters....63
3.2.1 Fixed Internal Configurations (Not User-Configurable)....64
3.3 Ports....65

3.3.1 Clock and Reset 65
3.3.2 Arbiter Interface 65
3.3.3 PWM Configuration 65
3.3.4 Monitor Configuration 66
3.3.5 Monitor Bus Output 66
3.3.6 Monitor Time Input 67
3.3.7 Debug Outputs 67
3.4 Functional Description 68
3.4.1 Block diagram 68
3.4.2 Round-Robin Arbiter 68
3.4.3 PWM Generator 68
3.4.4 Monitor Bus Common 69
3.4.5 Operation Flow 69
3.5 Timing Characteristics 69
3.6 Usage Examples 69
3.7 Design Notes 71
3.7.1 PWM Control Strategy 71
3.7.2 Fixed Configuration Benefits 71
3.7.3 Fairness in Round-Robin Mode 71
3.7.4 Monitor Bus Integration 71
3.8 Related Modules 71
3.9 Testing 72
3.10 References 72
3.11 Navigation 73

4 arbiter_wrr_pwm_monbus....74
4.1 Overview....74
4.2 Parameters....74
4.2.1 Fixed Internal Configurations (Not User-Configurable)....75
4.2.2 Derived Parameters (do not override)....76
4.3 Ports....76
4.3.1 Clock and Reset....76
4.3.2 Arbiter Configuration and Interface....76
4.3.3 PWM Configuration....77
4.3.4 Monitor Configuration....77
4.3.5 Monitor Bus Output....78
4.3.6 Monitor Time Input....78
4.3.7 Enhanced Debug Outputs....79
4.4 Functional Description....80
4.4.1 Block diagram....80
4.4.2 Weighted Round-Robin Arbiter....80
4.4.3 PWM Generator....81
4.4.4 Monitor Bus Common....81
4.4.5 Operation Flow....81
4.5 Timing Characteristics....82
4.6 Usage Examples....82
4.7 Design Notes....84
4.7.1 Weight Configuration....84
4.7.2 Fairness Deviation Threshold....84

4.7.3 Enhanced Debug Outputs....84
4.7.4 ACK Mode with Weighted Arbitration....85
4.8 Related Modules....85
4.9 Testing....85
4.10 References....86
4.11 Navigation....86
5 axi4ace_snoop_monitor_lite....87
5.1 Overview....87
5.2 Why a Separate Monitor Core....87
5.3 Block Diagram....88
5.4 Parameters....88
5.5 Ports....89
5.6 Packet Encoding....91
5.6.1 Completion Packet (PktTypeCompletion)....91
5.6.2 Error Packet (PktTypeError)....92
5.6.3 Timeout Packet (PktTypeTimeout)....92
5.6.4 Event Precedence....92
5.7 Drop-and-Count Discipline....93
5.8 Outstanding-Snoop Table....93
5.9 Wrappers that Instantiate This Core....93
5.10 Related Modules....93
5.11 References....94
5.12 Navigation....94
6 axi_monitor_addr_check....95

6.1 Overview....95
6.2 Parameters....96
6.2.1 Derived Parameters (do not override)....98
6.3 Ports....98
6.3.1 Clock and Reset....98
6.3.2 Side-Band Timestamp....98
6.3.3 Command Interface (Snoop)....99
6.3.4 Configuration Inputs....99
6.3.5 Monitor Bus Output....100
6.4 Functional Description....101
6.4.1 Block diagram....101
6.4.2 Internal Architecture....101
6.4.3 Event Encoding....102
6.5 Timing Characteristics....103
6.6 Usage Examples....103
6.7 Design Notes....104
6.7.1 Filtering Integration....104
6.7.2 Instantiation Pattern....104
6.7.3 Synthesis Behavior....104
6.7.4 Downstream FIFO....105
6.8 Related Modules....105
6.9 Testing....105
6.10 References....106
6.11 Navigation....107

7 AXI Monitor Base....108
7.1 Overview....108
7.2 Parameters....109
7.2.1 Identity and Sizing....109
7.2.2 Master Switches....111
7.2.3 Transaction-Table Shaping....112
7.2.4 ID-Range Filter....113
7.2.5 Timer LUT Sizing (CFI_*)....115
7.2.6 Synthesis-Cone Enables (ENABLE_*_LOGIC)....115
7.2.7 Derived Parameters (do not override)....117
7.3 Ports....117
7.3.1 Command Phase Interface (AW/AR)....117
7.3.2 Data Channel Interface (R/W)....117
7.3.3 Response Channel Interface (B)....118
7.3.4 Configuration Interface....118
7.3.5 Address-Range Checker Interface....120
7.3.6 Side-Band Timestamp Input....120
7.3.7 Monitor Bus Output....121
7.3.8 Performance-Window Control Inputs....122
7.3.9 Performance-Window Status and Counter Outputs....123
7.4 Functional Description....125
7.4.1 Block diagram....125
7.4.2 Architecture....125
7.4.3 Flow Control and the Saturation-Recovery Contract....126

7.4.4 Performance Monitoring....127
7.5 Timing Characteristics....130
7.6 Usage Examples....130
7.7 Design Notes....131
7.7.1 Transaction Table Sizing....131
7.7.2 Timeout Configuration....131
7.8 Related Modules....132
7.9 Testing....132
7.10 References....133
7.11 Navigation....134
8 AXI Monitor Filtered....135
8.1 Overview....135
8.2 Parameters....136
8.2.1 Forwarded to axi_monitor_base (pass-through)....139
8.2.2 Filter configuration and status (pass-through)....141
8.3 Ports....142
8.3.1 Observed AXI/AXIL Channels (inputs to the wrapped base)....142
8.3.2 Monitor Bus Output (filtered)....142
8.3.3 Reset and Synchronous Control....143
8.3.4 Filter Configuration....144
8.3.5 Configuration Status Output....145
8.3.6 Performance Window Control (Stage A of perfmon RFC)....145
8.3.7 Performance Counters (Stage B of perfmon RFC)....146
8.4 Functional Description....148

8.4.1 Block diagram....148
8.4.2 Architecture....148
8.5 Timing Characteristics....148
8.6 Usage Examples....149
8.6.1 Filter Strategy....149
8.7 Design Notes....150
8.8 Related Modules....150
8.9 Testing....150
8.10 References....151
8.11 Navigation....151
9 AXI Monitor Lite....152
9.1 Overview....152
9.2 Why it is small....153
9.3 Measured....155
9.3.1 In systems, on silicon (2026-09-26 and 27)....156
9.4 Adopted....157
9.5 Parameters....158
9.6 Ports....160
9.7 Functional Description....161
9.7.1 Block diagram....161
9.7.2 Attribution....161
9.7.3 Errors, once per transaction....162
9.7.4 Timeout....162
9.7.5 Latency threshold....163

9.7.6 Completion hold 163
9.7.7 Drop and count 163
9.8 Verification 163
9.9 Related 166
9.10 References 166
10 AXI Monitor Lite Wrappers 168
10.1 Overview 168
10.1.1 The eight AXIS wrappers (2026-09-27) 171
10.1.2 The eight ACE wrappers (2026-10-05) 172
10.1.3 The two CXS wrappers (2026-10-15) 173
10.1.4 What is not here 174
10.2 Monitor Parameters (every wrapper) 174
10.3 Monitor Ports (every wrapper) 176
10.4 Clock-Gated Wrappers 178
10.5 Functional Description 179
10.6 Usage Example 180
10.7 Design Notes 181
10.8 Related Modules 181
10.9 Testing 182
10.10 References 182
10.11 Navigation 183
11 AXI Monitor Reporter — Completion Cone 184
11.1 Overview 184
11.2 Parameters 185

11.3 Ports....185
11.3.1 Inputs (shared monitor state)....185
11.3.2 Outputs (packet fields to the top reporter)....186
11.4 Functional Description....187
11.4.1 Block diagram....187
11.4.2 Completion Scan....187
11.4.3 First-Match Selection....187
11.4.4 Emitted Fields....187
11.4.5 Ownership of State....188
11.5 Timing Characteristics....188
11.6 Usage Examples....188
11.7 Design Notes....189
11.7.1 Compile-Out Path....189
11.7.2 Combinational by Design....189
11.7.3 One Packet Per Cycle....189
11.8 Related Modules....189
11.9 Testing....190
11.10 References....190
11.10.1 Related work and specifications....190
11.10.2 Source Code....191
11.10.3 Documentation....191
11.11 Navigation....191
12 AXI Monitor Reporter — Debug State-Change Emitter....192
12.1 Overview....192

12.2 Parameters....193
12.3 Ports....193
12.3.1 Clock and Reset....193
12.3.2 Inputs (shared monitor state)....193
12.3.3 Direct-Inject Handshake....194
12.3.4 Outputs (packet fields)....194
12.4 Functional Description....195
12.4.1 Block diagram....195
12.4.2 Change Detection....195
12.4.3 Previous-State Flop....195
12.4.4 Packet Assembly....196
12.4.5 Direct-Inject and output_busy....196
12.5 Timing Characteristics....196
12.6 Usage Examples....196
12.7 Design Notes....197
12.7.1 pkt_taken Is Intentionally Unused....197
12.7.2 Sequential, Not Combinational....197
12.7.3 Compile-Out Path....197
12.7.4 State Tuple Encoding....198
12.8 Related Modules....198
12.9 Testing....198
12.10 References....199
12.10.1 Related work and specifications....199
12.10.2 Source Code....199

12.10.3 Documentation....199
12.11 Navigation....200
13 AXI Monitor Reporter — Error Cone....201
13.1 Overview....201
13.2 Parameters....202
13.3 Ports....202
13.3.1 Inputs (shared monitor state)....202
13.3.2 Outputs (packet fields to the top reporter)....203
13.4 Functional Description....204
13.4.1 Block diagram....204
13.4.2 Error / Orphan Scan....204
13.4.3 First-Match Selection....205
13.4.4 Emitted Fields....205
13.4.5 Ownership of State....205
13.5 Timing Characteristics....205
13.6 Usage Examples....206
13.7 Design Notes....206
13.7.1 Timeout De-Aliasing....206
13.7.2 Compile-Out Path....206
13.7.3 Raw Event Code Forwarding....206
13.8 Related Modules....207
13.9 Testing....207
13.10 References....208
13.10.1 Related work and specifications....208

13.10.2 Source Code....208
13.10.3 Documentation....208
13.11 Navigation....208
14 AXI Monitor Reporter (Dispatcher + 6 Sub-blocks)....210
14.1 Overview....210
14.2 Parameters....211
14.3 Ports....212
14.4 Functional Description....215
14.4.1 Block diagram....215
14.4.2 Architecture....216
14.4.3 Sub-blocks....216
14.4.4 Auto-Retire: Disabled Classes Never Leak Slots....218
14.5 Timing Characteristics....219
14.6 Usage Examples....219
14.7 Design Notes....219
14.8 Related Modules....219
14.9 Testing....220
14.10 References....220
14.11 Navigation....221
15 AXI Monitor Reporter — Performance Packets....222
15.1 Overview....222
15.2 Parameters....223
15.3 Ports....223
15.3.1 Clock and Reset....223

15.3.2 Control / Status Inputs....223
15.3.3 Packet Output....224
15.3.4 Lifetime Counters (Status / Debug)....225
15.4 Functional Description....226
15.4.1 Block diagram....226
15.4.2 Lifetime Counters....226
15.4.3 Cycle FSM....226
15.4.4 Output Multiplexer....227
15.4.5 Handshake....227
15.5 Timing Characteristics....227
15.6 Usage Examples....228
15.7 Design Notes....228
15.7.1 Completion + Performance Packets: Bandwidth, and the Two Ways to Suppress....228
15.7.2 pkt_taken Holds the Emit FSM — Do Not Tie It Off....229
15.7.3 Relationship to the Window-Bucket Perfmon....229
15.7.4 Counter Wrap....230
15.8 Related Modules....230
15.9 Testing....230
15.10 References....231
15.10.1 Related work and specifications....231
15.10.2 Source Code....231
15.10.3 Documentation....231
15.11 Navigation....232

16 AXI Monitor Reporter — Threshold Packets 233
16.1 Overview 233
16.2 Parameters 234
16.3 Ports 234
16.3.1 Clock and Reset 234
16.3.2 Transaction Table and Configuration 235
16.3.3 Handshake / Backpressure 235
16.3.4 Packet Output 236
16.4 Functional Description 237
16.4.1 Block diagram 237
16.4.2 Active-Count Detection 237
16.4.3 Per-Slot Latency Pipeline 237
16.4.4 Latency Event Selection 238
16.4.5 Edge-Sticky Flags 238
16.4.6 Output Multiplexer 238
16.5 Timing Characteristics 238
16.6 Usage Examples 238
16.7 Design Notes 239
16.7.1 Threshold Packets Bypass the FIFO 239
16.7.2 Active Count vs Latency Priority 239
16.7.3 Latency Pipeline Is the Area Cost 239
16.7.4 Read vs Write Latency 239
16.8 Related Modules 240
16.9 Testing 240

16.10 References....241
16.10.1 Related work and specifications....241
16.10.2 Source Code....241
16.10.3 Documentation....241
16.11 Navigation....241
17 AXI Monitor Reporter — Timeout Packets....242
17.1 Overview....242
17.2 Parameters....243
17.3 Ports....243
17.3.1 Inputs....243
17.3.2 Packet Output....244
17.4 Functional Description....245
17.4.1 Block diagram....245
17.4.2 Candidate Detection....245
17.4.3 Priority Encoding....245
17.4.4 Output Assignment....245
17.4.5 No Double-Reporting....246
17.5 Timing Characteristics....246
17.6 Usage Examples....246
17.7 Design Notes....247
17.7.1 Pure Combinational....247
17.7.2 Event Code Comes From the Slot....247
17.7.3 Shared Vectors Are Load-Bearing....247
17.8 Related Modules....247

17.9 Testing....248
17.10 References....248
17.10.1 Related work and specifications....248
17.10.2 Source Code....248
17.10.3 Documentation....249
17.11 Navigation....249
18 AXI Monitor Timeout....250
18.1 Overview....250
18.2 Parameters....250
18.3 Ports....251
18.3.1 Clock and Reset....251
18.3.2 Transaction Table....251
18.3.3 Timer....251
18.3.4 Timeout Configuration....252
18.3.5 Outputs....252
18.4 Functional Description....253
18.4.1 Block diagram....253
18.4.2 Architecture....253
18.4.3 Detection Semantics....253
18.5 Timing Characteristics....254
18.6 Usage Examples....255
18.7 Design Notes....255
18.8 Related Modules....255
18.9 Testing....256

18.10 References....256
18.11 Navigation....256
19 AXI Monitor Timer....257
19.1 Overview....257
19.2 Parameters....258
19.3 Ports....258
19.3.1 Global Clock and Reset....258
19.3.2 Timer Configuration....258
19.3.3 Timer Outputs....259
19.4 Functional Description....259
19.4.1 Block diagram....259
19.4.2 Global Timestamp Counter....260
19.4.3 Frequency-Invariant Timer Tick....260
19.4.4 Counter Freq Invariant Integration....262
19.4.5 Integration with Monitor Architecture....263
19.5 Timing Characteristics....264
19.6 Usage Examples....264
19.7 Design Notes....264
19.7.1 Timestamp Wraparound Handling....264
19.7.2 Frequency Selection Is Not a Granularity Trade-off....265
19.7.3 Dynamic Frequency Scaling....265
19.7.4 Zero Timestamp on Reset....266
19.7.5 Timer Tick Duty Cycle....266
19.7.6 Resource Utilization....266

19.7.7 Multiple Timer Instances....266
19.8 Related Modules....267
19.9 Testing....268
19.10 References....268
19.10.1 Related work and specifications....268
19.10.2 Specifications....268
19.10.3 Source Code....268
19.10.4 Configuration Guides....269
19.11 Navigation....269
20 AXI Monitor Transaction Manager....270
20.1 Overview....270
20.2 Parameters....271
20.3 Ports....273
20.4 Functional Description....276
20.4.1 Block diagram....276
20.4.2 Architecture....276
20.4.3 Transaction Lifecycle....277
20.4.4 Allocation and Same-ID Tracking....278
20.4.5 Address filtering....280
20.5 Timing Characteristics....280
20.6 Usage Examples....281
20.7 Design Notes....282
20.7.1 Synthesis Notes....282
20.7.2 Formal Properties....283

20.8 Related Modules....284
20.9 Testing....284
20.10 References....285
20.11 Navigation....285
21 AXIS Monitor Lite....286
21.1 Overview....286
21.2 Parameters....286
21.3 Ports....287
21.4 Functional Description....289
21.4.1 Block diagram....289
21.4.2 Events....289
21.4.3 Timing....291
21.4.4 Drop and count....291
21.5 Usage Examples....291
21.6 Design Notes....292
21.7 Related Modules....293
21.8 Testing....293
21.9 References....294
21.10 Navigation....294
22 CXS Monitor Lite....295
22.1 Overview....295
22.2 Events....295
22.3 Parameters....297
22.4 Ports....298

22.5 Wrappers....299
22.6 Design Notes....299
22.7 Testing....299
22.8 References....300
22.9 Navigation....300
23 Monitor Bus Round-Robin Arbiter....301
23.1 Overview....301
23.2 Parameters....301
23.3 Ports....302
23.3.1 Clock and Reset....302
23.3.2 Block Arbitration....302
23.3.3 Monitor Bus Inputs (Array of CLIENTS)....303
23.3.4 Monitor Bus Output (Aggregated)....303
23.3.5 Debug/Status Outputs....303
23.4 Functional Description....304
23.4.1 Block diagram....304
23.4.2 Arbiter Request and Grant ACK Logic....305
23.4.3 Round-Robin Arbiter Instance....305
23.4.4 Client Ready Signal Generation....305
23.4.5 Output Multiplexer....305
23.4.6 Optional Skid Buffers....306
23.5 Timing Characteristics....306
23.6 Usage Examples....307
23.7 Design Notes....308

23.7.1 ACK Mode Operation....308
23.7.2 Skid Buffer Depth Selection....308
23.7.3 Skid-Free Configuration....309
23.7.4 Formal properties....309
23.8 Related Modules....310
23.9 Testing....310
23.10 References....310
23.10.1 Related work and specifications....310
23.10.2 Specifications....311
23.10.3 Source Code....311
23.11 Navigation....311
24 MonBus Group — AXI4 / AXI4....312
24.1 Overview....312
24.2 Parameters....313
24.3 Ports....314
24.3.1 Clock, Reset, Clear....314
24.3.2 MonBus Ingress and Timestamp....314
24.3.3 S-Side — AXI4 Slave Read (error record drain)....315
24.3.4 M-Side — AXI4 Master Write (bulk trace capture)....316
24.3.5 Status and Egress Configuration....317
24.4 Functional Description....318
24.4.1 Block diagram....318
24.4.2 S-Side Error Drain (AXI4 slave read)....318
24.4.3 M-Side Bulk Capture (AXI4 master write)....318

24.4.4 Shared Egress Configuration....319
24.5 Timing Characteristics....319
24.6 Usage Examples....319
24.7 Design Notes....320
24.7.1 Both Sides Are Full AXI4....320
24.7.2 AW Sideband Defaults Live in the Wrapper....320
24.7.3 Skid Depths Are Tunable....321
24.8 Related Modules....321
24.9 Testing....321
24.10 References....321
24.10.1 Related work and specifications....321
24.10.2 Source Code....322
24.10.3 Documentation....322
24.11 Navigation....322
25 MonBus Group — AXI4 / AXIL....323
25.1 Overview....323
25.2 Parameters....324
25.3 Ports....325
25.3.1 Clock, Reset, Clear....325
25.3.2 MonBus Ingress and Timestamp....325
25.3.3 S-Side — AXI4 Slave Read (error record drain)....326
25.3.4 M-Side — AXIL Master Write (bulk trace capture)....327
25.3.5 Status and Egress Configuration....327
25.4 Functional Description....328

25.4.1 Block diagram....328
25.4.2 S-Side Error Drain (AXI4 slave read)....328
25.4.3 M-Side Bulk Capture (AXIL master write)....329
25.4.4 Shared Egress Configuration....329
25.5 Timing Characteristics....329
25.6 Usage Examples....330
25.7 Design Notes....331
25.7.1 AXIL Forces Single-Beat Writes....331
25.7.2 Core Still Drives AXI4 FUB Fields....331
25.7.3 Read Side Is Unchanged....331
25.8 Related Modules....331
25.9 Testing....332
25.10 References....332
25.10.1 Related work and specifications....332
25.10.2 Source Code....332
25.10.3 Documentation....333
25.11 Navigation....333
26 Monitor Bus Group — AXIL Slave-Read / AXI4 Master-Write....334
26.1 Overview....334
26.2 Parameters....335
26.3 Ports....337
26.3.1 Clock, Reset, and CAM Clear....337
26.3.2 Monitor-Bus Input + Timestamp....337
26.3.3 AXIL Slave Read — Err-FIFO Drain....337

26.3.4 AXI4 Master Write — Burst Bulk Capture....338
26.3.5 Interrupt....339
26.3.6 Configuration....340
26.3.7 Per-Protocol Filter Masks....340
26.3.8 Status / Debug....341
26.4 Functional Description....342
26.4.1 Block diagram....342
26.4.2 S-Side Err-Drain Read Protocol....342
26.4.3 M-Side Bulk-Capture Protocol (AXI4 Burst)....343
26.4.4 Interrupt....343
26.4.5 Shared Egress Packet Filter....343
26.4.6 Compression....343
26.5 Timing Characteristics....343
26.6 Usage Examples....344
26.7 Design Notes....345
26.7.1 When to Pick This Over AXIL/AXIL....345
26.7.2 AXI4 Leaf Default Tie-Offs....345
26.7.3 Why One Core + Four Wrappers....346
26.8 Related Modules....346
26.9 Testing....347
26.10 References....347
26.10.1 Related work and specifications....347
26.10.2 Source Code....347
26.10.3 Documentation....347

26.11 Navigation....348
27 Monitor Bus Group — AXIL Slave-Read / AXIL Master-Write....349
27.1 Overview....349
27.2 Parameters....350
27.3 Ports....352
27.3.1 Clock, Reset, and CAM Clear....352
27.3.2 Monitor-Bus Input + Timestamp....352
27.3.3 AXIL Slave Read — Err-FIFO Drain....352
27.3.4 AXIL Master Write — Bulk Capture....353
27.3.5 Interrupt....354
27.3.6 Configuration....354
27.3.7 Per-Protocol Filter Masks....354
27.3.8 Status / Debug....355
27.4 Functional Description....356
27.4.1 Block diagram....356
27.4.2 S-Side Err-Drain Read Protocol....357
27.4.3 M-Side Bulk-Capture Protocol....357
27.4.4 Interrupt....357
27.4.5 Shared Egress Packet Filter....357
27.4.6 Compression....358
27.5 Timing Characteristics....358
27.6 Usage Examples....358
27.7 Design Notes....360
27.7.1 Legacy Replacement....360

27.7.2 Why One Core + Four Wrappers....360
27.7.3 32-Bit Drain Serializer....360
27.8 Related Modules....360
27.9 Testing....361
27.10 References....361
27.10.1 Related work and specifications....361
27.10.2 Source Code....362
27.10.3 Documentation....362
27.11 Navigation....362
28 Monitor Bus LRU CAM....363
28.1 Overview....363
28.2 Parameters....364
28.3 Ports....365
28.3.1 Counting-Consumer Ports....366
28.4 Functional Description....367
28.4.1 Block diagram....367
28.4.2 Storage Model: Position-Indexed LRU....368
28.4.3 Actions....368
28.4.4 Per-Slot Update Mechanics....369
28.4.5 Per-Entry Timestamp Storage....370
28.4.6 Match Logic....370
28.5 Timing Characteristics....371
28.6 Usage Examples....371
28.7 Design Notes....372

28.8 Testing....372
28.9 Related Modules....373
28.10 References....373
28.11 Navigation....374
29 Monitor Bus LRU CAM (Pipelined)....375
29.1 Overview....375
29.2 Parameters....375
29.3 Ports....376
29.4 Functional Description....377
29.4.1 Block diagram....377
29.4.2 Forwarding (depth-1)....378
29.4.3 Self-Derived Action....379
29.4.4 Synchronous Clear....379
29.5 Timing Characteristics....380
29.6 Usage Examples....380
29.7 Design Notes....381
29.7.1 How It Differs from monbus_cam....381
29.8 Testing....381
29.9 Related Modules....382
29.10 References....382
29.11 Navigation....382
30 Monitor Bus Compressor....383
30.1 Overview....383
30.1.1 Why Compress Monitor Traces?....383

30.2 Parameters....384
30.3 Ports....384
30.3.1 Synchronous CAM Clear....385
30.4 Functional Description....386
30.4.1 Block diagram....386
30.4.2 Compression Techniques....386
30.4.3 Slot Bit Layouts....389
30.4.4 CAM Design....390
30.4.5 Encoder Decision Tree....391
30.4.6 Statistics Counters....392
30.5 Timing Characteristics....393
30.5.1 Per-template delta_ts....394
30.5.2 Input skid....394
30.5.3 pblock (Nexys A7 build only)....394
30.6 Usage Examples....394
30.7 Design Notes....395
30.8 Testing....395
30.8.1 Unit-level (the compressor in isolation)....395
30.8.2 Integration-level (compressor + AXIL writer + base/limit wrap)....396
30.8.3 Validation Numbers (locked dataset)....396
30.8.4 Test Suite Summary....397
30.9 Related Modules....397
30.10 References....398
30.11 Navigation....399

31 monbus_group_core 400
31.1 Overview 400
31.1.1 Key Features 400
31.2 Parameters 401
31.3 Ports 402
31.3.1 Clock, Reset, and Clear 402
31.3.2 MonBus Ingress and Timestamp 402
31.3.3 Status / IRQ / Debug 402
31.3.4 Address Window and Flush Configuration 403
31.3.5 Per-Protocol Filter Masks 403
31.3.6 Compressor Statistics 405
31.3.7 AXI4-Shaped Master-Write FUB (bulk capture) 406
31.3.8 AXI4-Shaped Slave-Read FUB (error drain) 407
31.4 Functional Description 408
31.4.1 Block diagram 408
31.4.2 Free-Running Timestamp 409
31.4.3 Packet Filtering 409
31.4.4 Error FIFO and Slave-Read Drain 409
31.4.5 Write Path — Raw Expander vs Compressor 409
31.4.6 Master-Write Burst Writer 410
31.5 Timing Characteristics 411
31.6 Usage Examples 411
31.7 Design Notes 412
31.7.1 Hold cfg_compress_en Stable 412

31.7.2 AXIL Builds Use the Same Core....412
31.7.3 FIFO Granularity Difference....412
31.7.4 Timing-Motivated Structure....412
31.7.5 4KB and Window Compliance....412
31.8 Related Modules....413
31.8.1 Used By....413
31.8.2 Uses....413
31.8.3 See Also....413
31.9 Testing....413
31.10 References....414
31.10.1 Related work and specifications....414
31.10.2 Source Code....415
31.10.3 Documentation....415
31.11 Navigation....415
32 monbus_group....416
32.1 Overview....416
32.2 Parameters....416
32.2.1 Runtime Config....418
32.3 Ports....419
32.4 Functional Description....420
32.4.1 Architecture....420
32.4.2 Beat Layout....421
32.4.3 Master-Write Behavior....422
32.4.4 Per-Protocol Filter Configuration....425

32.5 Usage Examples....426
32.5.1 Migration from monbus_axil_group....426
32.6 Design Notes....427
32.6.1 Why One Core Plus Four Wrappers....427
32.7 Related Modules....428
32.7.1 Sub-modules Instantiated....428
32.7.2 See Also....429
32.8 Testing....429
32.9 References....430
33 monbus_halfbeat_packer....432
33.1 Overview....432
33.1.1 Key Features....432
33.2 Parameters....433
33.3 Ports....433
33.3.1 Clock and Reset....433
33.3.2 Input — Beats from the Compressor....433
33.3.3 Output — Packed Beats to the Write FIFO....433
33.4 Functional Description....434
33.4.1 Block diagram....434
33.4.2 Beat Layout....434
33.4.3 Case Decode....434
33.4.4 Handshake and Ordering....435
33.4.5 Pending-Slot Register....435
33.4.6 Bit-Exactness....435

33.5 Timing Characteristics....436
33.6 Usage Examples....436
33.7 Design Notes....436
33.7.1 Requires the Compressor....436
33.7.2 Order Preservation Is Non-Negotiable....437
33.7.3 One-Slot Buffer Only....437
33.8 Related Modules....437
33.8.1 Used By....437
33.8.2 Uses....437
33.8.3 See Also....437
33.9 Testing....437
33.10 References....438
33.10.1 Related work and specifications....438
33.10.2 Source Code....438
33.10.3 Documentation....438
33.11 Navigation....438
34 Monitor Bus Legal-Set CAM....439
34.1 Overview....439
34.2 Parameters....439
34.3 Ports....440
34.4 Functional Description....440
34.5 Timing Characteristics....441
34.6 Usage Examples....441
34.7 Design Notes....442

34.8 Related Modules....442
34.9 Testing....442
34.10 References....442
34.11 Navigation....443
35 monbus_pkt_tally....444
35.1 Overview....444
35.2 Parameters....444
35.2.1 Derived Parameters (do not override)....445
35.3 Ports....445
35.4 Functional Description....447
35.4.1 Architecture....447
35.4.2 Data Model....447
35.4.3 Legal-Set (Dense) Binning....448
35.4.4 Snapshot Protocol....448
35.4.5 First-Event Latch....449
35.5 Timing Characteristics....449
35.6 Usage Examples....449
35.7 Design Notes....450
35.7.1 Why Count Instead of Log?....450
35.8 Related Modules....451
35.8.1 Building Blocks Reused....451
35.8.2 See Also....451
35.9 Testing....451
35.10 References....452

35.11 Navigation....452
36 monbus_wb4_axil4_group / monbus_wb4_axi4_group....453
36.1 Overview....453
36.2 monbus_wb4_rd_shim....453
36.3 Reading a record....454
36.4 Notes....454
36.5 Related....454
36.6 Test....454
36.7 References....454
37 Monitor Characterization....456
37.1 What was measured....456
37.1.1 Kintex-7 325T -2 (Genesys 2), clock constrained at 6.667 ns, out of context....457
37.1.2 Artix-7 100T -1 (Nexys A7), clock constrained at 10.000 ns, out of context....461
37.2 What the numbers say....463
37.3 Recommendations....464
37.4 Open findings from this sweep....464
37.5 Files....465
37.6 References....465
38 Ocelus plane map....466
38.1 The five planes....466
38.1.1 Shared reporter infrastructure....467
38.2 Build personas (examples, not prescriptions)....467

38.3 Data-flow contract 468
38.4 Plane architecture 470
38.5 Related 470
38.6 Navigation 471
39 The Ocelus Architecture 472
39.1 Planes 472
39.2 The one thing to understand first 472
39.3 Two coordinate systems and a topology 473
39.3.1 Precedence decides how many protocols exist 474
39.3.2 Fix the choice per project 475
39.3.3 Coordinate A – classification: what happened 475
39.3.4 Healthy classes vs fault classes 475
39.3.5 Coordinate B – identity: who it happened to 476
39.3.6 Topology – how packets reach the capture point 476
39.4 Adaptability: what is deliberately left open 478
39.5 Functional Description 478
39.5.1 1. Detect 479
39.5.2 2. Shape 479
39.5.3 3. Filter 480
39.5.4 4. Transport and capture 481
39.6 Choosing a capture strategy 481
39.6.1 Bulk trace 482
39.6.2 Compressed trace – monbus_compressor 482
39.6.3 Counting – monbus_pkt_tally 483

39.7 Instrumenting something that is not a bus protocol....483
39.8 Performance monitoring....484
39.9 Configuration cautions....485
39.10 Related Modules....486
39.11 References....486
39.12 Navigation....487
40 monitor_trans_cam....488
40.1 Overview....488
40.1.1 Key Features....488
40.2 Parameters....489
40.3 Ports....489
40.4 Functional Description....491
40.4.1 Block diagram....491
40.4.2 Architecture....491
40.4.3 Why Three Lookup Ports?....491
40.4.4 The “First-Match” Variant....492
40.4.5 3-Way Mutex Alloc Priority Encoder....492
40.4.6 Per-Slot Storage and Write Port....493
40.5 Timing Characteristics....493
40.6 Usage Examples....494
40.6.1 Caller Pattern (axi_monitor_trans_mgr)....494
40.7 Design Notes....495
40.7.1 Synthesis Notes....495
40.8 Related Modules....495

40.9 Testing.....496
40.10 References.....497
40.11 Navigation.....497

1 apb_monitor_addr_check

Module: `apb_monitor_addr_check.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: protocol plane (APB address-range filter; used by both full and lite monitors)

1.1 Overview

A configurable N-range address-violation filter for the APB monitor pipeline — the APB mirror of `axi_monitor_addr_check`. It watches the `cmd_valid`/`cmd_ready` handshake the `apb4_monitor` already snoops, and when an accepted command's `paddr` falls inside any of N configured `[low, high]` inclusive ranges it emits a `PktTypeError` MonBus packet with event code `APB_ERR_ADDR_RANGE` (`8'h08`).

APB peripherals tend to grow reserved or protected address windows that software must never touch, and flagging those accesses is the whole point of this block. It adds no logic to the APB datapath itself: it snoops the command handshake the monitor already observes, compares the accepted address against the programmed ranges, and reports which range was hit, the offending address, and whether the access was a read or a write.

- Up to N configurable inclusive `[low, high]` address ranges (default 4, up to 16 usable via the 4-bit range index)
- Per-range enable plus a global `cfg_addr_check_enable`
- Emits a standard MonBus error packet with the offending address, range index, and direction
- Preserves an `is_read` bit (APB has no separate AR/AW channels, so direction must be carried explicitly)
- Per-range pending mask latches hits and drains them one packet at a time under backpressure; repeat hits on one range coalesce newest-wins (see Operation)
- Side-band timestamp captured from the broadcast free-running counter on emission
- Exact-match support by setting a range's `low == high`

Typical uses: flagging accesses to reserved/protected APB register windows, security or safety guard-banding of peripheral address maps, debug tripwires on unexpected address regions during bring-up, and exact-address watchpoints (`low == high`). In short: programmable, per-range address-violation reporting on the MonBus that mirrors the AXI variant while carrying the APB-specific read/write direction bit.

1.2 Parameters

Parameter	Type	Default	Description
N_ADDR_RANGES	int	4	Number of configurable `[low, high]` ranges
ADDR_WIDTH	int	32	APB address width
UNIT_ID	logic [7:0]	8’h00	Unit id stamped into emitted packets
AGENT_ID	logic [15:0]	16’h0000	Agent id stamped into emitted packets
PROTOCOL	logic [3:0]	PROTOCOL_APB	Protocol tag on the emitted packet. `wb4_monitor` reuses the checker on its command queue with `PROTOCOL_WB`; the event code `8'h08` is `*_ERR_ADDR_RANGE` in both packages
M	int	ADDR_WIDTH	Internal address-width alias

1.3 Ports

1.3.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

1.3.2 Timestamp

Port	Direction	Width	Description
i_mon_time	input	monbus_timestamp_t	Free-running counter broadcast by the monbus_group family; sampled on emission

1.3.3 Snooped APB Command Stream

Port	Direction	Width	Description
cmd_paddr	input	M	Command address (APB `paddr`)
cmd_pwrite	input	1	Command direction (1 = write, 0 = read)
cmd_valid	input	1	Command valid (from the monitor's snoop of the APB handshake)
cmd_ready	input	1	Command ready; `valid && ready && enable` marks an accepted command

1.3.4 Range Configuration

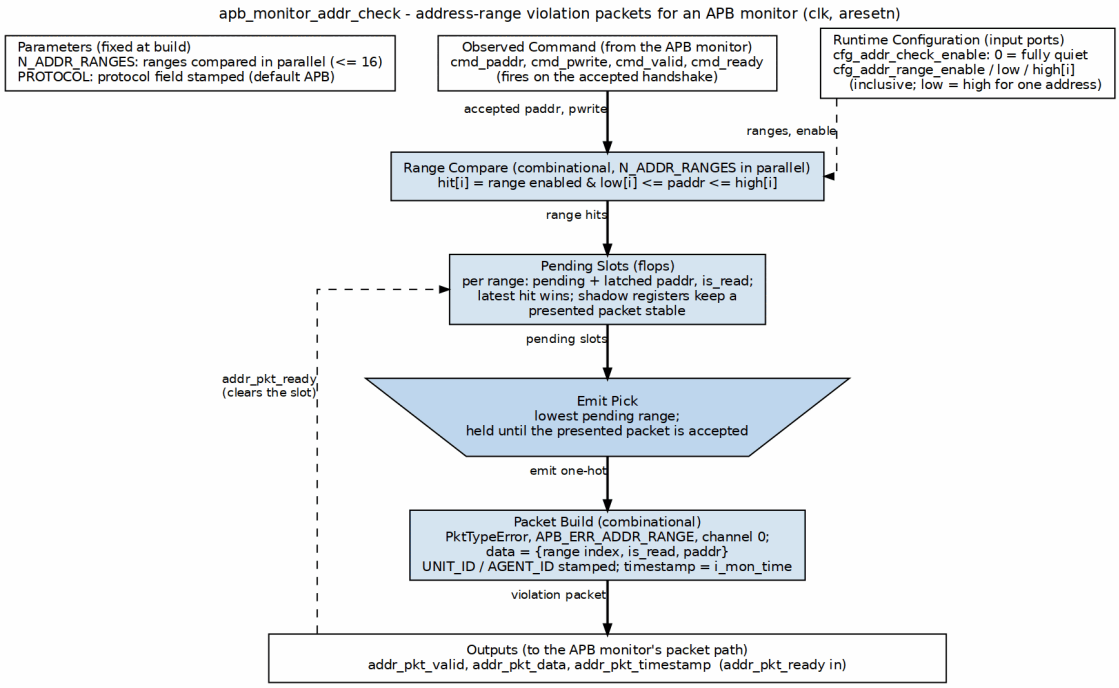
Port	Direction	Width	Description
cfg_addr_check_enable	input	1	Global enable for the checker (also gates output valid)
cfg_addr_range_enable	input	N_ADDR_RANGES	Per-range enable mask
cfg_addr_range_low	input	N_ADDR_RANGES × M	Per-range inclusive lower bound
cfg_addr_range_high	input	N_ADDR_RANGES × M	Per-range inclusive upper bound

1.3.5 Outgoing MonBus Packet

Port	Direction	Width	Description
addr_pkt_valid	output	1	Packet valid (a pending violation is ready to emit)
addr_pkt_ready	input	1	Downstream ready; `valid && ready` accepts the packet
addr_pkt_data	output	monitor_packet_t	Assembled MonBus error packet
addr_pkt_timestamp	output	monbus_timestamp_t	Timestamp side-band (`i_mon_time`)

1.4 Functional Description

1.4.1 Block diagram



apb_monitor_addr_check block diagram

Each accepted APB address is compared against every range at once; a hit in any enabled range is a violation. Each range has a pending slot where the latest hit wins. One packet drains per cycle, lowest range first, held stable while the bus stalls; the packet carries the range index, the direction and the address.

1.4.2 Combinational Range Hits

A command “fires” when `cmd_valid && cmd_ready && cfg_addr_check_enable`. For each range `i`, a one-hot hit is asserted when the range is enabled, the command fires, and `cmd_paddr` lies within `[cfg_addr_range_low[i], cfg_addr_range_high[i]]` inclusive. Setting a range’s low and high equal yields exact-match (watchpoint) semantics.

1.4.3 Pending Mask and Latched Snapshot

Each range has a pending bit. On a hit, the range’s pending bit sets and its address and direction are latched (`r_lat_addr[i] = cmd_paddr`, `r_lat_is_read[i] = !cmd_pwrite`). This decouples detection from emission so a range with a pending violation keeps its slot while the MonBus is backpressured. It does NOT preserve every violation: there is one latch per range, so repeated hits on the SAME range coalesce and the newest address wins. Two hits on range 0 during a stall yield one packet carrying the second address; the first address is not reported. This is the same lossy-but-honest behaviour the AXI sibling documents for its DEBUG ranges. Distinct

ranges do not coalesce with each other. A first-match priority encoder over `r_pending` selects the range to emit (`emit_oh` / `emit_idx`); `addr_pkt_valid` asserts when any range is pending and the checker is enabled. When the packet is accepted (`addr_pkt_valid && addr_pkt_ready`), the emitted range's pending bit clears. If a fresh hit and an accept collide on the same range in one cycle, the hit wins (the pending bit stays set), so a back-to-back violation is not dropped.

1.4.4 Packet Encoding

The emitted packet is the standard 128-bit MonBus format with a 64-bit `event_data` field, assembled via `create_monitor_packet`:

- **packet_type** = `PktTypeError` (`4'h0`)
- **protocol** = `PROTOCOL` parameter: `PROTOCOL_APB` (`4'h2`) by default, `PROTOCOL_WB` (`4'h5`) inside `wb4_monitor`
- **event_code** = `APB_ERR_ADDR_RANGE` (`8'h08`)
- **channel_id** = 0 (APB has no ID concept)
- **unit_id** = `UNIT_ID`, **agent_id** = `AGENT_ID`
- **event_data[63:60]** = range index (4 bits → up to 16 ranges)
- **event_data[59]** = `is_read` (1 = read, 0 = write)
- **event_data[58:0]** = offending `cmd_paddr` (zero-padded if `M < 59`, or its low 59 bits if `M >= 59`)

1.4.5 The is_read Carve-Out

The AXI variant drops the direction bit because AR and AW are separate channels — direction is implied by which monitor (read vs write) emitted the packet. APB has no such split: the same monitor sees both directions, so this block preserves an explicit `is_read` bit (carved out of the 60-bit address slot) so consumers can disambiguate read from write.

1.4.6 Timestamp Side-Band

Exactly as in the AXI variant, `i_mon_time` (the free-running counter broadcast by the `monbus_group` family) is driven straight out on `addr_pkt_timestamp`, so the consuming group can stamp the violation with a bus-consistent time.

1.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

1.6 Usage Examples

```
// Guard two reserved APB windows; report violations on the MonBus.
apb_monitor_addr_check #(
    .N_ADDR_RANGES  (4),
    .ADDR_WIDTH     (32),
    .UNIT_ID        (8'h10),
    .AGENT_ID       (16'h0001)
) u_apb_addr_check (
    .clk                    (pclk),
    .aresetn                (presetn),
    .i_mon_time             (mon_time),         // from the monbus
group

    // Snooped APB command handshake (from apb4_monitor)
    .cmd_paddr              (apb_paddr),
    .cmd_pwrite             (apb_pwrite),
    .cmd_valid              (cmd_valid),
    .cmd_ready              (cmd_ready),

    // Range config (range 0 = a window, range 1 = an exact
watchpoint).
    //
    // cfg_addr_range_low/high are PACKED 2D arrays, so an assignment
pattern
    // fills left-to-right starting at the HIGHEST index -- exactly
like a
    // concatenation. The leftmost item is range 3, the rightmost is
range 0.
    // Getting this backwards silently programs the wrong ranges: the
enable
    // mask lights up ranges 0-1 while the intended bounds sit on 3-2.
    //                             {  r3,       r2,       r1,
r0            }
    .cfg_addr_check_enable  (1'b1),
    .cfg_addr_range_enable  (4'b0011),
    .cfg_addr_range_low     ('{32'h0, 32'h0, 32'h0000_2000,
32'h0000_1000}),
    .cfg_addr_range_high    ('{32'h0, 32'h0, 32'h0000_2000,
```

```
32'h0000_1FFF}),

    // MonBus output (connect to the group's error-drain path)
    .addr_pkt_valid         (addr_pkt_valid),
    .addr_pkt_ready         (addr_pkt_ready),
    .addr_pkt_data          (addr_pkt_data),
    .addr_pkt_timestamp     (addr_pkt_timestamp)
);
```

1.7 Design Notes

1.7.1 Mirror of the AXI Variant

The block shares `axi_monitor_addr_check`'s *structure* – timestamp handling, the pending-mask/priority-encoder emission path, and the range-index/address packet layout – but it is **not** a behavioural mirror. The differences are load-bearing:

	`apb_monitor_addr_check`	`axi_monitor_addr_check`
Range meaning	**Blocklist** — an in-range **hit** raises `APB_ERR_ADDR_RANGE`	**Allowlist** — a **miss** (outside every enabled error range) raises `AXI_ERR_ADDR_RANGE`
Packet classes	Error only	Error *and* `PktTypeAddrMatch`
Per-range flavor	none	`ADDR_RANGE_IS_ERROR` selects DEBUG vs ERROR per range
Enable gating	`cfg_addr_check_enable`	plus `cfg_debug_enable` / `cfg_error_enable` per path
Channel id	hardwired `9'h0` (APB has no ID)	from `cmd_id`
`is_read`	preserved in the payload	dropped (recovered from `IS_READ` + unit/agent id)

The polarity inversion is the one that bites. Programming the APB ranges as if they were an allowlist — the AXI convention — flags every access *inside* your intended-legal window as a violation and lets

everything outside it pass silently. On APB, a configured range is a region you want to be told about.

1.7.2 Backpressure Safety

Detection latches into `r_pending` and emission drains one range per accepted packet, so a pending RANGE survives downstream backpressure – though repeat hits on that range coalesce to the newest address, so the count of packets is not the count of violations. The hit-versus-accept collision rule keeps the pending bit set when a new hit lands on a range being drained the same cycle.

A hit landing on a range whose beat is already being presented goes to a per-range **shadow slot** rather than overwriting the latched payload, and is installed when that beat is accepted. Rewriting a beat already on the bus would change the payload under a held valid, which the MonBus valid/ready contract forbids; the shadow keeps newest-wins for every beat except the one on the wire. The AXI sibling carries the same guard for the same reason.

1.7.3 Address Field Width

For `ADDR_WIDTH >= 59` the low 59 bits of the address are carried; for narrower buses the address is zero-extended into the 59-bit payload slot. The upper four `event_data` bits are always the range index.

1.7.4 Exact-Match Watchpoints

Program `cfg_addr_range_low[i] == cfg_addr_range_high[i]` to turn a range into a single-address watchpoint.

1.8 Related Modules

Used by: the `apb4_monitor` pipeline (address-range violation reporting).

Uses: `monitor_common_pkg` (`PktTypeError`, `PROTOCOL_APB`, `monbus_timestamp_t`, `create_monitor_packet`); `monitor_amba4_pkg` (`APB_ERR_ADDR_RANGE` event code); `reset_defs.svh` (`ALWAYS_FF_RST` / `RST_ASSERTED` reset macros).

See also: `axi_monitor_addr_check.sv` (AXI-side equivalent, no `is_read` bit); `apb4_monitor.sv` (the APB monitor this checker plugs into).

1.9 Testing

`val/amba/test_apb_monitor_addr_check.py` exercises this module. It collects 1 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb_monitor_addr_check.py -v
```

1.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023. developer.arm.com/documentation/ihi0024/e. — §3.4 “Error response” (PSLVERR); the range check here is a monitor policy on PADDR, not a rule of the specification.
- Boulé, M., Zilic, Z. “Efficient Automata-Based Assertion-Checker Synthesis of SEREs for Hardware Emulation.” *Proc. Asia and South Pacific Design Automation Conf. (ASP-DAC 2007)*, pp. 324-329, Jan. 2007. DOI: 10.1109/ASPDAC.2007.358006. — related work: checker circuits built in hardware for emulation and silicon; this checker is a hand-written parallel range compare.
- IEEE-ISTO 5001-2003. *The Nexus 5001 Forum Standard for a Global Embedded Processor Debug Interface*, Version 2.0, IEEE Industry Standards and Technology Organization, Dec. 2003 (defining non-peer standard). — §4.8 “Breakpoints/Watchpoints”: a comparable address-match event in a processor debug standard.
- Herveille, R. (ed.). *Wishbone B4: WISHBONE System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores*, OpenCores, 2010. — ch. 3 bus cycles; `wb4_monitor` reuses this checker on its command queue.

- RTL: `rtl/amba/ocelus/apb_monitor_addr_check.sv`
- Packet format: `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`
- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Design Guide: `docs/markdown/rtl-amba/index.md`
- Indexed in `rtl/amba/references/README.md` (local PDF: `HardwareAssertionCheckers__Boule2007.pdf`, `DebugTraceInterfaceStandard__IeeeIsto2003.pdf`, `WishboneB4Spec__OpenCores2010.pdf`)

Last Updated: 2026-07-15

1.11 Navigation

- ← Back to Shared Infrastructure Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

2 arbiter_monbus_common

Module: `arbiter_monbus_common.sv` **Location:** `rtl/amba/ocelus/` **Category:** Core Infrastructure **Status:** Production Ready

Plane: output plane

2.1 Overview

`arbiter_monbus_common` is the shared **telemetry block** that watches an arbiter and reports on it. It is instantiated inside `arbiter_rr_pwm_monbus` and `arbiter_wrr_pwm_monbus` as `u_monitor`.

It performs **no arbitration of its own.** Every arbiter-facing signal (`request`, `grant_valid`, `grant`, `grant_id`, `grant_ack`, `block_arb`) is an **input**: the module snoops the decisions an external arbiter has already made, measures them, and emits 128-bit monitor packets about what it saw. There is no request/grant output, no data mux, and no client stream inputs.

> **Looking for the N:1 monitor-bus merge?** That is a different module: `monbus_arbiter`, which takes N `monbus_valid/packet` streams and arbitrates them onto one. This page has described that module in the past; it does not describe this one.

What it does, in five verbs:

1. **Observe:** sample the arbiter's request/grant/ACK activity without perturbing it
2. **Detect:** identify starvation, ACK timeouts, protocol violations, and threshold crossings
3. **Measure:** maintain fairness-deviation and grant-efficiency metrics
4. **Report:** format findings as `monitor_packet_t` records and queue them for the monitor bus
5. **Expose:** drive silicon-debug status outputs (`debug_*`) for direct observation

Feature summary:

- **Snoops any arbiter** — RR or WRR, with or without the grant-ACK handshake
- **Per-client ACK timeout tracking** with a configurable threshold
- **Protocol violation detection** — multiple simultaneous grants, spurious ACKs, grant without request

- **Starvation detection** with per-client timing
- **Fairness deviation** measured against configured client weights
- **Grant efficiency** tracking (grants issued vs. completed)
- **128-bit monitor packets** emitted through an internal FIFO, plus a 64-bit side-band timestamp

---

2.2 Parameters

Parameter	Type	Default	Description
`CLIENTS`	int	4	Width of the **snooped arbiter's** client vectors (`request`, `grant`, `grant_ack`). Nothing is arbitrated here – this sizes what is observed
`WAIT_GNT_ACK`	int	0	1 = require a grant-ACK handshake
`WEIGHTED_MODE`	int	0	**Dead** — declared but referenced by no logic. Fairness analysis always uses the `cfg_max_thresh` weights, and this module performs no arbitration for a mode switch to modulate. Setting it changes nothing
`MON_AGENT_ID`	logic [15:0]	16'h0010	Monitor agent identifier (16-bit)
`MON_UNIT_ID`	logic [7:0]	8'h00	Monitor unit

Parameter	Type	Default	Description
			identifier (8-bit)
MON_FIFO_DEPTH	int	8	Monitor packet FIFO depth
MON_FIFO_ALMOST_MARGIN	int	1	Almost-full margin for the monitor FIFO
FAIRNESS_REPORT_CYCLES	int	256	How often the fairness deviation is **recomputed** (cycles). **Not a sliding window:** r_grant_counters and r_total_grants are cleared on reset alone, so the deviation is always computed from grants accumulated since reset. MIN_GRANTS_FOR_FAIRNESS is likewise compared against the lifetime total. After a traffic-phase or weight change the figure converges toward the new distribution rather than snapping to it
MIN_GRANTS_FOR_FAIRNESS	int	100	Minimum grants before a fairness

Parameter	Type	Default	Description
			report is valid
DEFAULT_ACK_TIMEOUT	int	64	**Dead** — declared and referenced by no logic. The effective threshold is whatever is driven on the cfg_mon_ack_timeout_thresh **port** (arbiter_rr_pwm_monbus hardwires 16'h40; arbiter_wrr_pwm_monbus passes its own port through). Overriding this parameter changes nothing

MAX_LEVELS (default 16) is an independent parameter, not a derived one: it sets the per-client weight resolution and therefore the width of the cfg_max_thresh port (CXMTW = CLIENTS * $clog2(MAX_LEVELS)). arbiter_wrr_pwm_monbus passes its own MAX_LEVELS down. Change it and the port width changes with it.

N ($clog2(CLIENTS)), MON_FIFO_COUNT_WIDTH, MAX_LEVELS_WIDTH and the weight-vector widths are derived from the parameters above. They are declared with parameter rather than localparam, so they are overridable in principle — don't, since they must stay consistent with what they are derived from.

2.2.1 Derived Parameters (do not override)

These are declared as parameter so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
MFCW	MON_FIFO_COUNT_WIDTH
MTW	MAX_LEVELS_WIDTH

2.3 Ports

All ports, from rtl/amba/ocelus/arbiter_monbus_common.sv. Every arbiter-facing signal is an INPUT: this module snoops and never drives the arbiter.

Clock and reset

Port	Dir	Width	Description
clk	In	1	Clock. Note this module uses clk/rst_n, not the AMBA aclk/aresetn
rst_n	In	1	Active-low reset

Snooped arbiter interface (inputs only)

Port	Dir	Width	Description
request	In	CLIENTS	Client request vector
grant_valid	In	1	Grant is valid this cycle
grant	In	CLIENTS	One-hot grant vector
grant_id	In	N	Binary-encoded grant ID
grant_ack	In	CLIENTS	Per-client grant acknowledge
block_arb	In	1	Arbiter is blocked (optional; tie 0 if unused)

Configuration

Port	Dir	Width	Description
cfg_mon_enable	In	1	Master enable for the monitor
cfg_mon_pkt_type_enable	In	16	Per-packet-type enable mask
cfg_mon_latency_thresh	In	16	Grant-latency threshold
cfg_mon_starvation_thresh	In	16	Per-client starvation threshold
cfg_mon_fairness_thresh	In	16	Fairness-deviation threshold
cfg_mon_active_thresh	In	16	Active-client-count threshold
cfg_mon_ack_timeout_thresh	In	16	ACK timeout threshold
cfg_mon_efficiency_thresh	In	16	Grant-efficiency threshold
cfg_mon_sample_period	In	8	Sampling period for the periodic metrics
cfg_max_thresh	In	CXMTW	Packed per-client weight/threshold vector
i_mon_time	In	monbus_timestamp_t	Shared timestamp input

Monitor bus output

Port	Dir	Width	Description
monbus_valid	Out	1	Packet valid
monbus_ready	In	1	Downstream accepts the packet
monbus_packet	Out	monitor_packet_t	128-bit monitor packet

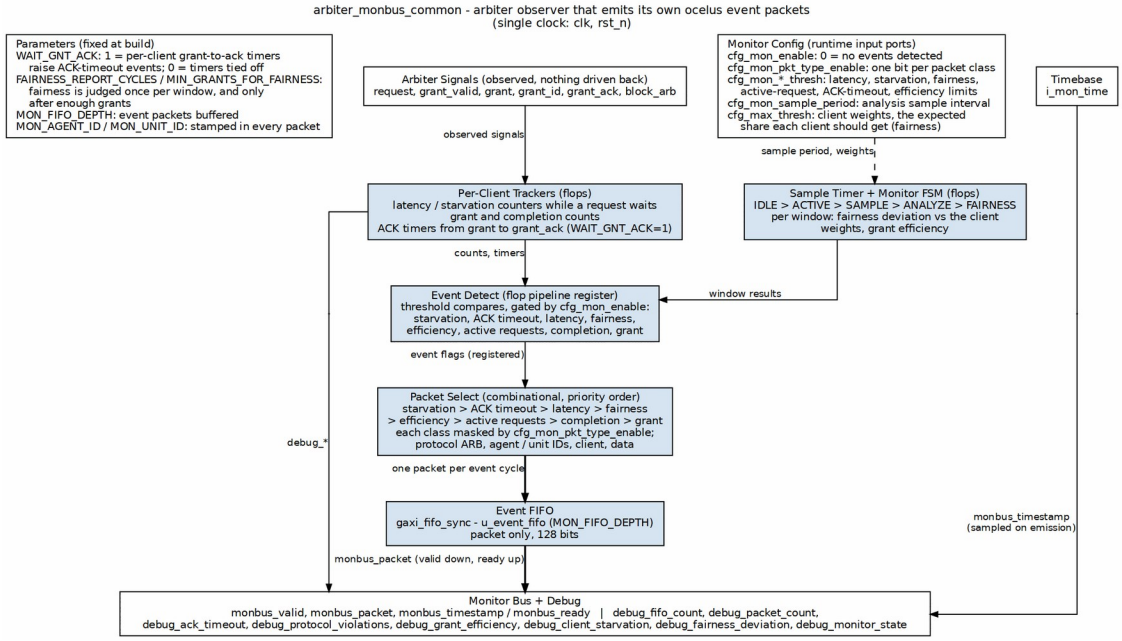
Port	Dir	Width	Description
monbus_timestamp	Out	monbus_timestamp_t	Side-band sampled time

Debug status outputs

Port	Dir	Width	Description
debug_fifo_count	Out	$clog2(MON_FIFO_DEPTH)+1	Occupancy of the packet FIFO
debug_packet_count	Out	16	Packets emitted
debug_ack_timeout	Out	CLIENTS	Per-client ACK timeout status
debug_protocol_violations	Out	16	Protocol-violation count
debug_grant_efficiency	Out	16	Grant efficiency, percent
debug_client_starvation	Out	CLIENTS	Per-client starvation flags
debug_fairness_deviation	Out	16	Fairness-deviation metric
debug_monitor_state	Out	3	Monitor internal state

2.4 Functional Description

2.4.1 Block diagram



arbiter_monbus_common block diagram

This module only observes: the arbiter's request, grant, ack and block signals come in at the top and nothing goes back. Per-client trackers count waiting time and grants and run the grant-to-ack timers. The sample timer and monitor FSM judge fairness against the client weights and grant efficiency once per window. The event register compares all of it against the thresholds. A priority select turns the highest-priority enabled event into one ocelus packet per cycle, and the event FIFO holds packets until the bus takes them. The timestamp is sampled from i_mon_time when a packet leaves, and the debug counters run down the left lane.

The single monitor-bus output carries this module's **own** event packets. It is not a merge of upstream streams — there are none.

This module is instantiated automatically within the higher-level monitor modules (`arbiter_rr_pwm_monbus`, `arbiter_wrr_pwm_monbus`); you configure its behavior through the top-level monitor parameters rather than instantiating it yourself. See the individual monitor pages for configuration examples.

2.5 Timing Characteristics

Metric	Value	Notes
Latency	1-2 cycles	Typical processing

Metric	Value	Notes
		delay
Throughput	1 operation/cycle	Maximum rate

2.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
arbiter_monbus_common #(
    .CLIENTS                (4),
    .WAIT_GNT_ACK           (0),
    .WEIGHTED_MODE          (0),
    .MON_AGENT_ID           (16'h0010),
    .MON_UNIT_ID            (8'h00),
    .MON_FIFO_DEPTH         (8),
    .MON_FIFO_ALMOST_MARGIN(1),
    .FAIRNESS_REPORT_CYCLES(256),
    .MIN_GRANTS_FOR_FAIRNESS(100),
    .DEFAULT_ACK_TIMEOUT    (64)
) u_arbiter_monbus_common (
    .clk                    (clk),
    .rst_n                  (rst_n),
    .cfg_max_thresh         (cfg_max_thresh),
    .request                (request),
    .grant_valid            (grant_valid),
    .grant                  (grant),
    .grant_id               (grant_id),
    .grant_ack              (grant_ack),
    .block_arb              (block_arb),
    .cfg_mon_enable         (cfg_mon_enable),
    .cfg_mon_pkt_type_enable(cfg_mon_pkt_type_enable),
    .cfg_mon_latency_thresh(cfg_mon_latency_thresh),
    .cfg_mon_starvation_thresh(cfg_mon_starvation_thresh),
    .cfg_mon_fairness_thresh(cfg_mon_fairness_thresh),
    .cfg_mon_active_thresh (cfg_mon_active_thresh),
    .cfg_mon_ack_timeout_thresh(cfg_mon_ack_timeout_thresh),
    .cfg_mon_efficiency_thresh(cfg_mon_efficiency_thresh),
    .cfg_mon_sample_period (cfg_mon_sample_period),
    .i_mon_time             (i_mon_time),
    .monbus_valid           (monbus_valid),
    .monbus_ready           (monbus_ready),
    .monbus_packet          (monbus_packet),
    .monbus_timestamp       (monbus_timestamp),
    .debug_fifo_count       (debug_fifo_count),
    .debug_packet_count     (debug_packet_count),
```

```
    .debug_ack_timeout     (debug_ack_timeout),
    .debug_protocol_violations(debug_protocol_violations),
    .debug_grant_efficiency(debug_grant_efficiency),
    .debug_client_starvation(debug_client_starvation),
    .debug_fairness_deviation(debug_fairness_deviation),
    .debug_monitor_state   (debug_monitor_state)
);
```

2.7 Design Notes

This module snoops and never drives. Every arbiter-facing port is an input; it observes request/grant/ACK activity and cannot perturb the arbitration it is measuring. That is the property that lets it be dropped into an existing arbiter without changing behaviour.

It uses `clk`/`rst_n`, not the AMBA `aclk`/`aresetn`. An easy mis-wire in a file where every neighbour uses the AMBA names.

The compliance model belongs to the DV framework, not here. Round-robin order, starvation and fairness verdicts are computed in `CocoTBFramework`'s `ArbiterCompliance`; several historical "violations" were defects in that model rather than in any arbiter (common BUG-006 (was COMMON-016) through common BUG-009 (was COMMON-019)).

2.8 Related Modules

Used by: `arbiter_rr_pwm_monbus`, `arbiter_wrr_pwm_monbus` (as `u_monitor`).

See also: axi_monitor_reporter

2.9 Testing

Coverage targets:

- Functional correctness of core logic
- Boundary conditions (min/max values)
- Error handling and recovery
- Interface protocol compliance

See: `val/amba/test_arbiter_monbus_common.py` for verification tests

2.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Sprunt, B. “The Basics of Performance-Monitoring Hardware.” *IEEE Micro*, 22(4):64-71, July/Aug. 2002. DOI: 10.1109/MM.2002.1028477. — related work: on-chip event counters for performance monitoring; the grant and fairness counters here report the same kind of data for an arbiter.
- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. “An Event-Based Monitoring Service for Networks on Chip.” *ACM Trans. Design Automation of Electronic Systems*, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126. — related work: probes that turn observed activity into event packets sent over the interconnect.
- Dally, W.J., Towles, B. *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004, ch. 18 “Arbitration”, §18.4.2 “Round-Robin Arbiter” and §18.4.4 “Weighted Round-Robin Arbiter”. — textbook treatment (secondary source) of the arbiters this block observes.

- **Monitor Architecture:** `docs/markdown/rtl-amba/overview.md`
- **Monitor Configuration Guide:** Monitor Base Configuration
- **Packet Format Specification:** `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

2.11 Navigation

- **← Back to Shared Infrastructure Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

3 arbiter_rr_pwm_monbus

Module: `arbiter_rr_pwm_monbus.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: output plane

3.1 Overview

Three blocks in one wrapper: a fair round-robin arbiter, a PWM generator that gates it, and the `arbiter_monbus_common` telemetry block watching the result. The internal sizing is fixed on purpose — 16-bit PWM resolution, 16-entry monitor FIFO, 256-cycle fairness reporting — so every instance behaves identically and there are fewer knobs to get wrong.

Use it where multiple masters need equal-priority access and you also want periodic arbitration windows: the PWM output drives the arbiter's `block_arb` input directly, so you get precise time-division control, while the monitor bus gives you real-time visibility into arbitration behavior, request latencies, and problems like starvation or protocol violations. The standardized internal configurations (PWM width, FIFO depth, fairness intervals) keep behavior consistent across instantiations, simplifying integration and reducing parameter proliferation.

- Fair round-robin arbitration across N clients
- PWM-based arbiter blocking for periodic control
- Standardized 16-bit PWM resolution
- Comprehensive monitoring via arbiter_monbus_common
- Optional ACK protocol support
- Fixed internal sizing for consistency (16-entry FIFO, 256-cycle fairness reporting)
- Configurable client count (1-64)
- Real-time performance and fairness tracking

3.2 Parameters

Parameter	Type	Default	Description
CLIENTS	int	4	Number of arbitration clients (1-64)
WAIT_GNT_ACK	int	0	ACK protocol enable (0=disable,

Parameter	Type	Default	Description
			1=enable)
MON_AGENT_ID	logic [15:0]	16’h0010	Monitor agent identifier (16-bit per monitor bus protocol)
MON_UNIT_ID	logic [7:0]	8’h00	Monitor unit identifier (8-bit per monitor bus protocol)
USE_MONITOR	bit	1	Synthesis-time monitor enable. When 0, omits monbus telemetry block (arbiter and PWM remain intact). When 1, full monitoring.

3.2.1 Fixed Internal Configurations (Not User-Configurable)

Configuration	Value	Rationale
PWM_WIDTH	16 bits	Adequate resolution for most use cases
MON_FIFO_DEPTH	16 entries	Optimal for monitoring scenarios
MON_FIFO_ALMOST_MARGIN	2 entries	Safety margin for FIFO
FAIRNESS_REPORT_CYCLES	256 cycles	Power-of-2 efficient interval
MIN_GRANTS_FOR_FAIRNESS	64 grants	Statistical significance threshold

3.3 Ports

3.3.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Clock signal
rst_n	input	1	Active-low asynchronous reset

3.3.2 Arbiter Interface

Port	Direction	Width	Description
request	input	CLIENTS	Client request signals
grant_valid	output	1	Grant is valid this cycle
grant	output	CLIENTS	One-hot grant vector
grant_id	output	$clog2(CLIENTS)	Binary encoded grant ID
grant_ack	input	CLIENTS	Grant acknowledge signals (ACK mode only)

3.3.3 PWM Configuration

Port	Direction	Width	Description
cfg_pwm_sync_rst_n	input	1	PWM synchronous reset (active-low)
cfg_pwm_start	input	1	Start PWM generation
cfg_pwm_duty	input	16	Duty cycle value (0 to period-1)
cfg_pwm_period	input	16	Period value (1 to 65535)

Port	Direction	Width	Description
cfg_pwm_repeat_count	input	16	Number of periods to generate (0=infinite)
cfg_pwm_sts_done	output	1	PWM sequence complete status
pwm_out	output	1	PWM output (controls arbiter blocking)

3.3.4 Monitor Configuration

Port	Direction	Width	Description
cfg_mon_enable	input	1	Global monitor enable
cfg_mon_pkt_type_enable	input	16	Packet type enable mask
cfg_mon_latency	input	16	Latency threshold (cycles)
cfg_mon_starvation	input	16	Starvation threshold (cycles)
cfg_mon_fairness	input	16	Fairness deviation threshold (percent)
cfg_mon_active	input	16	Active request count threshold
cfg_mon_period	input	8	Sample period for metrics

3.3.5 Monitor Bus Output

Port	Direction	Width	Description
monbus_valid	output	1	Monitor bus packet valid
monbus_ready	input	1	Monitor bus ready (from

Port	Direction	Width	Description
			downstream)
monbus_packet	output	128	`monitor_packet_t` event packet
monbus_timestamp	output	64	Side-band timestamp sampled at packet emission, travelling with the packet

3.3.6 Monitor Time Input

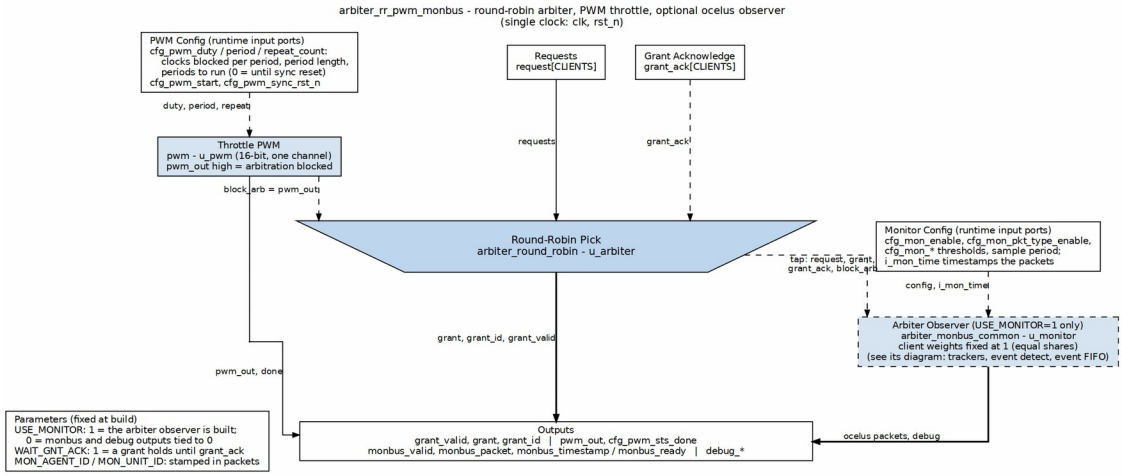
Port	Direction	Width	Description
i_mon_time	input	64	Free-running monitor-time broadcast from the `monbus_*_group` family. **Must be connected** — leaving it open floats the timestamp every emitted packet carries

3.3.7 Debug Outputs

Port	Direction	Width	Description
debug_fifo_count	output	$clog2(17)	Monitor FIFO fill level (0-16)
debug_packet_count	output	16	Total packets generated

3.4 Functional Description

3.4.1 Block diagram



arbiter_rr_pwm_monbus block diagram

Requests go straight down the center into the round-robin pick, and its heavy grant goes to the outputs. On the left, a single-channel pwm drives block_arb, so the PWM duty cycle sets how often arbitration is held off. On the right, when USE_MONITOR=1, arbiter_monbus_common taps the arbiter's signals and sends its own ocelus packets to the outputs. The monitor sees every client with weight 1.

The module integrates three key components.

3.4.2 Round-Robin Arbiter

Provides fair arbitration using the arbiter_round_robin module: - Equal priority for all clients - Sequential rotation through active requests - Optional ACK protocol (WAIT_GNT_ACK=1) - Block_arb input for external flow control

3.4.3 PWM Generator

Controls arbiter availability through periodic blocking: - 16-bit resolution (65536 discrete levels) - Configurable duty cycle and period - Repeat count for finite sequences - PWM output directly drives arbiter block_arb input

Duty Cycle Calculation: - pwm_out = 1 (arbiter blocked) for `cfg_pwm_duty` cycles - pwm_out = 0 (arbiter active) for `cfg_pwm_period - cfg_pwm_duty` cycles

Example: 25% arbiter availability:

```
cfg_pwm_period = 16'd100;      // 100-cycle period
cfg_pwm_duty   = 16'd75;       // Block for 75 cycles
// Result: arbiter active for 25 cycles, blocked for 75 cycles
```

3.4.4 Monitor Bus Common

Comprehensive monitoring via arbiter_monbus_common: - Default client weights = 1 (equal priority) - Tracks grant distribution, latencies, starvation - Fixed internal configurations ensure consistency - Event packets include request latency, starvation detection, active request count

3.4.5 Operation Flow

1. Clients assert request signals
2. PWM output modulates arbiter availability (pwm_out -> block_arb)
3. When arbiter active (pwm_out=0), round-robin arbiter grants one request
4. Monitor tracks arbitration events (grants, timeouts, fairness)
5. Event packets generated to monitor bus output
6. If ACK mode enabled, arbiter waits for grant_ack before next grant

---

3.5 Timing Characteristics

This module is **sequential**: the AST shows clocked processes, so it holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

3.6 Usage Examples

```
// 8-client round-robin arbiter with 30% availability window
arbiter_rr_pwm_monbus #(
    .CLIENTS      (8),
    .WAIT_GNT_ACK (0),        // No ACK protocol
    .MON_AGENT_ID (16'h0015),
    .MON_UNIT_ID  (8'h03)
) u_rr_arb (
    .clk                    (clk),
    .rst_n                  (rst_n),

    // Arbiter interface
    .request                (client_requests),
    .grant_valid            (grant_valid),
    .grant                  (grant_onehot),
```

```
    .grant_id               (grant_binary),
    .grant_ack              (8'h00),  // Not used (ACK disabled)

    // PWM configuration - 30% availability
    .cfg_pwm_sync_rst_n     (1'b1),
    .cfg_pwm_start          (pwm_start),
    .cfg_pwm_duty           (16'd70),    // Block for 70 cycles
    .cfg_pwm_period         (16'd100),   // Out of 100 cycles total
    .cfg_pwm_repeat_count   (16'd0),     // Infinite repeat
    .cfg_pwm_sts_done       (pwm_done),
    .pwm_out                (pwm_signal),

    // Monitor configuration
    .cfg_mon_enable         (1'b1),
    .cfg_mon_pkt_type_enable (16'h000F),  // Error, timeout,
completion, threshold
    .cfg_mon_latency        (16'd50),    // 50-cycle latency warning
    .cfg_mon_starvation     (16'd200),   // 200-cycle starvation
threshold
    .cfg_mon_fairness       (16'd15),    // 15% fairness deviation
    .cfg_mon_active         (16'd7),     // Warn if 7+ active
requests
    .cfg_mon_period         (8'd128),

    // Monitor bus output
    .monbus_valid           (mon_valid),
    .monbus_ready           (mon_ready),
    .monbus_packet          (mon_packet),
    .monbus_timestamp       (mon_timestamp),

    // Monitor time broadcast (from the monbus group) -- not optional
    .i_mon_time             (mon_time),

    // Debug
    .debug_fifo_count       (mon_fifo_level),
    .debug_packet_count     (mon_pkt_count)
);

// Start PWM generation
initial begin
    pwm_start = 1'b0;
    @(posedge clk);
    pwm_start = 1'b1;
    @(posedge clk);
    pwm_start = 1'b0;
end
```

3.7 Design Notes

3.7.1 PWM Control Strategy

The PWM output directly controls arbiter blocking: - **pwm_out** = **1**: Arbiter blocked (no grants issued) - **pwm_out** = **0**: Arbiter active (can issue grants)

This allows precise timing control for: - Time-division multiple access (TDMA) scheduling - Bandwidth reservation windows - Periodic maintenance operations - Energy-efficient arbitration (idle periods)

3.7.2 Fixed Configuration Benefits

The standardized internal configurations provide: - **Consistency**: All instances behave identically - **Predictability**: Known FIFO depths, fairness intervals - **Simplified Integration**: Fewer parameters to configure - **Verified Operation**: Tested configurations

If different internal sizes are needed, create a new variant rather than parameterizing.

3.7.3 Fairness in Round-Robin Mode

All clients have equal priority (weight = 1 in monitoring logic). Fairness deviation events are generated if any client receives significantly more or fewer grants than expected. The measurement is **cumulative since reset**, not a sliding window: `FAIRNESS_REPORT_CYCLES` (256) only paces how often the deviation is recomputed from the lifetime grant counters, which are cleared on reset alone.

Expected: Each client receives (1/CLIENTS) * 100% of grants Threshold: cfg_mon_fairness parameter (percent)

3.7.4 Monitor Bus Integration

See arbiter_monbus_common.md for complete monitoring details. Key points: - Uses PROTOCOL_ARB (3'b011) event encoding - Fixed 16-entry FIFO absorbs bursts, but does **not** guarantee zero loss: the event registers are overwritten every cycle and the FIFO write is not held when full, so a one-cycle event (grant, completion) that fires while the FIFO is full is dropped. Persistent conditions (starvation, thresholds) re-present until accepted - Per-client latency and starvation tracking - Grant efficiency monitoring

3.8 Related Modules

Used by: system arbitration hierarchies, multi-master interconnects, TDMA scheduling systems.

Uses: `arbiter_round_robin.sv` (core RR arbiter); `pwm.sv` (PWM generator); `arbiter_monbus_common.sv` (monitoring infrastructure).

See also: `arbiter_wrr_pwm_monbus.sv` (weighted variant); `monbus_arbiter.sv` (aggregates monitor bus streams).

3.9 Testing

`val/amba/test_arbiter_rr_pwm_monbus.py` exercises this module. It collects 8 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_arbiter_rr_pwm_monbus.py -v
```

3.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Gupta, P., McKeown, N. “Designing and Implementing a Fast Crossbar Scheduler.” *IEEE Micro*, 19(1):20-28, Jan./Feb. 1999. DOI: 10.1109/40.748793. — the masked/unmasked priority-encoder round-robin used by the `arbiter_round_robin` core (indexed in `rtl/common/references/README.md`).
- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. “An Event-Based Monitoring Service for Networks on Chip.” *ACM Trans. Design Automation of Electronic Systems*, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126. — related work for the attached telemetry block: event packets from a hardware probe.
- Dally, W.J., Towles, B. *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004, ch. 18 “Arbitration”, §18.4.2 “Round-Robin Arbiter” and §18.4.4 “Weighted Round-Robin Arbiter”. — textbook treatment (secondary source).
- Internal: docs/markdown/rtl-amba/index.md (AMBA subsystem requirements)
- Internal: docs/markdown/rtl-amba/monitor/arbiter_monbus_common.md (monitoring details)

- RTL: `rtl/amba/ocelus/arbiter_rr_pwm_monbus.sv`
- Tests: `val/amba/test_arbiter_rr_pwm_monbus.py`
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

Last Updated: 2025-10-24

3.11 Navigation

- **← Back to Shared Infrastructure Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

4 arbiter_wrr_pwm_monbus

Module: `arbiter_wrr_pwm_monbus.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: output plane

4.1 Overview

The weighted sibling of `arbiter_rr_pwm_monbus`: priority-based arbitration with configurable client weights, the same PWM flow control, and the same `arbiter_monbus_common` telemetry with standardized internal configurations. Higher-weight clients receive proportionally more grants while lower-weight clients still get guaranteed service — no starvation — and the monitoring infrastructure tracks whether the actual grant distribution matches the configured weights and reports fairness violations when it drifts.

The PWM integration adds temporal control of arbiter availability on top: TDMA windows or bandwidth reservation, with priority-based selection inside each active window.

- Weighted round-robin arbitration with configurable priorities
- Per-client weight thresholds (1 to `MAX_LEVELS-1`; the field is `$clog2(MAX_LEVELS)` bits, so `MAX_LEVELS` itself is not representable — see Design Notes)
- PWM-based arbiter blocking for periodic control
- Standardized 16-bit PWM resolution
- Comprehensive fairness deviation monitoring
- Optional ACK protocol support
- Fixed internal sizing (16-entry FIFO, 256-cycle fairness reporting)
- Enhanced debug outputs for priority tracking

4.2 Parameters

Parameter	Type	Default	Description
MAX_LEVELS	int	16	Maximum weight levels per client (1-64)
CLIENTS	int	4	Number of arbitration clients (1-64)

Parameter	Type	Default	Description
WAIT_GNT_ACK	int	0	ACK protocol enable (0=disable, 1=enable)
MON_AGENT_ID	logic [15:0]	16'h0010	Monitor agent identifier (16-bit per monitor bus protocol)
MON_UNIT_ID	logic [7:0]	8'h00	Monitor unit identifier (8-bit per monitor bus protocol)
USE_MONITOR	bit	1	Synthesis-time monitor enable. When 0, omits monbus telemetry block (arbiter and PWM remain intact). When 1, full monitoring.

4.2.1 Fixed Internal Configurations (Not User-Configurable)

Configuration	Value	Rationale
PWM_WIDTH	16 bits	Adequate resolution for most use cases
MON_FIFO_DEPTH	16 entries	Optimal for monitoring scenarios
MON_FIFO_ALMOST_MARGIN	2 entries	Safety margin for FIFO
FAIRNESS_REPORT_CYCLES	256 cycles	Power-of-2 efficient interval
MIN_GRANTS_FOR_FAIRNESS	64 grants	Statistical significance threshold

4.2.2 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`MAX_LEVELS_WIDTH`	`$clog2(MAX_LEVELS)`
`N`	`$clog2(CLIENTS)`
`CXMTW`	`CLIENTS * MAX_LEVELS_WIDTH`

4.3 Ports

4.3.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Clock signal
rst_n	input	1	Active-low asynchronous reset

4.3.2 Arbiter Configuration and Interface

Port	Direction	Width	Description
cfg_arb_max_thresh	input	CLIENTS * $clog2(MAX_LEVELS)	Per-client weight thresholds
request	input	CLIENTS	Client request signals
grant_valid	output	1	Grant is valid this cycle
grant	output	CLIENTS	One-hot grant vector
grant_id	output	$clog2(CLIENTS)	Binary encoded grant ID
grant_ack	input	CLIENTS	Grant acknowledge signals (ACK mode

Port	Direction	Width	Description
			only)

4.3.3 PWM Configuration

Port	Direction	Width	Description
cfg_pwm_sync_rst_n	input	1	PWM synchronous reset (active-low)
cfg_pwm_start	input	1	Start PWM generation
cfg_pwm_duty	input	16	Duty cycle value (0 to period-1)
cfg_pwm_period	input	16	Period value (1 to 65535)
cfg_pwm_repeat_count	input	16	Number of periods (0=infinite)
cfg_pwm_sts_done	output	1	PWM sequence complete status
pwm_out	output	1	PWM output (controls arbiter blocking)

4.3.4 Monitor Configuration

Port	Direction	Width	Description
cfg_mon_enable	input	1	Global monitor enable
cfg_mon_pkt_type_enable	input	16	Packet type enable mask
cfg_mon_latency_thresh	input	16	Latency threshold (cycles)

Port	Direction	Width	Description
cfg_mon_starvation_thresh	input	16	Starvation threshold (cycles)
cfg_mon_fairness_thresh	input	16	Fairness deviation threshold (percent)
cfg_mon_active_thresh	input	16	Active request count threshold
cfg_mon_ack_timeout_thresh	input	16	ACK timeout threshold (cycles)
cfg_mon_efficiency_thresh	input	16	Grant efficiency threshold (percent)
cfg_mon_sample_period	input	8	Sample period for metrics

4.3.5 Monitor Bus Output

Port	Direction	Width	Description
monbus_valid	output	1	Monitor bus packet valid
monbus_ready	input	1	Monitor bus ready (from downstream)
monbus_packet	output	128	`monitor_packet_t` event packet
monbus_timestamp	output	64	Side-band timestamp sampled at packet emission, travelling with the packet

4.3.6 Monitor Time Input

Port	Direction	Width	Description
i_mon_time	input	64	Free-running monitor-time

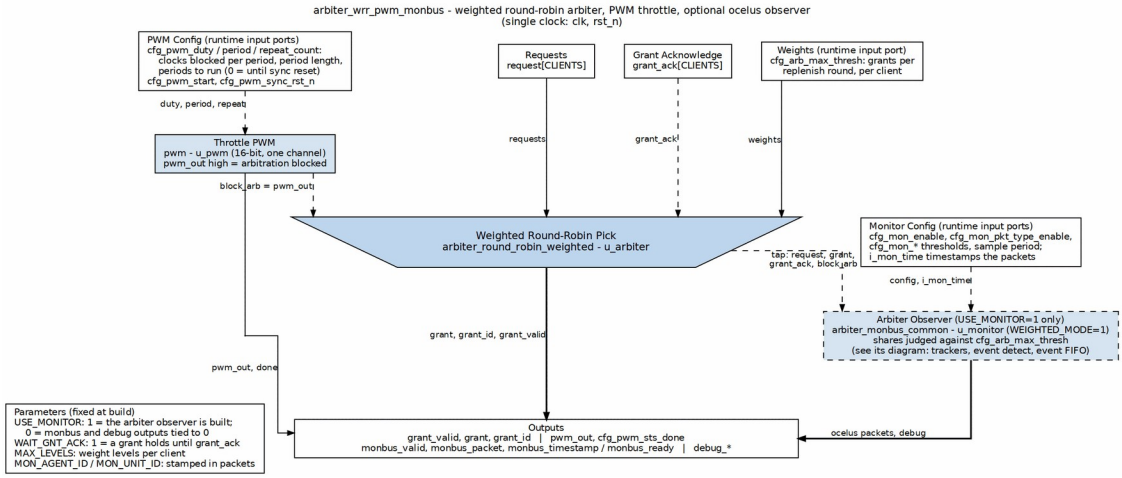
Port	Direction	Width	Description
			broadcast from the `monbus_*_group` family. **Must be connected** — leaving it open floats the timestamp every emitted packet carries

4.3.7 Enhanced Debug Outputs

Port	Direction	Width	Description
debug_fifo_count	output	$clog2(17)	Monitor FIFO fill level
debug_packet_count	output	16	Total packets generated
debug_ack_timeout	output	CLIENTS	Per-client ACK timeout status
debug_protocol_violations	output	16	Protocol violation count
debug_grant_efficiency	output	16	Grant efficiency percentage
debug_client_starvation	output	CLIENTS	Per-client starvation flags
debug_fairness_deviation	output	16	Maximum fairness deviation
debug_monitor_state	output	3	Monitor internal state

4.4 Functional Description

4.4.1 Block diagram



arbiter_wrr_pwm_monbus block diagram

The same structure as arbiter_rr_pwm_monbus, with the weighted pick: requests and the per-client weights (cfg_arb_max_thresh) enter arbiter_round_robin_weighted from the top, and the heavy grant goes to the outputs. The pwm on the left throttles arbitration through block_arb. The optional observer on the right (arbiter_monbus_common, WEIGHTED_MODE=1) judges each client's share against the same weights.

The module integrates four key components.

4.4.2 Weighted Round-Robin Arbiter

Provides priority-based arbitration using arbiter_round_robin_weighted: - Per-client weight configuration via cfg_arb_max_thresh - Higher weight = higher priority - Fair rotation within same weight level - Prevents starvation of low-priority clients - Optional ACK protocol support

Weight Encoding: cfg_arb_max_thresh is a concatenated vector:

```
cfg_arb_max_thresh = {weight[CLIENTS-1], ..., weight[1], weight[0]}
```

Where each weight is $clog2(MAX_LEVELS) bits wide.

Arbitration Algorithm (`arbiter_round_robin_weighted`, credit-based): 1. Each client's weight loads a per-client **credit counter** on replenish (`w_credit_counter[i] = w_client_weight[i]`) 2. A client is eligible while it has credit left (`w_has_crd[i] = (r_credit_counter[i] > 0) && ...`) 3. Selection among *eligible* clients is **plain round-robin** — no comparison of weights picks a winner 4. Granting spends a credit; when all requesting clients are out of credit the counters replenish and the cycle repeats

Weight therefore controls **how many grants** a client gets per replenishment round, not **which** client wins a given contest. A high-weight client does not lock out low-weight ones — they interleave, which is what makes the share figures below achievable.

4.4.3 PWM Generator

Controls arbiter availability (same as RR variant): - 16-bit resolution - Configurable duty cycle and period - PWM output directly drives arbiter block_arb

4.4.4 Monitor Bus Common

Comprehensive monitoring with enhanced fairness tracking: - **WEIGHTED_MODE** is a **dead parameter** in `arbiter_monbus_common` — declared and never referenced. Fairness analysis is weight-aware unconditionally (it always uses `cfg_max_thresh`); setting or clearing this changes nothing - Compares actual vs expected grant distribution based on configured weights - Detects fairness violations when distribution deviates from weight ratios

Fairness Calculation Example:

```
4 clients with weights [8, 4, 2, 1] (total weight = 15)
Expected distribution:
  Client 0: (8/15) * 100 = 53.3%
  Client 1: (4/15) * 100 = 26.7%
  Client 2: (2/15) * 100 = 13.3%
  Client 3: (1/15) * 100 = 6.7%

Actual distribution is measured over **all grants since reset**, not
over a
sliding window: `r_grant_counters` and `r_total_grants` in
`arbiter_monbus_common` are cleared only on reset.
`FAIRNESS_REPORT_CYCLES`
(256) paces how often the deviation is **recomputed** from those
lifetime
totals, and `MIN_GRANTS_FOR_FAIRNESS` is likewise compared against the
lifetime
total. After a traffic-phase or weight change the reported deviation
therefore
still reflects all prior history; it converges toward the new
distribution
rather than snapping to it.
Deviation = |actual% - expected%| for each client.
Event generated if max deviation > cfg_mon_fairness_thresh.
```

4.4.5 Operation Flow

1. Configure per-client weights via cfg_arb_max_thresh
2. Clients assert request signals
3. PWM output modulates arbiter availability

4. When active, the weighted arbiter grants a **credit-eligible** requester, chosen round-robin among those with credit remaining — weight budgets how many grants a client gets per replenishment round, it does not rank clients
5. Monitor tracks grants and calculates fairness metrics
6. Fairness violations reported if distribution deviates from weights

---

4.5 Timing Characteristics

This module is **sequential**: the AST shows clocked processes, so it holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

4.6 Usage Examples

```
// 4-client weighted arbiter with priorities [8, 4, 2, 1]
localparam int MAX_LEVELS = 16;
localparam int CLIENTS = 4;
localparam int WEIGHT_WIDTH = $clog2(MAX_LEVELS);  // 4 bits

// Configure client weights
logic [WEIGHT_WIDTH-1:0] client_weights [CLIENTS];
assign client_weights[0] = 4'd8;   // Highest priority
assign client_weights[1] = 4'd4;
assign client_weights[2] = 4'd2;
assign client_weights[3] = 4'd1;   // Lowest priority

wire [CLIENTS*WEIGHT_WIDTH-1:0] cfg_weights = {
    client_weights[3], client_weights[2],
    client_weights[1], client_weights[0]
};

arbiter_wrr_pwm_monbus #(
    .MAX_LEVELS             (MAX_LEVELS),
    .CLIENTS                (CLIENTS),
    .WAIT_GNT_ACK           (1),         // Enable ACK protocol
```

```
    .MON_AGENT_ID           (16'h0018),
    .MON_UNIT_ID            (8'h04)
) u_wrr_arb (
    .clk                    (clk),
    .rst_n                  (rst_n),

    // Arbiter configuration
    .cfg_arb_max_thresh     (cfg_weights),
    .request                (client_requests),
    .grant_valid            (grant_valid),
    .grant                  (grant_onehot),
    .grant_id               (grant_binary),
    .grant_ack              (grant_ack),

    // PWM configuration - 40% availability
    .cfg_pwm_sync_rst_n     (1'b1),
    .cfg_pwm_start          (pwm_start),
    .cfg_pwm_duty           (16'd60),    // Block for 60 cycles
    .cfg_pwm_period         (16'd100),   // Out of 100 total
    .cfg_pwm_repeat_count   (16'd0),     // Infinite
    .cfg_pwm_sts_done       (pwm_done),
    .pwm_out                (pwm_signal),

    // Monitor configuration
    .cfg_mon_enable         (1'b1),
    .cfg_mon_pkt_type_enable (16'h003F),
    .cfg_mon_latency_thresh  (16'd100),
    .cfg_mon_starvation_thresh (16'd500),
    .cfg_mon_fairness_thresh (16'd10),    // 10% deviation threshold
    .cfg_mon_active_thresh   (16'd4),
    .cfg_mon_ack_timeout_thresh (16'd200),
    .cfg_mon_efficiency_thresh (16'd75),
    .cfg_mon_sample_period   (8'd100),

    // Monitor bus output
    .monbus_valid           (mon_valid),
    .monbus_ready           (mon_ready),
    .monbus_packet          (mon_packet),
    .monbus_timestamp       (mon_timestamp),

    // Monitor time broadcast (from the monbus group) -- not optional
    .i_mon_time             (mon_time),

    // Enhanced debug outputs
    .debug_fifo_count       (mon_fifo_level),
    .debug_packet_count     (mon_pkt_count),
    .debug_ack_timeout      (mon_ack_timeouts),
```

```
    .debug_protocol_violations (mon_violations),
    .debug_grant_efficiency  (mon_efficiency),
    .debug_client_starvation (mon_starvation),
    .debug_fairness_deviation (mon_fairness_dev),
    .debug_monitor_state     (mon_state)
);
```

4.7 Design Notes

4.7.1 Weight Configuration

The cfg_arb_max_thresh parameter requires careful configuration:

Valid Range: Each client weight must be 1 to (MAX_LEVELS-1) **Zero Weights**: Zero-weight clients are effectively disabled **Dynamic Updates**: Weights can be changed runtime, but may cause temporary fairness violations during transition

Example Calculation (4 clients, MAX_LEVELS=16):

```
localparam WEIGHT_WIDTH = 4;  // $clog2(16) = 4
logic [WEIGHT_WIDTH-1:0] w0 = 4'd12;  // 75% of max
logic [WEIGHT_WIDTH-1:0] w1 = 4'd6;   // 37.5% of max
logic [WEIGHT_WIDTH-1:0] w2 = 4'd3;   // 18.75% of max
logic [WEIGHT_WIDTH-1:0] w3 = 4'd1;   // 6.25% of max

assign cfg_arb_max_thresh = {w3, w2, w1, w0};  // LSB is client 0
```

4.7.2 Fairness Deviation Threshold

The cfg_mon_fairness_thresh parameter should account for statistical variation:

Recommended Values: - 5% for strict fairness enforcement - 10% for typical systems (allows reasonable variation) - 20% for relaxed monitoring

Factors Affecting Deviation: - Short measurement windows (MIN_GRANTS_FOR_FAIRNESS) - Bursty request patterns - Weight ratio extremes (e.g., 16:1 vs 2:1) - PWM blocking periods

4.7.3 Enhanced Debug Outputs

This module exposes complete debug outputs from arbiter_monbus_common:

- **debug_client_starvation**: Bit vector showing starved clients
  - Use to identify clients not receiving service
  - Indicates potential weight configuration issues
- **debug_fairness_deviation**: Maximum deviation percentage

    - Real-time fairness metric without consuming monitor bus bandwidth
    - Connect to system debug registers for runtime visibility
- **debug_grant_efficiency**: Grant completion ratio
    - Low efficiency may indicate ACK timeout issues or downstream congestion

4.7.4 ACK Mode with Weighted Arbitration

When WAIT_GNT_ACK=1, the arbiter waits for grant_ack before issuing next grant. This can affect fairness:

Impact on Fairness: - Slow-ACK clients may receive fewer grants than their weight warrants - ACK timeout tracking helps identify problematic clients - Consider increasing cfg_mon_ack_timeout_thresh for clients with known slower response

---

4.8 Related Modules

Used by: priority-based interconnect arbiters, QoS-aware memory controllers, multi-tier scheduling hierarchies.

Uses: `arbiter_round_robin_weighted.sv` (core WRR arbiter); `pwm.sv` (PWM generator); `arbiter_monbus_common.sv` (monitoring; note WEIGHTED_MODE is inert).

See also: `arbiter_rr_pwm_monbus.sv` (equal-priority variant); `monbus_arbiter.sv` (aggregates monitor bus streams).

4.9 Testing

`val/amba/test_arbiter_wrr_pwm_monbus.py` exercises this module. It collects 8 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_arbiter_wrr_pwm_monbus.py -v
```

4.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Katevenis, M., Sidiropoulos, S., Courcoubetis, C. “Weighted Round-Robin Cell Multiplexing in a General-Purpose ATM Switch Chip.” *IEEE J. Selected Areas in Communications*, 9(8):1265-1279, Oct. 1991. DOI: 10.1109/49.105173. — interleaved weighted round robin, the credit scheme of the `arbiter_round_robin_weighted` core (indexed in `rtl/common/references/README.md`).
- Gupta, P., McKeown, N. “Designing and Implementing a Fast Crossbar Scheduler.” *IEEE Micro*, 19(1):20-28, Jan./Feb. 1999. DOI: 10.1109/40.748793. — the round-robin core underneath the weighting layer.
- Dally, W.J., Towles, B. *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004, ch. 18 “Arbitration”, §18.4.2 “Round-Robin Arbiter” and §18.4.4 “Weighted Round-Robin Arbiter”. — textbook treatment (secondary source).
- Internal: docs/markdown/rtl-amba/index.md (AMBA subsystem requirements)
- Internal: docs/markdown/rtl-amba/monitor/arbiter_monbus_common.md (monitoring details)
- RTL: `rtl/amba/ocelus/arbiter_wrr_pwm_monbus.sv`
- Tests: `val/amba/test_arbiter_wrr_pwm_monbus.py`
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

Last Updated: 2025-10-24

4.11 Navigation

- **← Back to Shared Infrastructure Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

5 axi4ace_snoop_monitor_lite

ACE snoop-channel lite monitor for `cache-ip`. Tracks outstanding snoops in AC-issue order and emits error, timeout, and completion packets on the monbus.

5.1 Overview

`axi4ace_snoop_monitor_lite` is the ACE snoop-channel counterpart to `axi_monitor_lite`. The standard `axi_monitor_lite` cannot monitor snoop channels because they have no transaction ID. This core keeps an in-order table of outstanding snoops, attributes each CR handshake and CDLAST beat to the oldest entry, and emits the same 128-bit `monitor_packet_t` records on the monbus.

The monitor is a TAP: it observes `ac_valid`/`ac_ready`, `cr_valid`/`cr_ready`, and `cd_valid`/`cd_ready`/`cd_last`, but never drives the observed channels.

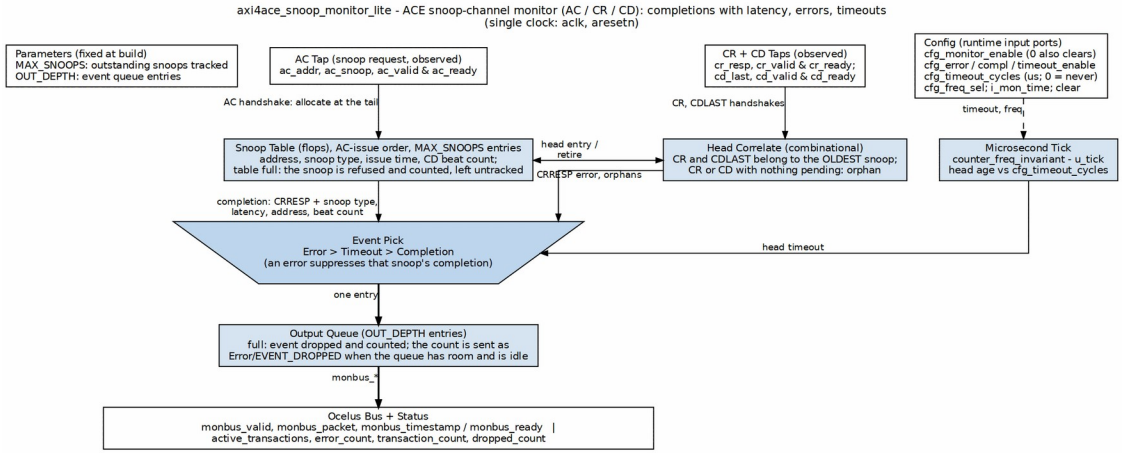
5.2 Why a Separate Monitor Core

ACE snoop channels differ from AXI4 channels in two ways that break per-ID tracking:

- **No ID field**: AC carries only address, snoop type, and protection; CR and CD carry no ID.
- **In-order attribution**: Responses and data must be returned in the order the ACs were issued.

Therefore the monitor cannot use a CAM or linked-list keyed by ID. It uses a circular FIFO of outstanding snoops in AC-issue order, with CR and CDLAST always attributed to the head entry.

5.3 Block Diagram



axi4ace_snoop_monitor_lite block diagram

The monitor only observes the snoop channels. Each AC handshake adds an entry at the tail of the snoop table in issue order, with no slot pick. CR and CDLAST always belong to the oldest outstanding snoop, so the head correlate retires the head entry, and a CR or CD with nothing pending is an orphan. Completions, errors and head timeouts go through the event pick (errors first) into the output queue and onto the ocelus bus.

5.4 Parameters

Parameter	Type	Default	Description
UNIT_ID	logic [7:0]	8'h01	Unit ID in every emitted packet
AGENT_ID	logic [15:0]	16'h000A	Agent ID in every emitted packet
MAX_SNOOPS	int	8	Outstanding-snoop table depth
OUT_DEPTH	int	4	monbus output queue depth (a `gaxi_fifo_sync`), any depth >= 2
ACLK_MHZ	int	100	Clock frequency in MHz for the frequency-invariant us tick
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	Frequency-invariant tick LUT lower bound
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Frequency-invariant tick LUT upper bound
CFI_NUM_FREQ_ENTRIES	int	16	Number of frequency entries in the LUT
CFI_FREQ_STRATEGY	int	0	Frequency strategy selector
ADDR_WIDTH	int	32	Snoop address width
DATA_WIDTH	int	32	Snoop data width (used only for beat counting; data itself is not captured)
AW	int	ADDR_WIDTH	Short alias for address width

Parameter	Type	Default	Description
DW	int	DATA_WIDTH	Short alias for data width
N	int	MAX_SNOOPS	Short alias for table depth
SW	int	(N>1)? $clog2(N):1	Table-index width
CW	int	$clog2(N+1)|Occu clog2(CFI_NUM_FREQ_ENTRIES): 1	`cfg_freq_sel` width
TS_WIDTH	int	16	Timestamp / latency width
AGE_WIDTH	int	16	Internal us-counter and age width

5.5 Ports

Port	Direction	Type	Description
aclk	input	logic	AXI clock
aresetn	input	logic	Active-low reset
clear	input	logic	Synchronous clear; also driven by `~cfg_monitor_enable` in wrappers
i_mon_time	input	`monitor_common_pkg::monbus_timestamp_t`	Free-running broadcast timestamp
cfg_monitor_enable	input	logic	0 = monitor inert, table held clear
cfg_error_enable	input	logic	Enable error packet emission
cfg_compl_enable	input	logic	Enable completion packet emission
cfg_timeout	input	logic	Enable timeout detection

Port	Direction	Type	Description
_enable			and packet emission
cfg_timeout_cycles	input	logic [15:0]	Timeout in microseconds; 0 = never, 0xFFFF = never
cfg_freq_sel	input	logic [SELW-1:0]	Frequency-invariant tick LUT index
monbus_valid	output	logic	monbus packet valid
monbus_ready	input	logic	monbus consumer ready
monbus_packet	output	`monitor_common_pkg::monitor_packet_t`	128-bit monitor packet
monbus_timestamp	output	`monitor_common_pkg::monbus_timestamp_t`	Side-band timestamp
active_transactions	output	logic [7:0]	Entries currently in the snoop table
error_count	output	logic [15:0]	Error + timeout packets emitted
transaction_count	output	logic [31:0]	Completion packets emitted
dropped_count	output	logic [15:0]	Events lost to monbus backpressure since last report
ac_addr	input	logic [AW-1:0]	AC address observation tap
ac_snoop	input	logic [3:0]	AC snoop type observation tap
ac_valid	input	logic	AC valid observation tap
ac_ready	input	logic	AC ready observation tap
cr_resp	input	logic [4:0]	CR response observation tap
cr_valid	input	logic	CR valid observation tap

Port	Direction	Type	Description
cr_ready	input	logic	CR ready observation tap
cd_last	input	logic	CD last-beat observation tap
cd_valid	input	logic	CD valid observation tap
cd_ready	input	logic	CD ready observation tap

5.6 Packet Encoding

All packets carry `protocol = PROTOCOL_AXI`.

5.6.1 Completion Packet (`PktTypeCompletion`)

Field	Value	Notes
event_code	`{CRRESP[3:0], ACSNOOP[3:0]}`	Lower 4 bits = snoop type that started the transaction; upper 4 bits = response code
channel_id	CDLAST beat count for this snoop	Saturated to 9 bits; the snoop type is already in `event_code`
event_data	`{latency[15:0], ACADDR[AW-1:0]}`	Latency (cycles) occupies `event_data[63:48]`; address occupies the lower bits. For `AW > 48` the top address bits share the latency field, matching the trade-off in `axi_monitor_lite`.
latency	`i_mon_time - AC handshake i_mon_time`	Cycles

5.6.2 Error Packet (`PktTypeError`)

Condition	`event_code`	Notes
CRRESP[1] set (Error)	`AXI_ERR_PROTOCOL`	Only when `cfg_error_enable` is high
CR handshake with no AC	`AXI_ERR_RESP_ORPHAN`	Response

Condition	event_code	Notes
pending		arrived while table empty
CDLAST handshake with no AC pending	AXI_ERR_DATA_ORPHAN	Data arrived while table empty

5.6.3 Timeout Packet (PktTypeTimeout)

Condition	event_code	Notes
Head entry older than cfg_timeout_cycles us	AXI_TIMEOUT_RESP	Only the head can age out; a CR or CD handshake on the head resets its progress stamp

5.6.4 Event Precedence

When multiple events are ready in the same cycle, only one packet is emitted. Priority is:

```
ERROR > TIMEOUT > COMPLETION
```

When CRRESP[1] is set and cfg_error_enable is high, an ERROR packet is emitted and the COMPLETION packet for that snoop is suppressed.

5.7 Drop-and-Count Discipline

The monitor never stalls the observed channels. Events that cannot be queued in the OUT_DEPTH output queue are dropped and counted. The count is emitted as an AXI_ERR_EVENT_DROPPED packet when the queue has room and no other event wants the bus.

A snoop that arrives when the table is full is counted as refused and left untracked. Any later CR/CD for that snoop reports as an orphan error. The monitor core itself does not expose refused_count; the wrappers that instantiate it may choose to expose or aggregate it.

5.8 Outstanding-Snoop Table

The table is a circular FIFO:

- r_rd_ptr points to the oldest outstanding snoop
- r_wr_ptr points to the next free tail

- CR and CDLAST are attributed to the head entry
- Only the head can timeout
- Multiple outstanding snoops are allowed

This matches ACE's per-channel ordering rule: CR responses and CD data must be returned in the same order as the AC addresses.

5.9 Wrappers that Instantiate This Core

The snoop-side `_monlite` wrappers instantiate `axi4ace_snoop_monitor_lite`. See axi_monitor_lite_wrappers.md for wrapper details.

Wrapper	Direction	Core wrapped	Monitor taps
`axi4ace_snoop_slave_monlite`	Cache side	`axi4ace_snoop_slave`	`m_axi_ac*` / `m_axi_cr*` / `m_axi_cd*`
`axi4ace_snoop_master_monlite`	CCU side	`axi4ace_snoop_master`	`m_axi_ac*` / `m_axi_cr*` / `m_axi_cd*`

5.10 Related Modules

- axi_monitor_lite — AXI/AXIL transaction lite monitor (per-ID tracking)
- axi_monitor_lite_wrappers — the two snoop-side wrappers that use this core
- axi4ace_snoop_slave — cache-side snoop responder transport
- axi4ace_snoop_master — CCU-side snoop initiator transport

5.11 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021. developer.arm.com/documentation/ihi0022/h. — Part D: §D3.6 snoop address channel, §D3.7 snoop response channel ("all snoop transactions are ordered"), §D3.8 snoop data channel (data in snoop-address order). The in-order attribution this monitor uses is that rule; no paper is needed for it.
- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. "An Event-Based Monitoring Service for Networks on Chip." *ACM Trans. Design*

Automation of Electronic Systems, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126. — related work: event-based hardware monitoring of an interconnect.

- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, sec. “ARM AMBA interconnect protocols suite”. — textbook background on ACE and the AMBA family (secondary source).
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

Plane: protocol plane (lite ACE snoop monitor)

5.12 Navigation

- **← Back to Monitor Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

6 axi_monitor_addr_check

Module: axi_monitor_addr_check.sv **Location:** rtl/amba/ocelus/ **Category:** Core Infrastructure **Status:** Production Ready (Formally Verified)

Plane: protocol plane (AXI address-range filter; used by both full and lite monitors)

6.1 Overview

axi_monitor_addr_check is a parallel address-range comparator instantiated within AXI monitor wrappers. It observes command-phase handshakes (AR/AW) and classifies each accepted address against N user-configured inclusive ranges [low, high]. Each range carries a **flavor** (ADDR_RANGE_IS_ERROR[i]) that selects one of two independent report paths:

- **DEBUG range** (ADDR_RANGE_IS_ERROR[i] = 0): a hit emits a PktTypeAddrMatch (4'h8) packet with event code AXI_ADDR_RANGE_MATCH = 8'h01, gated by cfg_debug_enable.
- **ERROR range** (ADDR_RANGE_IS_ERROR[i] = 1): the enabled ERROR ranges form an **allowlist**; a command whose address is in NONE of them emits a PktTypeError (4'h0) packet with event code AXI_ERR_ADDR_RANGE = 8'h0D, gated by cfg_error_enable.

DEBUG and ERROR ranges are evaluated independently, so a single command may produce both a match and a miss; two pending slots hold each and the output stream serialises them.

This is a **shared infrastructure module** used internally by AXI4/AXIL4/AXI5 monitor wrappers when parameterized with N_ADDR_RANGES > 0. It is not typically instantiated directly by users but is critical for address-space validation (allowlist enforcement) and targeted debug tracing. In practice it buys you four things:

1. **Allowlist enforcement:** ERROR ranges declare the *expected* address space; any access outside it raises an Error/ADDR_RANGE packet.
2. **Targeted debug tracing:** DEBUG ranges watch specific regions; a hit emits an AddrMatch packet without implying an error.
3. **Independent debug/error regions:** the two flavors can cover different address sets, so “watch this region” and “everything must stay inside that region” are configured separately.
4. **Design Verification:** verify address constraints in functional simulation and formal proof.

The ERROR/allowlist path is the checker built into the monitor whenever `N_ADDR_RANGES > 0`, **independent of the error reporter cone** (`ENABLE_ERROR_LOGIC`). It is therefore the most controllable way to **deliberately inject an error** for coverage: point an enabled ERROR range at a region the legitimate traffic never touches, and every access becomes an allowlist miss that emits `PktTypeError`/`AXI_ERR_ADDR_RANGE`. Because an error is a fault condition, this injection is expected to hang the traffic that provoked it – see Healthy classes vs fault classes for why that stall is the point, not a defect.

Feature summary:

- **Parallel Range Comparators:** N independent [low, high] inclusive range checkers
- **Two flavors per range:** `ADDR_RANGE_IS_ERROR` selects DEBUG (match) vs ERROR (allowlist-miss) behavior per range; default all-0 leaves the ERROR/miss path inert (feature unused by default)
- **Independent path enables:** `cfg_debug_enable` gates AddrMatch packets, `cfg_error_enable` gates Error packets; `cfg_addr_check_enable` is the master gate
- **Zero-Area Synthesis:** `N_ADDR_RANGES = 0` omits this module entirely (no gates, no regs) – done by the **parent**, not here: `axi_monitor_base` wraps the instance in `if (N_ADDR_RANGES > 0) gen_addr_check` and ties the packet stream off in `gen_no_addr_check`
- **Monbus Integration:** AddrMatch packets (`8'h8`/`8'h01`) and Error packets (`8'h0`/`8'h0D`)
- **Coalescing:** per-range latched address for MATCH; a single latched slot for MISS; one packet per cycle (MISS has emit priority)
- **Formal Verification:** all properties proven (prove + cover PASS)

6.2 Parameters

Parameter	Type	Default	Description
`N_ADDR_RANGES`	int	4	Number of address-range comparators (>= 1). Max 16 (4-bit range index).
`ADDR_WIDTH`	int	32	Address bus width (matches

Parameter	Type	Default	Description
			AXI_ADDR_WIDTH of parent monitor). Must be <= 60 (fits the event_data address field).
ID_WIDTH	int	6	Transaction ID width (clipped to 9 bits when copied into the packet's channel_id).
UNIT_ID	logic [7:0]	8'h00	8-bit unit identifier in monitor packets.
AGENT_ID	logic [15:0]	16'h0000	16-bit agent identifier in monitor packets.
IS_READ	bit	1	Build-time flag: 1 if this monitor watches reads (AR), 0 for writes (AW). Drives direction recovery at the consumer (see Event Encoding).
ADDR_RANGE_IS_ERROR	logic [N_ADDR_RANGES-1:0]	'0	Per-range flavor: bit i = 0 → range i is a DEBUG range (hit → AddrMatch); bit i = 1 → range i is an ERROR range (allowlist; miss → Error/ADDR_RAN

Parameter	Type	Default	Description
			GE). Default all-0 keeps the ERROR/miss path inert.

6.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`M`	`ADDR_WIDTH`
`IW`	`ID_WIDTH`

6.3 Ports

6.3.1 Clock and Reset

Port	Direction	Width	Description
`clk`	Input	1	AXI clock
`aresetn`	Input	1	AXI active-low reset

6.3.2 Side-Band Timestamp

Port	Direction	Width	Description
`i_mon_time`	Input	64	Free-running counter from `monbus group`, broadcast to every wrapper via the shared `mon_time_w` net. Sampled when `addr_pkt_valid` asserts and driven out on

Port	Direction	Width	Description
			addr_pkt_timestamp.

6.3.3 Command Interface (Snoop)

Port	Direction	Width	Description
cmd_valid	Input	1	Command valid (AR or AW handshake)
cmd_ready	Input	1	Command ready (slave accepting)
cmd_addr	Input	ADDR_WIDTH	Command address (araddr or awaddr)
cmd_id	Input	ID_WIDTH	Command transaction ID (arid or awid)

6.3.4 Configuration Inputs

Port	Direction	Width	Description
cfg_addr_check_enable	Input	1	Master enable for all comparators. 0 = no packets generated.
cfg_debug_enable	Input	1	Enables the MATCH path: DEBUG-range hits emit AddrMatch packets.
cfg_error_enable	Input	1	Enables the MISS path: an address outside the ERROR allowlist

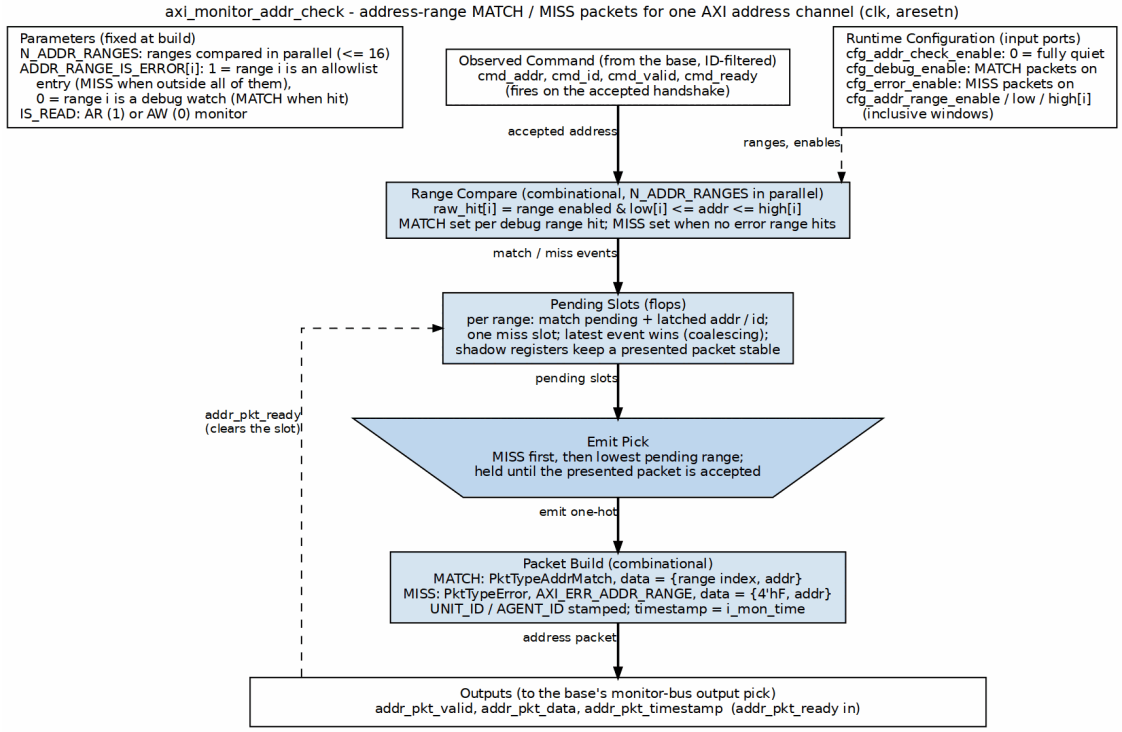
Port	Direction	Width	Description
			emits an Error/ADDR_RANGE packet.
cfg_addr_range_enable[N_ADDR_RANGES-1:0]	Input	N	Per-range enable bits. 1 = range active, 0 = range disabled.
cfg_addr_range_low[N_ADDR_RANGES-1:0][ADDR_WIDTH-1:0]	Input	N × ADDR_WIDTH	Inclusive low bound for each range.
cfg_addr_range_high[N_ADDR_RANGES-1:0][ADDR_WIDTH-1:0]	Input	N × ADDR_WIDTH	Inclusive high bound for each range.

6.3.5 Monitor Bus Output

Port	Direction	Width	Description
addr_pkt_valid	Output	1	Address-check packet valid (AddrMatch or Error)
addr_pkt_ready	Input	1	Downstream ready to accept packet
addr_pkt_data	Output	128	Monitor packet (monitor_packet_t, 128-bit format)
addr_pkt_timestamp	Output	64	Sampled i_mon_time paired atomically with addr_pkt_data

6.4 Functional Description

6.4.1 Block diagram



axi_monitor_addr_check block diagram

Each accepted command address is compared against every range at once. Debug (watch) ranges that hit raise MATCH events. An address outside every enabled error-allowlist range raises one MISS event. Events wait in pending slots, where the latest event wins. One packet drains per cycle: MISS first, then the lowest pending range, held stable while the bus stalls.

6.4.2 Internal Architecture

The module instantiates N parallel comparators and splits them by flavor:

1. **Per-Range Comparators:** Each range i computes `raw_hit[i] = cfg_addr_range_enable[i] && (cmd_addr >= cfg_addr_range_low[i]) && (cmd_addr <= cfg_addr_range_high[i])`.

2. **Flavor split:**

   - `debug_hit[i] = raw_hit[i] && !ADDR_RANGE_IS_ERROR[i]` — a hit in a DEBUG range.
   - `err_hit = |(raw_hit & ADDR_RANGE_IS_ERROR)` — the address is inside some enabled ERROR range (allowed).

- `err_ranges_exist = |(cfg_addr_range_enable & ADDR_RANGE_IS_ERROR)`.

3. **Per-command events** (qualified by `cmd_fire = cmd_valid && cmd_ready && cfg_addr_check_enable`):
   - `match_set[i] = cmd_fire && cfg_debug_enable && debug_hit[i]` → MATCH (AddrMatch packet).
   - `miss_set = cmd_fire && cfg_error_enable && err_ranges_exist && !err_hit` → MISS (Error packet).
4. **Pending + emit:** MATCH events coalesce per DEBUG range (`r_match_pending[i]`, latched address, latest-win); MISS events coalesce into a single slot (`r_miss_pending`). One packet drains per cycle — **MISS has priority**, then the lowest-index pending DEBUG range. Direction (read vs. write) is not embedded — the build-time `IS_READ` parameter fixes the channel, and consumers recover direction from `(UNIT_ID, AGENT_ID)`.

Because the two flavors are independent, a command that hits a DEBUG range and is also outside the ERROR allowlist sets BOTH pending slots; the two packets emit on successive cycles.

6.4.3 Event Encoding

Two packet types share the 128-bit `monitor_packet_t` layout; only `packet_type`, `event_code`, and the range-index nibble differ:

Field	MATCH (debug hit)	MISS (error allowlist)
`[127:124]` Packet Type	`4'h8` PktTypeAddrMatch	`4'h0` PktTypeError
`[108:105]` Protocol	`4'h0` PROTOCOL_AXI	`4'h0` PROTOCOL_AXI
`[104:97]` Event Code	`8'h01` AXI_ADDR_RANGE_MATCH	`8'h0D` AXI_ERR_ADDR_RANGE
`[96:88]` Channel ID	`cmd_id` clipped/zero-extended to 9 bits	same
`[87:72]` Agent ID	from `AGENT_ID`	from `AGENT_ID`
`[71:64]` Unit ID	from `UNIT_ID`	from `UNIT_ID`
`[63:60]` Range Index	matching DEBUG range (0..N-1)	`4'hF` (no-range sentinel)
`[59:0]` Address	full `cmd_addr`, zero-padded	full `cmd_addr`, zero-padded

`is_read` flag dropped: Earlier revisions reserved a bit in event_data for read-vs-write direction. The 128-bit layout drops it because each AXI monitor instance watches a single direction (set at build time by `IS_READ`); consumers recover direction from `(UNIT_ID, AGENT_ID)`. Note:

apb_monitor_addr_check still carries is_read since a single APB monitor sees both directions on the same channel.

Side-band timestamp: addr_pkt_timestamp carries the sampled i_mon_time paired atomically with the packet through the arbiter and into monbus group.

6.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via always_ff or the repository's ALWAYS_FF_RST macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

6.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
axi_monitor_addr_check #(
    .N_ADDR_RANGES         (4),
    .ADDR_WIDTH            (32),
    .ID_WIDTH              (6),
    .UNIT_ID               (8'h00),
    .AGENT_ID              (16'h0000),
    .IS_READ               (1'b1),
    .ADDR_RANGE_IS_ERROR   ('0)
) u_axi_monitor_addr_check (
    .clk                   (clk),
    .aresetn               (aresetn),
    .i_mon_time            (i_mon_time),
    .cmd_addr              (cmd_addr),
    .cmd_id                (cmd_id),
    .cmd_valid             (cmd_valid),
    .cmd_ready             (cmd_ready),
    .cfg_addr_check_enable (cfg_addr_check_enable),
    .cfg_debug_enable      (cfg_debug_enable),
    .cfg_error_enable      (cfg_error_enable),
    .cfg_addr_range_enable (cfg_addr_range_enable),
    .cfg_addr_range_low    (cfg_addr_range_low),
```

```
    .cfg_addr_range_high   (cfg_addr_range_high),
    .addr_pkt_valid        (addr_pkt_valid),
    .addr_pkt_ready        (addr_pkt_ready),
    .addr_pkt_data         (addr_pkt_data),
    .addr_pkt_timestamp    (addr_pkt_timestamp)
);
```

6.7 Design Notes

6.7.1 Filtering Integration

The checker produces two packet classes, each filtered by the parent monitor's existing drop masks (`axi_monitor_filtered` / `monbus_group_core`):

- **MATCH** (`PktTypeAddrMatch`, event `0x01`) — dropped by `cfg_axi_addr_mask[1]`.
- **MISS** (`PktTypeError`, event `0x0D` `AXI_ERR_ADDR_RANGE`) — dropped by `cfg_axi_error_mask[13]`.

No new mask wiring is needed — both event codes already have reserved bits in the per-class 16-bit masks.

Example: suppress the error/allowlist packets while keeping debug matches:

```
.cfg_axi_error_mask(16'h2000)  // bit 13 high = drop ADDR_RANGE error packets
```

6.7.2 Instantiation Pattern

The module is instantiated **inside** AXI monitor wrappers (`axi4_master_wr_mon`, etc.) by the monitor architecture. Users do not instantiate this module directly but configure it via wrapper parameters.

6.7.3 Synthesis Behavior

- **`N_ADDR_RANGES = 0`:** handled by the **instantiating parent**. `axi_monitor_base` generates the instance only under `N_ADDR_RANGES > 0`; its `gen_no_addr_check` branch drives `addr_pkt_valid = 0` and a zeroed packet, so from the monitor's arbiter the stream is constant 0 and the checker costs nothing.

  This module itself has **no `N == 0` guard** and must not be instantiated directly with 0 – its parameter is documented `>= 1`. At 0 the per-range vectors degenerate (`logic [-1:0]`), the range loops are empty so the enable terms are never driven, and `addr_pkt_valid` would be undriven (X in simulation) rather than a clean constant 0.

- **`N_ADDR_RANGES > 0`:** Full comparator logic synthesized. Area scales with N.

6.7.4 Downstream FIFO

The monbus output should be fed into a standard FIFO (e.g., `gaxi_fifo_sync`) to prevent backpressure stalls:

In practice the addr_check output is merged with the reporter's main packet stream by an arbiter (`monbus_arbiter`) that carries packet+timestamp atomically through a 192-bit skid. The arbiter's downstream FIFO sits at the `monbus group` boundary, sized for the aggregate of all per-wrapper streams. A standalone FIFO on the addr_check output is normally not needed.

6.8 Related Modules

- **axi_monitor_base** — Core monitor infrastructure that instantiates this module
- **axi_monitor_filtered** — packet-type and event-code filtering (sibling to addr_check)
- **axi4_master_wr_mon** — Wrapper that uses this module (N_ADDR_RANGES parameter)

6.9 Testing

All properties proven via formal verification (see `formal/amba/axi_monitor_addr_check/formal_axi_monitor_addr_check.sv`):

Property	Description	Status
P1: Reset quiet	After reset, `addr_pkt_valid` is deasserted.	PASS
P2: Master gate	When `cfg_addr_check_enable=0`, `addr_pkt_valid` stays low.	PASS
P3: Packet class	An emitted packet is PROTOCOL_AXI and is either MATCH (AddrMatch/`0x01`) or MISS (Error/`0x0D`) — nothing else.	PASS

Property	Description	Status
P4: MATCH validity	A MATCH packet's range_index points to an enabled **DEBUG** range (`ADDR_RANGE_IS_ERROR=0`) and the address lies within its `[low, high]`.	PASS
P5: MISS validity	A MISS packet carries the `0xF` sentinel, at least one ERROR range is enabled, and the address lies in **no** enabled ERROR range.	PASS
P6: Sticky valid	Once `addr_pkt_valid` asserts it stays asserted until the consumer accepts.	PASS
cover	Both emission and the emit+handshake are reachable (non-vacuous).	PASS

6.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. — §A2.3.1 handshake (the AR/AW acceptance this checker samples).
- Boulé, M., Zilic, Z. "Efficient Automata-Based Assertion-Checker Synthesis of SEREs for Hardware Emulation." *Proc. Asia and South Pacific Design Automation Conf. (ASP-DAC 2007)*, pp. 324-329, Jan. 2007. DOI: 10.1109/ASPDAC.2007.358006. — related work: hardware checker circuits; this block is a hand-written parallel range compare, formally verified in `formal/amba/axi_monitor_addr_check/`.

- IEEE-ISTO 5001-2003. *The Nexus 5001 Forum Standard for a Global Embedded Processor Debug Interface*, Version 2.0, IEEE Industry Standards and Technology Organization, Dec. 2003 (defining non-peer standard). — §4.8 “Breakpoints/Watchpoints”: a comparable address-match event in a processor debug standard.
- **Monitor Architecture:** `docs/markdown/rtl-amba/overview.md`
- **Monitor Configuration Guide:** Monitor Base Configuration
- **Packet Format Specification:** `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`
- **Formal Verification:** `formal/amba/axi_monitor_addr_check/`
- Indexed in `rtl/amba/references/README.md` (local PDF: `HardwareAssertionCheckers__Boule2007.pdf`, `DebugTraceInterfaceStandard__IeeeIsto2003.pdf`)

6.11 Navigation

- **← Back to Shared Infrastructure Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

7 AXI Monitor Base

Module: `axi_monitor_base.sv` **Location:** `rtl/amba/ocelus/` **Category:** Core Infrastructure
Status: Production Ready

Plane: protocol plane (full monitor core; the lite variant is axi_monitor_lite)

7.1 Overview

The `axi_monitor_base` module provides the core transaction tracking and event reporting behind every AXI/AXIL monitor.

This is a **shared infrastructure module** used internally by the AXI/AXIL monitors. You won't instantiate it directly — the wrappers do that — but it's the piece the whole monitor architecture hangs off, so it's worth understanding before you touch any wrapper.

Key features:

- Transaction-based tracking for AXI and AXI-Lite protocols
- Out-of-order transaction handling with ID-based tracking
- Data-before-address support (slave-side scenarios)
- 128-bit standardized monitor bus packet output, plus a 64-bit side-band timestamp
- Configurable performance metrics tracking
- Timeout detection and threshold monitoring
- Per-transaction state-change debug packets (via `ENABLE_DEBUG_LOGIC` + `cfg_debug_enable`; the verbosity-level/mask interface is dead — see the parameter notes)

Five jobs land in this module:

1. **Transaction Tracking:** maintains state for all outstanding AXI/AXIL transactions
2. **Event Detection:** identifies protocol errors, timeouts, threshold violations
3. **Packet Generation:** creates standardized 128-bit `monitor_packet_t` records paired with a 64-bit side-band timestamp
4. **Flow Control:** manages backpressure and transaction-table exhaustion
5. **Performance Metrics:** optional latency and throughput tracking

---

7.2 Parameters

7.2.1 Identity and Sizing

Parameter	Type	Default	Description
`UNIT_ID`	logic [7:0]	8'h09	8-bit unit identifier in monitor packets
`AGENT_ID`	logic [15:0]	16'h0063	16-bit agent identifier in monitor packets
`MAX_TRANSACTIONS`	int	16	Maximum outstanding transactions in the CAM
`ADDR_WIDTH`	int	32	Address bus width
`ID_WIDTH`	int	8	Transaction ID width (0 for AXIL). **Hard maximum 8** — `bus_transaction_t.id` is an 8-bit field, so a wider key would disagree with the payload and mis-attribute transactions; `axi_monitor_trans_mgr` refuses `ID_WIDTH > 8` at elaboration with an `$error`
`ADDR_BITS_IN_PKT`	int	38	**Inert.** Intended as the number of address LSBs carried in a packet, but the

Parameter	Type	Default	Description
			derived `ADDR_BITS` is never referenced: `bus_transaction_t.addr` is 32 bits, so packets carry the low **32** address bits regardless. With `ADDR_WIDTH > 32` the upper bits are lost at table-allocation time
`IS_READ`	bit	1	1=read monitor (R data channel, AR bursts), 0=write monitor (W data channel, AW bursts)
`IS_AXI`	bit	1	1=AXI protocol, 0=AXI-Lite
`INTR_FIFO_DEPTH`	int	8	Depth of the reporter's outgoing interrupt/event FIFO
`DEBUG_FIFO_DEPTH`	int	8	**Dead.** No debug-trace FIFO is instantiated; referenced by no logic
`N_ADDR_RANGES`	int	0	Number of address-range comparators. `0` removes the `axi_monitor_addr`

Parameter	Type	Default	Description
			`_check` block entirely (zero area)
`ADDR_RANGE_IS_ERROR`	logic [N_ADDR_RANGES-1:0]	`'0`	Per-range flavor forwarded to `axi_monitor_addr_check`: bit i = 0 -> DEBUG range (hit -> AddrMatch, `cfg_debug_enable`); bit i = 1 -> ERROR range (allowlist miss -> Error/ADDR_RANGE, `cfg_error_enable`). Default all-0.

7.2.2 Master Switches

Parameter	Type	Default	Description
`ENABLE_PERF_PACKETS`	bit	0	Master switch for the reporter's legacy perf-rollup packets. Setting it also defaults `ENABLE_PERF_LOGIC` on (see Performance Monitoring)
`ENABLE_DEBUG_MODULE`	bit	0	**Inert / reserved.** The debug-trace sub-module it was meant to switch on does not exist in this design; the parameter is kept only because the

Parameter	Type	Default	Description
			wrapper family plumbs it. The live debug path is `ENABLE_DEBUG_LOGIC` + `cfg_debug_enable` (the reporter's state-change emitter)

7.2.3 Transaction-Table Shaping

These size and shape the transaction table. Defaults reproduce the classic single-bank, age-ranked behaviour exactly.

Parameter	Type	Default	Description
`USE_WDATA_ORDERQ`	bit	0	**Write monitors only.** Recover AXI4's WID-less W-beat ordering with an AWID FIFO (push on the AW handshake, pop on W-last) instead of an O(N^2) oldest-select over the whole table. Required for banking on a write monitor
`NUM_BANKS`	int	1	Split the table into banks by low ID bits so the two O(N^2) structures (oldest-select, rank update) are restricted to same-

Parameter	Type	Default	Description
			bank pairs. Must be a power of 2 and divide `MAX_TRANSACTIONS`

Sizing rule — this one bites. Every transaction sharing an ID lands in the *same* bank, so per-ID concurrency is capped by the **bank** depth, not by `MAX_TRANSACTIONS`: `MAX_TRANSACTIONS / NUM_BANKS >= (IDs per bank) x (outstanding per ID)`. Undersize it and entries are refused rather than mis-tracked — which surfaces three layers up as missing packets.

`NUM_BANKS > 1` on a write monitor requires `USE_WDATA_ORDER_Q = 1`. The combination without it refuses to elaborate (`$error` in `axi_monitor_trans_mgr`): the WID-less fallback is a state predicate spanning every bank, so the same-bank oldest-select returns one winner *per bank* and a single W beat advances one transaction in each. There is no correct fallback, so the build is refused rather than silently double-counting.

7.2.4 ID-Range Filter

Lets several monitors snoop one ID-multiplexed bus, each owning a slice, so no single table has to hold the whole concurrency. Default OFF — bit-identical to an unfiltered build.

Parameter	Type	Default	Description
`ID_FILTER_ENABLE`	bit	0	Enable the filter
`ID_MATCH_BASE`	int	0	First ID owned by this instance
`ID_MATCH_COUNT`	int	0	How many IDs; `0` means all (no filter)
`ADDR_FILTER_ENABLE`	bit	0	Address-range packet filter, forwarded to `axi_monitor_trans_mgr`. 0 leaves it inert

The ID window can also be set **at runtime**, which matters because `ID_MATCH_BASE`/`ID_MATCH_COUNT` are elaboration constants — retargeting which master an instance watches otherwise means a rebuild:

Port	Description
`cfg_id_filter_enable`	High: use the `cfg_id_*` window below. Low: use the parameters, bit-identical to a build without this feature
`cfg_id_match_base`	First ID owned, runtime
`cfg_id_match_count`	How many; `0` means all, matching the parameter rule so a zeroed register block does not silently filter everything away

AXI-Lite has no transaction IDs, so the `axil4_*_mon` wrappers tie these off rather than exposing them — a filter keyed on a field the protocol lacks has nothing to match against.

The **address** filter is driven the same way. `ADDR_FILTER_ENABLE` only builds the logic; without these three ports driven it stays inert, which is the failure a reader hits when the parameter is the only thing documented:

Port	Width	Description
`cfg_addr_filter_enable`	1	High: suppress packets for transactions outside the window. Low: filter inert, regardless of the parameter
`cfg_addr_filter_low`	`ADDR_WIDTH`	Window base, inclusive
`cfg_addr_filter_high`	`ADDR_WIDTH`	Window limit, inclusive

The filter is live only when `ADDR_FILTER_ENABLE && cfg_addr_filter_enable`. The verdict is latched per table entry at ALLOCATION (`filtered_mask`), not re-evaluated at emission, so a transaction is judged once on its command address and its data and response beats follow that verdict — see `axi_monitor_trans_mgr`.

This gates the monitor's **observation** inputs only, never the datapath the wrapper drives, so a filtered instance stays transparent on the bus. All three channels filter on the same range deliberately: filtering the command alone would leave data/resp for other IDs arriving unmatched, and the unmatched path allocates orphans — the table would fill with other channels' traffic, which is the problem the filter exists to avoid.

7.2.5 Timer LUT Sizing (CFI_*)

Forwarded to axi_monitor_timer, which divides aclk down to a 1 us tick. The divisor **is** the clock frequency in MHz, so a table built for this design's clock gives an exact tick.

Parameter	Type	Default	Description
CFI_MIN_FREQ_MHZ	int	100	Lowest frequency in the LUT
CFI_MAX_FREQ_MHZ	int	100	Highest frequency in the LUT
CFI_NUM_FREQ_ENTRIES	int	16	LUT entries. Note it does **not** size this module's cfg_freq_sel, which is a fixed 4-bit port here – only axi_monitor_timer derives its select width from this
CFI_FREQ_STRATEGY	int	0	0 = LINEAR spacing, 1 = POW2

The defaults set **every entry to 100 MHz**, so the tick is exactly 1 us at 100 MHz regardless of cfg_freq_sel. Override to a real MIN..MAX range only if the design changes aclk at runtime.

7.2.6 Synthesis-Cone Enables (ENABLE_*_LOGIC)

Each detection cone can be compiled out to save area. These gate the **logic**, not just packet emission — a disabled cone synthesizes away entirely. Defaults keep the classic cones on and perf/debug off.

Parameter	Type	Default	Effect when 0
ENABLE_ERROR_LOGIC	bit	1	Drop the error-detection cone (orphans, response errors)
ENABLE_TIMEOUT_LOGIC	bit	1	Drop the timeout cone **and** the axi_monitor_time

Parameter	Type	Default	Effect when 0
			out instance
ENABLE_COMPL_LOGIC	bit	1	Drop the completion cone
ENABLE_THRESHOLD_LOGIC	bit	1	Drop the threshold cone (latency / active-count thresholds)
ENABLE_PERF_LOGIC	bit	= ENABLE_PERF_PACKETS	Drop the reporter's **legacy perf-rollup cone** (axi_monitor_reporter_perf: two 16-bit lifetime counters + the 5-state emit FSM). It does **not** gate the perfmon measurement window or its bucket counters — those are unconditional (see below)
ENABLE_DEBUG_LOGIC	bit	0	Drop the debug cone

Removed: the former CAM_PIPELINE / TRANS_CAM_PIPELINE parameters no longer exist. The transaction CAM is now **always pipelined** (one extra cycle of active_count latency). The block_ready margin is derived from monitor_common_pkg::cmd_entry_reserve() — see Flow Control and the Saturation-Recovery Contract. Overshoot past MAX_TRANSACTIONS is structurally impossible regardless of the margin: the CAM allocates from its exact combinational free vector.

7.2.7 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`IW`	`ID_WIDTH`

7.3 Ports

7.3.1 Command Phase Interface (AW/AR)

Port	Direction	Width	Description
`cmd_addr`	Input	ADDR_WIDTH	Command address value
`cmd_id`	Input	ID_WIDTH	Transaction ID
`cmd_len`	Input	8	Burst length (AXI only)
`cmd_size`	Input	3	Burst size (AXI only)
`cmd_burst`	Input	2	Burst type (AXI only)
`cmd_valid`	Input	1	Command valid
`cmd_ready`	Input	1	Command ready

7.3.2 Data Channel Interface (R/W)

Port	Direction	Width	Description
`data_id`	Input	ID_WIDTH	Data transaction ID
`data_last`	Input	1	Last data beat indicator
`data_resp`	Input	2	Response code (OKAY/EXOKAY/SLVERR/DEC

Port	Direction	Width	Description
			ERR)
`data_valid`	Input	1	Data valid
`data_ready`	Input	1	Data ready

7.3.3 Response Channel Interface (B)

Port	Direction	Width	Description
`resp_id`	Input	ID_WIDTH	Response transaction ID
`resp_code`	Input	2	Response code
`resp_valid`	Input	1	Response valid
`resp_ready`	Input	1	Response ready

7.3.4 Configuration Interface

Port	Direction	Width	Description
`clear`	Input	1	**Synchronous clear** — passes through to `axi_monitor_trans_mgr` to empty the transaction CAM and zero the active-count pipeline atomically, without a full `aresetn`. Pulse one cycle while the monitor is idle.
`cfg_freq_sel`	Input	4	Frequency selection for timeout scaling
`cfg_addr_cnt`	Input	16	Address phase timeout, in

Port	Direction	Width	Description
			microseconds (1 us tick)
`cfg_data_cnt`	Input	16	Data phase timeout, in **microseconds**
`cfg_resp_cnt`	Input	16	Response phase timeout, in **microseconds**
`cfg_error_enable`	Input	1	Enable error event packets
`cfg_compl_enable`	Input	1	Enable completion packets
`cfg_threshold_enable`	Input	1	Enable threshold packets
`cfg_timeout_enable`	Input	1	Enable timeout packets
`cfg_perf_enable`	Input	1	Enable performance metric packets
`cfg_debug_enable`	Input	1	Enable debug/trace packets (feeds the debug reporter sub-block)
`cfg_active_trans_threshold`	Input	16	Active-transaction count that triggers a threshold packet
`cfg_latency_threshold`	Input	32	Latency value that triggers a threshold packet
`cfg_debug_level`	Input	4	**Dead** — referenced by no logic (interface to

Port	Direction	Width	Description
			the absent debug sub-module)
cfg_debug_mask	Input	16	**Dead** — referenced by no logic

7.3.5 Address-Range Checker Interface

Active only when N_ADDR_RANGES > 0; otherwise these inputs are ignored and the axi_monitor_addr_check block is not synthesized. Address-range violation packets are the lowest-priority source on the monitor bus (reporter > debug > addr_check).

Port	Direction	Width	Description
cfg_addr_check_enable	Input	1	Master enable for the address-range checker
cfg_addr_range_enable	Input	N_ADDR_RANGES	Per-range enable bit vector
cfg_addr_range_low	Input	N_ADDR_RANGES x ADDR_WIDTH	Per-range low (inclusive) address bounds
cfg_addr_range_high	Input	N_ADDR_RANGES x ADDR_WIDTH	Per-range high (inclusive) address bounds

7.3.6 Side-Band Timestamp Input

Port	Direction	Width	Description
i_mon_time	Input	64	Free-running counter from the monbus_group family (any wrapper), sampled at packet emission

7.3.7 Monitor Bus Output

Port	Direction	Width	Description
monbus_valid	Output	1	Monitor packet valid
monbus_ready	Input	1	Monitor packet ready (from downstream)
monbus_packet	Output	128	Standardized monitor_packet_t
monbus_timestamp	Output	64	monbus_timestamp_t paired atomically with monbus_packet
block_ready	Output	1	Flow control: 1 = upstream may proceed, 0 = transaction-table occupancy is at the blocking threshold. Derived from active_count vs MAX_TRANSACTIONS; the reporter's INTR_FIFO_DEPTH has **no** path to it. See Flow Control and the Saturation-Recovery Contract
busy	Output	1	Monitor is busy indicator (active_count > 0)
active_count	Output	8	Number of live transaction-table entries (registered

Port	Direction	Width	Description
			pop-count of CAM occupancy; lags true occupancy by 1 cycle)
`perf_completed_count`	Output	16	**Lifetime** completion count from `axi_monitor_reporter_perf` – packets actually emitted, not window-scoped. Reads 0 when `ENABLE_PERF_LOGIC = 0`
`perf_error_count`	Output	16	**Lifetime** error/timeout count from the same block; 0 when `ENABLE_PERF_LOGIC = 0`

The base module multiplexes three internal sources onto the monbus output — reporter packets (highest priority), debug packets, and addr_check error packets — driving `monbus_timestamp` from the source's sampled timestamp (`i_mon_time` for reporter/debug; `addr_pkt_timestamp` from the addr_check submodule).

7.3.8 Performance-Window Control Inputs

These configure the measurement-window state machine. Tie the selectors to a reserved code (e.g. `3'b111`) and the pulses/force-close low at instances that don't use perfmon. See Performance Monitoring.

Port	Direction	Width	Description
`cfg_start_event_sel`	Input	3	Window **start** event source select
`cfg_end_event_sel`	Input	3	Window **end**

Port	Direction	Width	Description
			event source select
cfg_start_trigger	Input	1	Software/engine pulse: open the measurement window
cfg_end_trigger	Input	1	Software/engine pulse: close the measurement window
cfg_window_force_close	Input	1	Software override: force the window closed immediately

7.3.9 Performance-Window Status and Counter Outputs

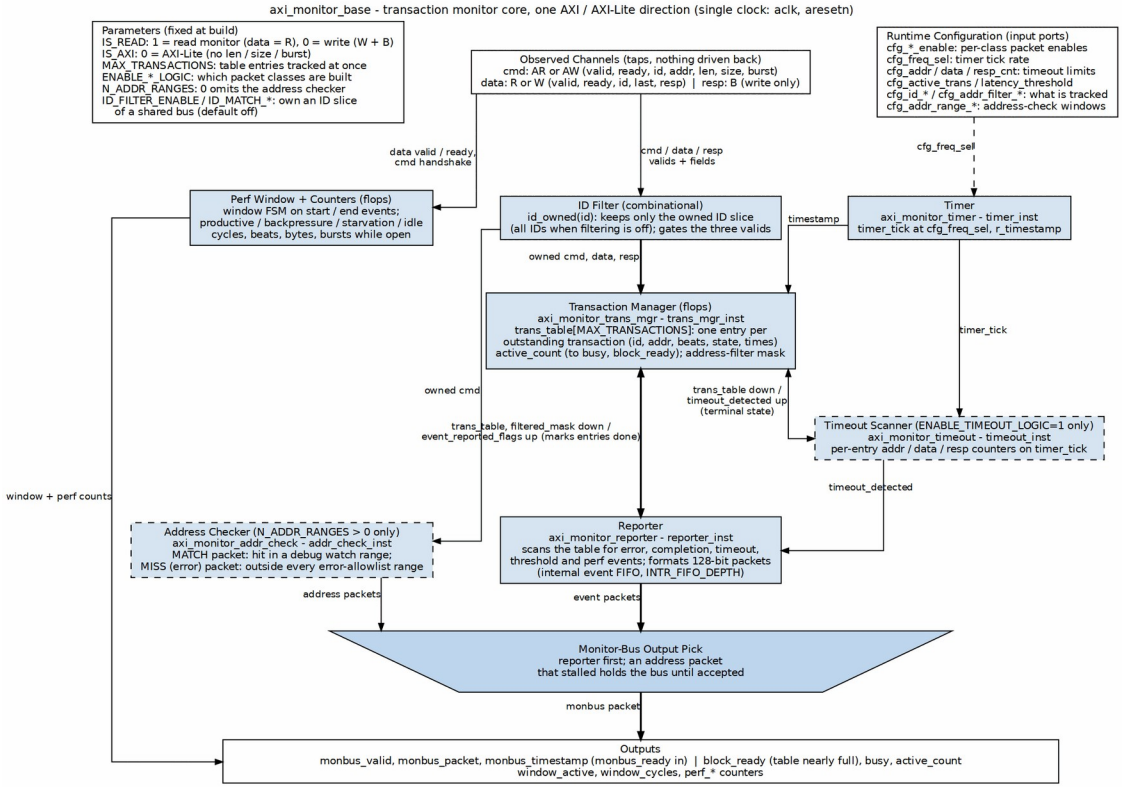
All counters accumulate only while window_active=1, reset at window-start, and hold their values through WIN_CLOSING/WIN_IDLE so the integrating block can sample a completed window's totals after window_active deasserts.

Port	Direction	Width	Description
window_active	Output	1	High while a measurement window is open
window_cycles	Output	32	Cycles elapsed inside the current window
perf_prod_cycles	Output	32	data_valid && data_ready cycles (productive beats)
perf_bp_cycles	Output	32	data_valid && ! data_ready cycles (back-pressure)
perf_starv_cycles	Output	32	!data_valid && data_ready cycles

Port	Direction	Width	Description
			(starvation)
perf_idle_cycles	Output	32	!data_valid && !data_ready cycles (idle)
perf_beat_count	Output	32	Data beats transferred (= perf_prod_cycles, 1 beat/cycle)
perf_byte_count	Output	64	Bytes transferred = beats x (1 << latched AXSIZE)
perf_burst_count	Output	32	Address-phase handshakes inside the window (AR for reads, AW for writes)

7.4 Functional Description

7.4.1 Block diagram



axi_monitor_base block diagram

The observed command, data and response channels enter at the top; nothing is driven back onto the bus. The ID filter keeps only this instance's slice of IDs, and the owned handshakes update the transaction manager's table, stamped by the timer. The timeout scanner and the reporter both read the table, and each feeds back into it: a timeout forces an entry terminal, and a reported event lets the manager free the entry. The reporter's event packets and the optional address checker's packets compete for the one monitor-bus output; the reporter wins, except that an address packet already presented and stalled holds the bus until it is accepted. The perf window counts handshake cycles on its own path to the outputs.

7.4.2 Architecture

The module coordinates four sub-components:

1. **Transaction Manager:** tracks transactions, manages the table
2. **Timeout Monitor:** detects stuck transactions
3. **Performance Tracker:** optional latency/throughput metrics
4. **Reporter:** generates standardized packets

7.4.3 Flow Control and the Saturation-Recovery Contract

This is the canonical statement of the monitor's flow-control behavior. The wrapper pages (`axi4_*_mon`, `axi5_*_mon`, `axil4_*_mon`) link here.

`block_ready` is a positive-enable flow-control output: 1 = the upstream handshake may proceed, 0 = stall. It is driven **solely by transaction-table occupancy** (`active_count` vs `MAX_TRANSACTIONS`); the reporter's packet FIFO (`INTR_FIFO_DEPTH`) has no path to it. In the `*_mon` wrappers, `block_ready` is an internal net ANDed into the upstream-facing ready signal — it is **not** a port on the wrappers.

The contract (single source of truth: `monitor_common_pkg::cmd_entry_reserve()` in `rtl/amba/includes/ocelus_common_pkg.sv`):

- **Command-entry cap.** The transaction manager caps command-originated entries (allocated by AW/AR activity) at `MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS)`. The reserve is 4 for `MAX_TRANSACTIONS >= 16` and 0 below. Orphan entries (data/response beats with no matching command) may still use every slot commands are not entitled to; orphans always drain via error reporting, so they cannot pin occupancy.
- **Reopen threshold strictly above the cap.** `block_ready` re-asserts at `active_count < MAX_TRANSACTIONS - (cmd_entry_reserve - 1)`. Because this threshold sits **strictly above** the command cap, a saturated table always drains back below the reopen point: even if every command entry is permanently in flight, occupancy cannot reach the threshold once the orphan entries drain. Blocking therefore **throttles, never deadlocks**.
- **The blocking margin covers all three allocators.** `active_count` is a registered pop-count that lags true occupancy by one cycle, and three independent allocators (address, data, response) can each take a slot in that stale cycle. The derived margin (`cmd_entry_reserve - 1 = 3`) covers all three, so a command can never be admitted against stale occupancy and then find no free slot. The reserve of 4 is the only value that satisfies both this and the reopen constraint above simultaneously; with the earlier reserve of 2 the margin derived to 1, and commands admitted in the stale cycle went untracked while their un-backpressureable data beats were silently discarded (measured as an observer-vs-in-core burst-count gap of 4096 vs 3073). `val/amba/test_axi_mon_block_ready.py` asserts no command is admitted without an allocation, on every wrapper.
- **Overshoot is impossible.** The CAM allocates from its exact combinational free vector, so the table can never hold more than `MAX_TRANSACTIONS`

entries. A command that handshakes while the cap is reached is simply not tracked (lossy-but-honest degrade), never stalled forever.

- **Small tables trade recovery for capacity.** Tables with `MAX_TRANSACTIONS < 16` get `cmd_entry_reserve = 0` and keep the full legacy allocation (with the legacy flat margin of 3 on `block_ready`). Small tables cannot spare slots — even one reserved slot at `MAX=8` measurably starves oversubscribed same-ID tracking — so they give up the recovery guarantee in exchange for full tracking capacity.

Both `axi_monitor_trans_mgr` (the cap) and this module (the `block_ready` margin) derive their constants from the same package function, so the two cannot drift. The contract is enforced by in-RTL formal properties in `axi_monitor_trans_mgr.sv` (`ap_cmd_entry_cap`, `ap_no_reopened_complete`, mutation-checked) and by a 100-seed deliberately-undersized-table stream sweep (`test_stream_core_mon_backpressure` in the STREAM project area, plus `val/amba/test_axi_monitor_trans_mgr.py::phase_saturation_recovers`).

History: before commit `cb29e226`, a stray non-last data beat could poison a terminal entry into an unclosable state, occupancy ratcheted to `MAX`, and the old flat `MAX-3` margin placed the reopen threshold exactly AT the saturation point — `block_ready` latched low forever and the monitored datapath wedged (the stream_core multi-channel hang). Commit `95c9490a` extended the fix to runtime-disabled packet classes (see the auto-retire notes in axi_monitor_reporter).

Sizing rule for shared masters: a monitor watching a bus shared by multiple channels/requesters must size `MAX_TRANSACTIONS` to cover the SUM of outstanding transactions — `NUM_CHANNELS x per-channel outstanding`, plus margin (the orphan reserve and the 1-cycle `active_count` lag). Sizing to the per-channel limit alone makes the monitor throttle the shared master from two channels up; this exact mistake shipped in stream_core (fixed in `95c9490a` by making the monitor table parametric on channel count).

Verilator note: transaction tables deeper than 64 require raising `--unroll-count` (default 64) in simulation builds, or the per-slot generate loops fail with BLKLOOPINIT errors. The stream sim builds use `--unroll-count 256`.

7.4.4 Performance Monitoring

`axi_monitor_base` is the **canonical home** of the monitor's performance subsystem. Every AXI/AXIL wrapper (`axi4_*_mon`, `axil4_*_mon`, and the `axi_monitor_filtered` wrapper) forwards these ports straight through to this core. Because the base is generic, the data channel it measures is chosen by `IS_READ`: the **R** channel and **AR** bursts for read monitors, the **W** channel and **AW** bursts for write monitors.

The perfmon logic is **always instantiated**: the measurement-window state machine and the bank of data-channel utilization and throughput counters are plain `always_ff` blocks in this module with no generate guard, so they cost their area in every build. `ENABLE_PERF_LOGIC` gates only the reporter's legacy perf-rollup sub-block (`axi_monitor_reporter_perf`), which is a different thing: two 16-bit lifetime counters and the FSM that emits `PktTypePerf` rollup packets. Building

with ENABLE_PERF_PACKETS=0 therefore removes the rollup packets, not the window. All window counters accumulate **only while a window is open** and hold their values between windows so the host can read a completed window's totals.

7.4.4.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**. Event sources are chosen by the 3-bit cfg_start_event_sel / cfg_end_event_sel selectors:

Code	Start event	End event
3'b000	cfg_start_trigger pulse (software/CSR)	cfg_end_trigger pulse
3'b001	first command handshake (cmd_valid && cmd_ready)	last data (reads: data_last beat; writes: response handshake)
3'b010	cfg_perf_enable rising edge	cfg_perf_enable falling edge
3'b011	first data handshake (data_valid && data_ready)	window_cycles saturation
3'b100	cfg_start_trigger pulse (external-trigger convention)	cfg_end_trigger pulse
others	never fires (reserved)	never fires (reserved)

cfg_window_force_close is a software override that closes an open window immediately regardless of the end-event selector.

The state machine has three states:

- **WIN_IDLE** — waiting for a start event; window_active=0.
- **WIN_ACTIVE** — window open; window_active=1; window_cycles free-runs (starting at 1 so the total is inclusive of the first cycle). All counters accumulate.
- **WIN_CLOSING** — one-cycle hold before re-arming; counters are frozen and stable for sampling.

7.4.4.2 Utilization Buckets (data channel)

Every cycle inside the window is classified by the data channel's valid/ready into exactly one of four mutually-exclusive buckets. The four buckets sum to window_cycles - 1 by construction (the start cycle seeds window_cycles to 1 while the buckets reset to 0), so utilization =

`perf_prod_cycles / (window_cycles - 1)` — **until a counter saturates.** `window_cycles` freezes at `32'hFFFF_FFFE` and each bucket sticks at `32'hFFFF_FFFF` rather than wrapping, so on a window longer than ~2^32 cycles (~43 s at 100 MHz) the identity and this formula both break. A reader seeing `32'hFFFF_FFFF`, or a bucket sum below `window_cycles - 1`, is looking at an overflowed window.

Counter	Condition	Meaning
`perf_prod_cycles`	`valid && ready`	productive beat transferred
`perf_bp_cycles`	`valid && !ready`	back-pressure (data offered, sink not ready)
`perf_starv_cycles`	`!valid && ready`	starvation (sink ready, no data)
`perf_idle_cycles`	`!valid && !ready`	idle

7.4.4.3 Throughput Counters

Counter	Width	Meaning
`perf_beat_count`	32	data beats transferred (= `perf_prod_cycles`, 1 beat/cycle)
`perf_byte_count`	64	bytes transferred = beats x (1 << latched AXSIZE). AXSIZE is captured (`cmd_size`) at the most recent address-phase handshake and assumed constant within a burst per the AXI4 mandate. 64-bit width prevents wrap on long windows at wide buses
`perf_burst_count`	32	address-phase handshakes inside the window (AR for read monitors, AW for write monitors)

The integrator computes average burst length as `perf_beat_count / perf_burst_count`.

Note: the perfmon counters are exposed on the module interface for the integrating block to sample; the reporter's `PerfWin` packet emission is staged separately (RFC Stage B+). The four-bucket model follows `DMA_UTILIZATION_MEASUREMENT.md` Section 3.

7.5 Timing Characteristics

Metric	Value	Notes
Latency	2-3 cycles	Event detection to packet output
Throughput	1 packet per 2 cycles	The reporter's registered output stage cannot load a new packet on the same cycle the previous one is accepted, so sustained rate is at most one packet every other cycle
Table Lookup	1 cycle	ID-based transaction lookup

7.6 Usage Examples

Not typically instantiated directly by users. Use the high-level monitors instead:

```
// User instantiates this:
axi4_master_rd_mon #(...) u_mon (...);

// Which internally uses:
// - axi4_master_rd (frontend)
// - axi_monitor_base (this module)
// - axi_monitor_filtered
```

7.7 Design Notes

7.7.1 Transaction Table Sizing

Design Scenario	MAX_TRANSACTIONS	Rationale
AXI4 Master (Burst)	16-32	Support multiple outstanding bursts
AXI4 Slave (Burst)	16-32	Handle out-of-order responses
AXI4-Lite Master	4-8	Single-beat, limited concurrency
AXI4-Lite Slave	4-8	Simple protocol, fewer outstanding
Shared master (multi-channel)	NUM_CHANNELS x per-channel outstanding + margin	Per-channel limit alone throttles the shared bus — see the sizing rule in Flow Control and the Saturation-Recovery Contract

Tables of 16 or more slots reserve `cmd_entry_reserve(MAX) = 4` slots (the saturation-recovery guarantee plus a blocking margin wide enough for all three same-cycle allocators); tables below 16 keep full legacy allocation. Tables deeper than 64 need `--unroll-count` raised above Verilator's default of 64 in sim builds.

7.7.2 Timeout Configuration

- **`cfg_addr_cnt`**: command phase timeout (AR/AW)
- **`cfg_data_cnt`**: data phase timeout (R/W)
- **`cfg_resp_cnt`**: response phase timeout (B)

Typical values:

- Burst traffic: `cfg_*_cnt = 4-8` (aggressive timeout)
- Memory controllers: `cfg_*_cnt = 10-15` (allow latency)
- External interfaces: a large `cfg_*_cnt` (the field is 16 bits, so up to 65535 us ~= 65 ms; `0xFFFF` is effectively "never time out")

7.8 Related Modules

- **axi_monitor_filtered**
- **axi_monitor_trans_mgr**
- **axi_monitor_reporter**
- **axi_monitor_timeout**

Used by:

- **axi4_master_rd_mon**
- **axi4_master_wr_mon**
- **axi4_slave_rd_mon**
- **axi4_slave_wr_mon**
- **axil4_master_rd_mon**
- **axil4_master_wr_mon**
- **axil4_slave_rd_mon**
- **axil4_slave_wr_mon**

See also:

- **Monitor Architecture:** `docs/markdown/rtl-amba/overview.md`
- **Monitor Configuration Guide:** Monitor Base Configuration
- **Packet Format Specification:** `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`

7.9 Testing

Key test scenarios:

1. **Transaction Table Management:**
   - Fill table to MAX_TRANSACTIONS
   - Verify table exhaustion handling
   - Test ID reuse after completion
2. **Out-of-Order Transactions:**
   - Issue multiple transactions with different IDs
   - Complete in random order
   - Verify correct ID matching
3. **Timeout Detection:**

    - Initiate transaction without completion
    - Verify timeout event at configured threshold
    - Check timeout packet contains correct ID
4. **Performance Metrics:**
    - Enable `ENABLE_PERF_PACKETS`
    - Verify latency calculations
    - Check throughput tracking

7.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. — §A2.3.1 handshake, §A3.3 transaction response (BRESP/RRESP), §A5.1 transaction identifiers, §A5.5 and §A5.6 response ordering: the rules the monitor tracks and checks.
- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. "An Event-Based Monitoring Service for Networks on Chip." *ACM Trans. Design Automation of Electronic Systems*, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126. — related work: hardware probes that generate events and send them as monitoring traffic.
- Goossens, K., Vermeulen, B., van Steeden, R., Bennebroek, M. "Transaction-Based Communication-Centric Debug." *Proc. 1st Int. Symp. Networks-on-Chip (NOCS 2007)*, pp. 95-106, May 2007. DOI: 10.1109/NOCS.2007.46. — related work: transaction-level monitors with a separate event interconnect.
- Abramovici, M., Bradley, P., Dwarakanath, K., Levin, P., Memmi, G., Miller, D. "A Reconfigurable Design-for-Debug Infrastructure for SoCs." *Proc. 43rd Design Automation Conf. (DAC 2006)*, pp. 7-12, July 2006. DOI: 10.1145/1146909.1146916. — related work: a debug fabric hosting checkers, transaction identifiers, triggers and event counters.
- Pagiamtzis, K., Sheikholeslami, A. "Content-Addressable Memory (CAM) Circuits and Architectures: A Tutorial and Survey." *IEEE J. Solid-State Circuits*, 41(3):712-727, Mar. 2006. DOI: 10.1109/JSSC.2005.864128. — for the CAM structure behind the outstanding-transaction table.

- Hopkins, A.B.T., McDonald-Maier, K.D. “Debug Support for Complex Systems On-Chip: A Review.” *IEE Proc. Computers and Digital Techniques*, 153(4):197-207, 2006. DOI: 10.1049/ip-cdt:20050194. — survey of on-chip debug support, for context.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, sec. “ARM AMBA interconnect protocols suite”. — textbook background on the AXI protocol family (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/common/references/ContentAddressableMemory__Pagiamtzis2006.pdf`)

7.11 Navigation

- **Back to Shared Infrastructure Index**
- **Back to rtl-amba Index**
- **Back to Main Documentation Index**

8 AXI Monitor Filtered

Module: `axi_monitor_filtered.sv` **Location:** `rtl/amba/ocelus/` **Category:** Core Infrastructure **Status:** Production Ready

Plane: protocol plane (source filter; used by both full and lite monitors)

8.1 Overview

`axi_monitor_filtered` is a **filtering wrapper around `axi_monitor_base`**. It instantiates one `axi_monitor_base` internally, taps the same AXI/AXIL command, data, and response channels, and then applies a configurable drop filter to the 128-bit monitor packets the base emits before forwarding them downstream. It does **not** take a monitor-bus input stream — it produces the monitor bus from the channels it observes.

This is a **shared infrastructure module** used internally by the AXI/AXIL monitors. You won't instantiate it directly, but it's the piece that keeps monitor-bus traffic manageable, so it's worth understanding.

Key features:

- **Wraps `axi_monitor_base`:** all transaction tracking, timeout, threshold, and perfmon logic lives in the base; this module only filters its output stream.
- **Level 1 — packet-type drop mask** (`cfg_axi_pkt_mask`): drop entire packet types by index.
- **Level 2 — error select** (`cfg_axi_err_select`): reserved for cross-routing; in the AXI wrapper it is used only for **configuration-conflict validation** (see `cfg_conflict_error`), not applied to the stream.
- **Level 3 — per-event-code masking:** one 16-bit drop mask per packet type (`cfg_axi_error_mask`, `cfg_axi_timeout_mask`, `cfg_axi_compl_mask`, `cfg_axi_thresh_mask`, `cfg_axi_perf_mask`, `cfg_axi_addr_mask`, `cfg_axi_debug_mask`) indexed by the packet's event code.
- **Configuration conflict detection:** `cfg_conflict_error` flags overlapping `cfg_axi_pkt_mask` / `cfg_axi_err_select` bits.
- **Bypass mode:** `ENABLE_FILTERING=0` passes every packet straight through.
- **Optional pipeline stage:** `ADD_PIPELINE_STAGE=1` registers the filtered output for timing closure.

Four reasons this block exists:

1. **Traffic Management:** reduces monitor bus congestion by dropping unwanted packet types and event codes at the source.
2. **Granular Control:** supports per-packet-type (Level 1) and per-event-code (Level 3) drop masking.
3. **Configuration Validation:** flags conflicting mask settings via `cfg_conflict_error`.
4. **Protocol Isolation:** drops any non-AXI-protocol packet (protocol field `!= 4'h0`), which should never occur inside an AXI monitor.

8.2 Parameters

Parameter	Type	Default	Description
`UNIT_ID` / `AGENT_ID`	logic	`8'h01` / `16'h000A`	Identity bits stamped into emitted monitor packets.
`MAX_TRANSACTIONS`	int	16	Outstanding transaction table depth (passed to `axi_monitor_base` -> `axi_monitor_trans_mgr`).
`ADDR_WIDTH` / `ID_WIDTH`	int	32 / 8	Address and AXI ID widths. `ID_WIDTH` has a **hard maximum of 8** (the transaction-table id field is 8 bits; wider is refused at elaboration with an `$error`). `ADDR_WIDTH > 32` is allowed but truncates the address carried in

Parameter	Type	Default	Description
			packets to the low 32 bits.
`IS_READ` / `IS_AXI`	bit	1 / 1	Direction (read vs write) and protocol family (AXI4 vs AXIL).
`ENABLE_PERF_PACKETS`	bit	1	Emit perf packets onto the monitor bus.
`ENABLE_DEBUG_MODULE`	bit	0	**Inert / reserved** – the debug-trace sub-module it names does not exist; forwarded to `axi_monitor_base` only because the wrapper family plumbs it. Real debug packets come from `ENABLE_DEBUG_LOGIC` + `cfg_debug_enable`.
Reporter sub-block enables			Each gates one of the six reporter sub-blocks (`axi_monitor_reporter_*`) at elaboration time. Pass-through to `axi_monitor_base`.
`ENABLE_ERROR_LOGIC`	bit	1	Error reporter (orphans, resp

Parameter	Type	Default	Description
			errors).
ENABLE_TIMEOUT_LOGIC	bit	1	Timeout reporter (addr/data/resp timers).
ENABLE_COMPL_LOGIC	bit	1	Completion reporter.
ENABLE_THRESHOLD_LOGIC	bit	1	Threshold reporter (latency / active-count thresholds).
ENABLE_PERF_LOGIC	bit	ENABLE_PERF_PACKETS	Gates axi_monitor_reporter_perf only: two 16-bit lifetime counters plus the 5-state emit FSM. It does **not** gate the Stage B window counters below — those are unconditional always_ff blocks in axi_monitor_base and are live in every build.
ENABLE_DEBUG_LOGIC	bit	0	Debug reporter.
ENABLE_FILTERING	bit	1	Master enable for filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
ADD_PIPELINE_STA	bit	0	Add register stage

Parameter	Type	Default	Description
GE			for timing.
N_ADDR_RANGES	int	0	Number of address-range comparators in the axi_monitor_addr_check sub-block (0 = the comparator block is not synthesised at all).
ADDR_RANGE_IS_ERROR	logic [N_ADDR_RANGES-1:0]	'0	Per-range flavor forwarded to the checker: 0 = DEBUG (hit -> AddrMatch), 1 = ERROR (allowlist miss -> Error/ADDR_RANGE). Default all-0.

8.2.1 Forwarded to axi_monitor_base (pass-through)

These are declared here and passed straight down; the base module's page is the authority on what each one does. They are listed so this wrapper's own page is a complete inventory.

Parameter	Type	Default	Description
USE_WDATA_ORDER_Q	bit	0	Write monitors: recover WID-less W-beat order with an AWID FIFO. **Required** when NUM_BANKS > 1 on a write monitor — the combination without it refuses to elaborate

Parameter	Type	Default	Description
NUM_BANKS	int	1	Bank the transaction table by low ID bits. Power of 2, must divide MAX_TRANSACTIONS; per-ID concurrency is capped by the **bank** depth
ID_FILTER_ENABLE	bit	0	Track only a slice of the IDs on a shared bus
ID_MATCH_BASE	int	0	First ID owned by this instance
ID_MATCH_COUNT	int	0	How many IDs; 0 = all (no filter)
ADDR_FILTER_ENABLE	bit	0	Address-range packet filter. Builds the logic only; the cfg_addr_filter_* ports below arm it
CFI_MIN_FREQ_MHZ	int	100	Timer LUT lower bound (MHz)
CFI_MAX_FREQ_MHZ	int	100	Timer LUT upper bound (MHz)
CFI_NUM_FREQ_ENTRIES	int	16	Timer LUT entries. Does **not** size this module's cfg_freq_sel, which is a fixed 4-bit port here
CFI_FREQ_STRATEGY	int	0	0 = LINEAR, 1 =

Parameter	Type	Default	Description
			POW2

See Transaction-Table Shaping, ID-Range Filter and Timer LUT Sizing in `axi_monitor_base` for the sizing rules and the elaboration error.

8.2.2 Filter configuration and status (pass-through)

Also declared here and passed straight down. The two filters are inert unless their `cfg_*_enable` is driven, whatever the parameters say:

Port	Dir	Width	Description
`cfg_id_filter_enable`	input	1	Use the runtime ID window instead of `ID_MATCH_BASE/COUNT`
`cfg_id_match_base`	input	`ID_WIDTH`	First ID owned, runtime
`cfg_id_match_count`	input	`ID_WIDTH+1`	How many; 0 = all
`cfg_addr_filter_enable`	input	1	Arm the address-range packet filter
`cfg_addr_filter_low`	input	`ADDR_WIDTH`	Window base, inclusive
`cfg_addr_filter_high`	input	`ADDR_WIDTH`	Window limit, inclusive
`block_ready`	output	1	Table near capacity: stop admitting new commands
`busy`	output	1	At least one transaction is being tracked
`active_count`	output	8	Registered count of occupied table entries

block_ready, busy and active_count are real outputs and must be connected or explicitly left open; they are how an integrator sees the monitor's own backpressure and occupancy.

8.3 Ports

8.3.1 Observed AXI/AXIL Channels (inputs to the wrapped base)

This module has **no monitor-bus input stream.** It taps the same command, data, and response channels as axi_monitor_base and passes them through unchanged. See axi_monitor_base for full descriptions.

Group	Ports
Command (AW/AR)	cmd_addr, cmd_id, cmd_len, cmd_size, cmd_burst, cmd_valid, cmd_ready
Data (W/R)	data_id, data_last, data_resp, data_valid, data_ready
Response (B)	resp_id, resp_code, resp_valid, resp_ready
Timer / enables / thresholds	cfg_freq_sel, cfg_addr_cnt, cfg_data_cnt, cfg_resp_cnt, cfg_error_enable, cfg_compl_enable, cfg_threshold_enable, cfg_timeout_enable, cfg_perf_enable, cfg_debug_enable, cfg_debug_level, cfg_debug_mask, cfg_active_trans_threshold, cfg_latency_threshold
Address-range checker (when N_ADDR_RANGES > 0)	cfg_addr_check_enable, cfg_addr_range_enable, cfg_addr_range_low, cfg_addr_range_high
Side-band time	i_mon_time (monbus_timestamp_t)

All of the above are passed through to the internal axi_monitor_base verbatim.

8.3.2 Monitor Bus Output (filtered)

Port	Direction	Width	Description
monbus_valid	Output	1	Filtered packet

Port	Direction	Width	Description
			valid (base valid AND not dropped)
monbus_ready	Input	1	Downstream ready
monbus_packet	Output	128	Filtered monitor_packet_t (unmodified; only passed or dropped)
monbus_timestamp	Output	64	monbus_timestamp_t paired atomically with monbus_packet

8.3.3 Reset and Synchronous Control

Port	Direction	Width	Description
aclk / aresetn	Input	1 / 1	Standard clock and active-low async reset.
clear	Input	1	**Synchronous clear** — passes through to axi_monitor_base -> axi_monitor_trans_mgr to empty the transaction CAM and zero the active-count pipeline without aresetn. Pulse one cycle while idle.

8.3.4 Filter Configuration

All filter masks use **drop semantics**: a set bit drops the corresponding packet type or event code. All masks are 16-bit.

Port	Direction	Width	Description
`cfg_axi_pkt_mask`	Input	16	**Level 1** packet-type drop mask. Bit `pkt_type` set -> drop all packets of that type
`cfg_axi_err_select`	Input	16	**Level 2** error-select. Reserved for cross-routing; in the AXI wrapper it is consumed only by the conflict check, not applied to the stream
`cfg_axi_error_mask`	Input	16	**Level 3** per-event-code drop mask for Error packets
`cfg_axi_timeout_mask`	Input	16	Level 3 drop mask for Timeout packets
`cfg_axi_compl_mask`	Input	16	Level 3 drop mask for Completion packets
`cfg_axi_thresh_mask`	Input	16	Level 3 drop mask for Threshold packets
`cfg_axi_perf_mask`	Input	16	Level 3 drop mask for Performance packets
`cfg_axi_addr_mask`	Input	16	Level 3 drop mask for Address-Match

Port	Direction	Width	Description
			packets
cfg_axi_debug_mask	Input	16	Level 3 drop mask for Debug packets

The Level 3 masks are indexed by the packet's event code (low nibble). For each packet the wrapper selects the mask matching its packet type, then drops the packet if `mask[event_code[3:0]]` is set.

8.3.5 Configuration Status Output

Port	Direction	Width	Description
cfg_conflict_error	Output	1	Asserted when cfg_axi_pkt_mask & cfg_axi_err_select is non-zero (overlapping type is both dropped and error-selected)

8.3.6 Performance Window Control (Stage A of perfmon RFC)

Port	Direction	Width	Description
cfg_start_event_sel	Input	3	Event selector that opens the perf window (e.g. command-handshake, first beat, software pulse).
cfg_end_event_sel	Input	3	Event selector that closes the perf window.
cfg_start_trigger	Input	1	Software-driven open pulse (combines with cfg_start_event_sel).

Port	Direction	Width	Description
cfg_end_trigger	Input	1	Software-driven close pulse.
cfg_window_force_close	Input	1	**Synchronous** software override: forces the measurement window closed on the next clock edge. Perf totals are **held**, not dropped – the counters keep their values through WIN_CLOSING/WIN_IDLE so a host can read the forced window, and reset only at the next window start.
window_active	Output	1	The perf window is currently open.
window_cycles	Output	32	Number of cycles the current window has been open.

Tie cfg_start_event_sel/cfg_end_event_sel to 3'b111 and the triggers + cfg_window_force_close to 1'b0 at instances that don't use perfmon.

8.3.7 Performance Counters (Stage B of perfmon RFC)

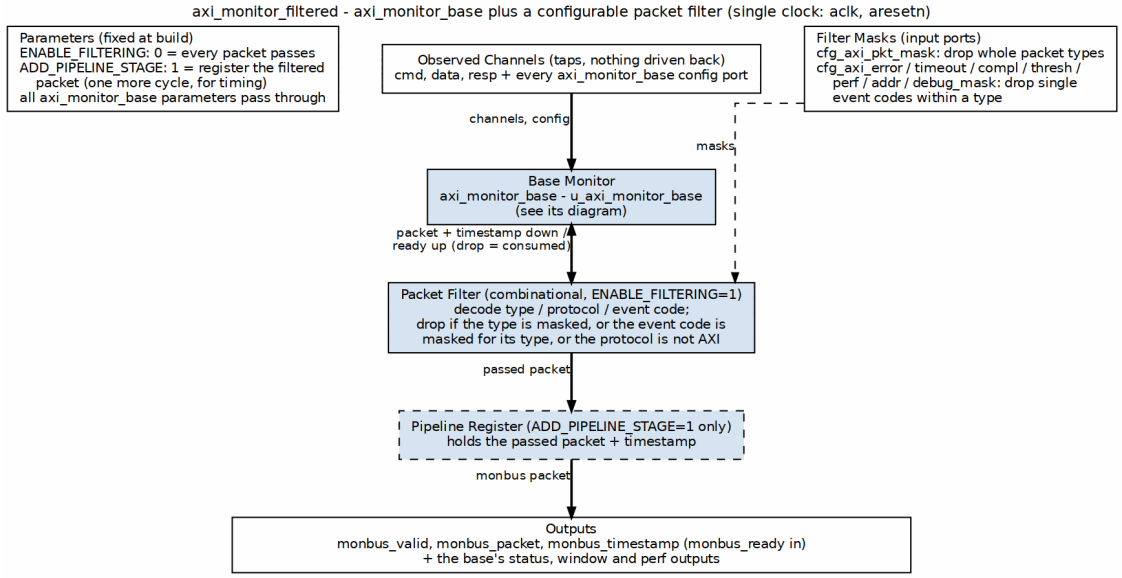
Port	Direction	Width	Description
perf_prod_cycles	Output	32	Productive cycles (valid + ready both high).
perf_bp_cycles	Output	32	Back-pressure

Port	Direction	Width	Description
			cycles (valid high, ready low).
`perf_starv_cycles`	Output	32	Starvation cycles (valid low, ready high).
`perf_idle_cycles`	Output	32	Idle cycles (both low).
`perf_beat_count`	Output	32	Beat handshakes accumulated this window.
`perf_byte_count`	Output	64	Bytes transferred this window: productive beats x (1 << latched `cmd_size`). `cmd_len` is not involved.
`perf_burst_count`	Output	32	Bursts (command handshakes) this window.
`perf_completed_count`	Output	16	**Lifetime** completion count from `axi_monitor_reporter_perf` (not window-scoped). Reads 0 when `ENABLE_PERF_LOGIC = 0`
`perf_error_count`	Output	16	**Lifetime** error/timeout count from the same block; 0 when `ENABLE_PERF_LOGIC = 0`

These counters latch on each `window_active` open and freeze on close; software reads them after the close edge.

8.4 Functional Description

8.4.1 Block diagram



axi_monitor_filtered block diagram

The base monitor (see its own diagram) does all tracking and packet generation. Its monitor-bus output passes through a two-level filter: a whole packet type can be dropped, or a single event code within a type. A dropped packet is still acknowledged back to the base, so the base never stalls on it. Passed packets go out directly, or through one optional register stage.

8.4.2 Architecture

The base monitor generates every packet; the filter then drops unwanted ones. A dropped packet is acknowledged back to the base (`base_monbus_ready`) so the base never stalls on packets that will be discarded. Level 2 (`cfg_axi_err_select`) is a validation-only input in this wrapper and is not shown in the drop path.

8.5 Timing Characteristics

Metric	Value	Notes
Filtering Latency	0-1 cycles	Combinatorial (0) or

Metric	Value	Notes
		registered (1)
Throughput	1 packet per 2 cycles	Limited by the reporter's registered output stage; the filter itself introduces no additional backpressure

8.6 Usage Examples

This module is instantiated automatically within higher-level monitor modules — users configure behavior through top-level monitor parameters.

8.6.1 Filter Strategy

`cfg_axi_pkt_mask` uses **drop semantics** — a bit set at index `pkt_type` drops that type. Leave a bit `0` to keep the type. Indices follow `monitor_common_pkg` (`PktTypeError`, `PktTypeCompletion`, `PktTypeThreshold`, `PktTypeTimeout`, `PktTypePerf`, `PktTypeAddrMatch`, `PktTypeDebug`). Tie the per-event Level 3 masks to `16'h0000` unless you need to suppress specific event codes.

Pass everything (no filtering):

```
.cfg_axi_pkt_mask   (16'h0000),  // drop nothing
// or set ENABLE_FILTERING = 0 at elaboration for full bypass
```

Suppress performance packets (typical functional run):

```
.cfg_axi_pkt_mask   (16'h0000 | (16'h1 << PktTypePerf)),  // drop only
Perf
```

Performance-only capture (drop completions to cut traffic):

```
.cfg_axi_pkt_mask   ((16'h1 << PktTypeCompletion) |
                     (16'h1 << PktTypeTimeout)),          // keep
Error + Perf
```

Suppress one Error event code (Level 3):

```
.cfg_axi_pkt_mask   (16'h0000),                  // keep all types
.cfg_axi_error_mask (16'h1 << ERR_EVENT_CODE),  // drop just that
error event
```

8.7 Design Notes

Two active filter levels, not three. Packet-type masks (`cfg_axi_pkt_mask`), then per-event-code masks. `cfg_axi_err_select` is RESERVED and routes nothing: this module uses it in exactly one expression, `cfg_conflict_error = |(cfg_axi_pkt_mask & cfg_axi_err_select)`, so its only effect is to raise that flag on an overlap. Its own port comment says "unused in this context".

A mask bit set means DROP. The sense catches people out; a mask of all ones emits nothing.

8.8 Related Modules

- **axi_monitor_base**

Used by:

- **axi4_master_rd_mon**
- **axi4_master_wr_mon**
- **axi4_slave_rd_mon**
- **axi4_slave_wr_mon**

See also:

- **Monitor Architecture:** `docs/markdown/rtl-amba/overview.md`
- **Monitor Configuration Guide:** Monitor Base Configuration
- **Packet Format Specification:** `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`

8.9 Testing

Key test scenarios:

1. **Level 1 masking:**
   - Generate all packet types from the base
   - Set individual `cfg_axi_pkt_mask` bits and verify only the masked types are dropped
2. **Level 3 masking:**
   - Set a per-type event mask (e.g. `cfg_axi_error_mask`) and verify only packets whose event code matches the set bit are dropped
3. **Configuration conflict:**

- Set the same bit in both `cfg_axi_pkt_mask` and `cfg_axi_err_select` and verify `cfg_conflict_error` asserts

4. **Packet integrity:**
   - Verify passed packets are bit-identical to the base output (filter only drops, never mutates)
   - Verify a dropped packet is acked to the base so it never stalls
   - Exercise `ADD_PIPELINE_STAGE=1` and confirm one-cycle latency with correct backpressure

8.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. "An Event-Based Monitoring Service for Networks on Chip." *ACM Trans. Design Automation of Electronic Systems*, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126. — related work: reconfigurable probes with a run-time programming model for what they report.
- Abramovici, M., Bradley, P., Dwarakanath, K., Levin, P., Memmi, G., Miller, D. "A Reconfigurable Design-for-Debug Infrastructure for SoCs." *Proc. 43rd Design Automation Conf. (DAC 2006)*, pp. 7-12, July 2006. DOI: 10.1145/1146909.1146916. — related work: programmable triggers and filters in a debug fabric.
- IEEE-ISTO 5001-2003. *The Nexus 5001 Forum Standard for a Global Embedded Processor Debug Interface*, Version 2.0, IEEE Industry Standards and Technology Organization, Dec. 2003 (defining non-peer standard). — §5.2: typed messages (TCODE), comparable to filtering on `pkt_type`/`event_code`.
- Indexed in `rtl/amba/references/README.md` (local PDF: `DebugTraceInterfaceStandard__IeeeIsto2003.pdf`)

8.11 Navigation

- **Back to Shared Infrastructure Index**
- **Back to rtl-amba Index**
- **Back to Main Documentation Index**

9 AXI Monitor Lite

Module: axi_monitor_lite.sv **Location:** rtl/amba/ocelus/axi_monitor_lite.sv
Category: Monitor Infrastructure **Status:** Production Ready (rtl/amba/monitors TASK-001, 2026-09-25); adopted by every consumer, measured in three board builds and validated on silicon, 2026-09-27

Plane: protocol plane (lite monitor; the full variant is axi_monitor_base)

9.1 Overview

axi_monitor_lite is the AXI transaction monitor rebuilt for gate count and timing: three quarters of what axi_monitor_base reports, for about a fifth of its LUTs. It sits behind the same axi4/axi5/axil4/axil5_{master,slave}_{rd,wr}_mon wrappers as their _monlite siblings (axi4_slave_rd_monlite and fifteen more), and emits the same 128-bit monitor_packet_t with the same UNIT_ID/AGENT_ID on the same monbus handshake – the arbiter, the group, the tally and the host tooling cannot tell which monitor produced a packet.

What it reports:

- **Error** packets: a SLVERR/DECERR response, a data or response beat with no owning transaction (orphan), a burst whose LAST came early or never came, a write burst arriving ahead of a second address, and EVENT_DROPPED (see below) – each with the transaction's ID in channel_id and its address in event_data, exactly as the full monitor packs them.
- **Completion** packets on a read's last beat or a write's B, with the address and, in event_data[63:48], the latency in cycles from the command handshake.
- **Timeout** packets when a transaction makes no progress for cfg_timeout_cnt microseconds, naming the phase (CMD for a stalled command channel, DATA, RESP).
- **Threshold** packets when the number of outstanding transactions crosses cfg_active_trans_threshold upward, or a completion's latency exceeds cfg_latency_threshold.
- **Address-range** packets from the optional checker (N_ADDR_RANGES > 0): Error/ADDR_RANGE for a miss against an error-flavoured range, AddrMatch for a hit on a match-flavoured one, gated by cfg_addr_match_enable.

What it does not do, by design: performance packets and windows (axi_bus_meter is the perf path), debug state-change packets, the report-time address filter, the ID-range filter, three

independent per-phase timers, and the `block_ready` admission stall. The lite never touches the traffic it watches. Two features first listed here as dropped came back on 2026-09-26, each because a consumer bound it: the address-range checker and the latency threshold (TASK-001 sections 11 and 13).

9.2 Why it is small

The full monitor tracks every transaction as a ~285-bit record and re-derives everything about it every cycle: three CAM lookup ports, an oldest-first attribution done with age ranks that every entry recomputes on any free, a per-entry state machine that a reporter then scans, a second copy of the table in the reporter, three 16-bit timers per entry and an 8-deep packet FIFO. Measured on `bridge_1x2_rd_mon` (Artix-7 100T -1, default preset, 16 slots) that is 3,249 LUTs and 1,628 FFs per read monitor, 2,370 of the LUTs in the per-slot cones.

The lite keeps one 90-bit entry per transaction and computes each event at the handshake that causes it:

Mechanism	Full monitor	Lite
Allocation	three-port CAM with free-slot encode and hit suppression	free-slot priority encode on the command handshake
R/B attribution to an entry	ID match, oldest-first by dense age ranks (O(N^2) age matrix)	the head of that ID's linked list (head, tail and a 3-bit next pointer per slot); a one-hot match, no age compare
W attribution (AXI4 W beats carry no ID)	age ranks over all live entries	an AW-order FIFO of slot indices: the head owns the beat
Timeout	three 16-bit timers per entry, all counting	one microsecond stamp per entry (a copy of a shared tick counter), one rotating subtract-and-compare
Latency	three 32-bit timestamps per entry	one 16-bit cycle stamp per entry, one shared subtractor at completion
Event detection	reporter scans the table	computed on the

Mechanism	Full monitor	Lite
	with priority encoders	handshake, no scan; the cycle's events are registered, and the packet is picked and formatted from flops the next cycle
Output	8-deep FIFO of 85-bit entries plus a table copy	`OUT_DEPTH`-deep `gaxi_fifo_sync` (default 4) of 66-bit entries (type, code, id, latency, address), unreset storage and a mux read; the constant fields are added at the output
Backpressure	admission stall (`block_ready`)	drop and count, reported as `Error/EVENT_DROPPED`
Address-range checker	`axi_monitor_addr_check`, N ranges, muxed onto the monbus	the same module, optional (`N_ADDR_RANGES`), its packets muxed onto the monbus with a presented-packet hold (Sean, 2026-09-26: STREAM's data ports use it)

An entry holds stamps, not counters: the only per-slot arithmetic is an 8-bit beat decrement, kept local so the beat that lands reads one flag ("was that the last expected") instead of an 8-bit value through a mux. The two 16-bit subtractors (latency, timeout age) sit after the read mux and are shared by every slot, and one id/address mux serves every packet class.

Five builds got here, each measured in the same fixture. The first had a serial running-max loop for "oldest matching entry" (89 LUT levels). The second had a 16-bit incrementer, a 16-bit subtractor and an 8-bit decrementer in every slot (10,120 LUTs, 38 levels at sixteen slots). The third moved the arithmetic behind the read mux but ordered by an allocation sequence stamp, which both wraps for a long-lived entry and puts a subtract in front of every tournament compare, and it still formatted and queued the packet in the same cycle as the attribution (6,339 LUTs for the bridge, 23 levels). The fourth ordered by dense rank and registered the events before the pick (5,575 LUTs, 12 levels, 0.6 ns short); its hierarchy report put 346 of the lite's 838 LUTs in a

generic skid buffer, and its critical path through the rank tournament. The fifth, described here, replaced the tournament with per-ID linked lists and the skid with a four-entry unreset queue.

9.3 Measured

Same fixture, same flow: `bridge_1x2_rd` regenerated with `mon_preset = "error_only"` (the full monitor, 16 slots, error+timeout+compl+threshold cones) and with `mon_preset = "lite"` (this block, 8 slots), synthesized and routed out of context through `projects/components/fabric-gen-ip/bridge/fpga/` on 2026-09-25, Vivado 2025.1. (Since 2026-09-26 the bridge generator builds the lite on every monitored port, so `bridge_1x2_rd_mon` regenerated today is a lite bridge too; the full-monitor column is the 2026-09-25 baseline.) Per-instance numbers are the hierarchical utilization of one read monitor inside `cpu_rd_adapter`.

	Full monitor	Lite	Lite / full
One read monitor, LUTs	3,249	677	21%
One read monitor, FFs	1,628	831	51%
Whole bridge (3 monitors + arbiter + group), LUTs, Artix-7 100T -1	12,625	5,339	42%
Whole bridge, FFs	7,433	5,270	71%
Bridge WNS at 10 ns, Artix-7 100T -1	-0.303 ns	-0.160 ns	
Bridge WNS at 6.667 ns, Kintex-7 325T -2	+0.212 ns	+1.092 ns	
Bridge without monitors, LUTs / FFs	826 / 767		

The flops are the table: eight 90-bit entries, the event register and the four-entry queue. The full monitor's 16-slot CAM is 800 FFs on its own, so halving the slots does most of the flop saving; the LUT saving is the design.

Neither bridge meets 10 ns on the Artix-7, and in neither case is the path in a monitor: the full bridge's worst path is inside `monitor_trans_cam`, the lite bridge's inside `monbus_axil4_axil4_group` (the address-window compare chain `s1_beats_to_limit`, 11 CARRY4 in a row), a block both bridges share. No path through `axi_monitor_lite` is among the twenty worst; all of them have at least 0.85 ns of slack at 10 ns. The group's chain is filed as its own item.

9.3.1 In systems, on silicon (2026-09-26 and 27)

The fixture number above is one monitor. What a design saves depends on how much of it is monitored, so the lite was measured where it is used: the three Genesys 2 STREAM builds (Kintex-7 325T-2, 203,800 LUTs), each rebuilt from HEAD after `make clean-all`, post-route, Vivado 2025.1; and two synth-only matrices that hold everything but the monitor constant.

Build	Full monitor	Lite	Saved
STREAM build-obs, 4ch, both observers on (same-day A/B, 2026-09-26)	184,047 LUTs, WNS +1.334 ns	71,671 LUTs, WNS +4.154 ns	112,376 LUTs, 61%
STREAM build-mon, 8ch, in-core monitors on (`stable/` 2026-09-09 vs 2026-09-26)	139,293 LUTs, WNS +1.513 ns	87,443 LUTs, WNS +3.764 ns	51,850 LUTs, 37%
STREAM build-perf, 8ch, monitors compiled out	67,956 LUTs	68,139 LUTs	nothing to save
STREAM perf with in-core monitors on, synth matrix,	143,914 LUTs, WNS -6.061 ns	91,063 LUTs, WNS +1.312 ns	52,851 LUTs, 37%

Build	Full monitor	Lite	Saved
100 MHz			
RAPIDS beats, 8ch, in-core monitors on, synth matrix	71,721 LUTs	65,974 LUTs	5,747 LUTs, 8%

Two rows carry the argument. On STREAM's perf clocking the full monitor does not close (-6.061 ns at 100 MHz) and the lite does (+1.312 ns): on that design the lite is what makes an instrumented perf build buildable at all, not merely a smaller one. RAPIDS shows the other end: it monitors only its descriptor read path, so the saving is 8%. The lite's advantage scales with the monitored surface, and the build-obs row – four monitors at 64 slots, 112,376 LUTs back – is what that looks like when the surface is large.

On the board the numbers held. STREAM build-obs with both observers on the lite (`host_obs_matrix` at tool defaults, per iteration): completion 4491 to 4412, address-match 4410 to 4402, error 4402 to 4402, threshold 4402 to 4419; the DUT's own bus meters bit-identical (R/W utilization 99.8%, 914.2 MB/s); the six-endpoint register walk identical. Perf and Debug packet classes read 0, by design – the perf data is in the meters and histograms, which never lived in the monitor. Timeout went from 7 to 55 per iteration: the full monitor parked a timed-out slot in `TRANS_ERROR` and leaked it, saturating at about 7 per reset; the lite frees the slot and keeps reporting. STREAM build-mon on the in-core lite walks 283 registers clean and provokes 7 of 7 monitor scenarios with nothing in the UNEXPECTED bin; RAPIDS beats on the in-core lite passes its golden-CRC smoke and an 8-channel characterization at 2.88 GB/s.

9.4 Adopted

Every consumer is on the lite, each switch measured and tested where it lives:

Consumer	What switched	Since
Generated bridges	every monitored port; the `mon_preset` cone presets became aliases of the lite	`e92a5ae2d`, 2026-09-26
STREAM in-core (`stream_core`, `scheduler_group_array`)	rd/wr data ports and the descriptor port	`8cce2ecce`, 2026-09-26
RAPIDS beats in-core (`scheduler_group_array`	the descriptor port	`8cce2ecce`, 2026-09-26

Consumer	What switched	Since
_beats)		
Genesys 2 STREAM bridges (bridge_stream_mon_axil, bridge_stream_char_axil)	regenerated onto the _monlite wrappers	0b65960d4, 2026-09-26
Observers (axi4_intf_master_observer, axi4_intf_slave_observer)	the rd and wr taps	78cddb5e2, 2026-09-27

No design under projects/ instantiates a full _mon wrapper any more (178 _monlite instantiations tree-wide); the full monitor remains available behind its own sixteen _mon_cg wrappers for a build that wants the Perf and Debug packet classes on the monbus.

What a consumer gives up, stated once: those two packet classes. Anything that derives its expected classes from the hardware is correct without change – the observers' OBS_CAPS0 reports both cones as not built. Anything with a hardcoded class list (STREAM's host_obs_matrix) reads those two rows as LOW until it learns to ask; that is the tool, not the monitor.

9.5 Parameters

Parameter	Type	Default	Description
UNIT_ID	logic [7:0]	8'h09	Unit id in every packet
AGENT_ID	logic [15:0]	16'h0063	Agent id in every packet
MAX_TRANSACTIONS	int	8	Table slots. A command that finds none free is counted (refused_count), not tracked; its R beats or its B then report as orphans

Parameter	Type	Default	Description
			(a refused write's W beats are consumed silently)
ADDR_WIDTH	int	32	Address width stored per entry and reported
ID_WIDTH	int	8	Transaction ID width; 0 for AXI-Lite (one bit is kept)
IS_READ	bit	1	1 = AR/R monitor, 0 = AW/W/B monitor
IS_AXI	bit	1	0 = AXI-Lite: no ID compare
TS_WIDTH	int	16	Cycle stamp width: ordering and latency (latency saturates at 2^16 cycles)
AGE_WIDTH	int	16	Microsecond age width for the timeout (`cfg_timeout_cnt` is 16 bits)
OUT_DEPTH	int	4	Output queue depth in 66-bit entries (a `gaxi_fifo_sync`), any depth >= 2
WSEQ_WIDTH	int	8	Write monitors: width of the AW/W burst sequence

Parameter	Type	Default	Description
			numbers (amba #105). AW may lead W by up to 2^(WSEQ_WIDTH-1)-1 bursts, refused AWs included (minimum width 2); unused on reads
N_ADDR_RANGES	int	0	Address-range checker windows (axi_monitor_addr_check, the full monitor's); 0 = not built, zero area
ADDR_RANGE_IS_ERROR	logic [N-1:0]	'0	Per range: 1 = a MISS is an Error/ADDR_RANGE packet, 0 = a HIT is an AddrMatch packet
CFI_*		as axi_monitor_timer	Frequency-invariant microsecond tick (counter_freq_invariant)

9.6 Ports

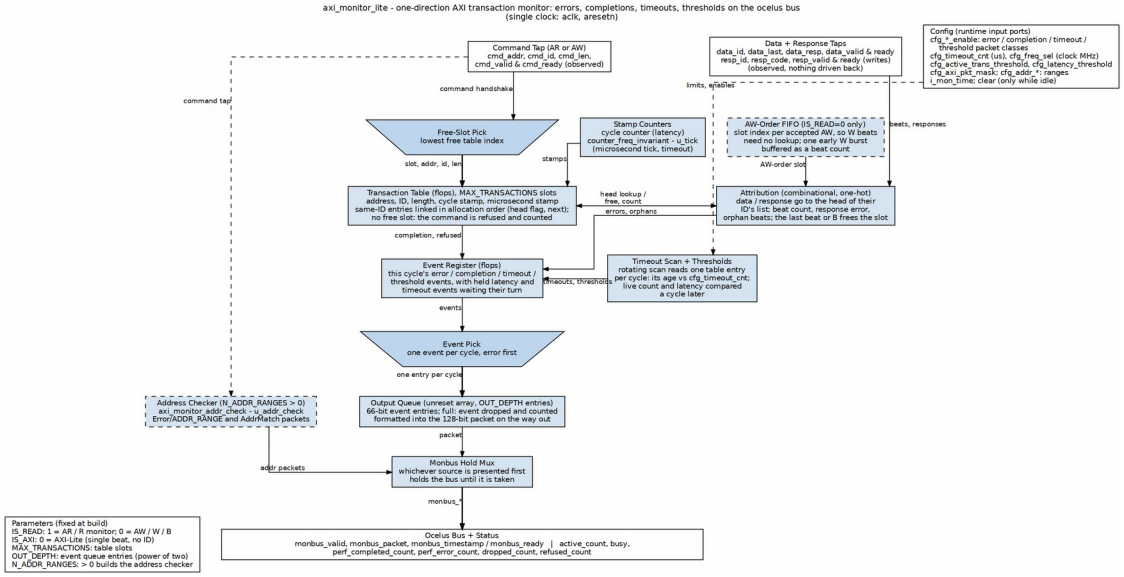
The command/data/response taps and the cfg_* names are the subset of axi_monitor_base's that the lite implements; the wrappers connect the same signals to both. cfg_timeout_cnt is one threshold for every phase (the wrappers feed it their cfg_timeout_cycles, microseconds, 0 = never). cfg_axi_pkt_mask[type] drops a packet type at the source.

Outputs beyond the monbus: active_count (live entries), busy, perf_completed_count / perf_error_count (16-bit saturating, the wrapper's transaction_count / error_count),

`dropped_count` (events lost to backpressure since reset or `clear`) and `refused_count` (commands that found no slot).

9.7 Functional Description

9.7.1 Block diagram



axi_monitor_lite block diagram

The monitor only observes: the command, data and response taps enter at the top and nothing is driven back. A command handshake takes the lowest free table slot and records the address, ID, length and two stamps. Same-ID entries are linked in allocation order, so a data beat or response goes straight to the head of its ID's list; writes follow the AW-order FIFO instead. Attribution, the rotating timeout scan and the threshold compares register their events, the event pick takes one per cycle (errors first), and the output queue formats each into a 128-bit packet. When N_ADDR_RANGES > 0, the address checker shares the bus through the hold mux.

9.7.2 Attribution

A read beat belongs to the oldest live entry whose ID matches `RID`; AXI returns same-ID responses in issue order, so oldest-matching is exact and the 16-bit allocation stamp orders it. A write beat belongs to the entry at the head of the AW-order FIFO, because AXI4 W beats carry no ID and follow AW issue order; the FIFO pops on `WLAST`. A B belongs to the oldest matching entry that has finished its data phase.

Refused AWs and burst numbering (amba #105). An AW that finds no free entry is refused and never enters the AW-order FIFO, but its W burst is still on the W channel in AW order. Every AW therefore takes a sequence number (`r_aw_seq`, tracked or refused), every finished W burst advances `r_w_seq`, and each FIFO entry carries its AW's number. A beat goes to the FIFO head only when the head's number is the burst now on W; a beat whose AW was seen but is not at the

head belongs to a refused AW and is consumed silently (its B then reports as RESP_ORPHAN, naming the ID). A refused AW also discards any early burst pending for it. Before this, a refused AW's W went to the next tracked entry or into the early-burst register, every later attribution shifted by one, and entries that never saw their own W stayed in the table: under load at MAX_TRANSACTIONS=12 the table wedged full and refused almost every command after (test_axi_mon_block_ready, 1024 admitted = 16 completed + 996 refused + 12 live). The FIFO pointers also wrap at MAX_TRANSACTIONS now: they were $clog2(N) bits and wrapped at 16, so at N=12 they walked onto entries 12..15, which do not exist.

A write burst that arrives before its AW (legal on a slave-side monitor) is buffered as a beat count and applied when the AW allocates: with e beats already seen, the entry expects len + 1 - e more. A second early burst before that AW is an Error/WRITE_BEFORE_ADDR and is dropped – unless an AW takes the first burst in that same cycle (absorbing it, or discarding it as refused), in which case the new beat simply starts the next early burst. A dropped burst is still counted in r_w_seq, so its AW, when it arrives, goes straight to its response phase and attribution stays aligned. Traffic that keeps W two or more bursts ahead of AW (legal, A2.6) therefore produces a WRITE_BEFORE_ADDR report for every burst past the first: lossy and reported, with the completions still correct.

A W beat in the same cycle as an AW, with no AW already awaiting data, belongs to the AW being allocated (AXI4 write data is in AW order) and is absorbed at allocation: a single-beat write goes straight to its response phase; a longer burst expects len more beats; a LAST on that first beat of a longer burst is an Error/BURST_LENGTH on the new entry. While an earlier early burst is still pending the same-cycle beat is a later transaction's and is counted as early instead. A same-cycle beat that continues a partial early burst the AW is absorbing is that burst's next beat and is absorbed with it; a same-cycle beat for a refused AW is consumed. Until 2026-09-28 the same-cycle beat was counted as early while its own AW queued for beats that had already passed, and the B then reported RESP_ORPHAN; the inherited same-cycle suite found it (rtl/amba/monitors TASK-002), along with an off-by-one in the absorbed count that made a legal two-beat write with one early beat report BURST_LENGTH.

9.7.3 Errors, once per transaction

A response error, an early or missing LAST, an orphan beat: each is reported the cycle it happens. An entry that has reported an error reports no more for that transaction and yields no completion; it still frees on its last beat or B, so the table never leaks on error.

9.7.4 Timeout

Each entry carries a microsecond age cleared on every beat. One rotating pointer compares one entry per cycle against cfg_timeout_cnt, so a stuck transaction is reported within MAX_TRANSACTIONS cycles of crossing – a rounding error against a microsecond tick – and reported once. A command channel that holds VALID without READY for the same threshold reports Timeout/CMD.

A timeout is a one-cycle event and an Error fired in the same cycle outranks it. Since 2026-09-28 one fired timeout the pick could not take is held with its own payload (code, id, address) and

offered on the following cycles, oldest first, so a sustained error stream delays a timeout rather than losing it (the inherited starvation suite had measured the loss at one error per cycle; rtl/amba/monitors TASK-004). A second timeout arriving while the hold is full is still lost and counted.

9.7.5 Latency threshold

A clean completion whose latency (cycle stamp at completion minus the cycle stamp at allocation) exceeds `cfg_latency_threshold` raises a `Threshold/LATENCY` packet. The compare is taken one stage after the completion, from the registered latency, and the event is then held with its own payload (id, address, latency) until the pick can take it – so the completed slot may be reallocated meanwhile without the packet naming the wrong transaction. Until 2026-09-28 the compare and the hold were decided in the completion cycle itself, which chained the RRESP decode, the slot pick, the subtract, the compare and the drop-count adder into 21 logic levels (rtl/amba/monitors ISSUE-002).

9.7.6 Completion hold

A completion that the pick could not queue – it lost to a higher class, or it won while the output queue was full – is held with its own payload (id, address, latency) and offered on the following cycles, oldest first, at the completion rung. This is the same contract the timeout and latency holds above have (rtl/amba/monitors TASK-004), extended to the completion class on 2026-10-08: coincidence is ordinary traffic (an erroring beat on one ID while a clean transaction completes on another), and arming a detection cone must change what is reported, never whether an unrelated class is delivered – the lite discipline's original complaint, board-measured as ~90 % of completions dropped while the error cone was armed and zero error packets were ever emitted (amba BUG-039). A second completion arriving while the hold is full and not draining is still lost and counted.

9.7.7 Drop and count

Events go into the output queue. If the queue is full when an event fires, or two events fire in one cycle (error > timeout > completion > threshold picks the one that goes), the rest are dropped and counted – except the one completion, the one timeout and the one latency event the holds above can keep. Once the queue has DRAINED and nothing else wants it, one `Error/EVENT_DROPPED` packet carries the count and the counter restarts – the report waits for an empty queue (the `axis_monitor_lite` rule) rather than merely a non-full one, because a report must never take the slot a live event needs while the bus is congested, which is exactly when drops happen (amba BUG-039). The consumer therefore always knows how many events it did not see.

9.8 Verification

- `val/amba/monitor-lite/test_axi_monitor_lite.py` through `axi4_slave_{rd,wr}_monlite`: exact packets for singles and bursts (address, id, non-zero latency), one `RESP_SLVERR` and no completion for an out-of-range access, one timeout naming the stuck phase, one active-count threshold, and a held monbus that drops events and then reports the count. A refused-command phase (amba #105) puts

more transactions than entries in flight on distinct IDs, twice so the AW queue wraps: completions + orphan reports must split the IDs exactly, nothing else may come out, and the table must be empty after. FUNC adds a `MAX_TRANSACTIONS=12` cell, the non-power-of-two depth. 12 cells at full.

- `val/amba/monitor-lite/test_<wrapper>_monlite.py`, sixteen tests derived from the `_mon` wrapper tests on the existing monitor TB classes: the same scenarios with the `_monlite` DUT. 105 cells at full with the above.
- `formal/amba/axi_monitor_lite`: under unconstrained taps, `active_count` never exceeds the table, `clear` empties it, and an offered packet is held unchanged until taken; covers reach a completion, an error, a timeout, a threshold and a full table (the proof is not vacuous). A second, write instance with a 3-entry table is built for the cover task only and reaches a write completion, including one after a refused command.
- The bridge's generated monitor stress tests run unchanged on `bridge_1x2_rd_lite_mon` (`mon_preset = "lite"`).
- The full monitor's own suites in `val/amba`, each with lite cells in its grid (rtl/amba/monitors TASK-002, 2026-09-28). Nothing is skipped by file; where the lite has no equivalent of a full-monitor signal the check is the lite's own contract instead:

Suite	Lite cells	What the lite is held to
`test_axi4_monitor`	6 (`axi_monitor_lite` core: AXI4/AXI-Lite x rd/wr, small table, 256 IDs)	all six phases: basic, bursts, response errors, orphans, sustained, zero-delay
`test_axi_monitor_wr_same_cycle`	2	same-cycle AW+W single and first-of-burst, control, AW during an open burst, a partial early burst (phase 5, new for both cores)
`test_axi_monitor_runtime_disable`	1	table drains with a class runtime-disabled and under backpressure; `refused_count` stays 0 (no `block_ready` to wedge)
`test_axi_mon_block_ready`	16 cells on 11 wrappers (`LiteRefuseCheck`)	the table filled and refused, `admitted == transaction_count +`

Suite	Lite cells	What the lite is held to
		`refused_count + live` exactly, the table EMPTY once the bus is idle (amba #105: the identity alone balanced against leaked entries), occupancy never above the depth. Depth follows the TB's bus-measured concurrency (the AXI-Lite and AXI5 read BFMs hold at most 4 in flight, the AXI-Lite write BFM 2), and `axil4_master_wr_monlite` is not claimed: its core taps behind the write skid and never sees two outstanding
`test_axi_monitor_soak`	1 (`monitor_soak_monlite`)	60k-200k cycles of random reads with errors, stalls and 15 % consumer backpressure: generated completions + errors + timeouts == delivered + drops reported + drops pending, exactly (60k cycles: 10,655 = 7,648 + 1,909 + 965 + 133)
`test_axi_monitor_pktgen`	2 (timeout starvation) + 1 (BUG-039 completion hold)	one stalled read against an SLVERR flood: accounting exact; the victim's timeout is delivered at a 1-in-2 error duty and LOST

Suite	Lite cells	What the lite is held to
		(counted) at 1-per-cycle – see rtl/amba/monitors TASK-004. Plus: five clean completions fired into a consumer-stalled depth-4 queue are all delivered with zero drop reports – the completion that finds the queue full is held, not dropped (amba BUG-039)

Every check reports its count, not a bare verdict.

9.9 Related

`axi_monitor_base` (the full monitor), the sixteen `*_monlite` wrappers (one per `*_mon`, e.g. `axi4_slave_rd_monlite`), `monbus_arbiter` and `monbus_group` (unchanged consumers), `vault/tracking/rtl/amba` rtl/amba/monitors TASK-001 (the review that sized this).

9.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Tamir, Y., Frazier, G.L. “High-Performance Multi-Queue Buffers for VLSI Communication Switches.” *Proc. 15th Annual Int. Symp. Computer Architecture (ISCA 1988)*, pp. 343-354, May 1988. DOI: 10.1109/ISCA.1988.5245. – a comparable structure: per-queue linked lists through one shared buffer pool, here per-ID lists through the shared slot pool.
- Dally, W.J., Towles, B. *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004, ch. 17 “Router Datapath Components”, §17.1.2 “Input Buffer Data Structures”. – textbook treatment (secondary source): head/tail pointers, next pointers and a free list.
- IEEE-ISTO 5001-2003. *The Nexus 5001 Forum Standard for a Global Embedded Processor Debug Interface*, Version 2.0, IEEE Industry Standards and Technology Organization, Dec. 2003 (defining non-peer standard). –

§4.4.3 and §4.5.3: trace overrun reported to the tool with an Error Message, comparable to `refused_count` and `Error/EVENT_DROPPED`.

- Goossens, K., Vermeulen, B., van Steeden, R., Bennebroek, M. “Transaction-Based Communication-Centric Debug.” *Proc. 1st Int. Symp. Networks-on-Chip (NOCS 2007)*, pp. 95-106, May 2007. DOI: 10.1109/NOCS.2007.46. – related work: monitoring at transaction granularity.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. – §A5.1 transaction identifiers, §A5.5 and §A5.6 response ordering (same-ID responses in order, which the per-ID list relies on).
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, sec. “ARM AMBA interconnect protocols suite”. – textbook background on the AXI protocol family (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LinkedListMultiQueueBuffer__Tamir1988.pdf`, `DebugTraceInterfaceStandard__IeeeIsto2003.pdf`)

10 AXI Monitor Lite Wrappers

Module: axi4_master_rd_monlite.sv / axi4_master_wr_monlite.sv / axi4_slave_rd_monlite.sv / axi4_slave_wr_monlite.sv / axi5_master_rd_monlite.sv / axi5_master_wr_monlite.sv / axi5_slave_rd_monlite.sv / axi5_slave_wr_monlite.sv / axil4_master_rd_monlite.sv / axil4_master_wr_monlite.sv / axil4_slave_rd_monlite.sv / axil4_slave_wr_monlite.sv / axil5_master_rd_monlite.sv / axil5_master_wr_monlite.sv / axil5_slave_rd_monlite.sv / axil5_slave_wr_monlite.sv / axi4ace_master_rd_monlite.sv / axi4ace_master_wr_monlite.sv / axi4ace_slave_rd_monlite.sv / axi4ace_slave_wr_monlite.sv / axi4ace_snoop_slave_monlite.sv / axi4ace_snoop_master_monlite.sv / axi4_master_rd_monlite_cg.sv / axi4_master_wr_monlite_cg.sv / axi4_slave_rd_monlite_cg.sv / axi4_slave_wr_monlite_cg.sv / axi5_master_rd_monlite_cg.sv / axi5_master_wr_monlite_cg.sv / axi5_slave_rd_monlite_cg.sv / axi5_slave_wr_monlite_cg.sv / axil4_master_rd_monlite_cg.sv / axil4_master_wr_monlite_cg.sv / axil4_slave_rd_monlite_cg.sv / axil4_slave_wr_monlite_cg.sv / axil5_master_rd_monlite_cg.sv / axil5_master_wr_monlite_cg.sv / axil5_slave_rd_monlite_cg.sv / axil5_slave_wr_monlite_cg.sv **Location:** rtl/amba/axi4/, rtl/amba/axi5/, rtl/amba/axil4/, rtl/amba/axil5/, rtl/amba/ace/ **Category:** Protocol wrappers with the lite monitor **Status:** Production Ready (rtl/amba/monitors TASK-001, 2026-09-26)

Plane: protocol plane (lite monitor wrappers)

10.1 Overview

Forty-eight wrappers, one page. Each <core>_monlite is the core wrapper (axi4_master_rd, axil5_slave_wr, axi4ace_snoop_slave, and so on) with the appropriate lite monitor watching its bus-side channels; each <core>_monlite_cg is that lite wrapper behind one amba_clock_gate_ctrl. They are the lite-monitor siblings of the _mon and _mon_cg wrappers: same core, same taps, same 128-bit monitor_packet_t on the same monbus with the same UNIT_ID/AGENT_ID, so the monbus arbiter, the AXI-Lite group, the tally and the host tooling cannot tell the two apart. The difference is what they cost and what they can be asked for: about a fifth of the monitor LUTs (677 against 3,249 per read monitor in the same bridge fixture) and the packet classes the lite emits – error, timeout, completion, threshold (active-count and latency), address-range match and miss, and the drop report.

Every core parameter and port is declared verbatim and passed through by name; only the monitor section differs from <core>. The files are generated from the core module headers and the lite tap wiring, which is why one page describes them all: the monitor section below is identical within each family, and the per-wrapper facts fit in three tables.

Table 1: The thirty-two wrappers

Family	Role	Channel	Lite wrapper	Clock-gated lite wrapper	Wraps	Full-monitor siblings
AXI4	master	read	`axi4_master_rd_monlite`	`axi4_master_rd_monlite_cg`	`axi4_master_rd`	`axi4_master_rd_mon`, `axi4_master_rd_mon_cg`
AXI4	master	write	`axi4_master_wr_monlite`	`axi4_master_wr_monlite_cg`	`axi4_master_wr`	`axi4_master_wr_mon`, `axi4_master_wr_mon_cg`
AXI4	slave	read	`axi4_slave_rd_monlite`	`axi4_slave_rd_monlite_cg`	`axi4_slave_rd`	`axi4_slave_rd_mon`, `axi4_slave_rd_mon_cg`
AXI4	slave	write	`axi4_slave_wr_monlite`	`axi4_slave_wr_monlite_cg`	`axi4_slave_wr`	`axi4_slave_wr_mon`, `axi4_slave_wr_mon_cg`
AXI5	master	read	`axi5_master_rd_monlite`	`axi5_master_rd_monlite_cg`	`axi5_master_rd`	`axi5_master_rd_mon`, `axi5_master_rd_mon_cg`
AXI5	master	write	`axi5_master_wr_monlite`	`axi5_master_wr_monlite_cg`	`axi5_master_wr`	`axi5_master_wr_mon`, `axi5_master_wr_mon_cg`
AXI5	slave	read	`axi5_slave_rd_monlite`	`axi5_slave_rd_monlite_cg`	`axi5_slave_rd`	`axi5_slave_rd_mon`,

Family	Role	Channel	Lite wrapper	Clock-gated lite wrapper	Wraps	Full-monitor siblings
						axi5_slave_rd_mon_cg
AXI5	slave	write	axi5_slave_wr_monlite	axi5_slave_wr_monlite_cg	axi5_slave_wr	axi5_slave_wr_mon, axi5_slave_wr_mon_cg
AXI4-Lite	master	read	axil4_master_rd_monlite	axil4_master_rd_monlite_cg	axil4_master_rd	axil4_master_rd_mon, axil4_master_rd_mon_cg
AXI4-Lite	master	write	axil4_master_wr_monlite	axil4_master_wr_monlite_cg	axil4_master_wr	axil4_master_wr_mon, axil4_master_wr_mon_cg
AXI4-Lite	slave	read	axil4_slave_rd_monlite	axil4_slave_rd_monlite_cg	axil4_slave_rd	axil4_slave_rd_mon, axil4_slave_rd_mon_cg
AXI4-Lite	slave	write	axil4_slave_wr_monlite	axil4_slave_wr_monlite_cg	axil4_slave_wr	axil4_slave_wr_mon, axil4_slave_wr_mon_cg
AXI5-Lite	master	read	axil5_master_rd_monlite	axil5_master_rd_monlite_cg	axil5_master_rd	axil5_master_rd_mon, axil5_master_rd_mon_cg
AXI5-Lite	master	write	axil5_master_wr_	axil5_master_wr_	axil5_master_wr	axil5_master_wr_

Family	Role	Channel	Lite wrapper	Clock-gated lite wrapper	Wraps	Full-monitor siblings
			monlite	monlite_cg		mon, axil5_master_wr_mon_cg
AXI5-Lite	slave	read	axil5_slave_rd_monlite	axil5_slave_rd_monlite_cg	axil5_slave_rd	axil5_slave_rd_mon, axil5_slave_rd_mon_cg
AXI5-Lite	slave	write	axil5_slave_wr_monlite	axil5_slave_wr_monlite_cg	axil5_slave_wr	axil5_slave_wr_mon, axil5_slave_wr_mon_cg

10.1.1 The eight AXIS wrappers (2026-09-27)

The stream families never had a _mon sibling to twin: the AXI monitor cores are transaction trackers and a stream has no transaction. These wrap axis_monitor_lite, the stream monitor built for them (rtl/amba/monitors TASK-003). The monitor TAPS the endpoint's FUB-side port (every monitor taps the FUB side, for timing) and drives nothing.

Table 2: The eight AXIS wrappers

Family	Role	Lite wrapper	Clock-gated lite wrapper	Wraps	Tapped port
AXI4-Stream	master	axis4_master_monlite	axis4_master_monlite_cg	axis4_master	fub_axis_* (upstream of the skid)
AXI4-Stream	slave	axis4_slave_monlite	axis4_slave_monlite_cg	axis4_slave	fub_axis_* (downstream of the skid)
AXI5-Stream	master	axis5_master_monlite	axis5_master_monlite_cg	axis5_master	fub_axis_* (upstream of the skid)
AXI5-Stream	slave	axis5_slave_monlite	axis5_slave_monlite_	axis5_slave	fub_axis_* (downstrea

Family	Role	Lite wrapper	Clock-gated lite wrapper	Wraps	Tapped port
			cg		m of the skid)

10.1.2 The eight ACE wrappers (2026-10-05)

The ACE family adds AXI Coherency Extensions to the AXI4 movers (AxSNOOP, AxDOMAIN[1:0], AxBAR[1:0], the 4-bit RRESP, auto-pulsed RACK/WACK on the masters and forwarded RACK/WACK on the slaves) and adds three snoop channels (AC/CR/CD) for cache-to-CCU traffic. Front-side ACE wrappers reuse axi_monitor_lite exactly as the AXI4 wrappers do; the read wrappers hand the monitor RRESP[1:0] only, so IsShared and PassDirty (RRESP[3:2]) never count as an error response. Snoop-side wrappers use the new axi4ace_snoop_monitor_lite because snoop channels have no transaction ID: CR and CDLAST are attributed to the oldest outstanding AC.

Table 3: The eight ACE wrappers

Family	Role	Channel	Lite wrapper	Wraps	Monitor core	Tapped port
ACE	master	read	axi4ace_master_rd_monlite	axi4ace_master_rd	axi_monitor_lite	fub_axi_* read channels
ACE	master	write	axi4ace_master_wr_monlite	axi4ace_master_wr	axi_monitor_lite	fub_axi_* write channels
ACE	slave	read	axi4ace_slave_rd_monlite	axi4ace_slave_rd	axi_monitor_lite	fub_axi_* read channels
ACE	slave	write	axi4ace_slave_wr_monlite	axi4ace_slave_wr	axi_monitor_lite	fub_axi_* write channels
ACE	snoop	slave (cache side)	axi4ace_snoop_slave_monlite	axi4ace_snoop_slave	axi4ace_snoop_monitor_lite	fub_ac* / fub_cr* / fub_cd*
ACE	snoop	master (CCU side)	axi4ace_snoop_master_monlite	axi4ace_snoop_master	axi4ace_snoop_monitor_lite	fub_ac* / fub_cr* / fub_cd*

10.1.3 The two CXS wrappers (2026-10-15)

CXS (Arm IHI 0079) is credited, not valid/ready, so neither the AXI cores nor `axis_monitor_lite` fit. These wrap `cxs_monitor_lite` (amba TASK-044). The monitor taps the endpoint's FUB handshake and its registered status outputs (`link_state`, credits, `credit_starved`, the sticky error flags), never the link pins. There are no `_cg` variants: clock gating is an FPGA-only concern here.

Table 4: The two CXS wrappers

Family	Role	Lite wrapper	Wraps	Tapped
CXS	transmitter	`cxs_master_monlite`	`cxs_master`	`fub_cxs_*` (upstream of the skid) + core status
CXS	receiver	`cxs_slave_monlite`	`cxs_slave`	`fub_cxs_*` (downstream of the receive FIFO) + core status

There are no `_cg` ACE wrappers in this release. The front-side ACE wrappers carry the same monitor port list as the AXI4 wrappers (with the core's ACE fields passed through). The snoop-side wrappers expose the smaller control set of `axi4ace_snoop_monitor_lite`: `clear`, `cfg_monitor_enable`, `cfg_error_enable`, `cfg_timeout_enable`, `cfg_compl_enable`, `cfg_timeout_cycles`, `cfg_freq_sel`, `i_mon_time`, plus the monbus and status outputs.

The AXIS wrappers' monitor section is the AXIS core's, not the AXI one's: parameters `USE_MONITOR`, `UNIT_ID`, `AGENT_ID`, `OUT_DEPTH`, `ACLK_MHZ`, `CFI_MIN/MAX_FREQ_MHZ` (no table, so no `MAX_TRANSACTIONS`, `ACTIVE_TRANS_THRESHOLD` or address ranges); control pins `cam_clear`, `cfg_monitor_enable`, the six class enables (`error`, `timeout`, `compl`, `credit`, `channel`, `stream`), `cfg_strb_check_enable`, `cfg_timeout_cycles` (microseconds, 0 = never), `cfg_freq_sel`, `cfg_axis_pkt_mask`, `cfg_stall_threshold` (cycles, 0 = off), `i_mon_time`; status `in_packet`, `packet_count`, `error_count`, `dropped_count`. The `_cg` twins take the family's own gating logic verbatim (AXIS4: `user_valid` / `axi_valid` terms; AXIS5: the registered wakeup term including `twakeup`) and add the monitor's activity – a queued packet or an open packet holds the clock – with the upstream READY and `monbus_valid` masked by `!cg_gating`. The AXIS5 `_cg` wrappers keep the `fub_axis_` / `m_axis_` / `s_axis_` names of their `_monlite`, not `axis5_master_cg`'s `axis5_` spelling, so each pair is pin-compatible with itself.

Which port is tapped matters for what a stall looks like. Every wrapper taps its FUB side. On a master that is upstream of the skid, so a slow consumer is invisible at the tap until the skid (default depth 4) is full; a single beat never stalls it. On a slave the FUB side is downstream of the skid, so a slow backend stalls the tap directly. The wrapper tests fill the skid before expecting a stall on a master wrapper.

10.1.4 What is not here

Compared with the _mon wrappers: no performance window or counters, no debug packets, no ID or address filters, no per-event masks, and no block_ready. The lite never stalls the port: an event it cannot deliver is dropped and counted (dropped_count, reported as an EVENT_DROPPED packet), and a command that finds no free table entry is counted (refused_count) and left untracked, so its beats report as orphans. Consumers that need the perf window keep a meter beside the lite (STREAM's axi_bus_meter, RAPIDS' descriptor-port meter), not inside it.

The ACE snoop-side wrappers additionally do not carry the full AXI monitor's transaction table: axi4ace_snoop_monitor_lite tracks snoops in AC-issue order because snoop channels have no ID, and it emits only error, timeout, and completion packets.

10.2 Monitor Parameters (every wrapper)

Parameter	Type	Default	Description
USE_MONITOR	bit	1'b1	0 = omit the monitor, tie its outputs
UNIT_ID	logic [7:0]	8'h01	Unit id in every packet
AGENT_ID	logic [15:0]	16'h000A	Agent id in every packet
MAX_TRANSACTIONS	int	8	table entries; a command finding none is counted, not tracked
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS / 2	Table occupancy that fires a threshold packet (rising edge)
OUT_DEPTH	int	4	monbus output queue (gaxi_fifo_sync), any depth the lite allows (>= 2; >= 1 on the AXIS/CXS lites). 16 on

Parameter	Type	Default	Description
			axi4_master_rd_monlite / axi4_master_wr_monlite (2026-10-08, amba BUG-039): 16 rides out the ~8-cycle observer-egress stall measured at ~1.1 events/cycle, the same margin the interface observers already run their AXIS taps at
N_ADDR_RANGES	int	0	address-range checker windows; 0 = not built
ADDR_RANGE_IS_ERROR	logic [(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	'0	per range: 1 = miss is an error, 0 = hit is a match
ACLK_MHZ	int	100	Clock in MHz; keeps the 1 us tick exact
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	Frequency-invariant tick LUT lower bound (cfg_freq_sel indexes it)
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Frequency-invariant tick LUT upper bound

All of <core>'s own parameters follow, unchanged.

10.3 Monitor Ports (every wrapper)

Port	Direction	Type	Description
cam_clear	input	logic	sync clear of the monitor's table (named as on the _mon wrapper: pin-compatible)
cfg_monitor_enable	input	logic	0 = monitor inert, table held clear
cfg_error_enable	input	logic	
cfg_timeout_enable	input	logic	
cfg_compl_enable	input	logic	
cfg_threshold_enable	input	logic	
cfg_timeout_cycles	input	logic [15:0]	MICROSECONDS of no progress; 0 = never
cfg_freq_sel	input	logic [3:0]	counter_freq_invariant LUT index
cfg_axi_pkt_mask	input	logic [15:0]	drop mask by packet type
cfg_latency_threshold	input	logic [31:0]	completion latency (cycles) above this -> Threshold/LATENCY
cfg_addr_check_enable	input	logic	
cfg_addr_match_enable	input	logic	hit in a match range -> AddrMatch packet
cfg_addr_range_enable	input	logic [(N_ADDR_RANGES > 0 ?	

Port	Direction	Type	Description
		N_ADDR_RANGES : 1)-1:0]	
cfg_addr_range_low	input	logic [(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0][AW-1:0]	
cfg_addr_range_high	input	logic [(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0][AW-1:0]	
i_mon_time	input	monitor_common_pkg::monbus_timestamp_t	
monbus_valid	output	logic	
monbus_ready	input	logic	
monbus_packet	output	monitor_common_pkg::monitor_packet_t	
monbus_timestamp	output	monitor_common_pkg::monbus_timestamp_t	
active_transactions	output	logic [7:0]	entries in the table
error_count	output	logic [15:0]	error + timeout packets emitted
transaction_count	output	logic [31:0]	completion packets emitted
dropped_count	output	logic [15:0]	events lost to monbus backpressure since the last report
refused_count	output	logic [15:0]	commands that found no free table entry

All of <core>'s own ports follow, unchanged. On the AXI-Lite families the fabric side is fub_axil_*, matching the _mon wrappers, so the two are pin-compatible on both sides.

10.4 Clock-Gated Wrappers

<core>_monlite_cg adds one parameter and four pins to the lite wrapper, and is built exactly as <core>_mon_cg is built around <core>_mon: the activity term, the request-side ready masks and the monitor-bus liveness terms are the _mon_cg ones verbatim.

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown, sizing cfg_cg_idle_count

Port	Direction	Width	Description
cfg_cg_enable	input	1	Enable clock gating
cfg_cg_idle_count	input	CG_IDLE_COUNT_WIDTH	Idle cycles before the clock stops
cg_gating	output	1	Gated clock is stopped
cg_idle	output	1	No activity observed

Activity is derived from VALID signals and outstanding work only, never from a peer's READY, so a consumer that parks its response-ready high while idle does not defeat gating. While cg_gating is high the request-side readys are held at 0, so nothing is accepted with the clock stopped:

Table 2: Ready masks per clock-gated wrapper

Wrapper	Request-side readys held at 0 while cg_gating
axi4_master_rd_monlite_cg	fub_axi_arready, m_axi_rready
axi4_master_wr_monlite_cg	fub_axi_awready, fub_axi_wready, m_axi_bready
axi4_slave_rd_monlite_cg	s_axi_arready, fub_axi_rready
axi4_slave_wr_monlite_cg	s_axi_awready, s_axi_wready,

Wrapper	Request-side readys held at 0 while `cg_gating`
	`fub_axi_bready`
`axi5_master_rd_monlite_cg`	`fub_axi_arready`, `m_axi_rready`
`axi5_master_wr_monlite_cg`	`fub_axi_awready`, `fub_axi_wready`, `m_axi_bready`
`axi5_slave_rd_monlite_cg`	`s_axi_arready`, `fub_axi_rready`
`axi5_slave_wr_monlite_cg`	`s_axi_awready`, `s_axi_wready`, `fub_axi_bready`
`axil4_master_rd_monlite_cg`	`fub_axil_arready`, `m_axil_rready`
`axil4_master_wr_monlite_cg`	`fub_axil_awready`, `fub_axil_wready`, `m_axil_bready`
`axil4_slave_rd_monlite_cg`	`s_axil_arready`, `fub_axil_rready`
`axil4_slave_wr_monlite_cg`	`s_axil_awready`, `s_axil_wready`, `fub_axil_bready`
`axil5_master_rd_monlite_cg`	`fub_axil_arready`, `m_axil_rready`
`axil5_master_wr_monlite_cg`	`fub_axil_awready`, `fub_axil_wready`, `m_axil_bready`
`axil5_slave_rd_monlite_cg`	`s_axil_arready`, `fub_axil_rready`
`axil5_slave_wr_monlite_cg`	`s_axil_awready`, `s_axil_wready`, `fub_axil_bready`

A packet parked on the monitor bus and any occupied table entry hold the block awake so the lite can retire the handshake, and the external `monbus_valid` is masked by `!cg_gating` so the consumer never sees a valid a stopped lite could not retire. The lite's output queue holds a parked packet across the wait.

10.5 Functional Description

The core is instantiated untouched; unlike `_mon` there is no gating of the command handshake, because the lite has no admission stall. The monitor taps the bus-side command, data and response channels (`m_*` on a master wrapper, `s_*` on a slave wrapper) through three valids gated by `cfg_monitor_enable`, which also holds the table clear while low. `cfg_timeout_cycles` is a microsecond count of no progress on an entry, passed at full width; 0 means never. `cfg_latency_threshold` turns a clean completion whose latency exceeds it into a `Threshold/LATENCY` packet. With `N_ADDR_RANGES > 0` the address checker is built inside the lite: a range flagged in `ADDR_RANGE_IS_ERROR` is an allowlist whose miss is an error, an

unflagged range is a watchpoint whose hit is an `AddrMatch` packet (gated by `cfg_addr_match_enable`).

`cam_clear` is legal only while the monitor is idle – no outstanding transactions, nothing queued (see the architecture page, Configuration cautions).

IDs. The command and response IDs are the bus's own (AxID, BID/RID), so the lite pairs each response with the oldest outstanding command of that ID. On the AXI5-Lite wrappers they were tied to 0 until amba #104, and every response was paired with the oldest command outstanding whichever it answered; a Subordinate may reorder responses across IDs (IHI 0022L B2.1.5). W carries no ID on AXI4 or AXI5-Lite and the write wrappers tie `data_id` to 0: the lite attributes W in AW order (A5.5) from its own AW-order queue, which also covers W that arrives ahead of its AW (A2.6).

The lite's own behaviour – per-ID linked-list attribution, stamps instead of counters, the registered event stage, the output queue and drop-and-count – is on the axi_monitor_lite page.

10.6 Usage Example

```
axi4_master_rd_monlite #(
    .AXI_ID_WIDTH         (8),
    .AXI_ADDR_WIDTH       (32),
    .AXI_DATA_WIDTH       (32),
    .UNIT_ID              (8'h02),
    .AGENT_ID             (16'h0014),
    .MAX_TRANSACTIONS     (8)
) u_rd_monlite (
    .aclk                 (aclk),
    .aresetn              (aresetn),
    // ... the axi4_master_rd ports, by name ...
    .cam_clear            (1'b0),
    .cfg_monitor_enable   (1'b1),
    .cfg_error_enable     (1'b1),
    .cfg_timeout_enable   (1'b1),
    .cfg_compl_enable     (1'b0),
    .cfg_threshold_enable (1'b0),
    .cfg_timeout_cycles   (16'd1000),
    .cfg_freq_sel         (4'd0),
    .cfg_axi_pkt_mask     (16'h0000),
    .cfg_latency_threshold(32'd0),
    .cfg_addr_check_enable(1'b0),
    .cfg_addr_match_enable(1'b0),
    .cfg_addr_range_enable('0),
    .cfg_addr_range_low   ('0),
    .cfg_addr_range_high  ('0),
    .i_mon_time           (mon_time),
```

```
    .monbus_valid          (monbus_valid),
    .monbus_ready          (monbus_ready),
    .monbus_packet         (monbus_packet),
    .monbus_timestamp      (monbus_timestamp),
    .active_transactions   (),
    .error_count           (),
    .transaction_count     (),
    .dropped_count         (dropped_count),
    .refused_count         (refused_count)
);
```

In a generated bridge every monitored port is a `_monlite` wrapper. STREAM's core and scheduler array and RAPIDS' scheduler group array instantiate them directly.

10.7 Design Notes

- `cam_clear` keeps the `_mon` wrapper's name so the two are pin-compatible on the monitor side; the lite has a table, not a CAM.
- `MAX_TRANSACTIONS` defaults to 8 here against 16 on `_mon`: size it to the port's real outstanding depth, since refused commands surface as orphans.
- Consumers that relied on `block_ready` to bound the table get drop-and-count instead; the count is always reported, the identity of the lost events is not.
- Filelists: `rtl/amba/filelists/<wrapper>.f`, each the core's closure plus `axi_monitor_lite.f` (or `axi4ace_snoop_monitor_lite.f` for the two snoop-side ACE wrappers), and `amba_clock_gate_ctrl.f` for the `_cg` variants; nothing from the full monitor family.

10.8 Related Modules

- axi_monitor_lite – the monitor itself
- axi_monitor_addr_check – the address checker built in behind `N_ADDR_RANGES`
- monitor_system_architecture – packet classes, monbus, configuration cautions

---

10.9 Testing

`val/amba/monitor-lite/` holds one test per wrapper (`test_<wrapper>.py`, the same TB and scenarios as the `_mon` / `_mon_cg` tests with the DUT swapped), the lite's own TB (`test_axi_monitor_lite.py`), the ACE snoop monitor's own TB (`test_axi4ace_snoop_monitor_lite.py`), and `test_monlite_cg_gating.py`, the six-phase BFM-driven structural gating test over all sixteen clock-gated wrappers at two idle counts and two shared delay profiles.

Two `val/amba` suites cover the wrappers as a family. `test_axi_mon_block_ready.py` saturates each AXI wrapper's table and checks that every admitted command is completed, refused or still live, and (since amba #105) that the table is empty once the bus is idle. `test_axil5_mon_reorder.py` puts every AXI5-Lite `_mon`/`_mon_cg`/`_monlite`/`_monlite_cg` wrapper behind a Subordinate that returns responses in reverse order across distinct IDs (and, on the write wrappers, holds AWREADY so W leads AW). Each completion must name its own (address, ID), in the bus's response order, and the table must drain (amba #104).

```
source env_python
make -C val/amba/monitor-lite run-all-full-parallel
```

Last Updated: 2026-10-12

The eight AXIS wrappers reuse the core's exact-packet suite (`AxisMonitorLiteTB`) end to end through the endpoint: framework AXIS master BFM on the upstream port, slave BFM downstream, MonbusSlave on the bus, every phase asserting class, code and payload and that nothing else came out. The one pin-driven phase (TVALID withdrawn) is skipped through a wrapper, since a skid buffer's output cannot be made to violate the rule; the `_cg` wrappers add a gating phase (idle gates, a packet wakes the clock and is reported exactly, idle re-gates). `val/amba/monitor-lite/test_axis{4,5}_{master,slave}_monlite[_cg].py`: 9 cells each at FULL; with the core's 12, 84/84 from a clean build, 2026-09-27.

10.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. – AXI4/AXI5 channels, §A2.3.1 handshake, §A5.1 transaction identifiers.

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021. developer.arm.com/documentation/ihi0022/h. – Part B ch. B1 (AXI4-Lite), Part D §D3.6-§D3.8 (ACE snoop channels) for the Lite and ACE wrappers.
- Tamir, Y., Frazier, G.L. “High-Performance Multi-Queue Buffers for VLSI Communication Switches.” *Proc. 15th Annual Int. Symp. Computer Architecture (ISCA 1988)*, pp. 343-354, May 1988. DOI: 10.1109/ISCA.1988.5245. – related to the per-ID linked lists inside the lite core.
- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. “An Event-Based Monitoring Service for Networks on Chip.” *ACM Trans. Design Automation of Electronic Systems*, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126. – related work: one hardware probe per observed port, emitting events.
- Indexed in `rtl/amba/references/README.md` (local PDF: `LinkedListMultiQueueBuffer__Tamir1988.pdf`)

10.11 Navigation

- **← Back to Monitor Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

11 AXI Monitor Reporter — Completion Cone

Module: `axi_monitor_reporter_compl.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: debug plane

11.1 Overview

The AXI Monitor Reporter Completion Cone is the completion-packet detection sub-block of `axi_monitor_reporter`. It scans the outstanding-transaction table for slots that have reached the `TRANS_COMPLETE` state but have not yet been reported, priority-encodes the first such slot, and emits the fields for a `PktTypeCompletion` MonBus packet. It was split out of the monolithic reporter so integrators who do not need completion packets can drop it with `ENABLE_COMPL_LOGIC=0` and pay zero LUT cost for the per-slot scan and priority encoder. The block is purely combinational.

Key features:

- Scans the shared transaction table for unreported `TRANS_COMPLETE` slots
- First-match priority encoder selects one slot per cycle
- Emits `PktTypeCompletion` packet fields with event code `EVT_TRANS_COMPLETE`
- Runtime-gated by `cfg_compl_enable`; compile-time removable via the parent's `ENABLE_COMPL_LOGIC`
- Reports the selected slot index (`sel_idx`) back to the top reporter for `event_reported` feedback
- Pure combinational — all state (table, `event_reported`) lives in the top reporter

The transaction manager tracks every outstanding AXI transaction and marks a slot `TRANS_COMPLETE` when it finishes cleanly. Someone must turn those completions into MonBus packets exactly once each. This cone does the detect-and-select half of that job: it finds the completed-but-unreported slots, picks the first, and hands the packet fields plus the chosen slot index up to the top reporter, which pushes the packet into the shared MonBus FIFO and sets the slot's `event_reported` bit so it is not emitted again. Factoring it into its own module means the (non-trivial) MAX_TRANSACTIONS-wide scan and encoder synthesize only when completion reporting is actually wanted.

Use cases:

- Emitting one completion packet per successfully-finished AXI transaction

- Completion-tracking test/verification configurations
- Transaction-throughput accounting on the MonBus

Key benefit: isolates the completion scan/encoder so it can be compiled out (`ENABLE_COMPL_LOGIC=0`) for zero area when completion packets are not needed.

11.2 Parameters

Parameter	Type	Default	Description
MAX_TRANSACTIONS	int	16	Depth of the shared transaction table scanned
IDX_W	int	`$clog2(MAX_TRANSACTIONS)`	Width of the selected-slot index

11.3 Ports

11.3.1 Inputs (shared monitor state)

Port	Direction	Width	Description
trans_table	Input	bus_transaction_tx MAX_TRANSACTIONS	The outstanding-transaction table
event_reported	Input	MAX_TRANSACTIONS	Per-slot “already reported” mask (from the top reporter)
cfg_compl_enable	Input	1	Runtime enable for completion reporting

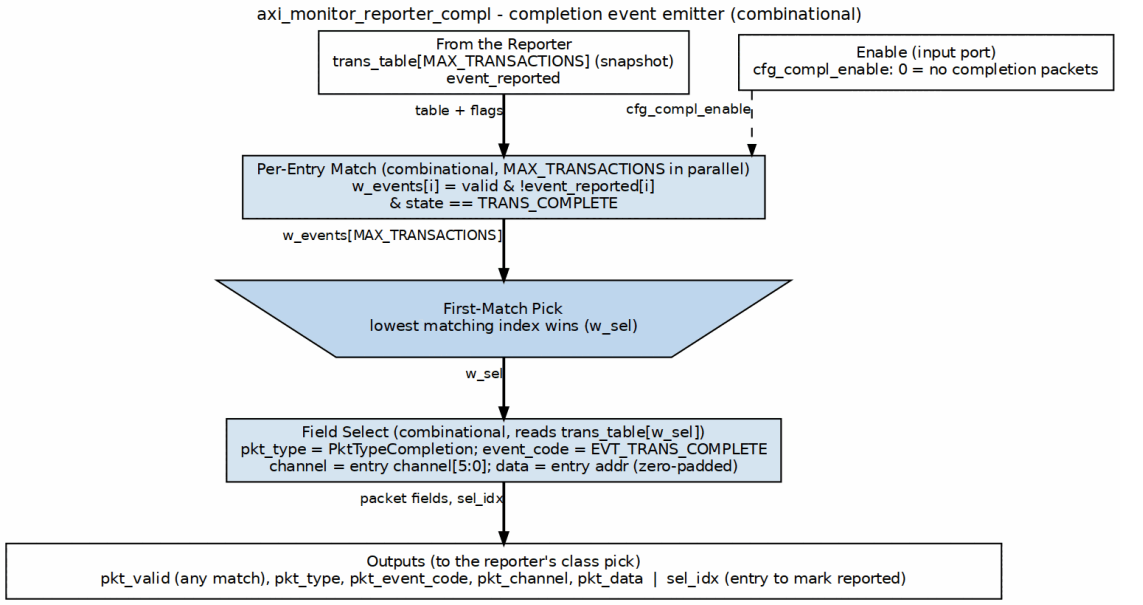
11.3.2 Outputs (packet fields to the top reporter)

Port	Direction	Width	Description
pkt_valid	Output	1	A completed-

Port	Direction	Width	Description
			unreported slot was found this cycle
pkt_type	Output	4	Packet type = `PktTypeCompletion`
pkt_event_code	Output	8	Event code = `EVT_TRANS_COMPLETE`
pkt_channel	Output	9	Channel id `{3'b0, slot.channel[5:0]}`
pkt_data	Output	64	Slot address, zero-padded to 64 bits
sel_idx	Output	IDX_W	Index of the selected slot (for `event_reported` feedback)

11.4 Functional Description

11.4.1 Block diagram



axi_monitor_reporter_compl block diagram

Every table entry is checked in parallel for a completed transaction that has not been reported yet. The lowest matching entry is picked, and its channel and address become a completion packet for the reporter's class pick; `sel_idx` tells the reporter which entry to mark.

11.4.2 Completion Scan

For every slot in `trans_table`, the slot qualifies as a completion event when it is `valid`, its `event_reported` bit is clear, its `state == TRANS_COMPLETE`, and `cfg_compl_enable` is set. All qualifying slots are collected into a one-hot-per-bit `w_events` vector.

11.4.3 First-Match Selection

A second loop scans `w_events` from index 0 upward and latches the first set bit into `w_sel`, asserting `w_has_event`. The `WIDTHTRUNC` lint is locally waived where the loop index is narrowed to `IDX_W`. This gives strict low-index-first priority so exactly one completion is emitted per cycle even when several slots complete together; the rest are picked up on subsequent cycles once the winner is marked reported.

11.4.4 Emitted Fields

- `pkt_valid` = `w_has_event`
- `sel_idx` = `w_sel` (tells the top reporter which slot to mark reported)
- `pkt_type` = `PktTypeCompletion`

- `pkt_event_code` = `EVT_TRANS_COMPLETE`
- `pkt_channel` = `{3'b0, trans_table[w_sel].channel[5:0]}`
- `pkt_data` = `pad_address(trans_table[w_sel].addr)` — the 32-bit slot address zero-extended to 64 bits

11.4.5 Ownership of State

The block holds no state of its own. The transaction table and the `event_reported` feedback mask both live in the top reporter (`axi_monitor_reporter`), which consumes `sel_idx` to set the reported bit after the packet is accepted. Keeping this cone combinational lets several such cones (completion, error, timeout, ...) share the same table snapshot each cycle.

11.5 Timing Characteristics

This module is **purely combinational** – it contains no `always_ff` and no latch, so it holds no state and adds no clock cycles. Its outputs settle a propagation delay after its inputs, and it introduces no latency into a pipeline that instantiates it.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

11.6 Usage Examples

```
// Instantiated inside axi_monitor_reporter, gated by
ENABLE_COMPL_LOGIC.
axi_monitor_reporter_compl #(
    .MAX_TRANSACTIONS (MAX_TRANSACTIONS)
) u_compl (
    .trans_table     (trans_table),
    .event_reported  (event_reported),
    .cfg_compl_enable(cfg_compl_enable),

    .pkt_valid       (compl_valid),
    .pkt_type        (compl_type),
    .pkt_event_code  (compl_event_code),
    .pkt_channel     (compl_channel),
    .pkt_data        (compl_data),
    .sel_idx         (compl_sel_idx)   // top reporter sets
event_reported[compl_sel_idx]
);
```

11.7 Design Notes

11.7.1 Compile-Out Path

The parent instantiates this cone under `ENABLE_COMPL_LOGIC`. Setting it to 0 removes the MAX_TRANSACTIONS-wide qualifier scan and priority encoder entirely, so a monitor build that never reports completions pays no LUT cost for them. `cfg_compl_enable` is the softer runtime mask when the logic is present.

11.7.2 Combinational by Design

All state lives upstream; this block is a pure combinational detect-and-select cone. Multiple packet cones can therefore examine the same table each cycle without duplicating storage, and the top reporter arbitrates which one wins the MonBus this cycle.

11.7.3 One Packet Per Cycle

The strict first-match encoder emits at most one completion per cycle. Simultaneous completions are serialized across cycles as each winner is marked reported.

11.8 Related Modules

Used by:

- **axi_monitor_reporter.sv** - Top reporter that pushes the packet into the MonBus FIFO and drives `event_reported`

Uses:

- **monitor_common_pkg** - `PktTypeCompletion`, `TRANS_COMPLETE`, `bus_transaction_t`
- **monitor_amba4_pkg** - `EVT_TRANS_COMPLETE` event code

See also:

- **axi_monitor_reporter_error.sv** - Error/orphan detection cone (sibling sub-block)
- **axi_monitor_reporter_debug.sv** - State-change debug emitter (sibling sub-block)
- **axi_monitor_base.sv** - The monitor scaffold that houses trans_mgr + reporter

---

11.9 Testing

No dedicated testbench for this module. It has no `val/**/test_axi_monitor_reporter_compl.py`. It is exercised indirectly, through the tests of modules that instantiate it (directly or further up):

- `axi4_master_rd_mon` – `val/**/test_axi4_master_rd_mon.py`
- `axi4_master_wr_mon` – `val/**/test_axi4_master_wr_mon.py`
- `axi4_slave_rd_mon` – `val/**/test_axi4_slave_rd_mon.py`
- `axi4_slave_wr_mon` – `val/**/test_axi4_slave_wr_mon.py`
- `axi5_master_rd_mon` – `val/**/test_axi5_master_rd_mon.py`

Indirect coverage exercises this module only in the configurations those parents elaborate. A parameter or mode no parent uses is untested.

Treat any behaviour described on this page as unverified by simulation.

11.10 References

11.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. — §A3.3 transaction response: the completion this cone reports.
- Goossens, K., Vermeulen, B., van Steeden, R., Bennebroek, M. “Transaction-Based Communication-Centric Debug.” *Proc. 1st Int. Symp. Networks-on-Chip (NOCS 2007)*, pp. 95-106, May 2007. DOI: 10.1109/NOCS.2007.46. — related work: transaction-level monitoring.
- Ciordas, C., Goossens, K., Basten, T., Rădulescu, A., Boon, A. “Transaction Monitoring in Networks on Chip: The On-Chip Run-Time Perspective.” *Proc. IEEE Int. Symp. Industrial Embedded Systems (IES 2006)*, pp. 1-10, Oct. 2006. DOI: 10.1109/IES.2006.357464. — related work: reconstructing transactions at run time in hardware.
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

11.10.2 Source Code

- RTL: `rtl/amba/ocelus/axi_monitor_reporter_compl.sv`

11.10.3 Documentation

- Packet format: `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`
- Monitor base: `docs/markdown/rtl-amba/monitor/axi_monitor_base.md`
- Known issues: `rtl/amba/KNOWN_ISSUES/`

Last Updated: 2026-07-15

11.11 Navigation

- **Back to Shared Infrastructure Index**
- **Back to rtl-amba Index**

12 AXI Monitor Reporter — Debug State-Change Emitter

Module: `axi_monitor_reporter_debug.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: debug plane

12.1 Overview

The AXI Monitor Reporter Debug Emitter is the state-change (trace) sub-block of `axi_monitor_reporter` — the sixth cone alongside error, timeout, completion, threshold, and perf. It gives integrators (and the compression-analysis flow) a packet stream that mirrors the live transaction-table FSM: one `PktTypeDebug` packet per (slot, state-change) event. It holds a per-slot previous-state vector, compares it against the live state each cycle, priority-encodes the first slot that transitioned, and emits a packet carrying that slot's address plus the `(prev_state, new_state)` tuple. Unlike the combinational completion/error cones, this block is sequential (it flops the previous state).

Key features:

- One `PktTypeDebug` packet per per-slot state transition, mirroring the trans_table FSM
- Per-slot 3-bit `prev_state` vector, `MAX_TRANSACTIONS` deep, flopped every cycle
- First-match priority encoder selects one changed slot per cycle
- Packs `(prev_state, new_state)` in the high bits of the 64-bit data field, address in the low bits
- Direct-inject path with an `output_busy` gate (bypasses the FIFO, like threshold/perf)
- Runtime-gated by `cfg_debug_enable`; compile-time removable via the parent's `ENABLE_DEBUG_LOGIC`

Deep debugging and compression research both benefit from a packet stream that reflects the internal monitor FSM rather than just terminal events. This block produces exactly that: whenever any transaction slot changes state, it emits a debug packet naming the slot's address and the state edge it took. Slot-free transitions are handled cleanly — when a slot is freed, its `prev_state` resets to `TRANS_IDLE` so the next allocation produces a fresh `IDLE -> ADDR_PHASE` edge rather than a spurious `IDLE -> IDLE`. The compression analysis can use either the state tuple or the address as its dictionary key, whichever carries more entropy.

Use cases:

- FSM-level trace of the transaction table for deep debugging
- Feeding the MonBus compression-analysis flow with a rich event stream
- Correlating state transitions with observed protocol behavior during bring-up

Key benefit: surfaces the live per-slot FSM as MonBus packets, giving debuggers and the compressor a per-transition trace that terminal-event cones cannot provide.

12.2 Parameters

Parameter	Type	Default	Description
MAX_TRANSACTIONS	int	16	Depth of the transaction table monitored
IDX_W	int	`$clog2(MAX_TRANSACTIONS)`	Width of the internal selected-slot index

12.3 Ports

12.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	Clock
aresetn	Input	1	Active-low asynchronous reset

12.3.2 Inputs (shared monitor state)

Port	Direction	Width	Description
trans_table	Input	bus_transaction_tx MAX_TRANSACTIONS	The outstanding-transaction table
cfg_debug_enable	Input	1	Runtime enable / mask for debug

Port	Direction	Width	Description
			packet emission

12.3.3 Direct-Inject Handshake

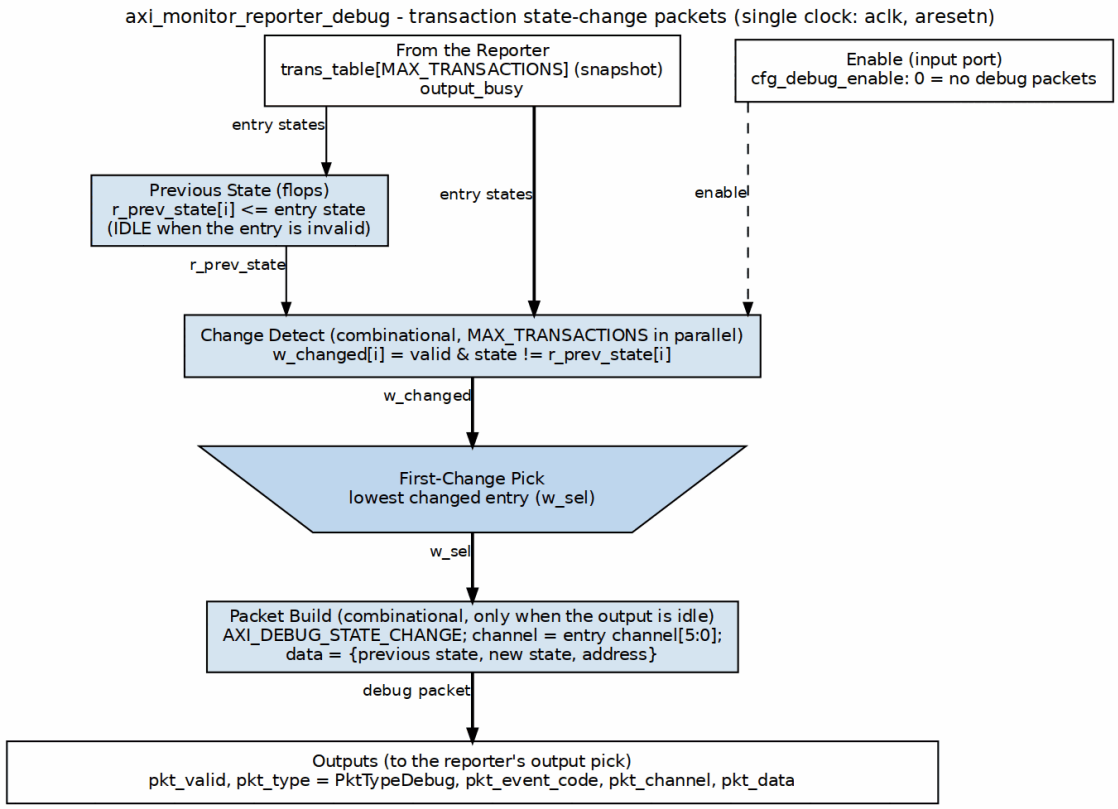
Port	Direction	Width	Description
output_busy	Input	1	High when the shared MonBus output is occupied; gates `pkt_valid`
pkt_taken	Input	1	Pulsed when this block's packet is accepted (currently unused — see Design Notes)

12.3.4 Outputs (packet fields)

Port	Direction	Width	Description
pkt_valid	Output	1	A state change was found and the output bus is free
pkt_type	Output	4	Packet type = `PktTypeDebug`
pkt_event_code	Output	8	Event code = `AXI_DEBUG_STATE_CHANGE`
pkt_channel	Output	9	Channel id `{3'b0, slot.channel[5:0]}`
pkt_data	Output	64	`{prev_state[2:0], new_state[2:0], 26'h0, addr[31:0]}`

12.4 Functional Description

12.4.1 Block diagram



axi_monitor_reporter_debug block diagram

Each entry's previous state is held in flops. An entry whose state changed is flagged, the lowest changed entry is picked, and a state-change packet carrying the old state, the new state and the address goes out when the output is idle.

12.4.2 Change Detection

When `cfg_debug_enable` is set, the block scans every slot: a slot is flagged changed when it is `valid` and its live `state` differs from the captured `r_prev_state[idx]`. All changed slots collect into `w_changed`, and a first-match priority encoder selects the lowest-index changed slot into `w_sel` (asserting `w_has_event`). When `cfg_debug_enable` is low, no slot is flagged, so no packets are produced.

12.4.3 Previous-State Flop

Each cycle `r_prev_state[idx]` snapshots the slot's live state — **except** when the slot is not valid, in which case it resets to `TRANS_IDLE`. This is the mechanism that makes freed slots behave: an

emptied slot returns to IDLE, so its next allocation registers as a real `IDLE -> ADDR_PHASE` transition rather than an aliased `IDLE -> IDLE` non-event.

12.4.4 Packet Assembly

When a change is found and the output bus is free (`w_has_event && !output_busy`):

- `pkt_valid` = 1
- `pkt_type` = `PktTypeDebug`
- `pkt_event_code` = `AXI_DEBUG_STATE_CHANGE`
- `pkt_channel` = `{3'b0, trans_table[w_sel].channel[5:0]}`
- `pkt_data` = `{3'(r_prev_state[w_sel]), 3'(trans_table[w_sel].state), 26'h0, trans_table[w_sel].addr}`

The `(prev, new)` state tuple sits in the top six bits and the 32-bit address in the low bits, leaving a 26-bit zero gap between them. The compression analysis can key on either field.

12.4.5 Direct-Inject and output_busy

Like the threshold and perf cones, debug packets bypass the shared FIFO and inject directly onto the MonBus, so the block must observe `output_busy` to avoid overwriting an in-flight packet — `pkt_valid` is gated by `!output_busy`.

12.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

12.6 Usage Examples

```
// Instantiated inside axi_monitor_reporter, gated by
ENABLE_DEBUG_LOGIC.
axi_monitor_reporter_debug #(
    .MAX_TRANSACTIONS (MAX_TRANSACTIONS)
) u_debug (
```

```
    .aclk             (aclk),
    .aresetn          (aresetn),

    .trans_table      (trans_table),
    .cfg_debug_enable (cfg_debug_enable),

    // Direct-inject arbitration with the top reporter
    .output_busy      (monbus_output_busy),
    .pkt_taken        (debug_pkt_taken),   // currently unused inside
the block

    .pkt_valid        (dbg_valid),
    .pkt_type         (dbg_type),
    .pkt_event_code   (dbg_event_code),
    .pkt_channel      (dbg_channel),
    .pkt_data         (dbg_data)
);
```

Note: enabling debug packets alongside completions/perf can congest the MonBus. See the AXI Monitor Configuration Guide before turning `cfg_debug_enable` on in a real run.

12.7 Design Notes

12.7.1 pkt_taken Is Intentionally Unused

The `r_prev_state` flop advances every cycle regardless of whether the emitted packet was accepted, so a missed transition simply gets aliased into the next change. `pkt_taken` is therefore not consumed today — it is kept on the port list (with an explicit `UNUSED` lint waiver) for symmetry with the threshold/perf cones and for future hooks such as backpressure on debug bursts.

12.7.2 Sequential, Not Combinational

Unlike the completion and error cones (pure combinational), this block flops `r_prev_state`, which is inherently required to detect edges. It therefore takes `aclk`/`aresetn`.

12.7.3 Compile-Out Path

Instantiated under `ENABLE_DEBUG_LOGIC` in the parent; setting it to 0 drops the block entirely. `cfg_debug_enable` is the runtime mask when the logic is present — emissions are gated but the logic still synthesizes.

12.7.4 State Tuple Encoding

The 3-bit state codes follow `transaction_state_t` (`TRANS_IDLE=0`, `ADDR_PHASE=1`, `DATA_PHASE=2`, `COMPLETE=3`, `ERROR=4`, `ORPHANED=5`), packed as `{prev, new}` in `pkt_data[63:58]`.

12.8 Related Modules

Used by:

- **axi_monitor_reporter.sv** - Top reporter that arbitrates the direct-inject MonBus path

Uses:

- **monitor_common_pkg** - `PktTypeDebug`, `transaction_state_t`, `TRANS_IDLE`, `bus_transaction_t`
- **monitor_amba4_pkg** - `AXI_DEBUG_STATE_CHANGE` event code
- **reset_defs.svh** - `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

See also:

- **axi_monitor_reporter_compl.sv** - Completion detection cone (sibling sub-block)
- **axi_monitor_reporter_error.sv** - Error/orphan detection cone (sibling sub-block)
- **axi_monitor_base.sv** - The monitor scaffold that houses trans_mgr + reporter

---

12.9 Testing

No dedicated testbench for this module. It has no `val/**/test_axi_monitor_reporter_debug.py`. It is exercised indirectly, through the tests of modules that instantiate it (directly or further up):

- `axi4_master_rd_mon` – `val/**/test_axi4_master_rd_mon.py`
- `axi4_master_wr_mon` – `val/**/test_axi4_master_wr_mon.py`
- `axi4_slave_rd_mon` – `val/**/test_axi4_slave_rd_mon.py`
- `axi4_slave_wr_mon` – `val/**/test_axi4_slave_wr_mon.py`
- `axi5_master_rd_mon` – `val/**/test_axi5_master_rd_mon.py`

Indirect coverage exercises this module only in the configurations those parents elaborate. A parameter or mode no parent uses is untested.

Treat any behaviour described on page as unverified by simulation.

12.10 References

12.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- IEEE-ISTO 5001-2003. *The Nexus 5001 Forum Standard for a Global Embedded Processor Debug Interface*, Version 2.0, IEEE Industry Standards and Technology Organization, Dec. 2003 (defining non-peer standard). — §4.4 and §4.5 program and data trace messaging: a comparable trace stream in a processor debug standard.
- Hopkins, A.B.T., McDonald-Maier, K.D. “Debug Support for Complex Systems On-Chip: A Review.” *IEE Proc. Computers and Digital Techniques*, 153(4):197-207, 2006. DOI: 10.1049/ip-cdt:20050194. — survey of on-chip trace and debug support.
- Indexed in `rtl/amba/references/README.md` (local PDF: `DebugTraceInterfaceStandard__IeeeIsto2003.pdf`)

12.10.2 Source Code

- RTL: `rtl/amba/ocelus/axi_monitor_reporter_debug.sv`

12.10.3 Documentation

- Packet format: `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`
- Monitor base: `docs/markdown/rtl-amba/monitor/axi_monitor_base.md`
- Configuration: `docs/user-guides/AXI_Monitor_Configuration_Guide.md`

Last Updated: 2026-07-15

12.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

13 AXI Monitor Reporter — Error Cone

Module: `axi_monitor_reporter_error.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: error plane

13.1 Overview

The AXI Monitor Reporter Error Cone is the error-packet detection sub-block of `axi_monitor_reporter`. It scans the outstanding-transaction table for unreported transactions in the `TRANS_ERROR` (genuine error, not a timeout) or `TRANS_ORPHANED` state, priority-encodes the first match, and emits the fields for a `PktTypeError` MonBus packet. It was split out of the monolithic reporter so integrators can drop it with `ENABLE_ERROR_LOGIC=0` and pay zero LUT cost for the per-slot scan and priority encoder. The block is purely combinational.

`error` is a **fault class** (taxonomy): in correct operation it never fires. **An error, by definition, hangs the system** — a transaction that returns `SLVERR`/`DECERR` or is never answered does not retire, so exercising this cone means *deliberately injecting a fault* (a slave forced to return a bad response), and the traffic that provoked it is expected to wedge. That is exactly what this cone exists to catch: it emits the `PktTypeError` packet as the hang happens. (For the address-range/allowlist flavor of error injection, which lives in `axi_monitor_addr_check` (generate-gated by `N_ADDR_RANGES > 0`, so it is built only when ranges are configured) and does *not* require this cone, see that page.)

Key features:

- Scans for unreported `TRANS_ERROR` (non-timeout) and `TRANS_ORPHANED` slots
- Uses `timeout_detected` to keep genuine errors distinct from timeouts (which the timeout cone claims)
- First-match priority encoder selects one slot per cycle
- Emits `PktTypeError` with the slot's own `event_code.raw_code`
- Runtime-gated by `cfg_error_enable`; compile-time removable via the parent's `ENABLE_ERROR_LOGIC`
- Pure combinational — table, `event_reported`, and threshold state stay in the top reporter

The transaction manager marks a slot `TRANS_ERROR` when a transaction returns SLVERR/DECERR or violates protocol, and `TRANS_ORPHANED` when data or a response arrives with no matching command. These must become MonBus error packets exactly once each. This cone performs the detect-and-select half: it finds the error/orphan slots that have not yet been reported, excludes any slot already flagged as a timeout so the timeout cone can own those without aliasing, picks the

first match, and hands the packet fields plus the chosen slot index to the top reporter for FIFO push and `event_reported` feedback. Splitting it out lets the wide scan and encoder be compiled away when error reporting is not required.

Use cases:

- Emitting SLVERR / DECERR / protocol-violation error packets
- Reporting orphaned data or response beats (no matching command)
- Functional-verification configurations that must catch bus errors

Key benefit: isolates the error/orphan scan/encoder for `ENABLE_ERROR_LOGIC=0` compile-out, and cleanly separates genuine errors from timeouts via the `timeout_detected` mask.

13.2 Parameters

Parameter	Type	Default	Description
MAX_TRANSACTIONS	int	16	Depth of the shared transaction table scanned
IDX_W	int	`$clog2(MAX_TRANSACTIONS)`	Width of the selected-slot index

13.3 Ports

13.3.1 Inputs (shared monitor state)

Port	Direction	Width	Description
trans_table	Input	bus_transaction_tx MAX_TRANSACTIONS	The outstanding-transaction table
event_reported	Input	MAX_TRANSACTIONS	Per-slot “already reported” mask (from the top reporter)
timeout_detected	Input	MAX_TRANSACTIONS	Per-slot timeout mask; excludes

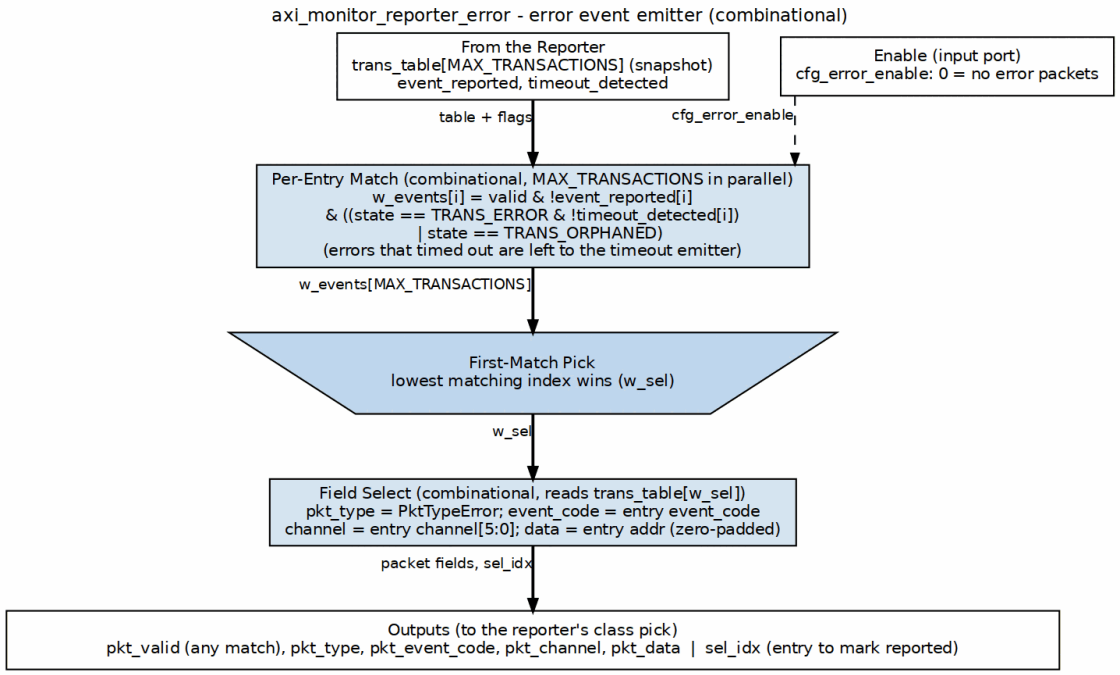
Port	Direction	Width	Description
			timeout slots from the error scan
cfg_error_enable	Input	1	Runtime enable for error reporting

13.3.2 Outputs (packet fields to the top reporter)

Port	Direction	Width	Description
pkt_valid	Output	1	An unreported error/orphan slot was found this cycle
pkt_type	Output	4	Packet type = `PktTypeError`
pkt_event_code	Output	8	Event code = selected slot's `event_code.raw_code`
pkt_channel	Output	9	Channel id `{3'b0, slot.channel[5:0]}`
pkt_data	Output	64	Slot address, zero-padded to 64 bits
sel_idx	Output	IDX_W	Index of the selected slot (for `event_reported` feedback)

13.4 Functional Description

13.4.1 Block diagram



axi_monitor_reporter_error block diagram

Every table entry is checked in parallel for an error, or an orphaned transaction, that has not been reported. Errors that also timed out are left to the timeout emitter. The lowest matching entry is picked, and its event code, channel and address become an error packet; `sel_idx` tells the reporter which entry to mark.

13.4.2 Error / Orphan Scan

For every slot, the slot qualifies as an error event when it is `valid`, its `event_reported` bit is clear, `cfg_error_enable` is set, and **either**:

- `state == TRANS_ERROR` **and** its `timeout_detected` bit is clear (a genuine error, not a timeout), **or**
- `state == TRANS_ORPHANED` (data/response with no matching command).

Qualifying slots collect into `w_events`. Consulting `timeout_detected` is what keeps this cone from aliasing timeouts: a transaction that errored because it timed out is left for the timeout sub-block to report, so the same slot is never emitted twice under two packet types.

13.4.3 First-Match Selection

A second loop takes the first set bit of `w_events` into `w_sel` and asserts `w_has_event` (with the local `WIDTHTRUNC` lint waiver on the index narrowing). Strict low-index-first priority means exactly one error is emitted per cycle; additional errors are serialized on later cycles as each winner is marked reported upstream.

13.4.4 Emitted Fields

- `pkt_valid` = `w_has_event`
- `sel_idx` = `w_sel`
- `pkt_type` = `PktTypeError`
- `pkt_event_code` = `trans_table[w_sel].event_code.raw_code` — the specific AXI error code captured by the trans manager (e.g. `AXI_ERR_RESP_SLVERR`, `AXI_ERR_RESP_DECERR`, `AXI_ERR_DATA_ORPHAN`, `AXI_ERR_RESP_ORPHAN`)
- `pkt_channel` = `{3'b0, trans_table[w_sel].channel[5:0]}`
- `pkt_data` = `pad_address(trans_table[w_sel].addr)`

Unlike the completion and debug cones, the error cone forwards the transaction's own captured raw error code rather than a fixed constant, so the packet carries the precise error kind.

13.4.5 Ownership of State

The block is stateless. The transaction table, the `event_reported` feedback, the `timeout_detected` mask, and the threshold flags all live in the top reporter, which consumes `sel_idx` to set the reported bit after acceptance. Keeping the cone combinational lets it share the table snapshot with the sibling cones each cycle.

13.5 Timing Characteristics

This module is **purely combinational** – it contains no `always_ff` and no latch, so it holds no state and adds no clock cycles. Its outputs settle a propagation delay after its inputs, and it introduces no latency into a pipeline that instantiates it.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

13.6 Usage Examples

```
// Instantiated inside axi_monitor_reporter, gated by
ENABLE_ERROR_LOGIC.
axi_monitor_reporter_error #(
    .MAX_TRANSACTIONS (MAX_TRANSACTIONS)
) u_error (
    .trans_table      (trans_table),
    .event_reported   (event_reported),
    .timeout_detected (timeout_detected),  // keeps timeouts out of
the error scan
    .cfg_error_enable (cfg_error_enable),

    .pkt_valid        (err_valid),
    .pkt_type         (err_type),
    .pkt_event_code   (err_event_code),
    .pkt_channel      (err_channel),
    .pkt_data         (err_data),
    .sel_idx          (err_sel_idx)        // top reporter sets
event_reported[err_sel_idx]
);
```

13.7 Design Notes

13.7.1 Timeout De-Aliasing

The `timeout_detected` input is the mechanism that prevents a timed-out transaction (which the trans manager may also mark `TRANS_ERROR`) from being reported by both this cone and the timeout cone. Only errors with a clear timeout bit are claimed here.

13.7.2 Compile-Out Path

Instantiated under `ENABLE_ERROR_LOGIC` in the parent; setting it to 0 removes the wide scan and encoder for zero LUT cost. `cfg_error_enable` is the runtime mask when the logic is present.

13.7.3 Raw Event Code Forwarding

Because `pkt_event_code` comes from the slot's `event_code.raw_code`, this cone reports the exact error type (SLVERR, DECERR, orphan variants, protocol errors, ...) rather than a single generic error code.

13.8 Related Modules

Used by:

- **axi_monitor_reporter.sv** - Top reporter that pushes the packet into the MonBus FIFO and drives `event_reported`

Uses:

- **monitor_common_pkg** - `PktTypeError`, `TRANS_ERROR`, `TRANS_ORPHANED`, `bus_transaction_t`
- **monitor_amba4_pkg** - AXI error event codes (via the slot's `event_code`)

See also:

- **axi_monitor_reporter_compl.sv** - Completion detection cone (sibling sub-block)
- **axi_monitor_reporter_debug.sv** - State-change debug emitter (sibling sub-block)
- **axi_monitor_base.sv** - The monitor scaffold that houses trans_mgr + reporter

---

13.9 Testing

No dedicated testbench for this module. It has no `val/**/test_axi_monitor_reporter_error.py`. It is exercised indirectly, through the tests of modules that instantiate it (directly or further up):

- `axi4_master_rd_mon` – `val/**/test_axi4_master_rd_mon.py`
- `axi4_master_wr_mon` – `val/**/test_axi4_master_wr_mon.py`
- `axi4_slave_rd_mon` – `val/**/test_axi4_slave_rd_mon.py`
- `axi4_slave_wr_mon` – `val/**/test_axi4_slave_wr_mon.py`
- `axi5_master_rd_mon` – `val/**/test_axi5_master_rd_mon.py`

Indirect coverage exercises this module only in the configurations those parents elaborate. A parameter or mode no parent uses is untested.

Treat any behaviour described on this page as unverified by simulation.

13.10 References

13.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. — §A3.3.1 write response and §A3.3.2 read response (SLVERR, DECERR): the error codes this cone reports.
- Boulé, M., Zilic, Z. "Efficient Automata-Based Assertion-Checker Synthesis of SEREs for Hardware Emulation." *Proc. Asia and South Pacific Design Automation Conf. (ASP-DAC 2007)*, pp. 324-329, Jan. 2007. DOI: 10.1109/ASPDAC.2007.358006. — related work: protocol checkers built as hardware.
- Abramovici, M., Bradley, P., Dwarakanath, K., Levin, P., Memmi, G., Miller, D. "A Reconfigurable Design-for-Debug Infrastructure for SoCs." *Proc. 43rd Design Automation Conf. (DAC 2006)*, pp. 7-12, July 2006. DOI: 10.1145/1146909.1146916. — related work: assertion checkers and transaction identifiers in a debug fabric.
- Indexed in `rtl/amba/references/README.md` (local PDF: `HardwareAssertionCheckers__Boule2007.pdf`)

13.10.2 Source Code

- RTL: `rtl/amba/ocelus/axi_monitor_reporter_error.sv`

13.10.3 Documentation

- Packet format: `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`
- Monitor base: `docs/markdown/rtl-amba/monitor/axi_monitor_base.md`
- Known issues: `rtl/amba/KNOWN_ISSUES/`

Last Updated: 2026-07-15

13.11 Navigation

- **Back to Shared Infrastructure Index**

- **Back to rtl-amba Index**

14 AXI Monitor Reporter (Dispatcher + 6 Sub-blocks)

Modules: - `axi_monitor_reporter.sv` — thin top-level dispatcher - `axi_monitor_reporter_error.sv` — error-packet detection (combinational) - `axi_monitor_reporter_timeout.sv` — timeout-packet detection (combinational) - `axi_monitor_reporter_compl.sv` — completion-packet detection (combinational) - `axi_monitor_reporter_threshold.sv` — threshold-packet detection (16 latency flops + edge flags) - `axi_monitor_reporter_perf.sv` — legacy perf-rollup packets (completion/error lifetime counters + 5-state FSM) - `axi_monitor_reporter_debug.sv` — debug-packet generation

Location: `rtl/amba/ocelus/` **Category:** Core Infrastructure **Status:** Production Ready (refactored to sub-blocks 2026-06-06)

Plane: shared reporter infrastructure (not a plane; multiplexes the error, debug and perf reporter cones)

14.1 Overview

The `axi_monitor_reporter` family generates Monitor-bus packets. The top-level `axi_monitor_reporter.sv` is a **thin dispatcher** that multiplexes one packet stream out of (up to) six packet-type-specific sub-blocks onto the monbus output — three classes through the shared FIFO, three straight into the 128-bit output register (see below). It also reports each emitted event back to `axi_monitor_trans_mgr` so the transaction table can release entries.

Per-packet-type detection lives in the six sub-blocks. The dispatcher gates each sub-block via an `ENABLE_*_LOGIC` parameter, so integrators can drop any combination at elaboration time and pay zero LUT/FF cost for the unused detection cones.

> The bridge case (`ENABLE_ERROR_LOGIC=1`, all others `0`) synthesises away the timeout, completion, threshold, perf and debug detection cones entirely.

Key features:

- 128-bit standardized `monitor_packet_t` formatting (via package helpers)
- Packet type multiplexing across six sub-blocks (error / timeout / compl / threshold / perf / debug) with per-type elaboration gates
- Protocol identification (AXI4, AXI5, APB, AXIS, CORE)
- Event code and data field population from the active sub-block
- Unit ID and Agent ID insertion (caller-configured constants)
- Packet valid/ready handshaking
- Internal monbus FIFO for packet queuing

- Event-reported feedback to `axi_monitor_trans_mgr` (closes the transaction-table loop documented in FIX-001)

What the dispatcher actually provides:

1. **Per-type detection gating** — only the enabled sub-blocks consume LUT/FF, so a single-purpose deployment (e.g. error-only on the bridge) is lean.
2. **Packet formatting** — pulls type / protocol / event code / event data / channel id from the active sub-block and packs into the 128-bit `monitor_packet_t`.
3. **Routing IDs** — inserts the static `UNIT_ID` / `AGENT_ID` so the downstream arbiter and the host can route packets back to their source.
4. **Queuing** — buffers up to `INTR_FIFO_DEPTH` **error, timeout and completion** packets when the downstream monbus is back-pressured. Threshold, perf and debug packets bypass the queue entirely (see above). Note that this FIFO's fill level has **no** path to the monitor's `block_ready` flow control — that is driven purely by transaction-table occupancy.
5. **Event acknowledge** — drives `event_reported_*` back to `axi_monitor_trans_mgr` so the transaction table can release its entry once the packet is accepted into the FIFO (the queued classes are the ones that mark entries reported).
6. **Auto-retire** — releases terminal entries whose packet class cannot report (see below), so disabled classes never leak table slots.

14.2 Parameters

Parameter	Type	Default	Description
`MAX_TRANSACTIONS`	int	16	Transaction-table size shared with `axi_monitor_trans_mgr`
`ADDR_WIDTH`	int	32	Address width carried in event_data
`UNIT_ID`	logic [7:0]	`8'h09`	8-bit unit identifier (static)
`AGENT_ID`	logic [15:0]	`16'h0063`	16-bit agent

Parameter	Type	Default	Description
			identifier (static)
IS_READ	bit	1	1 = read-channel monitor, 0 = write-channel
INTR_FIFO_DEPTH	int	8	Reporter packet FIFO depth
ENABLE_ERROR_LOGIC	bit	1	Instantiate the error detection sub-block
ENABLE_TIMEOUT_LOGIC	bit	1	Instantiate the timeout detection sub-block
ENABLE_COMPL_LOGIC	bit	1	Instantiate the completion detection sub-block
ENABLE_THRESHOLD_LOGIC	bit	1	Instantiate the threshold detection sub-block
ENABLE_PERF_LOGIC	bit	(alias)	Instantiate the perf sub-block; defaults to ENABLE_PERF_PACKETS for legacy compat
ENABLE_DEBUG_LOGIC	bit	0	Instantiate the debug sub-block

14.3 Ports

All ports, from rtl/amba/ocelus/axi_monitor_reporter.sv.

Clock and reset

Port	Dir	Width	Description
aclk	In	1	Clock
aresetn	In	1	Active-low asynchronous reset

Transaction-table inputs

Port	Dir	Width	Description
trans_table	In	bus_transaction_t[MAX_TRANSACTIONS]	The transaction table, read as a whole. This is the port behind the reporter's second full copy of the table (r_trans_table_local) – the reason a monitored interface costs twice the table area
timeout_detected	In	MAX_TRANSACTIONS	Per-slot timeout flags from axi_monitor_timeout
filtered_mask	In	MAX_TRANSACTIONS	Per-slot mask of entries the ID/address filters have excluded

Runtime configuration

Port	Dir	Width	Description
cfg_error_enable	In	1	Emit error packets
cfg_compl_enable	In	1	Emit completion packets

Port	Dir	Width	Description
cfg_threshold_enable	In	1	Emit threshold packets
cfg_timeout_enable	In	1	Emit timeout packets
cfg_perf_enable	In	1	Emit performance packets
cfg_debug_enable	In	1	Runtime mask for the debug emitter; live only when ENABLE_DEBUG_LOGIC=1
active_trans_threshold	In	16	Active-transaction count that trips a threshold packet
latency_threshold	In	32	Latency that trips a threshold packet

Monitor bus output

Port	Dir	Width	Description
monbus_valid	Out	1	Packet valid
monbus_ready	In	1	Downstream accepts the packet
monbus_packet	Out	monitor_packet_t	128-bit monitor packet

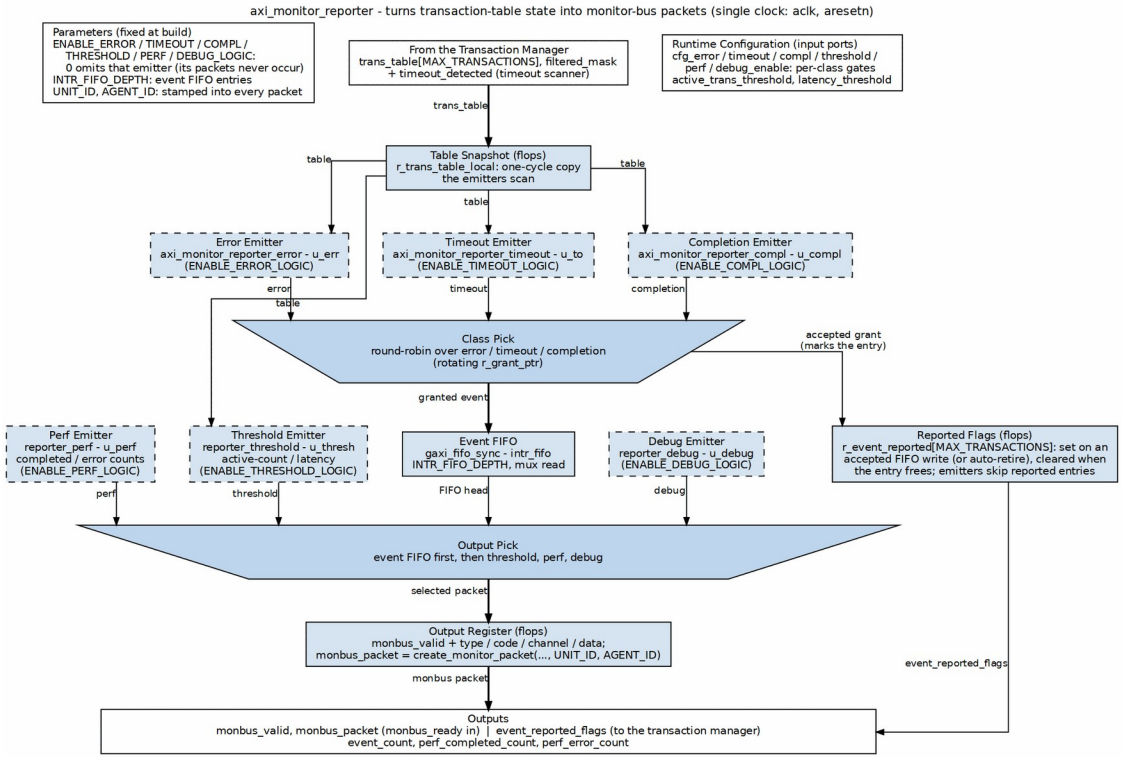
Status outputs

Port	Dir	Width	Description
event_count	Out	16	Total events emitted
perf_completed_count	Out	16	Completed transactions counted
perf_error_count	Out	16	Error events

Port	Dir	Width	Description
			counted
`event_reported_flags`	Out	`MAX_TRANSACTIONS`	Per-slot “already reported” feedback to the transaction manager, so one event is not emitted twice

14.4 Functional Description

14.4.1 Block diagram



axi_monitor_reporter block diagram

The reporter takes a one-cycle snapshot of the table. Three emitters each find the lowest entry with an error, a timeout or a completion; a round-robin class pick chooses one per cycle and writes it into the event FIFO. That write marks the entry reported, which is what lets the transaction manager free it. The output pick then takes the FIFO head first, then threshold, perf

and debug packets, which do not queue. The chosen packet is registered and assembled with the unit and agent IDs.

14.4.2 Architecture

Packet Format (128-bit `monitor_packet_t`):

Bits	Width	Field
[127:124]	4	Packet Type (error / completion / timeout / perf / etc.)
[123:109]	15	Reserved (forward-compat slack)
[108:105]	4	Protocol (AXI / AXIS / APB / ARB / CORE)
[104:97]	8	Event Code (protocol-specific)
[96:88]	9	Channel ID (AXI ID or channel index)
[87:72]	16	Agent ID
[71:64]	8	Unit ID
[63:0]	64	Event Data (full address, latency, counter value, etc.)

The reporter drives `monbus_packet` (128b) and `monbus_timestamp` (64b) together so the side-band timestamp travels paired with each packet through the arbiter and into the `monbus_group` family.

14.4.3 Sub-blocks

Sub-block	Gate parameter	Generates `pkt_type`	Logic shape
`axi_monitor_reporter_error`	`ENABLE_ERROR_LOGIC`	`PktTypeError`	combinational
`axi_monitor_reporter_timeout`	`ENABLE_TIMEOUT_LOGIC`	`PktTypeTimeout`	combinational
`axi_monitor_reporter_compl`	`ENABLE_COMPL_LOGIC`	`PktTypeCompletion`	combinational
`axi_monitor_reporter_threshold`	`ENABLE_THRESHOLD_LOGIC`	`PktTypeThreshold`	16 latency flops + edge detect

Sub-block	Gate parameter	Generates pkt_type	Logic shape
axi_monitor_reporter_perf	ENABLE_PERF_LOGIC (alias ENABLE_PERF_PACKETS)	PktTypePerf	two 16-bit lifetime counters + 5-state FSM (the perfmon *window* counters live in axi_monitor_base and are not gated by this parameter)
axi_monitor_reporter_debug	ENABLE_DEBUG_LOGIC (default 0)	PktTypeDebug	event-encoded debug points

Each sub-block presents the same "raise a request with packet payload" contract to the dispatcher, but the dispatcher routes them **two different ways**:

- **Error, timeout and completion** contend for the FIFO write port, in that priority order, and are queued.
- **Threshold, perf and debug** never touch the FIFO. They load the 128-bit output register directly, and only when it and the FIFO read port are both idle (`!monbus_valid && !w_fifo_rd_valid`) — which is what their `output_busy` input tells them.

That asymmetry is why an error is buffered while the other three are not. It does not mean all three are lossy — they differ, and the difference matters when you are reading a capture:

- **Perf** DEFERS. Its FSM holds while its packet is unaccepted (`if (!(pkt_valid && !pkt_taken)) r_state <= w_next_state;`), so a rollup that loses arbitration is emitted later carrying the then-current counts. Nothing is lost, but the timestamp is the emission, not the event.
- **Threshold** also defers while the crossing persists — the crossed flags latch on `pkt_taken`, not on detection. A crossing that lifts during congestion is the only case that goes unreported.
- **Debug** is genuinely lossy. `r_prev_state` advances every cycle regardless of acceptance (`pkt_taken` is unused in that block), so a state change that cannot be emitted is gone.

None of these sub-blocks are intended to be instantiated directly by integrators — they are private to the reporter family.

14.4.4 Auto-Retire: Disabled Classes Never Leak Slots

The transaction manager frees a terminal-state slot only once its `event_reported` flag is set, and the normal producer of that flag is an accepted FIFO write. A terminal entry whose reporting sub-block is **compiled out** (`ENABLE_*_LOGIC = 0`) or **runtime-disabled** (`cfg_*_enable = 0`) would therefore never be marked, never freed, and would permanently leak a table slot — before commit `95c9490a`, the documented "performance mode" (`ENABLE_COMPL_LOGIC=1` + `cfg_compl_enable=0`) leaked every completed entry until the table pinned and `block_ready` wedged the monitored bus after roughly `MAX_TRANSACTIONS` transactions.

The reporter now **auto-retires** such entries. The semantics:

- An entry marked in `filtered_mask` (the address-range packet filter in `axi_monitor_trans_mgr`) retires on the same rule the moment it is terminal. A filtered entry will never be emitted, so nothing is owed for it. Its packet is separately suppressed before the FIFO write mux, via effective valids that drop a filtered slot — suppressing the packet **without** this retire arm would leak the slot, which is the same failure the disabled-class arms exist to prevent.
- An entry in a terminal state (`TRANS_COMPLETE` / `TRANS_ERROR` / `TRANS_ORPHANED`) whose claiming packet class is unavailable — compiled out **or** runtime-disabled — is marked reported immediately, **without emitting a packet and without bumping `event_count` or the perf completion/error counters** (those count only packets actually emitted).
- The class-to-entry mapping mirrors the reporter cones' claim predicates exactly: `TRANS_COMPLETE` retires when the completion cone is unavailable; `TRANS_ERROR` retires on the **timeout** cone's availability when `timeout_detected` is set for that slot, else on the **error** cone's; `TRANS_ORPHANED` retires on the error cone's. (A naive "both cones unavailable" formula under-retires: with errors disabled but timeouts enabled, a genuine-error slot is claimable by neither cone.)
- The check is **continuous**, not edge-triggered: an entry that completed while its class was enabled but whose packet lost the FIFO-full race is still unmarked when the class is later disabled, and retires then. Accepted, documented consequence: **toggling an enable mid-flight may drop that one entry's packet — it can never leak the slot.**

Runtime-disabling a packet class is therefore safe and makes the monitor passive for that class. If you want to keep marking and counting while suppressing emission, use the packet-type drop mask (`cfg_axi_pkt_mask` in axi_monitor_filtered) downstream of the reporter instead.

Directed tests: `val/amba/test_axi_monitor_runtime_disable.py` (fails on pre-`95c9490a` RTL) and `val/amba/test_axi_monitor_pktgen.py`.

14.5 Timing Characteristics

Metric	Value	Notes
Latency	1-2 cycles	Typical processing delay
Throughput	1 packet per 2 cycles	The registered output stage cannot reload on the same cycle its packet is accepted, so sustained output is at most one packet every other cycle even with the FIFO full

14.6 Usage Examples

This module is instantiated automatically within higher-level monitor modules — `axi_monitor_base` owns it, and users configure behavior through top-level monitor parameters. Configuration is typically handled at the top-level monitor instantiation; see the individual monitor documentation for configuration examples.

14.7 Design Notes

This module is where half the monitor's area goes. It keeps `r_trans_table_local`, a second full copy of `bus_transaction_t x MAX_TRANSACTIONS`, alongside the transaction manager's. Budget a monitored interface as a multiple of the unmonitored one, not a percentage on top; `MAX_TRANSACTIONS` is the knob and the cost is linear in it.

`event_reported_flags` is feedback, not status. It tells the transaction manager which slots have already emitted, so one event is not reported twice. Leaving it unconnected reintroduces a defect the family has already had.

Packet classes contend for one bus. The monbus sustains at most one packet per two cycles; enabling completion and performance together under heavy traffic drops packets. `cfg_axi_pkt_mask` suppresses a class while keeping its marking and counting.

14.8 Related Modules

- **axi_monitor_base**
- **arbiter_monbus_common**

Used by:

- **axi_monitor_base**

See also:

- **Monitor Architecture:** `docs/markdown/rtl-amba/overview.md`
- **Monitor Configuration Guide:** Monitor Base Configuration
- **Packet Format Specification:** `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`

14.9 Testing

- Functional correctness of core logic
- Boundary conditions (min/max values)
- Error handling and recovery
- Interface protocol compliance

See: `val/amba/test_axi_monitor_pktgen.py` and `val/amba/test_axi_monitor_runtime_disable.py` for verification tests

14.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. “An Event-Based Monitoring Service for Networks on Chip.” *ACM Trans. Design Automation of Electronic Systems*, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126. — related work: probes that packetize observed events.
- IEEE-ISTO 5001-2003. *The Nexus 5001 Forum Standard for a Global Embedded Processor Debug Interface*, Version 2.0, IEEE Industry Standards and Technology Organization, Dec. 2003 (defining non-peer standard). — §5.2: typed messages (TCODE), comparable to the per-class reporter cones and `pkt_type`.
- Sprunt, B. “The Basics of Performance-Monitoring Hardware.” *IEEE Micro*, 22(4):64-71, July/Aug. 2002. DOI: 10.1109/MM.2002.1028477. — related work for the performance and threshold cones: on-chip event counters.

- Goossens, K., Vermeulen, B., van Steeden, R., Bennebroek, M. “Transaction-Based Communication-Centric Debug.” *Proc. 1st Int. Symp. Networks-on-Chip (NOCS 2007)*, pp. 95-106, May 2007. DOI: 10.1109/NOCS.2007.46. — related work: reporting at transaction granularity.
- Indexed in `rtl/amba/references/README.md` (local PDF: `DebugTraceInterfaceStandard__IeeeIsto2003.pdf`)

14.11 Navigation

- **Back to Shared Infrastructure Index**
- **Back to rtl-amba Index**
- **Back to Main Documentation Index**

15 AXI Monitor Reporter — Performance Packets

Module: `axi_monitor_reporter_perf.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: perf plane

15.1 Overview

The `axi_monitor_reporter_perf` module is the performance-packet emitter for the AXI/AXIL monitor family. It is one of the per-packet-type sub-blocks that the top-level `axi_monitor_reporter` dispatches to. It owns two lifetime counters — completed transactions and error transactions — and a small cycle FSM that periodically publishes count-rollup packets of type `PktTypePerf` onto the monitor bus (MonBus).

This block was split out of the original monolithic reporter so integrators who do not need performance counters can drop it (compile it out with `ENABLE_PERF_LOGIC=0` at the reporter level) and reclaim the counter area.

Key features:

- Lifetime completion and error transaction counters (16-bit each)
- Counters driven from mark-reported masks supplied by the top reporter
- 5-state cycle FSM that paces packet publication
- Emits one packet at a time via a simple `pkt_valid` / `pkt_taken` handshake
- Backpressure aware: only advances when the output bus is free (`output_busy` low)
- Emits `AXI_PERF_COMPLETED_COUNT` and `AXI_PERF_ERROR_COUNT` event codes
- Counters exposed as ports for status / debug read-back

Performance analysis of an AXI interface needs a periodic summary of how many transactions have completed and how many ended in error over the life of the monitor. This block accumulates those two counts and, when the output path is idle, walks a fixed FSM that emits a completion-count packet followed by an error-count packet.

Use cases:

- Long-run throughput and error-rate characterization of an AXI/AXIL master or slave
- Coarse “health” telemetry streamed to a host over the MonBus
- Regression sanity: confirming the number of completions matches the stimulus

- Cross-checking the window-bucket perfmon path (Stage A/B) against a lifetime rollup

Key benefit: real area savings when disabled (the counters and FSM are removed), while providing a lightweight lifetime performance rollup that runs in parallel with the window-bucket perfmon counters in `axi_monitor_base`.

15.2 Parameters

Parameter	Type	Default	Description
`MAX_TRANSACTIONS`	int	16	Number of transaction slots; sets the width of the `error_marked_mask` / `compl_marked_mask` inputs

15.3 Ports

15.3.1 Clock and Reset

Port	Direction	Width	Description
`aclk`	Input	1	Monitor clock
`aresetn`	Input	1	Active-low asynchronous reset

15.3.2 Control / Status Inputs

Port	Direction	Width	Description
`cfg_perf_enable`	Input	1	Enable performance packet generation (also gates the FSM)
`output_busy`	Input	1	Output path busy

Port	Direction	Width	Description
			(FIFO has data or `monbus_valid` asserted); FSM stalls while high
`pkt_taken`	Input	1	Strobed by the top reporter when this block's packet is accepted. **Load-bearing** — it holds the emit FSM; see Design Notes
`error_marked_mask`	Input	MAX_TRANSACTIONS	Per-slot bit set the cycle an error event is marked-reported into the FIFO
`compl_marked_mask`	Input	MAX_TRANSACTIONS	Per-slot bit set the cycle a completion event is marked-reported into the FIFO

15.3.3 Packet Output

Port	Direction	Width	Description
`pkt_valid`	Output	1	A performance packet is available this cycle
`pkt_type`	Output	4	Packet type — constant `PktTypePerf` (4'h4)
`pkt_event_code`	Output	8	Event code: `AXI_PERF_COMPLETED_COUNT` (8'h7) or `AXI_PERF_ERROR_C`

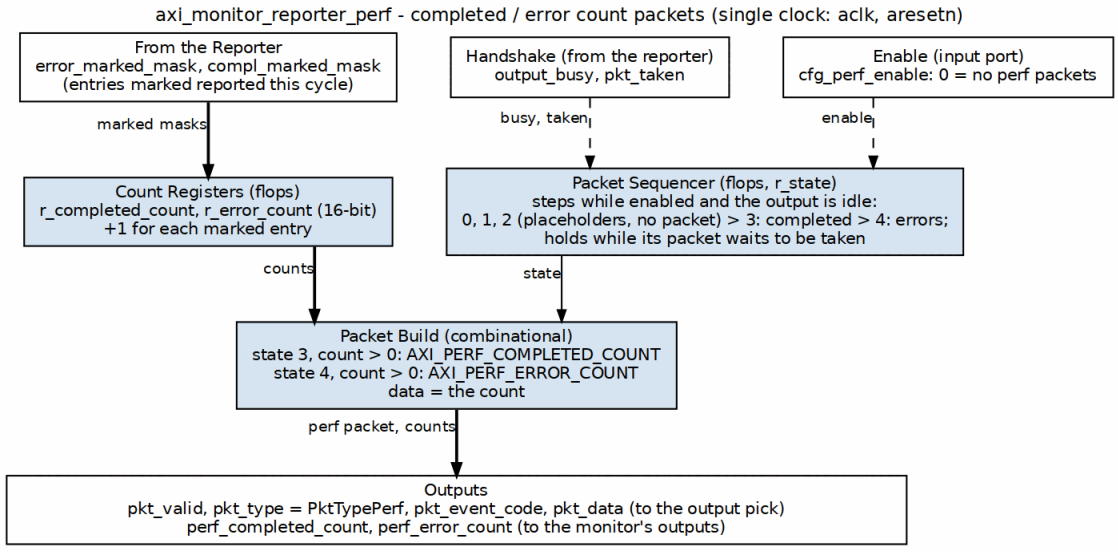
Port	Direction	Width	Description
			OUNT (8'h8)
pkt_channel	Output	9	Channel field (unused here — driven to 0)
pkt_data	Output	64	Zero-extended count value being reported

15.3.4 Lifetime Counters (Status / Debug)

Port	Direction	Width	Description
perf_completed_count	Output	16	Running total of completed transactions
perf_error_count	Output	16	Running total of error transactions

15.4 Functional Description

15.4.1 Block diagram



axi_monitor_reporter_perf block diagram

Two counters tally completed and errored transactions from the reporter's marked masks. A small sequencer steps while the output is idle and offers one count packet at a time: completed, then errors. Its first three states are placeholders that emit nothing.

15.4.2 Lifetime Counters

Two 16-bit registers, `r_completed_count` and `r_error_count`, accumulate over the life of the monitor. On every clock, the block scans the `error_marked_mask` and `compl_marked_mask` inputs across all `MAX_TRANSACTIONS` slots and increments the relevant counter for each asserted bit. Because the counters advance directly from the mark masks (not from `pkt_taken`), they track every event the top reporter accepted into its FIFO regardless of whether a rollup packet is emitted. Entries **auto-retired** because their packet class is disabled do NOT feed the mark masks, so these counters count packets actually emitted, not transactions observed. Timeout packets roll up into `r_error_count` (timeout slots sit in `TRANS_ERROR`).

Both counters are exposed on the port list for status read-back, and — since commit `95c9490a` — are plumbed through `axi_monitor_base` and `axi_monitor_filtered` to drive every `*_mon` wrapper's `error_count` / `transaction_count` status outputs (which read 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`).

15.4.3 Cycle FSM

A 5-state counter (`r_state`) paces packet publication and matches the original monolithic reporter's behavior one-for-one. The FSM only advances while `cfg_perf_enable` is asserted and `output_busy` is low:

State	Name	Action
3'h0	ADDR_LATENCY	No packet (placeholder state)
3'h1	DATA_LATENCY	No packet (placeholder state)
3'h2	TOTAL_LATENCY	No packet (placeholder state)
3'h3	COMPLETED_COUNT	Assert `w_gen_completed` if `r_completed_count > 0`
3'h4	ERROR_COUNT	Assert `w_gen_errors` if `r_error_count > 0`, then wrap to 3'h0

The three latency states are placeholders retained for behavioral compatibility; they emit no packet. Only the completed-count and error-count states can produce output.

15.4.4 Output Multiplexer

The output mux prioritizes the completion-count packet over the error-count packet. When `w_gen_completed` is set, `pkt_valid` asserts with event code `AXI_PERF_COMPLETED_COUNT` and `pkt_data` carrying the zero-extended completed count. Otherwise, when `w_gen_errors` is set, `pkt_valid` asserts with `AXI_PERF_ERROR_COUNT` and the error count. `pkt_type` is always `PktTypePerf` and `pkt_channel` is always 0.

15.4.5 Handshake

The block presents one packet at a time. The top reporter samples `pkt_valid`, forwards the packet fields onto the MonBus, and strobes `pkt_taken` when the packet is accepted. Publication rate is naturally throttled by the FSM (which only steps when the output is not busy), so the emitter cannot flood the bus.

15.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

15.6 Usage Examples

This block is not instantiated directly by users; it is instantiated inside `axi_monitor_reporter`. The pattern is:

```
axi_monitor_reporter_perf #(
    .MAX_TRANSACTIONS (MAX_TRANSACTIONS)
) u_reporter_perf (
    .aclk                   (aclk),
    .aresetn                (aresetn),

    .cfg_perf_enable        (cfg_perf_enable),
    .output_busy            (w_output_busy),      // FIFO data or
monbus_valid
    .pkt_taken              (w_perf_pkt_taken),   // top reporter
```

```
accepted our packet
    .error_marked_mask    (w_error_marked_mask),
    .compl_marked_mask    (w_compl_marked_mask),

    .pkt_valid            (w_perf_pkt_valid),
    .pkt_type             (w_perf_pkt_type),
    .pkt_event_code       (w_perf_pkt_event_code),
    .pkt_channel          (w_perf_pkt_channel),
    .pkt_data             (w_perf_pkt_data),

    .perf_completed_count (perf_completed_count),
    .perf_error_count     (perf_error_count)
);
```

15.7 Design Notes

15.7.1 Completion + Performance Packets: Bandwidth, and the Two Ways to Suppress

Enabling both completion packets (`cfg_compl_enable`) and performance packets (`cfg_perf_enable`) simultaneously can congest the monitor bus under heavy traffic (the reporter sustains at most one packet per two cycles). The usual split is a functional-debug config (error + completion + timeout) and a performance config (error + perf, completions suppressed). There are two suppression mechanisms with different semantics:

- **Runtime disable** (`cfg_compl_enable = 0` with `ENABLE_COMPL_LOGIC = 1`): the monitor is passive for that class. Since commit `95c9490a` this is **safe** — terminal entries of a disabled class auto-retire (are marked reported without emitting a packet and without bumping this block's counters), so the transaction table never leaks and `block_ready` never wedges. Before that commit this exact configuration leaked every completed entry and stalled the monitored bus after roughly `MAX_TRANSACTIONS` transactions. See the auto-retire section in axi_monitor_reporter.
- **Packet-type drop mask** (`cfg_axi_pkt_mask` in axi_monitor_filtered): completions are still detected, marked, and **counted** (this block's `perf_completed_count` keeps advancing, and the wrapper's `transaction_count` output stays live); only the emission is dropped downstream of the reporter. Use this when you want the lifetime counters while suppressing the packet stream.

See `docs/user-guides/AXI_Monitor_Configuration_Guide.md`.

15.7.2 pkt_taken Holds the Emit FSM — Do Not Tie It Off

pkt_taken does not gate the *counters* (those update from the mark masks unconditionally), but it **does** gate the FSM state register:

```
// Hold the state whenever we are presenting a packet that was not
// accepted (threshold beats perf in the top reporter's output mux).
// Advancing regardless silently dropped the packet.
if (!(pkt_valid && !pkt_taken)) begin
    r_state <= w_next_state;
end
```

The hold exists because threshold outranks perf in the top reporter's output mux: without it, a perf packet that lost that arbitration was generated, never emitted, and the FSM walked past it.

So the port must be driven correctly. Tie it **low** and the FSM deadlocks the first time it presents a packet — the state holds forever. Tie it **high** and the silently-dropped-packet bug returns.

(The "retained for a future hook, tied to an unused net" pattern this section used to describe belongs to axi_monitor_reporter_debug, a different module, whose own page documents it correctly.)

15.7.3 Relationship to the Window-Bucket Perfmon

This block emits the legacy PktTypePerf count-rollup packets that summarize completion/error counts over the monitor's lifetime. It runs in parallel with the Stage A/B window-bucket perfmon counters in axi_monitor_base. Those buckets are **readable as counters only** — nothing packetizes them onto the MonBus yet, so no PktTypePerfWin / PktTypePerfHist packets are emitted by any module today. The two mechanisms are complementary, not redundant.

15.7.4 Counter Wrap

Both counters are 16-bit and wrap on overflow. For very long runs the host should account for wrap or read the counters periodically via the status ports.

15.8 Related Modules

Used by:

- **axi_monitor_reporter.sv** — instantiates this block as its performance-packet sub-emitter
- **axi_monitor_base.sv** — top-level monitor scaffold that wires the reporter into the MonBus

Uses:

- **monitor_common_pkg** — `PktTypePerf` and shared packet definitions
- **monitor_amba4_pkg** — `AXI_PERF_COMPLETED_COUNT` / `AXI_PERF_ERROR_COUNT` event codes
- **reset_defs.svh** — reset macros (`ALWAYS_FF_RST`, `RST_ASSERTED`)

See also:

- **axi_monitor_reporter_threshold.sv** — threshold-crossing packet emitter (sibling)
- **axi_monitor_reporter_timeout.sv** — timeout packet emitter (sibling)

15.9 Testing

No dedicated testbench for this module. It has no `val/**/test_axi_monitor_reporter_perf.py`. It is exercised indirectly, through the tests of modules that instantiate it (directly or further up):

- `axi4_master_rd_mon` – `val/**/test_axi4_master_rd_mon.py`
- `axi4_master_wr_mon` – `val/**/test_axi4_master_wr_mon.py`
- `axi4_slave_rd_mon` – `val/**/test_axi4_slave_rd_mon.py`
- `axi4_slave_wr_mon` – `val/**/test_axi4_slave_wr_mon.py`
- `axi5_master_rd_mon` – `val/**/test_axi5_master_rd_mon.py`

Indirect coverage exercises this module only in the configurations those parents elaborate. A parameter or mode no parent uses is untested.

Treat any behaviour described on this page as unverified by simulation.

15.10 References

15.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Sprunt, B. “The Basics of Performance-Monitoring Hardware.” *IEEE Micro*, 22(4):64-71, July/Aug. 2002. DOI: 10.1109/MM.2002.1028477. — related work: on-chip event counters for performance monitoring, in processors.

- Abramovici, M., Bradley, P., Dwarakanath, K., Levin, P., Memmi, G., Miller, D. “A Reconfigurable Design-for-Debug Infrastructure for SoCs.” *Proc. 43rd Design Automation Conf. (DAC 2006)*, pp. 7-12, July 2006. DOI: 10.1145/1146909.1146916. — related work: event counters in a reconfigurable debug fabric.
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

15.10.2 Source Code

- RTL: `rtl/amba/ocelus/axi_monitor_reporter_perf.sv`
- Parent: `rtl/amba/ocelus/axi_monitor_reporter.sv`
- Packages: `rtl/amba/includes/ocelus_common_pkg.sv`, `rtl/amba/includes/ocelus_amba4_pkg.sv`

15.10.3 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Monitor Base: `docs/markdown/rtl-amba/monitor/axi_monitor_base.md`
- Configuration: `docs/user-guides/AXI_Monitor_Configuration_Guide.md`
- Packet Format: `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`

Last Updated: 2026-07-15

15.11 Navigation

- **Back to Shared Infrastructure Index**
- **Back to rtl-amba Index**

16 AXI Monitor Reporter — Threshold Packets

Module: `axi_monitor_reporter_threshold.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: error plane

16.1 Overview

The `axi_monitor_reporter_threshold` module is the threshold-crossing packet emitter for the AXI/AXIL monitor family. It is one of the per-packet-type sub-blocks dispatched by the top-level `axi_monitor_reporter`. It watches two conditions — the number of currently active transactions crossing a configured limit, and any transaction's measured latency exceeding a configured limit — and emits `PktTypeThreshold` packets when either crossing occurs.

The block was split out of the original monolithic reporter so integrators can drop it (`ENABLE_THRESHOLD_LOGIC=0`) and recover real area, because it owns the per-slot latency pipeline (16 x 32-bit latency flops plus 16 threshold flags).

`threshold` is a **fault class** (taxonomy) — an early-warning fault, one step below a hard timeout. Exercise it by **injecting a slow slave**: hold responses long enough that latency crosses `LATENCY_THRESH` but stays under the timeout window (or push enough concurrent traffic to cross the active-count limit). It does not occur under healthy traffic, so like the other faults it must be provoked deliberately.

Key features:

- Active-transaction-count threshold detection (instantaneous condition)
- Per-slot latency threshold detection with a registered latency pipeline
- Edge-sticky flags to fire once per crossing rather than every cycle
- Read/write latency measurement selectable via `IS_READ`
- Internal arbitration: active-count wins over latency events
- One packet at a time via `pkt_valid` / `pkt_taken` handshake
- Emits `AXI_THRESH_ACTIVE_COUNT` and `AXI_THRESH_LATENCY` event codes

Monitoring a bus for congestion and latency outliers requires two distinct checks: how many transactions are in flight right now, and whether any individual transaction took too long. This block performs both. The active count is computed combinationally by scanning the transaction table; latency is computed per slot in a registered pipeline (splitting the wide subtract-and-compare across a cycle to close timing) and compared against the configured latency threshold.

Use cases:

- Detecting bus congestion when outstanding-transaction count exceeds a budget
- Flagging latency-outlier transactions that exceed a service-level target
- Feeding threshold-crossing telemetry to a host for QoS monitoring
- Regression checks that latency stays within expected bounds

Key benefit: real area savings when disabled (the 16-deep latency pipeline is removed), plus a timing-friendly split of the latency compare so the wide carry chain does not sit on the packet-generation critical path.

16.2 Parameters

Parameter	Type	Default	Description
`MAX_TRANSACTIONS`	int	16	Number of transaction slots scanned for active count and latency
`IS_READ`	bit	1'b1	1 = measure read latency (data - addr timestamp); 0 = measure write latency (resp - addr timestamp)
`IDX_W`	int	`$clog2(MAX_TRANSACTIONS)`	Derived width of the selected-slot index

16.3 Ports

16.3.1 Clock and Reset

Port	Direction	Width	Description
`aclk`	Input	1	Monitor clock
`aresetn`	Input	1	Active-low asynchronous reset

16.3.2 Transaction Table and Configuration

Port	Direction	Width	Description
trans_table	Input	bus_transaction_t [MAX_TRANSACTIONS]	Live outstanding-transaction table (valid, state, timestamps, channel)
cfg_threshold_enable	Input	1	Enable threshold detection
active_trans_threshold	Input	16	Active-transaction-count limit; crossing above it fires a packet
latency_threshold	Input	32	Per-transaction latency limit (in timestamp ticks)

16.3.3 Handshake / Backpressure

Port	Direction	Width	Description
output_busy	Input	1	Output bus busy (FIFO read or monbus_valid); threshold packets inject directly, so this prevents overwriting an in-flight one
pkt_taken	Input	1	Pulsed by the top reporter when this block's packet was accepted; **sets** the edge-sticky flag so the same crossing cannot re-fire. The flag

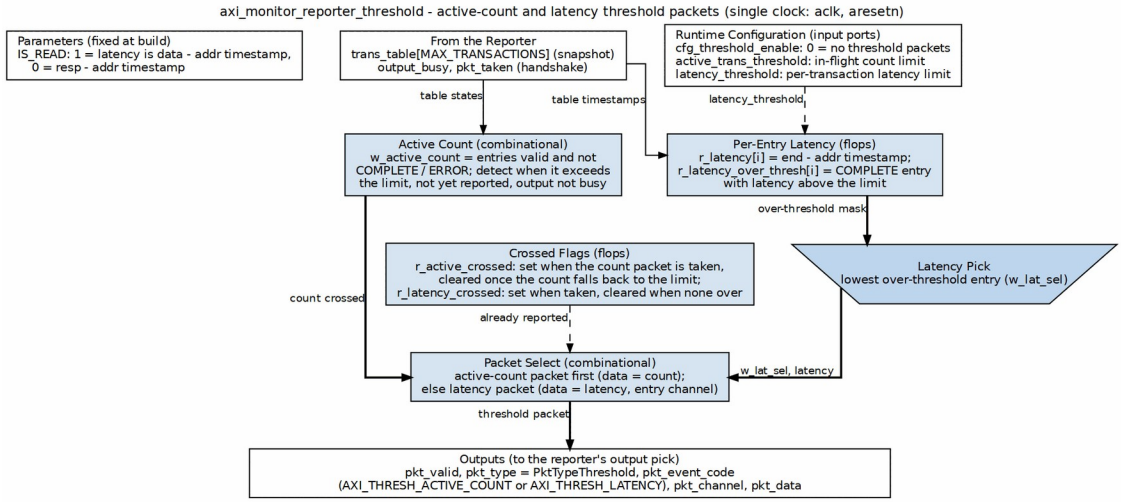
Port	Direction	Width	Description
			clears only when the underlying condition lifts

16.3.4 Packet Output

Port	Direction	Width	Description
pkt_valid	Output	1	A threshold packet is available this cycle
pkt_type	Output	4	Packet type — constant PktTypeThreshold (4'h2)
pkt_event_code	Output	8	Event code: AXI_THRESH_ACTIVE_COUNT (8'h0) or AXI_THRESH_LATENCY (8'h1)
pkt_channel	Output	9	Channel of the selected transaction (0 for active-count events)
pkt_data	Output	64	Zero-extended active count, or the zero-extended latency value

16.4 Functional Description

16.4.1 Block diagram



axi_monitor_reporter_threshold block diagram

Two checks run side by side. The active count tallies entries still in flight and compares the total with its limit. The latency path registers each completed entry's latency and flags those over the limit, and a pick chooses the lowest one. A crossed flag per check makes one crossing send one packet; the active-count packet goes first.

16.4.2 Active-Count Detection

A combinational loop scans `trans_table` and counts slots that are `valid` and in a state other than `TRANS_COMPLETE` or `TRANS_ERROR` — that is, transactions genuinely in flight. `w_active_detect` asserts when threshold detection is enabled, the count strictly exceeds `active_trans_threshold`, the edge flag `r_active_crossed` is not already set, and the output is not busy. Because this reflects an instantaneous condition, it takes priority in the output mux.

16.4.3 Per-Slot Latency Pipeline

For each slot, a registered pipeline computes latency from the live transaction table: `data_timestamp - addr_timestamp` for reads (`IS_READ=1`) or `resp_timestamp - addr_timestamp` for writes. The result is stored in `r_latency[idx]`, and a companion flag `r_latency_over_thresh[idx]` is set when the slot is valid, in `TRANS_COMPLETE` state, and its latency exceeds `latency_threshold`. It is deliberately **not** qualified by the latency edge flag — folding the flag in here would make the condition self-clearing, since the flag would suppress the very term that keeps it set. `r_latency_crossed` gates only the output mux. Registering the subtract and compare splits what would otherwise be a 16-wide carry chain plus output mux across a clock cycle.

16.4.4 Latency Event Selection

A combinational priority encoder (`w_lat_sel` / `w_has_lat`) picks the first slot with `r_latency_over_thresh` asserted, off the registered pipeline so it does not recreate the wide combinational path.

16.4.5 Edge-Sticky Flags

Two flags prevent re-firing on the same crossing every cycle:

- `r_active_crossed` — set when an active-count packet is accepted (`pkt_taken` with matching type/event code); cleared automatically when the active count drops back to or below the threshold.
- `r_latency_crossed` — set when a latency packet is accepted; gates further latency detections until re-armed.

16.4.6 Output Multiplexer

Active-count beats latency. When `w_active_detect` is asserted, `pkt_valid` fires with `AXI_THRESH_ACTIVE_COUNT`, `pkt_data` = zero-extended active count, and `pkt_channel` = 0. Otherwise, when a latency event is pending and the output is free (`w_has_lat && ! output_busy`), `pkt_valid` fires with `AXI_THRESH_LATENCY`, `pkt_data` = the selected slot's latency, and `pkt_channel` = that slot's channel (low 6 bits, zero-extended).

16.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

16.6 Usage Examples

This block is instantiated inside `axi_monitor_reporter`, not by users directly:

```
axi_monitor_reporter_threshold #(
    .MAX_TRANSACTIONS (MAX_TRANSACTIONS),
```

```
    .IS_READ          (IS_READ)
) u_reporter_threshold (
    .aclk                   (aclk),
    .aresetn                (aresetn),

    .trans_table            (w_trans_table),
    .cfg_threshold_enable   (cfg_threshold_enable),
    .active_trans_threshold (cfg_active_trans_threshold),
    .latency_threshold      (cfg_latency_threshold),

    .output_busy            (w_output_busy),
    .pkt_taken              (w_thresh_pkt_taken),

    .pkt_valid              (w_thresh_pkt_valid),
    .pkt_type               (w_thresh_pkt_type),
    .pkt_event_code         (w_thresh_pkt_event_code),
    .pkt_channel            (w_thresh_pkt_channel),
    .pkt_data               (w_thresh_pkt_data)
);
```

16.7 Design Notes

16.7.1 Threshold Packets Bypass the FIFO

Unlike error/completion packets, threshold packets inject directly into the output rather than routing through the reporter FIFO. That is why the block needs `output_busy` — to avoid overwriting a packet already in flight on the MonBus.

16.7.2 Active Count vs Latency Priority

Active count is an instantaneous, whole-table condition; latency is a per-slot event that has already been captured in the pipeline flag. The mux deliberately favors the active-count crossing so the more time-sensitive congestion signal is never starved by a queue of latency reports.

16.7.3 Latency Pipeline Is the Area Cost

The 16 x 32-bit latency registers plus 16 threshold flags are the reason disabling this block (`ENABLE_THRESHOLD_LOGIC=0`) yields a meaningful area reduction. When the feature is not needed, dropping the block removes the entire pipeline.

16.7.4 Read vs Write Latency

`IS_READ` selects which timestamp pair defines latency. Instantiate a read-monitor threshold block with `IS_READ=1` and a write-monitor block with `IS_READ=0` so the measured interval matches the protocol phase of interest.

16.8 Related Modules

Used by:

- **axi_monitor_reporter.sv** — instantiates this block as its threshold-packet sub-emitter
- **axi_monitor_base.sv** — top-level monitor scaffold

Uses:

- **monitor_common_pkg** — `PktTypeThreshold`, `bus_transaction_t`, transaction states
- **monitor_amba4_pkg** — `AXI_THRESH_ACTIVE_COUNT` / `AXI_THRESH_LATENCY` event codes
- **reset_defs.svh** — reset macros

See also:

- **axi_monitor_reporter_perf.sv** — performance packet emitter (sibling)
- **axi_monitor_reporter_timeout.sv** — timeout packet emitter (sibling)
- **axi_monitor_trans_mgr.sv** — maintains the `trans_table` consumed here

16.9 Testing

No dedicated testbench for this module. It has no `val/**/test_axi_monitor_reporter_threshold.py`. It is exercised indirectly, through the tests of modules that instantiate it (directly or further up):

- `axi4_master_rd_mon` – `val/**/test_axi4_master_rd_mon.py`
- `axi4_master_wr_mon` – `val/**/test_axi4_master_wr_mon.py`
- `axi4_slave_rd_mon` – `val/**/test_axi4_slave_rd_mon.py`
- `axi4_slave_wr_mon` – `val/**/test_axi4_slave_wr_mon.py`
- `axi5_master_rd_mon` – `val/**/test_axi5_master_rd_mon.py`

Indirect coverage exercises this module only in the configurations those parents elaborate. A parameter or mode no parent uses is untested.

Treat any behaviour described on this page as unverified by simulation.

16.10 References

16.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Sprunt, B. "The Basics of Performance-Monitoring Hardware." *IEEE Micro*, 22(4):64-71, July/Aug. 2002. DOI: 10.1109/MM.2002.1028477. — related work: on-chip performance counters, in processors.
- Abramovici, M., Bradley, P., Dwarakanath, K., Levin, P., Memmi, G., Miller, D. "A Reconfigurable Design-for-Debug Infrastructure for SoCs." *Proc. 43rd Design Automation Conf. (DAC 2006)*, pp. 7-12, July 2006. DOI: 10.1145/1146909.1146916. — related work: triggers and event counters in a debug fabric.
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

16.10.2 Source Code

- RTL: `rtl/amba/ocelus/axi_monitor_reporter_threshold.sv`
- Parent: `rtl/amba/ocelus/axi_monitor_reporter.sv`
- Packages: `rtl/amba/includes/ocelus_common_pkg.sv`, `rtl/amba/includes/ocelus_amba4_pkg.sv`

16.10.3 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Monitor Base: `docs/markdown/rtl-amba/monitor/axi_monitor_base.md`
- Packet Format: `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`

Last Updated: 2026-07-15

16.11 Navigation

- **Back to Shared Infrastructure Index**
- **Back to rtl-amba Index**

17 AXI Monitor Reporter — Timeout Packets

Module: `axi_monitor_reporter_timeout.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: error plane

17.1 Overview

The `axi_monitor_reporter_timeout` module is the timeout-packet emitter for the AXI/AXIL monitor family. It is one of the per-packet-type sub-blocks dispatched by the top-level `axi_monitor_reporter`. It is a pure combinational cone that scans the transaction table for unreported error slots the timeout detector has flagged, priority-encodes the first match, and drives the corresponding `PktTypeTimeout` packet fields.

The block was split out of the original monolithic reporter so integrators can drop it (`ENABLE_TIMEOUT_LOGIC=0`).

`timeout` is a **fault class** (taxonomy): it never fires in correct operation. Exercise it by **injecting a non-responsive slave** — hold the R/B response beats past the configured timeout window so the transaction stays outstanding long enough for the timeout detector to flag it. Like all faults, this stalls the traffic; the cone's value is emitting `PktTypeTimeout` so the capture records *which* transaction hung.

Key features:

- Pure combinational detection (no internal state)
- Scans for valid, unreported, error-state slots flagged as timed out
- Priority-encodes the first matching slot
- Emits the packet's event code, channel, and address directly from the table
- Exposes the selected slot index (`sel_idx`) for the top reporter's mark-reported feedback
- Shares the `timeout_detected` vector with the error sub-block to avoid double-reporting

When a transaction stalls, the `axi_monitor_timeout` block asserts a per-slot `timeout_detected` bit and the transaction manager moves that slot into the `TRANS_ERROR` state. This reporter block turns that condition into a MonBus packet: it finds the first slot that is valid, in error, timed out, and not yet reported, and emits a timeout packet describing it (event code, channel, and address).

Use cases:

- Reporting stuck AXI transactions (missing R/B response, hung handshake)

- Surfacing protocol hangs to a host IRQ handler for recovery
- Regression detection of deadlock or backpressure faults
- Correlating a timeout with the offending address and channel

Key benefit: zero-state, low-cost detection that shares its `timeout_detected` and `event_reported` vectors with the error emitter, guaranteeing each stuck transaction is reported exactly once.

17.2 Parameters

Parameter	Type	Default	Description
`MAX_TRANSACTIONS`	int	16	Number of transaction slots scanned
`IDX_W`	int	`$clog2(MAX_TRANSACTIONS)`	Derived width of the selected-slot index

17.3 Ports

17.3.1 Inputs

Port	Direction	Width	Description
`trans_table`	Input	`bus_transaction_t [MAX_TRANSACTIONS]`	Live outstanding-transaction table (valid, state, event_code, channel, addr)
`event_reported`	Input	MAX_TRANSACTIONS	Per-slot bit indicating the slot's event was already emitted (suppresses re-reporting)
`timeout_detected`	Input	MAX_TRANSACTIONS	Per-slot timeout flags from `axi_monitor_time`

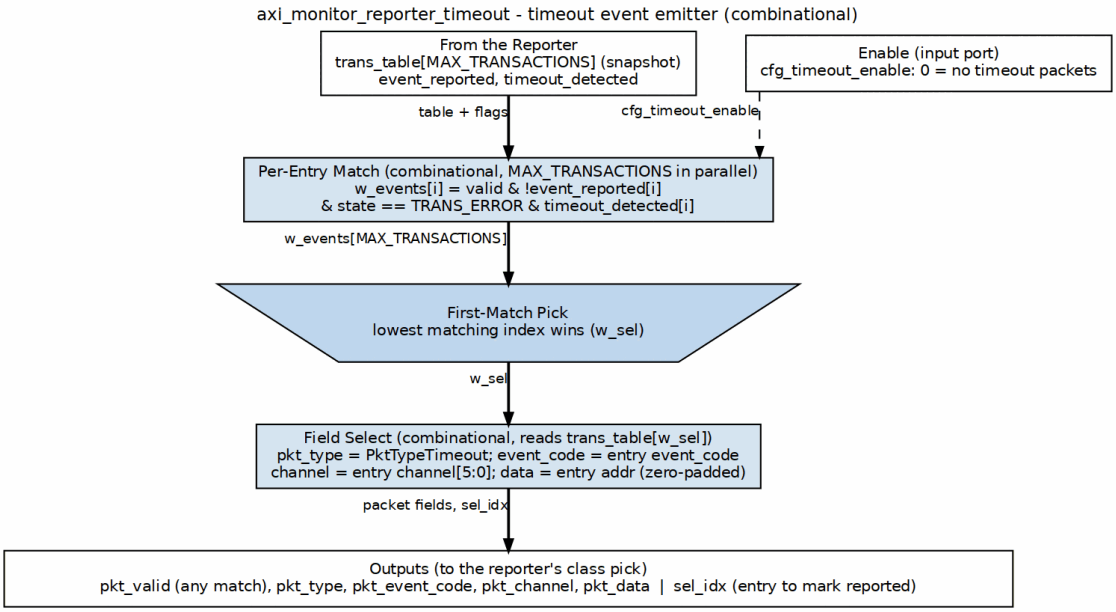
Port	Direction	Width	Description
			out
cfg_timeout_enable	Input	1	Enable timeout packet generation

17.3.2 Packet Output

Port	Direction	Width	Description
pkt_valid	Output	1	A timeout packet is available (a matching slot was found)
pkt_type	Output	4	Packet type — constant PktTypeTimeout (4'h3)
pkt_event_code	Output	8	Event code taken from the selected slot's event_code.raw_code
pkt_channel	Output	9	Channel of the selected slot (low 6 bits, zero-extended)
pkt_data	Output	64	Zero-extended address of the selected slot
sel_idx	Output	IDX_W	Index of the selected slot, returned to the top reporter for mark-reported feedback

17.4 Functional Description

17.4.1 Block diagram



axi_monitor_reporter_timeout block diagram

Every table entry is checked in parallel for an error state caused by a timeout that has not been reported. The lowest matching entry is picked, and its event code, channel and address become a timeout packet; `sel_idx` tells the reporter which entry to mark.

17.4.2 Candidate Detection

A combinational loop builds a one-hot-eligible mask `w_events`: a slot qualifies when it is `valid`, its `event_reported` bit is clear, its state is `TRANS_ERROR`, timeout reporting is enabled (`cfg_timeout_enable`), and the slot's `timeout_detected` bit is set. This intersection ensures only genuinely-timed-out, not-yet-reported error slots are candidates.

17.4.3 Priority Encoding

A second loop scans `w_events` from index 0 upward and latches the first asserted slot into `w_sel`, asserting `w_has_event`. This gives a deterministic, lowest-index-first selection.

17.4.4 Output Assignment

The outputs are pure continuous assignments off the selected slot:

- `pkt_valid = w_has_event`
- `sel_idx = w_sel`
- `pkt_type = PktTypeTimeout`

- `pkt_event_code = trans_table[w_sel].event_code.raw_code`
- `pkt_channel = {3'b0, trans_table[w_sel].channel[5:0]}`
- `pkt_data = pad_address(trans_table[w_sel].addr)` (zero-extended to 64 bits)

17.4.5 No Double-Reporting

The error sub-block masks the same `timeout_detected` slots, so a stuck transaction that is reported as a timeout here is not also reported as a plain error. The top reporter uses `sel_idx` to set the slot's `event_reported` bit, retiring the candidate.

17.5 Timing Characteristics

This module is **purely combinational** – it contains no `always_ff` and no latch, so it holds no state and adds no clock cycles. Its outputs settle a propagation delay after its inputs, and it introduces no latency into a pipeline that instantiates it.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

17.6 Usage Examples

This block is instantiated inside `axi_monitor_reporter`, not by users directly:

```
axi_monitor_reporter_timeout #(
    .MAX_TRANSACTIONS (MAX_TRANSACTIONS)
) u_reporter_timeout (
    .trans_table        (w_trans_table),
    .event_reported     (w_event_reported),
    .timeout_detected   (w_timeout_detected),
    .cfg_timeout_enable (cfg_timeout_enable),

    .pkt_valid          (w_timeout_pkt_valid),
    .pkt_type           (w_timeout_pkt_type),
    .pkt_event_code     (w_timeout_pkt_event_code),
    .pkt_channel        (w_timeout_pkt_channel),
    .pkt_data           (w_timeout_pkt_data),
    .sel_idx            (w_timeout_sel_idx)
);
```

17.7 Design Notes

17.7.1 Pure Combinational

The block holds no state; all edge-tracking and mark-reported bookkeeping lives in the top reporter. This keeps the timeout cone cheap and lets the top reporter arbitrate timeout packets against the other sub-emitters within one cycle.

17.7.2 Event Code Comes From the Slot

`pkt_event_code` is not a fixed constant — it is the slot's captured `event_code.raw_code`, so the packet carries the specific timeout reason recorded when the transaction manager moved the slot to `TRANS_ERROR`.

17.7.3 Shared Vectors Are Load-Bearing

Correct single-reporting depends on the top reporter driving `event_reported` and the timeout detector driving `timeout_detected` consistently. Historically, a missing `event_reported` feedback path caused transaction-table exhaustion; that feedback is now in place (see `rtl/amba/KNOWN_ISSUES/axi_monitor_reporter.md`).

17.8 Related Modules

Used by:

- **axi_monitor_reporter.sv** — instantiates this block as its timeout-packet sub-emitter
- **axi_monitor_base.sv** — top-level monitor scaffold

Uses:

- **monitor_common_pkg** — `PktTypeTimeout`, `bus_transaction_t`, `TRANS_ERROR`
- **monitor_amba4_pkg** — AMBA event-code definitions
- (No sequential logic — no reset macros required)

See also:

- **axi_monitor_timeout.sv** — produces the `timeout_detected` vector consumed here
- **axi_monitor_reporter_perf.sv** — performance packet emitter (sibling)
- **axi_monitor_reporter_threshold.sv** — threshold packet emitter (sibling)

17.9 Testing

No dedicated testbench for this module. It has no `val/**/test_axi_monitor_reporter_timeout.py`. It is exercised indirectly, through the tests of modules that instantiate it (directly or further up):

- `axi4_master_rd_mon` – `val/**/test_axi4_master_rd_mon.py`
- `axi4_master_wr_mon` – `val/**/test_axi4_master_wr_mon.py`
- `axi4_slave_rd_mon` – `val/**/test_axi4_slave_rd_mon.py`
- `axi4_slave_wr_mon` – `val/**/test_axi4_slave_wr_mon.py`
- `axi5_master_rd_mon` – `val/**/test_axi5_master_rd_mon.py`

Indirect coverage exercises this module only in the configurations those parents elaborate. A parameter or mode no parent uses is untested.

Treat any behaviour described on this page as unverified by simulation.

17.10 References

17.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. — the specification defines no transaction timeout, so the timeout window is a monitor policy; per-transaction watchdogs are a standard technique with no paper.
- Vermeulen, B. “Functional Debug Techniques for Embedded Systems.” *IEEE Design & Test of Computers*, 25(3):208-215, May/June 2008. DOI: 10.1109/MDT.2008.66. — survey of functional silicon debug, for context on detecting hangs.
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

17.10.2 Source Code

- RTL: `rtl/amba/ocelus/axi_monitor_reporter_timeout.sv`
- Parent: `rtl/amba/ocelus/axi_monitor_reporter.sv`

- Packages: `rtl/amba/includes/ocelus_common_pkg.sv`, `rtl/amba/includes/ocelus_amba4_pkg.sv`

17.10.3 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Monitor Base: `docs/markdown/rtl-amba/monitor/axi_monitor_base.md`
- Known Issues: `rtl/amba/KNOWN_ISSUES/`
- Packet Format: `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`

Last Updated: 2026-07-15

17.11 Navigation

- **Back to Shared Infrastructure Index**
- **Back to rtl-amba Index**

18 AXI Monitor Timeout

Module: `axi_monitor_timeout.sv` **Location:** `rtl/amba/ocelus/` **Category:** Core Infrastructure **Status:** Production Ready

Plane: protocol plane (timeout detector; used by both full and lite monitors)

18.1 Overview

The `axi_monitor_timeout` module provides configurable timeout detection for stuck transactions — the block that notices when a protocol phase stops making progress.

This is a **shared infrastructure module** used internally by the AXI/AXIL monitors. You won't instantiate it directly, but it's central to how the monitor catches hung traffic.

Key features:

- Per-phase timeout detection (AR/AW, R/W, B)
- Configurable timeout thresholds
- Frequency scaling for timeout counts
- Timeout event reporting with transaction ID
- Active transaction tracking
- Timeout clear on transaction completion

What it's for:

1. **Per-Phase Monitoring:** separate timeouts for AR/AW, R/W, B channels
2. **Configurable Thresholds:** adjustable timeout values per phase
3. **Event Reporting:** generates timeout packets with transaction details
4. **Active Tracking:** monitors all outstanding transactions
5. **Frequency Scaling:** adapts timeout counts to system clock frequency

---

18.2 Parameters

Parameter	Type	Default	Description
`MAX_TRANSACTIONS`	int	16	Number of transactions to monitor
`ADDR_WIDTH`	int	32	Address width

Parameter	Type	Default	Description
			for reporting
IS_READ	bit	1	1 = read channel, 0 = write channel

18.3 Ports

18.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	Monitor clock
aresetn	1	Input	Active-low asynchronous reset

18.3.2 Transaction Table

Port	Width	Direction	Description
trans_table	bus_transaction_t x MAX_TRANSACTIONS	Input	The transaction manager's table, read directly as an unpacked array. This module only reads it; axi_monitor_trans_mgr owns the entries.

18.3.3 Timer

Port	Width	Direction	Description
timer_tick	1	Input	One-cycle pulse from the frequency-invariant timer. Every threshold below counts these, not aclk cycles, so a timeout means the same wall-clock interval at any bus frequency.

18.3.4 Timeout Configuration

Each threshold is a count of `timer_tick` pulses for one phase of a transaction.

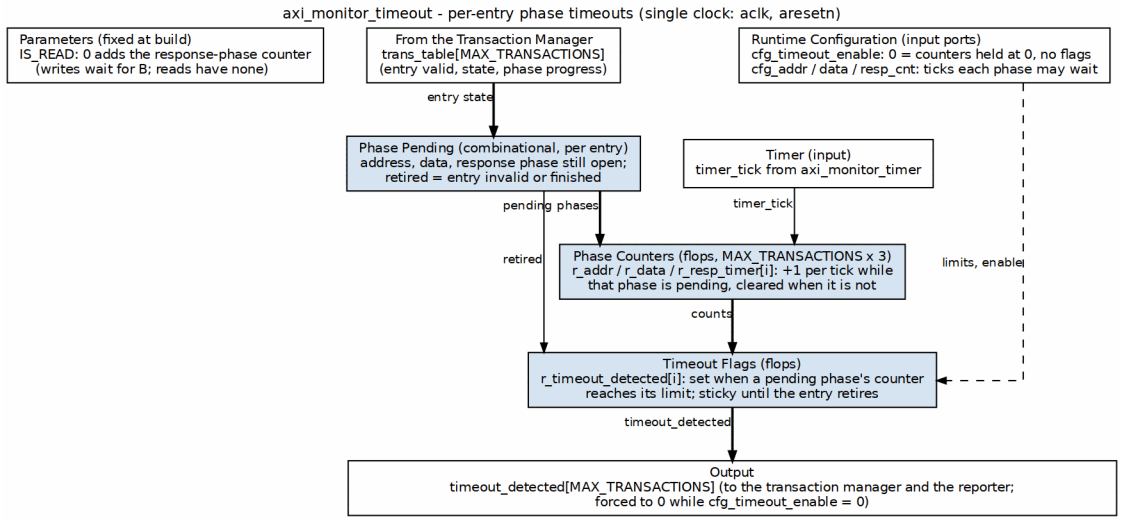
Port	Width	Direction	Description
`cfg_addr_cnt`	16	Input	Address-phase timeout threshold
`cfg_data_cnt`	16	Input	Data-phase timeout threshold
`cfg_resp_cnt`	16	Input	Response-phase timeout threshold
`cfg_timeout_enable`	1	Input	Master enable for timeout detection. Low does not merely mask the output: the per-phase timers and all recorded detections are flushed, so the block is inert rather than holding stale state. A detection registered while disabled would resurface the instant the enable returned – a detection with no `timer_tick` behind it, which the formal property `ap_no_set_without_tick` rejects.

18.3.5 Outputs

Port	Width	Direction	Description
`timeout_detected`	`MAX_TRANSACTIONS`	Output	One bit per table slot, high for a transaction that has timed out. Consumed by the transaction manager to retire the entry.

18.4 Functional Description

18.4.1 Block diagram



axi_monitor_timeout block diagram

For every table entry, three phase counters (address, data and, for writes, response) advance on each timer tick while that phase is still open, and clear when it is not. When a pending phase's counter reaches its configured limit, the entry's timeout flag sets and stays set until the entry retires. The flags go to the transaction manager, which makes the entry terminal, and to the reporter, which turns it into a timeout packet.

18.4.2 Architecture

Each transaction has 3 independent timeout counters for different phases. The timers are dedicated per-slot state owned by this module (`r_addr_timer` / `r_data_timer` / `r_resp_timer`, **16-bit** each — `TIMER_W = 16`, sized to hold the full microsecond threshold); the rest of the transaction record is read live off `trans_table`. Timers count `timer_tick` events (from the frequency-invariant timer, scaled by `cfg_freq_sel`) while their phase is pending, and fire at `timer >= cfg_addr_cnt / cfg_data_cnt / cfg_resp_cnt`.

18.4.3 Detection Semantics

The per-phase "still waiting" conditions, read straight off the live table:

Phase	Condition
Address	`valid && state == TRANS_ADDR_PHASE && !cmd_received` (command issued, not yet accepted)
Data	`valid && state in {TRANS_ADDR_PHASE,`

Phase	Condition
	`TRANS_DATA_PHASE} && cmd_received && !data_completed`
Response	write monitors only: `valid && state == TRANS_DATA_PHASE && data_completed && !resp_received`

Notable, deliberate properties:

- **"Command accepted, first beat never arrives" is detectable.** The data condition intentionally does NOT require `data_started`. With that term (the pre-`cb29e226` behavior) this stall class could never fire: the entry sat in `TRANS_ADDR_PHASE` forever, pinned its table slot, and enough of them held `block_ready` low permanently. A transaction whose command just handshook gets the full `cfg_data_cnt` window for its first beat, same as every later beat.
- **Runtime disable flushes state.** When `cfg_timeout_enable` is 0, all timers and detection flags are cleared, not merely masked. A stale detection computed against old timer state can therefore never resurface the instant the enable comes back (enforced by the `ap_no_set_without_tick` formal property). Disable means inert.
- **Detection is sticky until the slot retires** — the flag is held through `TRANS_ERROR` on purpose (that is the state a detected timeout puts the transaction into, and the reporter uses this vector to split timeout packets from genuine-error packets); it clears when the slot empties or reaches `TRANS_COMPLETE`.
- **This module cannot modify the table.** It exports `timeout_detected` per slot; `axi_monitor_trans_mgr` consumes that vector (its `i_timeout_detected` input) and moves the entry to `TRANS_ERROR` with `EVT_CMD_TIMEOUT` / `EVT_DATA_TIMEOUT` / `EVT_RESP_TIMEOUT` so it becomes cleanup-eligible.

18.5 Timing Characteristics

Metric	Value	Notes
Latency	1-2 cycles	Typical processing delay
Throughput	1 operation/cycle	Maximum rate

18.6 Usage Examples

This module is instantiated automatically within higher-level monitor modules — `axi_monitor_base` owns it, and users configure behavior through top-level monitor parameters. See the individual monitor documentation for configuration examples.

18.7 Design Notes

These are phase DURATION limits, not stall detectors. Each timer is zeroed only while its phase is not pending (`if (!w_data_pending[idx]) r_data_timer[idx] <= '0;`); a beat handshake does not reset it. A long burst that is making steady progress still trips `EVT_DATA_TIMEOUT` once the phase as a whole exceeds the threshold. Size `cfg_timeout_cycles` for your longest legitimate burst, not for your worst tolerable inter-beat gap.

The thresholds are microseconds, not clocks. `timer_tick` comes from `counter_freq_invariant`, so a timeout means the same wall-clock time at any `aclk` – provided `ACLK_MHZ` matches the real frequency. Leave it at 100 on a 90 MHz part and every timeout is wrong, silently.

`TIMER_W` is 16 deliberately. These counters were 4 bits, and every wrapper squashed the host's 16-bit request into them with a saturating truncation, so any value >= 16 became 15 and the entire configurable range collapsed onto 1..15 us.

18.8 Related Modules

- **axi_monitor_base**

Used by:

- **axi_monitor_base**

See also:

- **Monitor Architecture:** `docs/markdown/rtl-amba/overview.md`
- **Monitor Configuration Guide:** Monitor Base Configuration
- **Packet Format Specification:** `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`

18.9 Testing

- Functional correctness of core logic
- Boundary conditions (min/max values)
- Error handling and recovery
- Interface protocol compliance

See: `val/amba/test_axi_monitor_trans_mgr.py` (timeout-to-terminal transitions) and the `*_mon` wrapper suites for verification tests

18.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. — the specification defines no transaction timeout, so the window is a monitor policy; per-transaction watchdogs are a standard technique with no paper.
- Vermeulen, B. “Functional Debug Techniques for Embedded Systems.” *IEEE Design & Test of Computers*, 25(3):208-215, May/June 2008. DOI: 10.1109/MDT.2008.66. — survey of functional silicon debug, for context on detecting hangs.
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

18.11 Navigation

- **Back to Shared Infrastructure Index**
- **Back to rtl-amba Index**
- **Back to Main Documentation Index**

19 AXI Monitor Timer

Module: `axi_monitor_timer.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: protocol plane (latency timer; used by both full and lite monitors)

19.1 Overview

The AXI Monitor Timer provides frequency-invariant timing for the AXI monitor infrastructure by generating consistent timer ticks regardless of the system clock frequency. It combines a cycle-accurate global timestamp counter with a frequency-invariant timer tick generator to support both precise latency measurement and consistent timeout detection across different clock frequencies.

Key features:

- Frequency-invariant timer tick generation
- Fixed 1 MHz (1 us) tick; the 4-bit `cfg_freq_sel` selects the **clock frequency** the divider is matched to, not the tick period
- Global timestamp counter (32-bit cycle counter)
- Uses counter_freq_invariant for portable timing
- Zero-latency timestamp (combinational output)
- Consistent timeout behavior across frequencies
- Single clock domain operation
- Minimal resource utilization

The Timer module solves a portability problem that bites every monitor architecture eventually: timeout thresholds specified in cycles become frequency-dependent, so they need reconfiguration every time the clock frequency changes. By using frequency-invariant timer ticks, timeout thresholds can be specified in abstract “ticks” that represent consistent time periods regardless of the underlying clock frequency.

The global timestamp counter provides cycle-accurate timing for latency measurement and performance analysis, while the timer tick output enables timeout detection with configurable resolution.

19.2 Parameters

Runtime frequency selection is via `cfg_freq_sel`. The module also exposes elaboration parameters that size the clock-frequency-invariant (CFI) counter table:

Parameter	Type	Default	Description
CFI_MIN_FREQ_MHZ	int	5	Minimum frequency (MHz) in the CFI table
CFI_MAX_FREQ_MHZ	int	220	Maximum frequency (MHz) in the CFI table
CFI_NUM_FREQ_ENTRIES	int	16	Number of frequency entries
CFI_FREQ_STRATEGY	int	0	Frequency-spacing strategy selector

SEL_WIDTH is derived from CFI_NUM_FREQ_ENTRIES and sizes cfg_freq_sel.

19.3 Ports

19.3.1 Global Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	Clock signal
aresetn	Input	1	Active-low asynchronous reset

19.3.2 Timer Configuration

Port	Direction	Width	Description
cfg_freq_sel	Input	4	Frequency selection for timer tick (0-15)

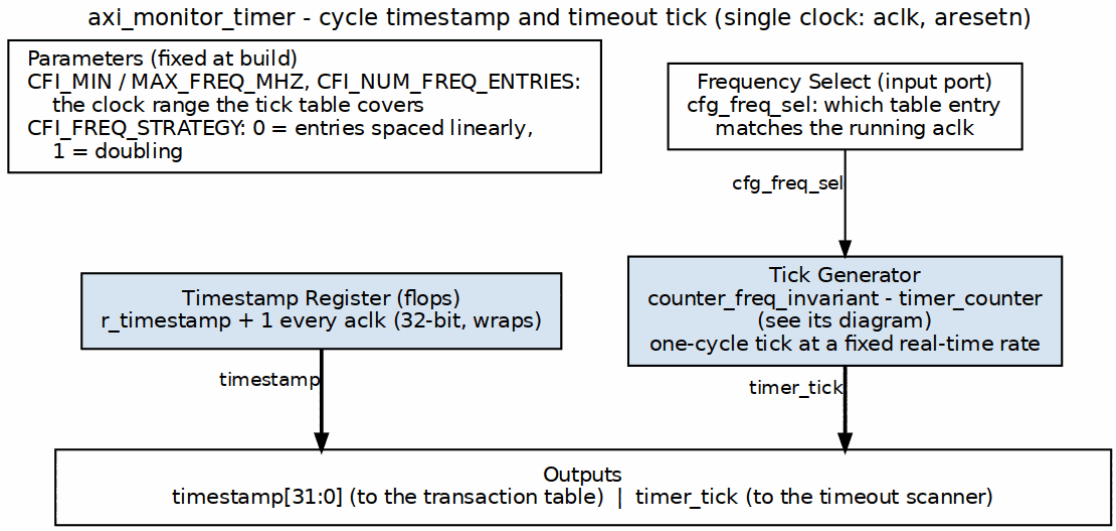
19.3.3 Timer Outputs

Port	Direction	Width	Description
timer_tick	Output	1	Timer tick signal

Port	Direction	Width	Description
			(frequency-invariant)
timestamp	Output	32	Global timestamp counter (cycle count)

19.4 Functional Description

19.4.1 Block diagram



axi_monitor_timer block diagram

Two independent outputs. The timestamp is a 32-bit register that counts every `aclk`. The tick comes from `counter_freq_invariant` (see its diagram), which uses `cfg_freq_sel` to pick the divider that makes the tick a fixed real time apart whatever the clock frequency; the timeout counters advance on it.

The Timer module implements two independent timing functions that work together to support the monitor infrastructure.

19.4.2 Global Timestamp Counter

The timestamp counter provides cycle-accurate time reference:

Implementation:

- 32-bit up-counter
- Increments every clock cycle
- Resets to zero on aresetn assertion
- Registered output (r_timestamp)

Purpose:

- Transaction timestamp marking (addr_timestamp, data_timestamp, resp_timestamp)
- Latency calculation (timestamp differences)
- Performance metric computation
- Event correlation across transactions

Range and Wraparound:

- Maximum count: 2^32 - 1 = 4,294,967,295 cycles
- At 1 GHz: ~4.3 seconds before wraparound
- At 100 MHz: ~43 seconds before wraparound
- Wraparound handled naturally in latency calculations (32-bit arithmetic)

Usage Example:

```
// Transaction manager records timestamp
addr_timestamp <= timestamp;  // When address phase completes

// Later, calculate latency
latency = data_timestamp - addr_timestamp;  // Handles wraparound
```

19.4.3 Frequency-Invariant Timer Tick

The timer tick provides consistent periodic signal across frequencies:

Implementation:

- Uses counter_freq_invariant module from rtl/common/
- Configured via cfg_freq_sel (4-bit selection: 0-15)
- Generates single-cycle pulse (timer_tick) at configured interval
- Combinational output (w_timer_tick)

Frequency Selection:

`cfg_freq_sel` does **not** select a tick period. The tick rate is **fixed at 1 MHz — one tick per microsecond** — and `cfg_freq_sel` indexes a lookup table of *clock frequencies in MHz* so the

divider can produce that 1 us tick from whatever clock this instance actually runs on. That is the whole point of the “frequency-invariant” name: a timeout of N is N microseconds on every clock.

The table is built at elaboration from `CFI_MIN_FREQ_MHZ`, `CFI_MAX_FREQ_MHZ`, `CFI_NUM_FREQ_ENTRIES` and `CFI_FREQ_STRATEGY`. With this module’s defaults (5-220 MHz, 16 entries, LINEAR) the entries are `freq[i] = 5 + (220 - 5) * i / 15`:

cfg_freq_sel	Clock (MHz)	Divisor (cycles/tick)	cfg_freq_sel	Clock (MHz)	Divisor
0	5	5	8	119	119
1	19	19	9	134	134
2	33	33	10	148	148
3	48	48	11	162	162
4	62	62	12	177	177
5	76	76	13	191	191
6	91	91	14	205	205
7	105	105	15	220	220

The divisor **is** the frequency in MHz, because dividing an F-MHz clock by F gives 1 MHz. So the rule for setting it is simply: **pick the entry closest to your actual `aclk` frequency.**

> **Instantiated through `axi_monitor_base`, the table is flat.** The base module defaults `CFI_MIN_FREQ_MHZ = CFI_MAX_FREQ_MHZ = 100`, so every entry is 100 and the tick is exactly 1 us at 100 MHz *regardless of `cfg_freq_sel`*. Override the CFI parameters only if the design changes `aclk` at runtime. A monitor running at a clock the table cannot express ticks at the wrong rate, and every timeout scales with it.

Purpose:

- Timeout detection timing base
- Consistent timeout duration across frequencies
- Periodic performance metric reporting
- Threshold comparison timing

Tick Characteristics:

- Single cycle pulse (asserts high for 1 cycle)
- Precise period (no jitter or drift)
- Synchronous to aclk
- Immediately reflects cfg_freq_sel changes

19.4.4 Counter Freq Invariant Integration

The module instantiates counter_freq_invariant with specific configuration:

Instance Configuration:

```
counter_freq_invariant #(
    .COUNTER_WIDTH    (1),                    // Only need tick output
    .MIN_FREQ_MHZ     (CFI_MIN_FREQ_MHZ),
    .MAX_FREQ_MHZ     (CFI_MAX_FREQ_MHZ),
    .NUM_FREQ_ENTRIES (CFI_NUM_FREQ_ENTRIES),
    .FREQ_STRATEGY    (CFI_FREQ_STRATEGY)
) timer_counter (
    .clk         (aclk),
    .rst_n       (aresetn),
    .sync_reset_n(1'b1),
    .freq_sel    (cfg_freq_sel),
    .tick        (w_timer_tick),
    .o_counter   ()                           // counter output unused
);
```

Design Rationale:

- `COUNTER_WIDTH=1`: only the tick pulse is needed, not the counter value
- The CFI_* parameters are forwarded so the frequency table is sized by this module's parameters rather than hard-coded
- `sync_reset_n` tied high: no need for a synchronous reset
- `o_counter` left open: only the tick matters (note the `o_` prefix — there is no port named `counter`)

counter_freq_invariant Module:

- Located in `rtl/common/counter_freq_invariant.sv`
- Divides an arbitrary clock down to a **1 MHz tick**
- `freq_sel` indexes a table of clock frequencies in MHz; the entry is used directly as the divisor
- Supports LINEAR (uniform) and POW2 (doubling) table spacing
- See `rtl/common/` documentation for detailed behavior

19.4.5 Integration with Monitor Architecture

The Timer module provides fundamental timing services to the entire monitor infrastructure.

19.4.5.1 Output Integration

To Transaction Manager (axi_monitor_trans_mgr):

- timestamp output provides time reference
- Used to mark transaction phase completion times
- Enables latency calculation in reporter

To Timeout Module (axi_monitor_timeout):

- timer_tick output drives timeout detection
- Timeout module increments counters on each tick
- Enables frequency-invariant timeout thresholds

To Reporter Module (axi_monitor_reporter):

- Indirect use via transaction table timestamps
- Reporter calculates latencies from timestamp differences
- Performance metrics derived from timestamp data

19.4.5.2 Configuration Strategy

There is one rule: **set `cfg_freq_sel` to the table entry closest to your actual `aclk` frequency.** The tick is 1 us at that setting, and every timeout threshold is then read directly in microseconds.

aclk	Default table (5-220 MHz, LINEAR)	Resulting tick
100 MHz	`cfg_freq_sel = 4'd7` (105 MHz — nearest entry)	1.05 us (5% long)
200 MHz	`cfg_freq_sel = 4'd14` (205 MHz)	1.025 us (2.5% long)
50 MHz	`cfg_freq_sel = 4'd3` (48 MHz)	0.96 us (4% short)

The tick period is **divisor / f_clk** — the selected entry is a cycle count, so a divisor above your real clock rate makes the tick (and every timeout built on it) run *long*, and one below makes it run *short*. The residual error is the gap between your clock and the nearest table entry; a table built for one exact frequency (`CFI_MIN = CFI_MAX = aclk_mhz`, which is what `axi_monitor_base` does at 100 MHz) has none.

Sizing timeouts. Because the tick is 1 us, the timeout counts are microseconds directly — no cycle arithmetic:

```
cfg_addr_cnt = 100    // 100 us on the command phase
cfg_data_cnt = 500    // 500 us on the data phase
```

The counts are 16-bit, so the range is 1..65535 us (~65 ms), and `16'hFFFF` is the conventional "effectively never" setting.

19.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

19.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
axi_monitor_timer #(
    .CFI_MIN_FREQ_MHZ      (5),
    .CFI_MAX_FREQ_MHZ      (220),
    .CFI_NUM_FREQ_ENTRIES  (16),
    .CFI_FREQ_STRATEGY     (0)
) u_axi_monitor_timer (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    .cfg_freq_sel          (cfg_freq_sel),
    .timer_tick            (timer_tick),
    .timestamp             (timestamp)
);
```

19.7 Design Notes

19.7.1 Timestamp Wraparound Handling

The 32-bit timestamp counter wraps to zero after ~4.3 seconds at 1 GHz:

Latency Calculation: Latency calculations naturally handle wraparound through unsigned arithmetic:

```
latency = end_timestamp - start_timestamp;  // Works across wraparound
```

Example:

```
start_timestamp = 0xFFFF_FFFF (max value)
end_timestamp   = 0x0000_0005 (after wraparound)
latency = 0x0000_0005 - 0xFFFF_FFFF = 0x0000_0006 (6 cycles)
```

Limitations:

- Maximum measurable latency: 2^31 cycles (half of counter range)
- Exceeding this causes incorrect results
- In practice, transaction latencies << 2^31 cycles

Mitigation: If longer-term timestamps needed, extend timestamp to 64 bits (requires parameter addition).

19.7.2 Frequency Selection Is Not a Granularity Trade-off

`cfg_freq_sel` is a **clock-matching** control, not a resolution control. Every valid setting produces the same 1 us tick; a setting that does not match `aclk` does not buy finer or coarser timeouts, it simply makes every timeout wrong by the ratio between the selected entry and the real clock. There is no power/precision trade-off to tune here.

Timeout *resolution* is fixed at 1 us. If you need finer resolution than that, the lever is the CFI table (build it for a sub-multiple of the real clock so the tick is shorter), not `cfg_freq_sel`.

19.7.3 Dynamic Frequency Scaling

When the system clock changes at runtime (DFS):

Timestamp counter: keeps counting *cycles*, so latency measurements are in cycles and their absolute-time meaning shifts with the clock.

Timer tick: the divisor is whatever `cfg_freq_sel` selected, so after a clock change the tick is no longer 1 us — it is off by the ratio of the new clock to the selected table entry, and every timeout scales with it.

Recommendation: re-point `cfg_freq_sel` at the table entry matching the new frequency whenever the clock changes; this is exactly the case the CFI table exists for, and it is why the default table spans 5-220 MHz rather than being built for a single frequency. (`axi_monitor_base` instantiates a flat 100 MHz table by default, which is correct for a fixed-clock design and wrong for a DFS one — override the CFI_* parameters there.)

19.7.4 Zero Timestamp on Reset

The timestamp counter resets to zero on aresetn assertion:

Implications:

- First transaction after reset has timestamp near zero
- Timestamps from before reset invalid after reset

- No timestamp preservation across resets

Best Practice: Reset all monitors simultaneously to ensure consistent timestamp domain across the system.

19.7.5 Timer Tick Duty Cycle

The timer_tick output is a single-cycle pulse:

Characteristics:

- High for exactly 1 cycle
- Low for (tick_period - 1) cycles
- Duty cycle = 1 / tick_period

Usage: Modules should use timer_tick as an enable/trigger signal, not as a clock. It's synchronous to aclk but much slower, making it unsuitable as a clock source.

19.7.6 Resource Utilization

The timer module has minimal resource cost:

Registers:

- 32-bit timestamp counter
- 1-bit counter in counter_freq_invariant
- Prescaler state in counter_freq_invariant

Combinational Logic:

- Increment logic for timestamp
- Frequency divider in counter_freq_invariant

19.7.7 Multiple Timer Instances

Each monitor instance can have its own timer if needed:

Advantages of Separate Timers:

- Independent frequency selection per monitor
- No timing constraints between monitors
- Simpler physical design (local timing)

Advantages of Shared Timer:

- Single timestamp domain across monitors
- Easier event correlation

- Reduced resource utilization

Recommendation: Use shared timer unless monitors operate at different frequencies or in different clock domains.

19.8 Related Modules

Used by:

- axi_monitor_base.sv (primary user)
- axi_monitor_filtered.sv (via axi_monitor_base)
- All AXI/AXIL monitor variants

Uses:

- counter_freq_invariant.sv (from rtl/common/)
- Provides frequency-invariant counting capability

Critical interfaces:

- **To axi_monitor_trans_mgr:**
  - timestamp output (32-bit time reference)
- **To axi_monitor_timeout:**
  - timer_tick output (timeout trigger)

Related infrastructure:

- axi_monitor_trans_mgr.sv (timestamp consumer)
- axi_monitor_timeout.sv (timer tick consumer)
- axi_monitor_reporter.sv (indirect via timestamps)

19.9 Testing

No dedicated testbench for this module. It has no `val/**/test_axi_monitor_timer.py`. It is exercised indirectly, through the tests of modules that instantiate it (directly or further up):

- `axi4_master_rd_mon` – `val/**/test_axi4_master_rd_mon.py`
- `axi4_master_wr_mon` – `val/**/test_axi4_master_wr_mon.py`
- `axi4_slave_rd_mon` – `val/**/test_axi4_slave_rd_mon.py`
- `axi4_slave_wr_mon` – `val/**/test_axi4_slave_wr_mon.py`

- `axi5_master_rd_mon` – `val/**/test_axi5_master_rd_mon.py`

Indirect coverage exercises this module only in the configurations those parents elaborate. A parameter or mode no parent uses is untested.

Treat any behaviour described on this page as unverified by simulation.

19.10 References

19.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- IEEE-ISTO 5001-2003. *The Nexus 5001 Forum Standard for a Global Embedded Processor Debug Interface*, Version 2.0, IEEE Industry Standards and Technology Organization, Dec. 2003 (defining non-peer standard). — §4.11 timestamping: a comparable optional timestamp in a debug standard. The 1 us prescaler (`counter_freq_invariant`) is a standard technique, indexed as such in `rtl/common/references/README.md`.
- Indexed in `rtl/amba/references/README.md` (local PDF: `DebugTraceInterfaceStandard__IeeeIsto2003.pdf`)

19.10.2 Specifications

- Base Module: axi_monitor_base.md
- Timeout Module: axi_monitor_timeout.md
- Transaction Manager: axi_monitor_trans_mgr.md

19.10.3 Source Code

- RTL: `rtl/amba/ocelus/axi_monitor_timer.sv`
- Counter: `rtl/common/counter_freq_invariant.sv`
- Documentation: `docs/markdown/rtl-common/counter_freq_invariant.md`
- Tests: `val/amba/test_axi4_master_rd_mon.py` (integration)

19.10.4 Configuration Guides

- Monitor Base Configuration - Timeout configuration
- `docs/markdown/rtl-amba/index.md` - Subsystem requirements

Last Updated: 2025-10-24

19.11 Navigation

- **Back to axi_monitor_base**
- **Previous: axi_monitor_timeout**
- **Back to Shared Infrastructure Index**
- **Back to rtl-amba Index**

20 AXI Monitor Transaction Manager

Module: `axi_monitor_trans_mgr.sv` **Location:** `rtl/amba/ocelus/` **Category:** Core Infrastructure **Status:** Production Ready (CAM-backed revision, 2026-06-08)

Plane: protocol plane (transaction manager for the full monitor)

20.1 Overview

`axi_monitor_trans_mgr` tracks outstanding AXI transactions through their addr → data → resp lifecycle. It exposes a registered table of in-flight transactions (`trans_table[N]`) for downstream consumers (reporter, debug, timeout) and feeds the monitor bus with state-change events.

This is shared infrastructure — every AXI4 / AXI5 / AXI-Lite monitor uses it internally. You don't instantiate it directly; you configure its behaviour through the top-level monitor wrapper.

The current revision delegates per-transaction keying + storage to the shared `monitor_trans_cam` module. The previous in-place revision, once parked in `mon_temp/`, was deleted in `d246a72d` along with its equivalence test and the `TRANS_MGR_VARIANT` rollback knob — there is no legacy variant anymore. Don't go looking for the escape hatch; it isn't there.

What it does for you:

- Tracks up to `MAX_TRANSACTIONS` outstanding (default 16)
- 3 independent ID lookups per cycle (addr / data / resp) via CAM
- Out-of-order transaction support
- **Multiple outstanding same-ID transactions**, each in its own slot, with oldest-first (rank-based) attribution of data/response beats
- Same-cycle AW+W first-beat capture for write monitors (combinational bypass)
- Command-entry cap implementing the saturation-recovery contract (see axi_monitor_base)
- Burst beat counting
- Per-phase timestamps for latency reporting
- Orphan-data / orphan-resp detection (per AXI4 / AXI-Lite rules)
- Timeout-to-terminal-state transition via `i_timeout_detected` feedback
- State-change events for downstream packet generation
- Active-transaction counter (exact CAM occupancy, registered pop-count)
- Cleanup-when-event-reported handshake with the reporter
- AXI4 / AXI4-Lite / read / write variants via parameters

20.2 Parameters

Parameter	Type	Default	Description
MAX_TRANSACTIONS	int	16	Transaction table depth
ADDR_WIDTH	int	32	Width of the address bus tracked. **Values above 32 truncate the REPORTED address:** bus_transaction_t.addr is a 32-bit field, so allocation stores 32'(cmd_addr) and packets carry only the low 32 bits. Tracking itself is unaffected (the CAM keys on ID, not address), and ADDR_WIDTH=64 is a supported, tested configuration — but a 64-bit integrator should not expect full addresses in monitor packets
ID_WIDTH	int	8	Width of the AXI ID. **Hard maximum 8** —

Parameter	Type	Default	Description
			`bus_transaction_t.id` is 8 bits, so a wider key would disagree with the payload and mis-attribute transactions. `ID_WIDTH > 8` is refused at elaboration with an `$error`, deliberately: there is no safe degraded behaviour
`IS_READ`	bit	1	1 for read monitors, 0 for write
`IS_AXI`	bit	1	1 for AXI4, 0 for AXI-Lite
`USE_WDATA_ORDER_Q`	bit	0	Write monitors: attribute W beats via an AWID FIFO instead of the table-wide state predicate. **Required when `NUM_BANKS > 1`**
`NUM_BANKS`	int	1	Generate the CAM this many times, `MAX_TRANSACTIONS/NUM_BANKS` deep each. Power of 2, must divide the table
`ENABLE_PERF_PACKETS`	bit	0	Reserved — perf

Parameter	Type	Default	Description
			packet generation hook
ADDR_FILTER_ENABLE	bit	0	Address-range packet filter. 0 leaves filtered_mask permanently zero and the build bit-identical. See Address filtering for why this suppresses packets rather than refusing admission

The AW and IW short-alias parameters are retained for API stability with prior revisions; they default to ADDR_WIDTH and ID_WIDTH.

20.3 Ports

The module exports a registered trans_table of bus_transaction_t entries (see monitor_amba4_pkg.sv) plus aggregate status:

```
module axi_monitor_trans_mgr
    import monitor_common_pkg::*;
    import monitor_amba4_pkg::*;
#(
    parameter int MAX_TRANSACTIONS = 16,
    parameter int ADDR_WIDTH       = 32,
    parameter int ID_WIDTH         = 8,
    parameter bit IS_READ          = 1'b1,
    parameter bit IS_AXI           = 1'b1,
    ...
) (
    input  logic                          aclk,
    input  logic                          aresetn,

    // Synchronous clear: empty the transaction CAM and zero the
    // active-count pipeline on the next edge (no full reset needed).
```

```
    // Pulse one cycle while idle.
    input  logic                            clear,

    // Address channel
    input  logic                            cmd_valid,
    input  logic                            cmd_ready,
    input  logic [IW-1:0]                   cmd_id,
    input  logic [AW-1:0]                   cmd_addr,
    input  logic [7:0]                      cmd_len,
    input  logic [2:0]                      cmd_size,
    input  logic [1:0]                      cmd_burst,

    // Data channel
    input  logic                            data_valid,
    input  logic                            data_ready,
    input  logic [IW-1:0]                   data_id,
    input  logic                            data_last,
    input  logic [1:0]                      data_resp,

    // Response channel (write only)
    input  logic                            resp_valid,
    input  logic                            resp_ready,
    input  logic [IW-1:0]                   resp_id,
    input  logic [1:0]                      resp_code,

    input  logic [31:0]                     timestamp,
    input  logic [MAX_TRANSACTIONS-1:0]     i_event_reported_flags,

    // Timeout feedback from axi_monitor_timeout. The timeout block
detects
    // the stall but cannot modify the table; trans_mgr consumes this
vector
    // and moves the flagged entry to TRANS_ERROR with the appropriate
    // EVT_*_TIMEOUT code so it becomes cleanup-eligible instead of
leaking.
    input  logic [MAX_TRANSACTIONS-1:0]     i_timeout_detected,

    // Address-range packet filter (active when ADDR_FILTER_ENABLE=1).
    // Inclusive [low, high]; a command whose address falls OUTSIDE
the
    // window has its packets suppressed. See "Address filtering".
    input  logic                            cfg_addr_filter_enable,
    input  logic [AW-1:0]                   cfg_addr_filter_low,
    input  logic [AW-1:0]                   cfg_addr_filter_high,

    // One bit per slot: 1 = suppress this entry's packets. Consumed
```

```
by
    // axi_monitor_reporter, which also retires filtered entries so
they do
    // not leak their slot.
    output logic [MAX_TRANSACTIONS-1:0]   filtered_mask,

    output bus_transaction_t
trans_table[MAX_TRANSACTIONS],
    output logic [7:0]                    active_count
);
```

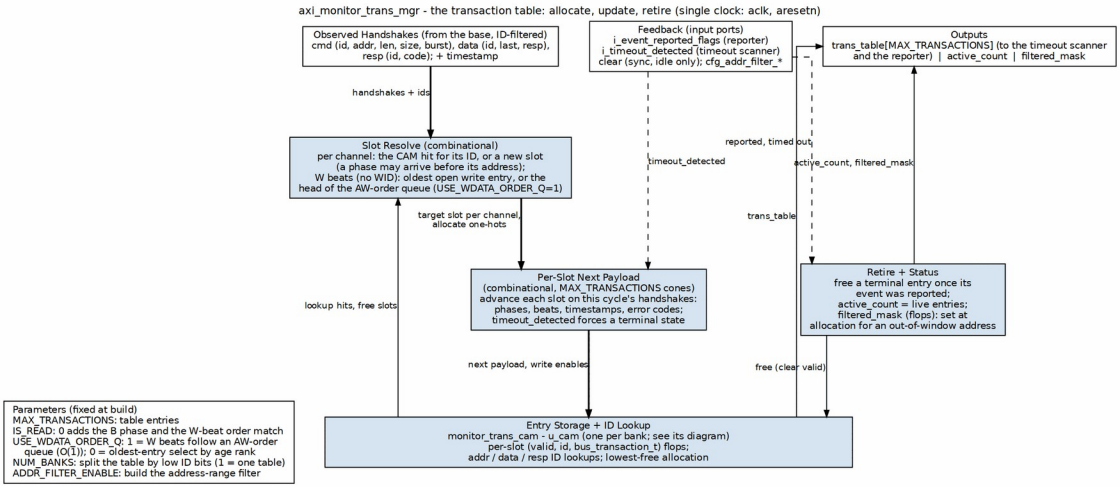
There is no per-slot state-change output, and none is planned: the reporters scan `trans_table` themselves. To act on a slot moving, compare `trans_table[i].state` — which is what `axi_monitor_reporter_debug` does to raise its `PktTypeDebug` packets.

The `bus_transaction_t` struct (defined in `monitor_amba4_pkg.sv`) contains: - State flags: `valid`, `cmd_received`, `data_started`, `data_completed`, `resp_received`, `event_reported`, `eos_seen` - FSM state: `state` (TRANS_IDLE / ADDR_PHASE / DATA_PHASE / COMPLETE / ERROR / ORPHANED) - Captured fields: `addr`, `id`, `len`, `size`, `burst`, `channel` - Timers and timestamps: `addr_timer`, `data_timer`, `resp_timer`, `addr_timestamp`, `data_timestamp`, `resp_timestamp` - Beat tracking: `expected_beats`, `data_beat_count` - `event_code` union (axi_error / axi_timeout / etc.)

Total: 285 bits per entry. With `MAX_TRANSACTIONS=16`, the trans_table is ~4.5 Kb of registered state.

20.4 Functional Description

20.4.1 Block diagram



axi_monitor_trans_mgr block diagram

The entries live in `monitor_trans_cam`, drawn as the storage bar; the three ID lookups and the free-slot vector read it straight up. Slot Resolve maps each handshake to an entry: the CAM hit for its ID, or a newly allocated slot, since a data or response phase may arrive before its address. Write data carries no ID, so W beats go to the oldest open write entry, or to the head of the AW-order queue when `USE_WDATA_ORDER_Q=1`. Every slot computes its next payload in parallel and writes it back. An entry is freed once it is terminal and the reporter has reported its event, and the table climbs the right edge to the outputs.

20.4.2 Architecture

The split of responsibilities is the whole story here. The trans_mgr owns:

- The **WID-less write data-channel match** (state predicate over the payload, not an id match — the CAM only sees ids), resolved oldest-first.
- The **per-slot age ranks** (`r_age`) and the `pick_oldest` selector that attribute each data/response beat to the oldest matching entry.
- The per-slot **next-payload computation** (the per-phase if/else chain that says "what should slot i's bus_transaction_t look like next cycle").
- The **wants_alloc** derivation (hit suppression + the command-entry cap; see Allocation and Same-ID Tracking).
- Cleanup eligibility, event_reported feedback, the registered `active_count` pop-count, and the `filtered_mask` verdict.

The CAM owns:

- Per-slot `(valid, id, payload)` storage.
- The 3 parallel ID lookups (`addr_match_oh`, `data_match_oh`, `resp_match_oh`).
- The free-slot vector and 3-way priority-encoded alloc one-hots.

This split makes the parallel-match shape that closes 100 MHz on the xc7a100t-1 (`(* keep = "true" *)` per-bit match vectors, per-slot generate-loop storage) explicit and reusable.

20.4.3 Transaction Lifecycle

A typical AXI4 read transaction, walked phase by phase:

```
                            addr_alloc fires (free CAM slot picked)
                            valid       = 1
                            state       = TRANS_ADDR_PHASE
   cmd_valid handshake      id          = cmd_id
   (cmd_id, cmd_addr, ...)  cmd_received= cmd_ready
                            expected_beats = cmd_len + 1
                            addr_timestamp = timestamp

                            addr_update fires (the SAME entry, still
```

```
   cmd held valid       awaiting its handshake: cmd_received=0)
   across stalled cycles      cmd_received <= 1 on the handshake cycle
                              addr_timer  <= 0
                              addr_timestamp <= timestamp

                              data_update fires (oldest matching open
entry)
   data_valid handshake     data_started <= 1
   (data_id == cmd_id,        data_beat_count++
    data_last, data_resp)     state        <= TRANS_DATA_PHASE
                              if data_last:
                                data_completed <= 1
                                state        <= TRANS_COMPLETE
                              if data_resp[1]:  # RESP error
                                state        <= TRANS_ERROR
                                event_code   <= EVT_RESP_*

                              (later, reporter handles the event)
                              cleanup fires
   event_reported flag      valid <= 0       # slot returned to free
pool
   from reporter              # CAM sees the entry as free on next
cycle
```

A second AR/AW with the **same ID** allocates its **own slot** — same-ID outstanding transactions are legal AXI4 and are never merged. Data and response beats are attributed oldest-first (see the next section).

For AXI4 writes the data phase uses a state predicate match (not id), since AXI4 W has no WID. For AXI-Lite writes, the data channel CAN allocate an orphan slot if data arrives before the AW handshake.

A transaction flagged by `i_timeout_detected` (and not already terminal) is moved to `TRANS_ERROR` with an event code derived from its progress: `EVT_CMD_TIMEOUT` if the command never handshook, `EVT_RESP_TIMEOUT` if data completed but the B response is outstanding, else `EVT_DATA_TIMEOUT`. This makes timed-out entries cleanup-eligible instead of leaking their slot.

20.4.4 Allocation and Same-ID Tracking

20.4.4.1 Separate slot per same-ID transaction

The pre-CAM design merged a new command into any existing entry with the same ID. That was a **defect** (issue #41 defect 1, fixed): AXI4 permits several outstanding transactions with the same ID, so a second AR/AW must get its own slot. Allocation is now suppressed only for the one legitimate case — a command held valid across several stalled cycles must not allocate a fresh slot every cycle. The actual derivations:

```
// A "pending" entry is the same-ID entry still awaiting its handshake
// (cmd_received=0) and not being freed this cycle.
addr_hit_any     = |w_addr_pend_oh;
addr_wants_alloc = cmd_valid && !addr_hit_any && w_cmd_headroom;

// Reads: allocate an orphan when a data beat matches no entry.
data_wants_alloc = data_valid && data_ready && !data_hit_any;
// IS_READ
data_wants_alloc = data_valid && data_ready && !IS_AXI && !
data_hit_any; // write
resp_wants_alloc = !IS_READ && resp_valid && resp_ready && !
resp_hit_any;
```

`w_cmd_headroom` is the command-entry cap: command-originated entries may occupy at most `MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS)` slots (reserve = 4 for tables of 16+, 0 below). A command seen while the cap is reached is simply not tracked until a command entry retires. This cap is one half of the saturation-recovery contract; the other half is the `block_ready` reopen threshold in `axi_monitor_base` — see the canonical description in axi_monitor_base.

20.4.4.2 Oldest-first (rank-based) attribution

Because same-ID entries coexist, an incoming beat can match several slots. AXI4 orders same-ID data/responses by issue order, so each beat belongs to the **oldest** matching entry that is still open. Slot index carries no ordering information (allocation takes the lowest free index, so a recycled low slot can be younger than a live high slot); instead each slot carries a dense rank `r_age[i]` (0 = oldest live entry), and a `pick_oldest()` function selects the lowest-rank candidate. Two tiers: prefer the oldest match whose data/response phase is still open; if every match has already closed, a stray **last** beat falls back to the oldest match (re-running completion — harmless, and it reconstructs untracked bursts under ID oversubscription), while a stray **non-last** beat is absorbed with no update at all (the pre-fix behavior re-opened a terminal entry into an unclosable state — the production saturation-wedge mechanism, guarded by the in-RTL formal property `ap_no_reopened_complete`).

20.4.4.3 Write-data attribution by AWID FIFO (USE_WDATA_ORDER_Q)

AXI4 W beats carry no WID, so the entry a beat belongs to cannot come from the beat. With `USE_WDATA_ORDER_Q=1` the manager records the **AWID** on each AW handshake and pops it on W-LAST; the head AWID keys the write-data candidate set, and `pick_oldest()` then resolves among that ID's entries.

This is required whenever the table is banked. `pick_oldest()` compares **same-bank only** — the cross-bank comparators are constant-folded away at elaboration — and that is sound precisely because candidates are ID-matched and all of an ID's entries live in one bank. The legacy write path built its candidates from a state predicate over the *whole* table, which is not ID-matched, so at `NUM_BANKS=B` the select returned one winner **per bank** and a single W beat advanced up to B

transactions. NUM_BANKS > 1 on a write monitor therefore refuses to elaborate without this parameter ($error), because the combination has no correct fallback.

Bus requirement — W must not lead AW. Attribution is by AW order, so the AW naming a beat must already have been seen. Same-cycle AW+W is supported (below); W strictly *before* its AW has no AWID to attribute it to and is treated as a stray. This is the restriction commercial VIPs commonly impose.

Regression: val/amba/test_axi_monitor_trans_mgr_wr_bank.py (attribution at NUM_BANKS 1 and 4, plus the refusal of the illegal combination).

20.4.4.4 Same-cycle AW+W bypass (write monitors)

The WID-less write predicate runs over registered entries, so a W beat arriving in the **same cycle** as its AW used to match nothing and be silently lost (the entry then waited for data forever and the B response fabricated an EVT_PROTOCOL error on legal traffic — routine for single-beat writes from skid-buffered masters). Fixed in 95c9490a: when no registered entry matches, the beat binds combinationally to the slot the command path touches this cycle — either the slot being allocated for this AW (via a local mirror of the CAM's allocation pick, cross-checked by the ap_bypass_alloc_mirror formal property) or the pending same-ID entry, state-qualified to TRANS_ADDR_PHASE so orphan adoption keeps its legacy behavior. An entry whose AW has not yet handshaked is held in TRANS_ADDR_PHASE even after taking the beat, preserving address-phase timeout coverage (formal property ap_wr_data_phase_has_cmd).

20.4.4.5 Early write data (W before its AW, write monitors)

AXI4 lets a master present W beats before the AW they belong to, and a W beat carries no ID. Until 2026-09-28 a beat that found no entry awaiting data was dropped outright (data_wants_alloc is !IS_AXI-gated for writes, so no orphan was allocated either); the AW then waited for data that had already passed and its B fabricated an EVT_PROTOCOL, losing the completion (amba BUG-037; rtl/amba/KNOWN_ISSUES/axi_monitor_early_write_data_dropped.md). Now a small FIFO (EARLY_BURSTS = 4) holds completed early bursts as beat counts, plus the one burst still open, and each write allocation absorbs the oldest as its data phase. AXI4 write data is in AW order, so the next AW is the owner by definition. While early data is pending the same-cycle bypass is off (that beat is a later transaction's and is queued too), and the bypass never binds to a pending AW whose data phase is already complete. Beyond four completed bursts ahead of their addresses the excess is still dropped.

20.4.5 Address filtering

ADDR_FILTER_ENABLE adds an address window, cfg_addr_filter_low..cfg_addr_filter_high inclusive, gated by cfg_addr_filter_enable. A command whose address falls **outside** the window sets that slot's bit in filtered_mask, and the reporter suppresses every packet for it.

The verdict is taken **at allocation** and held for the life of the slot. It has to be: the address appears only on the command channel, so once the entry exists there is nothing left to re-test it against.

The filter suppresses packets; it does **not** refuse admission. That looks like the weaker choice and is the correct one. Address exists only on the command channel, so refusing to allocate would leave that transaction's data and response beats arriving with no entry to match. Unmatched data is deliberately ungated — a monitor must never stall returning data — so those beats would be reported as orphan errors, and a filter meant to cut packet traffic would increase it.

Cost of deciding here rather than at admission is `MAX_TRANSACTIONS` flops (16 at the default). Admission filtering would need a counter per *possible* ID — `2**ID_WIDTH` counters, 1280 flops at `ID_WIDTH=8`/`MAX_TRANSACTIONS=16` — because one ID can hold several outstanding transactions whose addresses straddle the window, so a flag per ID is not enough.

Because the verdict is latched, widening the window at runtime does not un-filter entries already in the table; only transactions admitted afterwards see the new window. That is what makes the window safe to reprogram live — an entry's fate is fixed when it is admitted, so the retire accounting cannot be corrupted underneath it.

The consumer must retire filtered entries as well as silence them. A slot is freed only once `event_reported` is set, and the only producer of that flag is an accepted monbus write — so suppressing the packet without retiring the entry leaks the slot. `axi_monitor_reporter` handles this by folding filtered terminal entries into its auto-retire path.

20.5 Timing Characteristics

Everything downstream of the trans_mgr is registered, so the externally visible latencies are short and fixed:

Metric	Value	Notes
Throughput	1 transaction-event/cycle	Per-phase handshake; up to 3 phases (addr / data / resp) can act per cycle
`trans_table` latency	1 cycle	Output is registered
`active_count` latency	1 cycle	Registered pop-count of CAM occupancy
Resource	~5 Kb storage	16 × 285-bit struct, in the CAM

20.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
axi_monitor_trans_mgr #(
    .MAX_TRANSACTIONS      (16),
    .ADDR_WIDTH            (32),
    .ID_WIDTH              (8),
    .IS_READ               (1'b1),
    .IS_AXI                (1'b1),
    .ENABLE_PERF_PACKETS   (1'b0),
    .USE_WDATA_ORDER_Q     (1'b0),
    .NUM_BANKS             (1),
    .AW                    (ADDR_WIDTH),
    .IW                    (ID_WIDTH)
) u_axi_monitor_trans_mgr (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    .clear                 (clear),
    .cmd_valid             (cmd_valid),
    .cmd_ready             (cmd_ready),
    .cmd_id                (cmd_id),
    .cmd_addr              (cmd_addr),
    .cmd_len               (cmd_len),
    .cmd_size              (cmd_size),
    .cmd_burst             (cmd_burst),
    .data_valid            (data_valid),
    .data_ready            (data_ready),
    .data_id               (data_id),
    .data_last             (data_last),
    .data_resp             (data_resp),
    .resp_valid            (resp_valid),
    .resp_ready            (resp_ready),
    .resp_id               (resp_id),
    .resp_code             (resp_code),
    .timestamp             (timestamp),
    .i_event_reported_flags(i_event_reported_flags),
    .i_timeout_detected    (i_timeout_detected),
    .cfg_addr_filter_enable(cfg_addr_filter_enable),
    .cfg_addr_filter_low   (cfg_addr_filter_low),
    .cfg_addr_filter_high  (cfg_addr_filter_high),
    .filtered_mask         (filtered_mask),
    .trans_table           (trans_table),
    .active_count          (active_count)
);
```

20.7 Design Notes

20.7.1 Synthesis Notes

The CAM-backed revision preserves the 2026-04-23 WNS fix:

Construct	Rationale
Per-slot `always_comb` for next-payload (in a generate loop)	N independent small cones; synth cannot fuse them across slots
Per-slot CAM storage via `monitor_trans_cam` (generate-loop `always_ff`)	Same property at the registered storage layer
`(* keep = "true" *)` on CAM match vectors	Prevents Vivado from fusing match-result usage into the update cones, which would re-introduce the 12-LUT-level WNS issue
`active_count` = registered pop-count of `cam_entry_valid`	Derived directly from live CAM occupancy, **not** an alloc-minus-cleanup accumulator. The former accumulator could desync from true occupancy and underflow to 0xFF under legal AXI (found by the SymbiYosys proof — see `rtl/amba/KNOWN_ISSUES/axi_monitor_active_count_underflow.md`); a registered pop-count is structurally bounded to [0, N]. The valid bits are already registers, so the adder tree sits cleanly between flops; `active_count` lags occupancy by 1 cycle, which the `block_ready` margin absorbs.
Synchronous `clear` zeroes `active_count` alongside the CAM	`clear` invalidates every CAM slot and zeroes the registered count and age ranks on the same edge, so an empty table is never published with a stale nonzero `active_count`.

The combined effect is that no signal in the trans_mgr has more than ~6 LUT levels between flops at typical configurations — closes 100 MHz on xc7a100t-1 with margin.

20.7.2 Formal Properties

The saturation-recovery and same-cycle-bypass contracts are encoded as in-RTL formal properties (under `ifdef FORMAL`, flattened into the SymbiYosys proofs and mutation-checked):

Property	Guarantee
`ap_no_reopened_complete`	No read entry is ever in `TRANS_DATA_PHASE` with `data_completed` set — the unclosable "poison" state behind the production saturation wedge is unreachable
`ap_wr_data_phase_has_cmd`	A write entry in `TRANS_DATA_PHASE` always has its command handshake recorded (the same-cycle bypass cannot create timeout-coverage holes)
`ap_bypass_alloc_mirror`	The bypass's local allocation mirror always agrees with the CAM's own pick
`ap_cmd_entry_cap`	Command-originated entries never exceed `N - cmd_entry_reserve(N)` (vacuous by design when the reserve is 0, i.e. `N < 16`)

20.8 Related Modules

Module	Role
`monitor_trans_cam`	Per-slot keying + storage with 3 lookup ports and alloc priority encoder
`axi_monitor_base`	Top-level monitor wrapper that instantiates trans_mgr + timer + reporter
`axi_monitor_reporter`	Scans `trans_table`, gated by `filtered_mask`, to generate monbus packets
`axi_monitor_timeout`	Watches the per-phase timers in `trans_table` for timeout events

Further reading:

- **Monitor Architecture:** `docs/markdown/rtl-amba/overview.md`
- **Monitor Configuration Guide:** `axi_monitor_base.md`
- **Packet Format Specification:** `monitor_package_spec.md`
- **Saturation-recovery contract:** `axi_monitor_base.md` and `monitor_common_pkg::cmd_entry_reserve()`

20.9 Testing

Test	Coverage
`val/amba/test_axi_monitor_trans_mgr.py`	Directed trans_mgr suite: same-ID slot separation, oldest-first attribution, stray-beat absorption, `phase_saturation_recovers` (block_ready reopen), timeout-to-terminal transitions
`val/amba/test_monitor_trans_cam.py`	The CAM sub-module in isolation
`val/amba/test_axi4_master_rd_mon.py` and all *_mon* tests	End-to-end through the full monitor stack

Run the directed suite to validate trans_mgr changes:

```
pytest val/amba/test_axi_monitor_trans_mgr.py -v
```

20.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. — §A5.1 transaction identifiers, §A5.3 request ordering, §A5.5 and §A5.6 response ordering: the lifecycle and same-ID ordering this table tracks.
- Pagiamtzis, K., Sheikholeslami, A. “Content-Addressable Memory (CAM) Circuits and Architectures: A Tutorial and Survey.” *IEEE J. Solid-State*

Circuits, 41(3):712-727, Mar. 2006. DOI: 10.1109/JSSC.2005.864128. — for the CAM structure (`monitor_trans_cam`).

- Goossens, K., Vermeulen, B., van Steeden, R., Bennebroek, M. “Transaction-Based Communication-Centric Debug.” *Proc. 1st Int. Symp. Networks-on-Chip (NOCS 2007)*, pp. 95-106, May 2007. DOI: 10.1109/NOCS.2007.46. — related work: transaction-level monitoring.
- Ciordas, C., Goossens, K., Basten, T., Rădulescu, A., Boon, A. “Transaction Monitoring in Networks on Chip: The On-Chip Run-Time Perspective.” *Proc. IEEE Int. Symp. Industrial Embedded Systems (IES 2006)*, pp. 1-10, Oct. 2006. DOI: 10.1109/IES.2006.357464. — related work: reconstructing transactions at run time in hardware.
- Abramovici, M., Bradley, P., Dwarakanath, K., Levin, P., Memmi, G., Miller, D. “A Reconfigurable Design-for-Debug Infrastructure for SoCs.” *Proc. 43rd Design Automation Conf. (DAC 2006)*, pp. 7-12, July 2006. DOI: 10.1145/1146909.1146916. — related work: transaction identifiers in a debug fabric.
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/common/references/ContentAddressableMemory__Pagiamtzis2006.pdf`)

20.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

21 AXIS Monitor Lite

Module: `axis_monitor_lite.sv` **Location:** `rtl/amba/ocelus/axis_monitor_lite.sv`
Category: Monitor Infrastructure **Status:** Built, formally checked and verified in simulation, with the eight `axis{4,5}_{master,slave}_monlite[_cg]` wrappers (rtl/amba/monitors TASK-003, closed 2026-09-27). Standalone area from yosys only; no Vivado fixture build yet.

Plane: protocol plane (lite AXI-Stream monitor)

21.1 Overview

The first AXI4-Stream monitor in `rtl/amba/ocelus`. The `_mon` family and its `_monlite` twin cover AXI4, AXI5 and both AXI-Lite generations because `axi_monitor_base` and `axi_monitor_lite` are transaction trackers: a command handshake opens a table entry, data beats and a response close it. A stream has no command, no response and no outstanding-transaction notion, so there was nothing for those cores to track and nothing for a wrapper to wrap. Until this block the only stream-side measurement was `axis_bus_meter` (counters, no packets), and the only AXIS monbus emitter anywhere was the per-port tap inside `axis4_intf_observer`, written for that block and not reusable.

What a stream does have is packets (TLAST-delimited runs of beats), stalls (TVALID held without TREADY), bubbles (TVALID withdrawn inside a packet), and a per-beat TID and TDEST that may change under a packet. This block watches those and emits them as the AXIS packet classes the monbus package has always defined – Credit, Channel and Stream – plus the shared Error, Timeout and Completion classes. Same 128-bit `monitor_packet_t`, same side-band timestamp, same monbus handshake, same UNIT/AGENT ids: the arbiter, group, tally and host tooling see one more producer and nothing new.

The event set and payload layouts are the observer's tap, lifted unchanged so that the observer can later instantiate this core instead of carrying its own copy. What the block adds over the tap is the lite's delivery contract: a 4-deep output queue that takes two events a cycle instead of a one-packet hold register, drop-and-count with the count reported as `Error/EVENT_DROPPED`, the frequency-invariant microsecond tick for the timeouts, the lite's cfg pin set, and `clear`.

This is a tap. Nothing here drives or gates the stream; `axis_tready` is an input like `axis_tvalid`, and a packet the monbus will not take is dropped and counted.

21.2 Parameters

Parameter	Default	Meaning
`UNIT_ID`	`8'h09`	unit field of every packet
`AGENT_ID`	`16'h0064`	agent field of every packet

Parameter	Default	Meaning
DATA_WIDTH	32	sizes axis_tstrb only; the monitor never sees TDATA
ID_WIDTH	8	TID width; 0 is legal (a 1-bit tie-off)
DEST_WIDTH	4	TDEST width; 0 is legal
AGE_WIDTH	16	microsecond age counter (cfg_timeout_cnt is 16 bits)
OUT_DEPTH	4	output queue depth, any depth >= 1
CFI_MIN_FREQ_MHZ, CFI_MAX_FREQ_MHZ, CFI_NUM_FREQ_ENTRIES, CFI_FREQ_STRATEGY	5, 220, 16, 0	the counter_freq_invariant table behind the microsecond tick, as on axi_monitor_lite

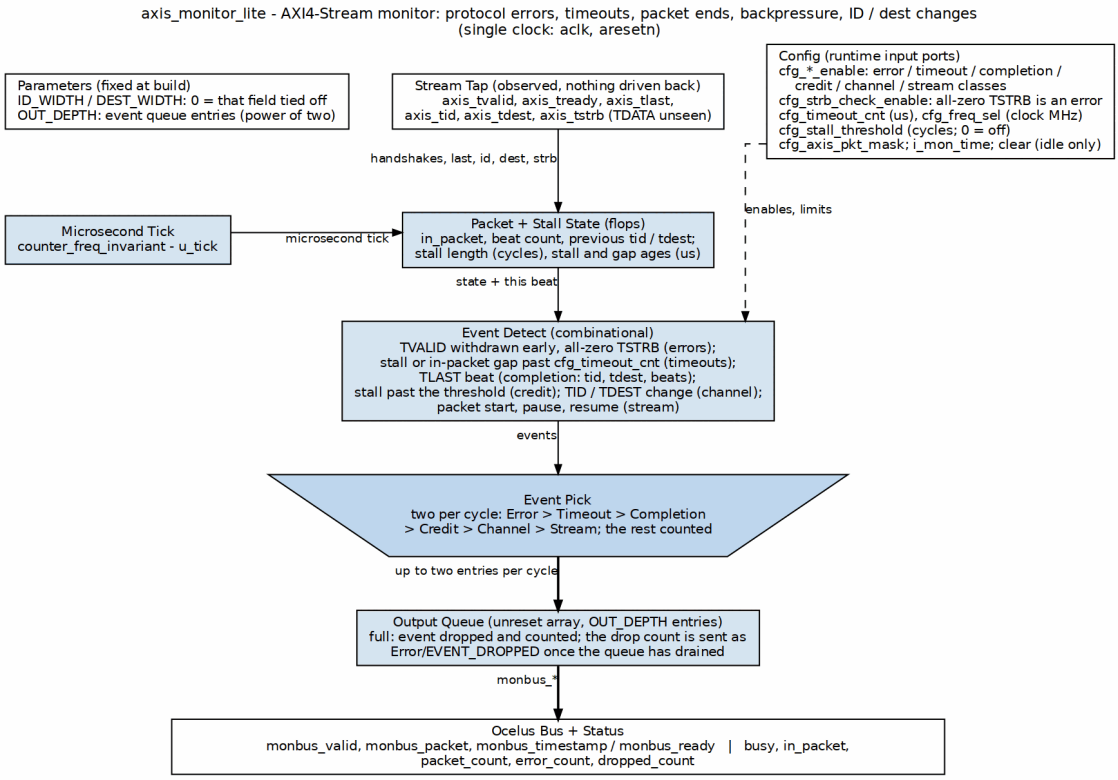
21.3 Ports

Group	Ports	Notes
Clock, reset, clear	aclk, aresetn, clear	clear is synchronous and legal only while idle: no packet open, no stall running, nothing queued (the same rule as axi_monitor_lite, amba ISSUE-002)
Time	i_mon_time	side-band timestamp, presented with every packet
Stream tap	axis_tvalid, axis_tready, axis_tlast, axis_tid, axis_tdest, axis_tstrb	all inputs
Configuration	cfg_freq_sel, cfg_timeout_cnt,	cfg_timeout_cnt is microseconds, 16'hFFFF

Group	Ports	Notes
	cfg_error_enable, cfg_timeout_enable, cfg_compl_enable, cfg_credit_enable, cfg_channel_enable, cfg_stream_enable, cfg_strb_check_enable, cfg_stall_threshold, cfg_axis_pkt_mask	= never; cfg_stall_threshold is cycles, 0 = off; the mask drops a packet type at the source
Monitor bus	monbus_valid, monbus_ready, monbus_packet, monbus_timestamp	standard monbus handshake
Status	busy, in_packet, packet_count, error_count, dropped_count	error_count excludes drop reports; dropped_count is since the last report

21.4 Functional Description

21.4.1 Block diagram



axis_monitor_lite block diagram

The monitor only observes the stream tap; nothing goes back onto the stream. The packet and stall state tracks whether a packet is open, its beat count, the previous TID and TDEST, and how long a stall or an in-packet gap has lasted (in microseconds from the tick). Event detect turns that into the AXIS event classes. The event pick takes up to two per cycle by priority, and the output queue holds them for the bus, counting and later reporting anything it had to drop.

21.4.2 Events

Priority when several fire in one cycle: Error > Timeout > Completion > Credit > Channel > Stream. The two highest go into the queue; anything beyond that, and anything offered to a full queue, is counted, never silently discarded.

Class / code	Fires when	`event_data[63:0]`
Error / `VALID_TIMING`	TVALID withdrawn before the handshake (the AXIS rule: once asserted, TVALID holds	`{stall_cycles[31:0], packets[31:0]}`

Class / code	Fires when	event_data[63:0]
	until TREADY)	
Error / STRB_INVALID	an accepted beat whose TSTRB is all zero, with cfg_strb_check_enable (legal AXIS, a bug on a DMA stream)	{beats_now[31:0], packets[31:0]}
Timeout / HANDSHAKE	TVALID held without TREADY for cfg_timeout_cnt microseconds; once per stall	{stall_cycles[31:0], age_us[15:0], cfg[15:0]}
Timeout / PACKET	inside a packet, no beat accepted for cfg_timeout_cnt microseconds; once per gap	{beats[31:0], age_us[15:0], cfg[15:0]}
Completion / STREAM_END	the TLAST beat	{tid[15:0], tdest[15:0], beats[31:0]}
Credit / BACKPRESSURE	a stall of cfg_stall_threshold cycles; once per stall	{stall_cycles[31:0], cfg_stall_threshold[31:0]}
Channel / ID_CHANGE, DEST_CHANGE	TID or TDEST differs from the previous accepted beat inside a packet	{old[15:0], new[15:0], beats_now[31:0]}
Stream / START	the first beat of a packet	{tid[15:0], tdest[15:0], packets[31:0]}
Stream / PAUSE, RESUME	TVALID low inside a packet, and back again	{beats[31:0], packets[31:0]}
Error / EVENT_DROPPED	the drop count, when the queue has drained and nothing else wants it	dropped_count

channel_id carries the beat's TID on every packet.

Two rules differ from the observer's tap, both learned from the exact-packet tests:

- **Two events a cycle.** On a stream, events coincide: the beat that ends a bubble is a RESUME and, if it carries TLAST, a STREAM_END; the first beat after a pause may also change TID; a one-beat packet is a START and an END in the same cycle. The observer's one-per-cycle pick dropped and counted the loser every time, so its drop count reported ordinary traffic. Here the two highest-priority candidates are both queued (a second write port; the queue is flops rather than LUTRAM at this depth), and the drop count means what it says. Forcing the second push off fails the suite.
- **The drop report goes only into an empty queue.** The AXI lite's rule is "when the queue has room". Here a report pushed into a queue that is merely not full takes the slot the next live event needs while the bus is congested, which is exactly when drops happen, and each idle cycle adds another report. So the report waits for the queue to drain; the count keeps accumulating, saturating, until then.

21.4.3 Timing

Age is counted in microsecond tick edges since the stall or gap began, so a timeout of `cfg_timeout_cnt = N` fires after between `N-1` and `N` full tick periods. The tick is entry `cfg_freq_sel` of the `counter_freq_invariant` table; a wrapper that knows its clock sets `CFI_MIN_FREQ_MHZ = CFI_MAX_FREQ_MHZ = ACLK_MHZ` and entry 0 is then one microsecond of that clock.

Events are decided in the cycle they happen and registered, with the values their payloads need, before the pick: the two picks, the payload muxes, the queue writes and the drop count all run from those flops a cycle later, so a packet leaves the block one cycle after its cause, plus queue occupancy. Until 2026-09-29 all of that happened in the deciding cycle, which chained the age and stall compares into the pick, the payload mux and the drop-count adder – 16 to 19 logic levels, 10.4 ns on an Artix-7 -1 – and the block missed 10 ns on that part (rtl/amba/monitors ISSUE-003). The AXI lite made the same split the day before.

21.4.4 Drop and count

Every candidate the pick cannot queue in a cycle – the losers of the priority pick and anything offered to a full queue – adds to `dropped_count`. When the queue has drained and no event fires, one `Error/EVENT_DROPPED` packet carries the count and zeroes it. The count saturates rather than wrapping.

21.5 Usage Examples

```
axis_monitor_lite #(
    .UNIT_ID          (8'h09),
```

```
    .AGENT_ID         (16'h0064),
    .DATA_WIDTH       (64),
    .ID_WIDTH         (8),
    .DEST_WIDTH       (4),
    .CFI_MIN_FREQ_MHZ (100),
    .CFI_MAX_FREQ_MHZ (100)
) u_axis_mon (
    .aclk (aclk), .aresetn (aresetn), .clear (1'b0), .i_mon_time
(mon_time),
    // tap: both directions are inputs
    .axis_tvalid (m_axis_tvalid), .axis_tready (m_axis_tready),
    .axis_tlast  (m_axis_tlast),  .axis_tid    (m_axis_tid),
    .axis_tdest  (m_axis_tdest),  .axis_tstrb  (m_axis_tstrb),
    .cfg_freq_sel (4'd0), .cfg_timeout_cnt (16'd10),      // 10 us
    .cfg_error_enable (1'b1), .cfg_timeout_enable
(1'b1), .cfg_compl_enable (1'b1),
    .cfg_credit_enable (1'b1), .cfg_channel_enable
(1'b1), .cfg_stream_enable (1'b0),
    .cfg_strb_check_enable (1'b0), .cfg_stall_threshold
(32'd64), .cfg_axis_pkt_mask (16'h0),
    .monbus_valid (mon_valid), .monbus_ready (mon_ready),
    .monbus_packet (mon_packet), .monbus_timestamp (mon_ts),
    .busy (), .in_packet (), .packet_count (), .error_count
(), .dropped_count ()
);
```

Stream packets (`START`, `PAUSE`, `RESUME`) fire on every packet and every bubble; on a busy link leave `cfg_stream_enable` low or mask the type, and read the counters instead.

21.6 Design Notes

- **One open packet, not a per-TID table.** A stream that interleaves several TIDs under one TLAST is reported as `Channel/ID_CHANGE`, not tracked per ID. That is the lite's trade and the observer's: the consumers in this repo (STREAM, RAPIDS data ports) do not interleave.
- **Everything is a handshake or a level.** No counters per packet beyond the 32-bit beat count, one shared microsecond counter, two stamps. The pick is shallow enough not to need the lite's registered event stage.
- **Queue = two interleaved `gaxi_fifo_sync` banks.** `gaxi_fifo_sync` has one write port and the pick queues up to two events a cycle, so entry k goes to bank k mod 2 and reads alternate between the banks: the same entries in the same order as a two-write queue. A registered occupancy count is the only capacity gate, so capacity is exactly `OUT_DEPTH` at any depth (each bank holds ceil(OUT_DEPTH/2), at least the 2 gaxi needs). Storage is unreset and

the read is a mux, as before. The count cannot come from gaxi: in mux mode its `count` is combinational from the next-state pointers, and gating writes on it would close a loop.

- **Standalone area** (yosys generic `synth` on the sv2v-flattened core, NAND-2 equivalents from `bin/yosys_to_nand_equiv.py`, default parameters): about 45 k NAND2 and 539 flops, against the AXI lite's 91 k and 1,209 in the same flow. The second queue push accounts for roughly 13 k of that; the one-push version measured 32 k. No Vivado numbers yet.

21.7 Related Modules

- `axi_monitor_lite` – the AXI transaction monitor whose delivery contract this block shares
- `axis4_intf_observer` (projects/components/utility-ip/misc) – the tap this block's event set comes from; a candidate to instantiate this core
- `axis{4,5}_{master,slave}_monlite` and `_monlite_cg` – the eight wrappers that tap this core onto the stream endpoints (axi_monitor_lite_wrappers, Table 2)
- `axis_bus_meter` – stream throughput and backpressure counters, the perf path (this block emits no perf packets)
- `monitor_common_pkg`, `monitor_amba4_pkg` – packet format and the AXIS event codes

21.8 Testing

Formal (`formal/amba/axis_monitor_lite/`, sv2v-flattened like the AXI lite, `make prove cover`): BMC to depth 24 over free stream, monbus and cfg inputs. Asserted: a presented packet holds until taken; every packet carries the AXIS protocol code and this unit/agent; `packet_count` moves by exactly one on a TLAST handshake and not otherwise; `in_packet` follows the TLAST run; `clear` zeroes every counter; `dropped_count` only ever falls to zero (a report or a clear); `busy` whenever a packet is queued or open. Cover to depth 40 reaches all seven packet classes the block emits, EVENT_DROPPED included.

Simulation. `val/amba/monitor-lite/test_axis_monitor_lite.py` drives the fixture `tb_axis_monitor_lite.sv` (the core tapping a stream between the framework AXIS master and slave BFMs) through `AxisMonitorLiteTB`. Each phase asserts the class, code and payload of every packet and that nothing else came out: packets of 1 to 16 beats (START and STREAM_END on every one), TID/TDEST change under a packet, source bubbles (PAUSE/RESUME per gap), a 300-cycle held tready (one BACKPRESSURE at 50 cycles, one HANDSHAKE timeout at 2 us, then the completion), a 300-cycle gap inside a packet (one PACKET timeout, no HANDSHAKE), a zero-strobe beat armed and unarmed, TVALID withdrawn (pin-driven, the one protocol violation a BFM

cannot produce), the type mask, a held monbus (delivered + reported dropped == issued), and `clear`. The same class drives the eight wrappers end to end (see the wrappers page): through an endpoint a phase waits for the skid to drain before judging, and on a slave wrapper the stall phase fills the skid first, since a single beat never stalls an upstream tap.

Cells: core GATE 1 / FUNC 6 / FULL 12; wrappers 9 each at FULL. 84/84 at FULL from a clean build, 2026-09-27. Three mutations of the RTL were caught: the credit firing every stall cycle (stall phase), the change detector comparing against the packet's first beat (channel phase), and the second queue push disabled (packets phase, START lost on one-beat packets).

21.9 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI-Stream Protocol Specification*, ARM IHI 0051B, Apr. 2021. developer.arm.com/documentation/ihi0051/b. – §2.2 handshake signaling, §2.6 packet boundaries (TLAST), §4.2 transfer ordering.
- IEEE-ISTO 5001-2003. *The Nexus 5001 Forum Standard for a Global Embedded Processor Debug Interface*, Version 2.0, IEEE Industry Standards and Technology Organization, Dec. 2003 (defining non-peer standard). – §4.4.3 and §4.5.3: overrun reported rather than stalling, comparable to the dropped-event count.
- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. "An Event-Based Monitoring Service for Networks on Chip." *ACM Trans. Design Automation of Electronic Systems*, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126. – related work: event-based hardware monitoring.
- Indexed in `rtl/amba/references/README.md` (local PDF: `DebugTraceInterfaceStandard__IeeeIsto2003.pdf`)

21.10 Navigation

- rtl-amba index
- axi_monitor_lite
- axi_monitor_lite_wrappers

22 CXS Monitor Lite

Module: cxs_monitor_lite.sv **Location:** rtl/amba/ocelus/ **Wrappers:** cxs_master_monlite, cxs_slave_monlite (rtl/amba/cxs/) **Status:** New (2026-10-15)

Plane: protocol plane (lite CXS monitor)

22.1 Overview

cxs_monitor_lite is the Ocelus monitor for a CXS link (Arm IHI 0079, Issues A-D). It works in the lite discipline of axis_monitor_lite:

- Events are decided and registered, and up to two are queued per cycle.
- The output queue is 4 deep: two interleaved gaxi_fifo_sync banks (unreset storage, mux read), so two entries can be queued per cycle at any depth.
- Events the bus cannot take are dropped and counted, and the count is reported as Error/EVENT_DROPPED.
- Timeouts are in microseconds, independent of the clock frequency.

A separate core is needed because CXS is not a valid/ready stream. Flits move on credits, packets are delimited inside flits by CXSCNTL, and a four-phase ACTIVEREQ/ACTIVEACK handshake brings the link up and down. Every packet is tagged PROTOCOL_CXS (4'h6), with event codes from monitor_cxs_pkg.

The block is a **tap**: every port is an input and it drives nothing. Following the house rule for monitors, it never sits on the external link pins. It watches two things:

- the endpoint's **FUB handshake** (fub_cxs_valid/ready with the flit's CXSCNTL and stream fields), for packets, streams and backpressure;
- the endpoint's **registered status outputs** (link_state, credits / credits_outstanding, credit_starved, protocol_error, parity_error), for everything that exists only on the link.

22.2 Events

Priority when several fire in one cycle runs from the top of the table down. Each timeout and Credit event reports once per episode.

Class	Code	When	event_data
Error	CXS_ERR_PROTOCOL (0)	protocol_error rose	{credits[16], 14'b0, link_state[2], flits[32]}
Error	CXS_ERR_PARITY (1)	parity_error rose	as PROTOCOL
Error	CXS_ERR_ENDERROR (2)	a flit ended packets with ENDERROR	{enderrs[8], ends[8], 16'b0, packets[32]}
Timeout	CXS_TIMEOUT_CREDIT (0)	credit_starved for cfg_timeout_cnt us	{starve_cycles[32], age_us[16], cfg[16]}
Timeout	CXS_TIMEOUT_LINK (1)	ACTIVATE or DEACTIVATE for cfg_timeout_cnt us	{credits[16], 14'b0, link_state[2], age_us[16], cfg[16]}
Timeout	CXS_TIMEOUT_PACKET (2)	a packet open with no flit for cfg_timeout_cnt us	{flits_in_packet[32], age_us[16], cfg[16]}
Completion	CXS_COMPL_PACKET_END (0)	a flit that ends one or more packets (CXS-Lite: every flit)	{prcltype[8], srcid[8], tgtid[8], ends[8], packets[32]}
Credit	CXS_CREDIT_STARVED (0)	credit_starved for cfg_stall_threshold cycles	{cycles[32], threshold[32]}
Credit	CXS_CREDIT_BACKPRESSURE (1)	FUB valid without ready for cfg_stall_threshold cycles	{cycles[32], threshold[32]}
Channel	CXS_CHAN_STREAM_CHANGE (0)	CXSPRCLTYPE/ SRCID/TGTID differ from the previous flit	{prev_stream[24], 8'b0, stream[24], 8'b0}

Class	Code	When	event_data
		(2.5.2)	
Stream	CXS_STREAM_LINK_ACTIVATE / LINK_UP / LINK_DEACTIVATE / LINK_DOWN (0-3)	the link entered ACTIVATE / RUN / DEACTIVATE / STOP	{credits[16], 14'b0, prev_link_state[2], flits[32]}
Error	CXS_ERR_EVENT_DROPPED (0xE)	the drop count, sent into an empty queue	count[16]

packets is the running total of packets ended, including this flit. channel_id carries {prcltype[2:0], tgtid[5:0]} on the flit events. A stream is the triple {CXSPRCLTYPE, CXSSRCID, CXSTGTID}, each field zero-extended to 8 bits.

credit_starved comes from the endpoint and means something different on each side:

- **Transmitter:** a flit is waiting in RUN with no credit to send it.
- **Receiver:** the link is in RUN, no credit is outstanding, and none can be granted because the FIFO is full of flits the FUB has not taken. In that case the consumer is the bottleneck.

22.3 Parameters

Parameter	Default	Description
UNIT_ID / AGENT_ID	8'h0A / 16'h00C0	Packet identity
CXS_DATA_WIDTH, CXS_MAX_PKT_PER_FLIT, CXS_START_ALIGN	256, 2, 16	The tapped interface; sizes the CXSCNTL decode (cxs_pkg)
CXS_SRCID_WIDTH, CXS_TGTID_WIDTH, CXS_MAX_CREDIT	0, 0, 15	Port widths of the stream fields and the credit count
OUT_DEPTH	4	Output queue entries, any depth >= 1 (two interleaved gaxi_fifo_sync banks)
CFI_*	5..220 MHz, 16 entries	Microsecond tick (counter_freq_invarian

Parameter	Default	Description
		t)

22.4 Ports

Port	Dir	Description
aclk, aresetn, clear	in	clear is synchronous: it forgets open packets and episodes and zeroes the counters. Legal only while idle
i_mon_time	in	Side-band time, copied to monbus_timestamp
fub_valid, fub_ready, fub_cntl, fub_prcltype, fub_srcid, fub_tgtid	in	The FUB handshake
link_state, credits, credit_starved, protocol_error, parity_error	in	The endpoint's registered status
cfg_freq_sel, cfg_timeout_cnt	in	Microsecond tick select; timeout in us (16'hFFFF = never)
cfg_{error,timeout,compl ,credit,channel,stream}_e nable	in	Class enables
cfg_stall_threshold	in	Credit-event threshold in cycles; 0 = Credit events off
cfg_cxs_pkt_mask	in	bit[packet_type] = 1 drops that class
monbus_valid/ready/ packet/timestamp	out/in	Ocelus bus
busy, packet_count, flit_count, error_count, dropped_count	out	Status

22.5 Wrappers

`cxs_master_monlite` and `cxs_slave_monlite` declare every parameter and port of their endpoint verbatim (generated from it) and pass them through unchanged. They add the Ocelus parameters (`USE_MONITOR`, `UNIT_ID`, `AGENT_ID`, `OUT_DEPTH`, `ACLK_MHZ`, `CFI_*`) and the lite control set: `cam_clear`, `cfg_monitor_enable` (0 holds the monitor clear), the class enables, `cfg_timeout_cycles` (microseconds, **0** = **never**), `cfg_freq_sel`, `cfg_cxs_pkt_mask`, `cfg_stall_threshold` and `i_mon_time`. They have no `_cg` variants, because clock gating is an FPGA-only concern here. See the wrappers page (Table 4).

22.6 Design Notes

- **Drops in normal traffic are legal, and counted.** One flit can raise PACKET_END, ENDERROR and STREAM_CHANGE in the same cycle. Back-to-back flits that end packets produce events as fast as the monbus can drain them, so a full-rate link with every class enabled will drop events. The lite contract counts every drop and reports it, so delivered + reported always equals issued. Mask Completion (`cfg_cxs_pkt_mask`) when only errors and link events are wanted.
- **Errors are edges of sticky flags.** The endpoint's `protocol_error` and `parity_error` are sticky, so each is reported once, when it rises. Reset or a rebuild re-arms them.
- **No formal proof yet.** `axis_monitor_lite` has an sv2v-flattened SymbiYosys harness; this block has simulation coverage only, including the seven mutations below.

22.7 Testing

`val/amba/monitor-lite/test_cxs_{master,slave}_monlite.py` drive the endpoint traffic and its flit scoreboard (`CXSMasterTB` / `CXSSlaveTB`) with a `MonbusSlave` on the Ocelus bus (`bin/TBClasses/cxs/cxs_monlite_tb.py`). They run the same configuration grid as the endpoint tests, Issues A-D and CXS-Lite.

- **Exact:** PACKET_END, ENDERROR and STREAM_CHANGE are predicted flit by flit from the packetizer's output. Delivered packets must be an in-order subsequence of the prediction with exact payloads, and delivered + reported dropped must equal issued.
- **Link events:** the sequence must be ACTIVATE, UP, DEACTIVATE, DOWN repeated, with as many ACTIVATEs as the wire-level CXSMonitor counted.

- **Bounded:** Credit events must fire exactly at the threshold. Each of the three timeouts must fire, with age at least the configured microseconds. The LINK timeout must fire exactly once, naming ACTIVATE.
- **Also covered:** the type mask, a held monbus (drop accounting), `clear`, and PROTOCOL / PARITY errors, provoked last and reported exactly once each.

Seven mutations of the core were caught: a wrong PACKET_END payload field, LINK_UP and DEACTIVATE codes swapped, the drop count never reported, the LINK timeout re-firing, the STARVED threshold off by one, STREAM_CHANGE suppressed, and PROTOCOL reported on the level instead of the edge.

22.8 References

- Arm. *AMBA CXS Protocol Specification*, ARM IHI 0079D, Sep. 2025 (and Issues A-C). developer.arm.com/documentation/ihi0079/d: 2.1.2 credits, 2.5.2 flit streams, 4.3 CXSCNTL, chapter 5 link activation. Citation only.

22.9 Navigation

- **Back to Monitor Index**
- **CXS Modules**

23 Monitor Bus Round-Robin Arbiter

Module: `monbus_arbiter.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: output plane

23.1 Overview

`monbus_arbiter` merges monitor bus packet streams from multiple clients — AXI monitors, APB monitors, arbiter monitors, whatever you've got — into a single output stream, using fair round-robin arbitration with an ACK protocol. Optional skid buffers on the inputs and the output improve timing closure and give you elasticity against backpressure.

The ACK protocol is the point: a granted client gets to finish its packet before the arbiter moves on to the next requester, so multi-cycle transfers never fragment. Combined with the integrated buffering and proper backpressure handling, packets don't get lost on the way through.

- Round-robin arbitration for monitor bus packet streams
- ACK mode operation (grants held until acknowledged)
- Optional input skid buffers per client (2..8 entries)
- Optional output skid buffer (2..8 entries)
- 128-bit packet + 64-bit side-band timestamp, carried atomically through a 192-bit skid
- Parameterizable client count (2-64; 1 does not elaborate)
- Skid buffers optional (disabling them removes the skid registers; the arbitration cycle itself remains, since `grant_valid` is registered)

23.2 Parameters

Parameter	Type	Default	Description
CLIENTS	int	4	Number of monitor bus clients (**2**-64). `CLIENTS=1` does **not** elaborate: `N = $clog2(1) = 0` makes `grant_id` a `logic [-1:0]` and

Parameter	Type	Default	Description
			the priority encoder's generic branch does an illegal `i[-1:0]` part-select
INPUT_SKID_ENABLE	int	1	Enable skid buffers on input interfaces
OUTPUT_SKID_ENABLE	int	1	Enable skid buffer on output interface
INPUT_SKID_DEPTH	int	2	Depth of input skid buffers (2..8 inclusive)
OUTPUT_SKID_DEPTH	int	2	Depth of output skid buffer (2..8 inclusive)
`SKID_DATA_WIDTH`	int	`MONBUS_PKT_WIDTH + MONBUS_TS_WIDTH`	Width of the packed skid-buffer payload.

23.3 Ports

23.3.1 Clock and Reset

Port	Direction	Width	Description
axi_aclk	input	1	Clock signal
axi_aresetn	input	1	Active-low asynchronous reset

23.3.2 Block Arbitration

Port	Direction	Width	Description
block_arb	input	1	Block arbitration

Port	Direction	Width	Description
			when asserted

23.3.3 Monitor Bus Inputs (Array of CLIENTS)

Port	Direction	Width	Description
monbus_valid_in[CLIENTS]	input	CLIENTS	Per-client packet valid signals
monbus_ready_in[CLIENTS]	output	CLIENTS	Per-client ready signals
monbus_packet_in[CLIENTS]	input	CLIENTS × 128	Per-client `monitor_packet_t` packets
monbus_timestamp_in[CLIENTS]	input	CLIENTS × 64	Per-client `monbus_timestamp_t`, sampled at each client's emission time

23.3.4 Monitor Bus Output (Aggregated)

Port	Direction	Width	Description
monbus_valid	output	1	Aggregated packet valid
monbus_ready	input	1	Aggregated ready from downstream
monbus_packet	output	128	Aggregated `monitor_packet_t`
monbus_timestamp	output	64	Aggregated `monbus_timestamp_t`, paired atomically with `monbus_packet`

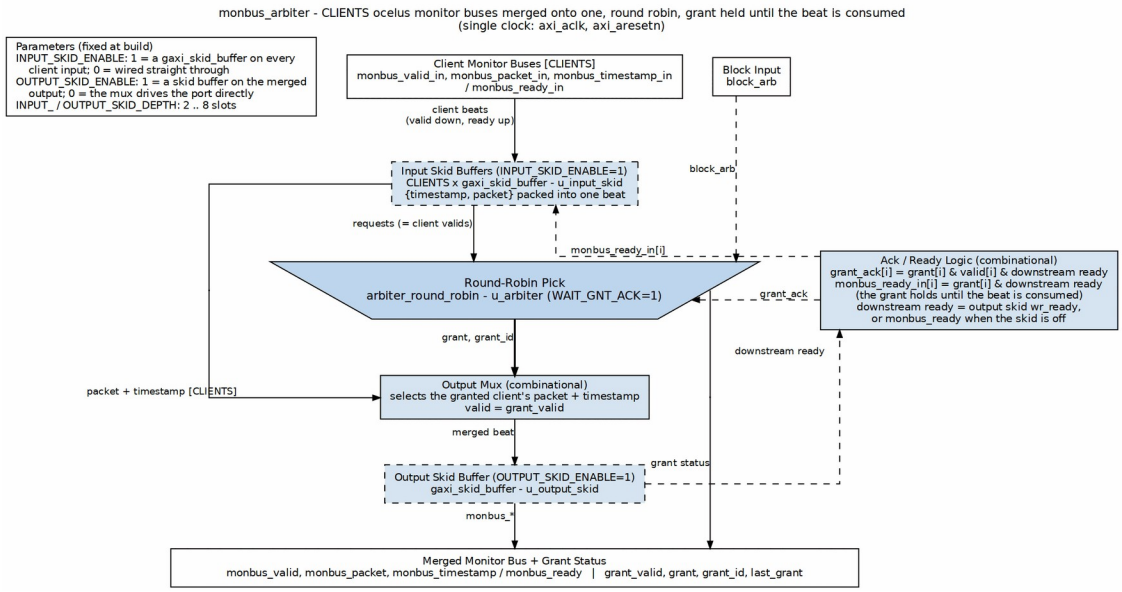
23.3.5 Debug/Status Outputs

Port	Direction	Width	Description
grant_valid	output	1	Grant is active this cycle

Port	Direction	Width	Description
grant	output	CLIENTS	One-hot grant vector
grant_id	output	$clog2(CLIENTS)	Binary encoded grant ID
last_grant	output	CLIENTS	Previous grant (for round-robin rotation)

23.4 Functional Description

23.4.1 Block diagram



monbus_arbiter block diagram

Each client's monitor bus enters at the top and, when INPUT_SKID_ENABLE=1, lands in its own gaxi_skid_buffer. The skid valids are the requests into the round-robin pick (arbiter_round_robin in ACK mode). The heavy grant drives the output mux, which forwards the granted client's packet and timestamp through the optional output skid to the merged bus at the bottom. The client beats bypass the pick down the left lane. On the right, the ack/ready logic acknowledges a grant only when the beat is actually consumed (grant, valid and downstream ready), so a grant holds under backpressure. The same logic returns each client's ready.

23.4.2 Arbiter Request and Grant ACK Logic

The module maps monitor bus protocol onto arbiter protocol.

Request Mapping: each client's valid becomes an arbiter request — `request[i] = int_monbus_valid_in[i]`, i.e. the signal *after* the optional input skid, not the port itself.

Grant ACK Logic: ACK occurs only when the beat is actually **consumed** — grant, valid, and downstream ready together:

```
grant_ack[i] = grant[i] && int_monbus_valid_in[i] && int_monbus_ready;
```

The `int_monbus_ready` term is load-bearing and must not be dropped. Without it (`grant[i] && int_monbus_valid_in[i]` alone) the ack fires on every cycle the sink is back-pressuring, so the grant rotates continuously while **zero** transfers take place — breaking the grant-hold contract. That was a real defect; `val/amba/test_monbus_arbiter_grant_hold.py` is the regression that pins it.

23.4.3 Round-Robin Arbiter Instance

Uses arbiter_round_robin with WAIT_GNT_ACK=1:

- Fair rotation through active requests
- Grant held until acknowledged
- Block_arb input for external control

23.4.4 Client Ready Signal Generation

Each client's ready signal asserts when:

1. That client is currently granted AND
2. Downstream (internal output) is ready to accept data

```
int_monbus_ready_in[i] = grant[i] && int_monbus_ready;
```

This ensures only the granted client can transfer data, preventing collisions.

> **With the default `INPUT_SKID_ENABLE=1`, this is not what the port does.** The rule above governs the *internal* ready behind the skid buffer. The actual port `monbus_ready_in[i]` is the skid's `wr_ready`, which asserts whenever the skid has room — **independently of any grant**. A client can therefore be accepted while ungranted; the skid holds the beat until its grant comes. Only with `INPUT_SKID_ENABLE=0` do the port and the internal signal coincide.

23.4.5 Output Multiplexer

Selects data from the granted client:

```
if (grant_valid)
    int_monbus_packet = int_monbus_packet_in[grant_id];  // post-skid
```

23.4.6 Optional Skid Buffers

All skid buffers in this arbiter carry the **packet and timestamp atomically** in a 192-bit payload (`MONBUS_PKT_WIDTH + MONBUS_TS_WIDTH = 128 + 64`). The arbiter never separates a packet from its sampled timestamp, so consumers downstream of the `monbus_group` family see a coherent (pkt, ts) tuple even after multiple levels of skid.

Input Skid Buffers (per client):

- Uses gaxi_skid_buffer instances configured for 192-bit data
- Provides elasticity for clients with bursty traffic
- Improves timing closure by breaking long paths
- Depth configurable: 2..8 entries inclusive (odd depths are legal)

Output Skid Buffer:

- Buffers aggregated stream before output (also 192-bit)
- Prevents backpressure propagation to arbiter
- Same depth options as input buffers

When to Enable Skid Buffers:

- Enable INPUT_SKID if clients have high-latency paths or bursty traffic
- Enable OUTPUT_SKID if downstream consumer has variable latency
- Disable both for minimum latency. That removes the skid registers only; it is NOT a combinational pass-through. `monbus_valid` comes from `arbiter_round_robin`'s `grant_valid`, which is registered, so a request still costs one arbitration cycle. See "Skid-Free Configuration".

23.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

23.6 Usage Examples

```
// Aggregate monitor bus streams from 4 clients
monbus_arbiter #(
    .CLIENTS             (4),
    .INPUT_SKID_ENABLE   (1),    // Enable input buffers
    .OUTPUT_SKID_ENABLE  (1),    // Enable output buffer
    .INPUT_SKID_DEPTH    (4),    // 4-entry input buffers
    .OUTPUT_SKID_DEPTH   (4)     // 4-entry output buffer
) u_monbus_arb (
    .axi_aclk            (clk),
    .axi_aresetn         (rst_n),
    .block_arb           (1'b0),  // Not blocked

    // Connect 4 client monitor bus streams (packet + timestamp per
client).
    // NOTE the asymmetry: the three INPUT arrays take assignment
patterns,
    // which are legal rvalues for an unpacked-array input.
monbus_ready_in is
    // an OUTPUT, so its connection must be an lvalue -- an assignment
pattern
    // there does not compile. Declare the array and connect it whole:
    //     logic mon_ready [4];
    //     assign {mon0_ready, mon1_ready, mon2_ready, mon3_ready} =
    //            {mon_ready[0], mon_ready[1], mon_ready[2],
mon_ready[3]};
    .monbus_valid_in     ('{mon0_valid, mon1_valid, mon2_valid,
mon3_valid}),
    .monbus_ready_in     (mon_ready),
    .monbus_packet_in    ('{mon0_packet, mon1_packet, mon2_packet,
mon3_packet}),
    .monbus_timestamp_in ('{mon0_ts,     mon1_ts,     mon2_ts,
mon3_ts}),

    // Aggregated output stream (packet + timestamp paired atomically)
    .monbus_valid        (agg_valid),
    .monbus_ready        (agg_ready),
    .monbus_packet       (agg_packet),
    .monbus_timestamp    (agg_ts),

    // Debug outputs
    .grant_valid         (arb_grant_valid),
    .grant               (arb_grant_onehot),
    .grant_id            (arb_grant_id),
    .last_grant          (arb_last_grant)
);
```

```
// Downstream consumer is typically the monbus_group family, which
expects the
// 128-bit packet on monbus_packet and the 64-bit timestamp on
// monbus_timestamp. No additional FIFO is needed at this boundary —
// raw-mode groups have no ingress skid of their own; only the
// compressed build adds one (u_comp_in_skid in monbus_group_core).
```

23.7 Design Notes

23.7.1 ACK Mode Operation

The ACK mode is what guarantees clean packet transfer:

Grant Assertion: When arbiter selects a client, grant[i] asserts **Client Response**: Client's monbus_valid_in[i] must be high **Grant ACK**: `grant_ack[i] = grant[i] && int_monbus_valid_in[i] && int_monbus_ready` — the ready term is required (see above) **Hold Grant**: Arbiter holds grant[i] until grant_ack[i] asserts **Next Client**: After ACK, arbiter moves to next requesting client

This prevents:

- Packet fragmentation (grant switching mid-transfer)
- Lost packets (grant removed before client ready)
- Unfair arbitration (clients getting multiple back-to-back grants)

23.7.2 Skid Buffer Depth Selection

Choose depth based on system characteristics:

2 Entries (Minimum):

- Minimal buffering for timing closure only
- Use when clients have low, consistent latency

4 Entries (Recommended):

- Good balance of buffering and area
- Handles typical backpressure scenarios
- Recommended for most systems

6-8 Entries:

- High buffering for very bursty traffic

- Use when downstream has high variable latency
- Higher area cost

23.7.3 Skid-Free Configuration

For minimum latency:

```
.INPUT_SKID_ENABLE  (0),
.OUTPUT_SKID_ENABLE (0)
```

This removes the skid registers, but it does **not** make the valid path combinational. `monbus_valid` is driven from the arbiter's `grant_valid`, which is a REGISTERED output of `arbiter_round_robin`, so a request still takes an arbitration cycle to appear as a grant:

- monbus_valid_in[i] -> request[i] -> (registered grant) -> monbus_valid
- monbus_packet_in[grant_id] -> monbus_packet (combinational mux on grant_id)
- monbus_ready -> monbus_ready_in[grant_id] (combinational, gated by grant)

Use when:

- Timing is not critical
- Clients and downstream have matched latencies
- Minimum latency required for event ordering

23.7.4 Formal properties

The module carries no assertions (repository rule: no assertions in RTL). Three contracts are proved in the formal harness `formal/amba/monbus_arbiter/` (`make prove`, `make cover`), each mutation-checked:

Grant One-Hot: the grant vector is one-hot whenever `grant_valid`. **Grant ID Consistency**: `grant[grant_id]` is set whenever `grant_valid`. **Ready Exclusivity**: a client that is not granted never sees ready.

The harness builds the arbiter with the input skids disabled, so the ready property is checked directly on the `monbus_ready_in` port.

23.8 Related Modules

Used by:

- System-level monitor bus aggregation hierarchies

- Multi-subsystem monitoring infrastructures

Uses:

- arbiter_round_robin.sv (ACK-mode arbiter)
- gaxi_skid_buffer.sv (optional input/output buffering)

Related:

- arbiter_monbus_common.sv (generates monitor bus packets)
- arbiter_rr_pwm_monbus.sv (arbiter with integrated monitoring)

23.9 Testing

The grant-hold defect described above is pinned by `val/amba/test_monbus_arbiter_grant_hold.py` — run it after any change to the ACK or ready logic.

23.10 References

23.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Gupta, P., McKeown, N. “Designing and Implementing a Fast Crossbar Scheduler.” *IEEE Micro*, 19(1):20-28, Jan./Feb. 1999. DOI: 10.1109/40.748793. — the round-robin core (`arbiter_round_robin`) this arbiter uses (indexed in `rtl/common/references/README.md`).
- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. “An Event-Based Monitoring Service for Networks on Chip.” *ACM Trans. Design Automation of Electronic Systems*, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126. — related work: monitoring traffic from many probes merged onto shared links.
- Dally, W.J., Towles, B. *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004, ch. 18 “Arbitration”, §18.4.2 “Round-Robin Arbiter” and §18.4.4 “Weighted Round-Robin Arbiter”. — textbook treatment (secondary source).

- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

23.10.2 Specifications

- Internal: docs/markdown/rtl-amba/index.md (AMBA subsystem requirements)
- Internal: docs/markdown/rtl-amba/includes/ocelus_package_spec.md (monitor bus protocol)

23.10.3 Source Code

- RTL: `rtl/amba/ocelus/monbus_arbiter.sv`
- Tests: `val/amba/test_monbus_arbiter_grant_hold.py` (if exists)

Last Updated: 2025-10-24

23.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

24 MonBus Group — AXI4 / AXI4

Module: `monbus_axi4_axi4_group.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: output plane

24.1 Overview

`monbus_axi4_axi4_group` is the AXI4-slave-read + AXI4-master-write wrapper around `monbus_group_core`. It pairs a full AXI4 slave-read leaf (the error/interrupt drain port, supporting burst reads) with a full AXI4 master-write leaf (the bulk-capture port, emitting large write bursts). Both leaves are true AXI4, so their FUB interfaces pass straight through to the core's AXI4-shaped FUB ports; the wrapper only supplies safe constant defaults for the AXI4 sideband fields the core does not produce.

An SoC monitoring subsystem needs to expose captured monitor packets to two AXI4 consumers: a CPU that reads error records over a slave port, and a DMA-style write path that dumps a bulk trace to memory. This wrapper wires the two AXI4 leaf protocol adapters (`axi4_slave_rd`, `axi4_master_wr`) to the shared capture core, giving you a drop-in AXI4/AXI4 capture block.

Use it for:

- AXI4-connected monitoring block where both the error-read and trace-write masters are AXI4
- High-throughput trace capture that benefits from long master-write bursts
- CPU IRQ handler reading error records via AXI4 burst reads

The key benefit: true AXI4 on both sides with zero protocol translation loss — the leaves pass through to the core, so burst geometry, IDs, and full-size bursts are available end to end.

Feature summary:

- Full AXI4 on both the error-drain (slave read) and bulk-capture (master write) sides
- Burst AR support on the slave-read drain (walk multiple error records per burst)
- Large write bursts on the master-write side (`MAX_BURST_BEATS` up to 256)
- Thin structural adapter — all logic lives in `monbus_group_core`
- Configurable skid depths on all five AXI channels
- Optional runtime compression (`cfg_compress_en`) with `USE_COMPRESSION`
- Full per-protocol filter mask set (AXI / AXIS / CORE) passed to the core

24.2 Parameters

Parameter	Type	Default	Description
FIFO_DEPTH_ERR	int	64	Error FIFO depth in 192-bit records
FIFO_DEPTH_WRITE	int	96	Write FIFO depth in 64-bit beats
ADDR_WIDTH	int	32	Address width for both AXI4 ports
AXI_ID_WIDTH_S	int	8	Slave-read ID width
AXI_ID_WIDTH_M	int	8	Master-write ID width
AXI_USER_WIDTH	int	1	AXI user-signal width
MAX_BURST_BEATS	int	64	Max beats per master-write sub-burst
FLUSH_TIMEOUT_CYCLES	int	1024	Cycles before a timeout-triggered flush
NUM_PROTOCOLS	int	3	Informational only
USE_COMPRESSION	int	0	0 = raw 3-beat records; 1 = elaborate the compressor
SKID_DEPTH_AR	int	2	Slave-read AR channel skid depth
SKID_DEPTH_R	int	4	Slave-read R channel skid depth
SKID_DEPTH_AW	int	2	Master-write AW

Parameter	Type	Default	Description
			channel skid depth
SKID_DEPTH_W	int	4	Master-write W channel skid depth
SKID_DEPTH_B	int	2	Master-write B channel skid depth

24.3 Ports

24.3.1 Clock, Reset, Clear

Port	Direction	Width	Description
axi_aclk	input	1	Clock
axi_aresetn	input	1	Active-low asynchronous reset
cam_clear	input	1	Synchronous compressor CAM + stats clear

24.3.2 MonBus Ingress and Timestamp

Port	Direction	Width	Description
monbus_valid	input	1	Incoming packet valid
monbus_ready	output	1	Ready to accept
monbus_packet	input	monitor_packet_t (128)	Monitor packet
monbus_timestamp	input	monbus_timestamp_t (64)	Side-band timestamp
mon_time_out	output	monbus_timestamp_t (64)	Free-running counter for wrappers'

Port	Direction	Width	Description
			i_mon_time

24.3.3 S-Side — AXI4 Slave Read (error record drain)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH_S	Read address ID
s_axi_araddr	input	ADDR_WIDTH	Read address
s_axi_arlen	input	8	Burst length (beats − 1)
s_axi_arsize	input	3	Burst size
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock
s_axi_arcache	input	4	Cache attributes
s_axi_arprot	input	3	Protection
s_axi_arqos	input	4	QoS
s_axi_arregion	input	4	Region
s_axi_aruser	input	AXI_USER_WIDTH	User
s_axi_arvalid	input	1	AR valid
s_axi_arready	output	1	AR ready
s_axi_rid	output	AXI_ID_WIDTH_S	Read data ID
s_axi_rdata	output	64	Read data (sliced error record)
s_axi_rresp	output	2	Read response
s_axi_rlast	output	1	Read last
s_axi_ruser	output	AXI_USER_WIDTH	User (tied to 0)
s_axi_rvalid	output	1	R valid
s_axi_rready	input	1	R ready

24.3.4 M-Side — AXI4 Master Write (bulk trace capture)

Port	Direction	Width	Description
m_axi_awid	output	AXI_ID_WIDTH_M	Write address ID
m_axi_awaddr	output	ADDR_WIDTH	Write address
m_axi_awlen	output	8	Burst length (beats − 1)
m_axi_awsize	output	3	Burst size
m_axi_awburst	output	2	Burst type
m_axi_awlock	output	1	Lock
m_axi_awcache	output	4	Cache attributes
m_axi_awprot	output	3	Protection
m_axi_awqos	output	4	QoS
m_axi_awregion	output	4	Region
m_axi_awuser	output	AXI_USER_WIDTH	User
m_axi_awvalid	output	1	AW valid
m_axi_awready	input	1	AW ready
m_axi_wdata	output	64	Write data
m_axi_wstrb	output	8	Write strobes
m_axi_wlast	output	1	Write last
m_axi_wuser	output	AXI_USER_WIDTH	User
m_axi_wvalid	output	1	W valid
m_axi_wready	input	1	W ready
m_axi_bid	input	AXI_ID_WIDTH_M	Write response ID
m_axi_bresp	input	2	Write response
m_axi_buser	input	AXI_USER_WIDTH	User (ignored)
m_axi_bvalid	input	1	B valid

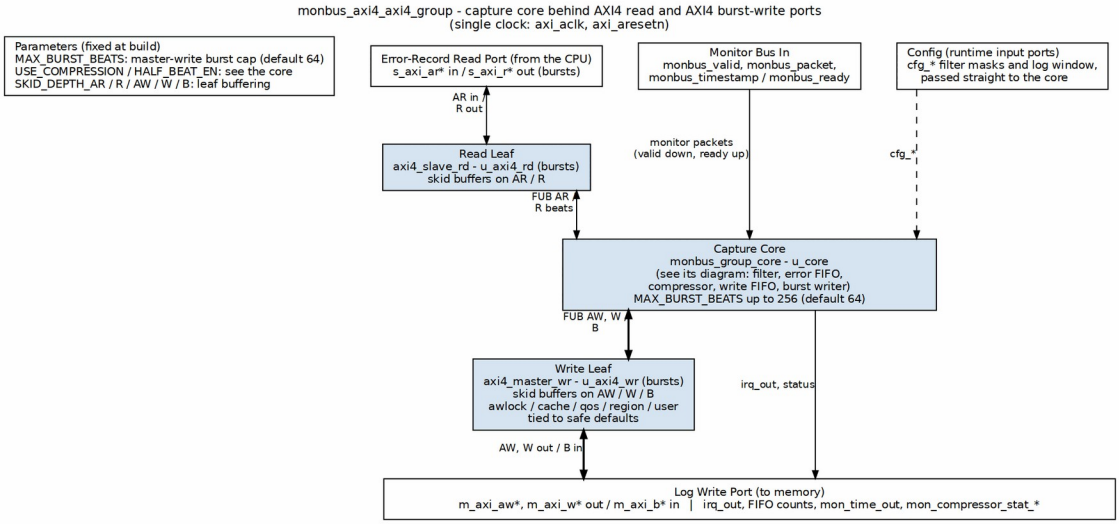
Port	Direction	Width	Description
m_axi_bready	output	1	B ready

24.3.5 Status and Egress Configuration

Port	Direction	Width	Description
irq_out	output	1	Error FIFO non-empty
cfg_base_addr	input	ADDR_WIDTH	Capture window base
cfg_limit_addr	input	ADDR_WIDTH	Capture window limit
cfg_flush_watermark	input	16	Flush-trigger beat count
cfg_compress_en	input	1	Runtime compression enable
cfg_axi_* / cfg_axis_* / cfg_core_*	input	16 each	Per-protocol filter masks (drop / err-select / per-event-code); see monbus_group_core
err_fifo_full	output	1	Error FIFO full
write_fifo_full	output	1	Write FIFO full
err_fifo_count	output	16	Error FIFO occupancy (records)
write_fifo_count	output	16	Write FIFO occupancy (beats)
mon_compressor_stat_*	output	32 each	Compressor statistics (0 in raw mode)

24.4 Functional Description

24.4.1 Block diagram



monbus_axi4_axi4_group block diagram

A structural wrapper over monbus_group_core, drawn as one box. The CPU's AXI4 read port at the top left reaches the core's error-record read FUB through an axi4_slave_rd leaf (bursts). The core's log writes leave through an axi4_master_wr leaf (bursts up to MAX_BURST_BEATS) to the memory port at the bottom, with the AXI4 fields the core does not produce tied to safe defaults.

24.4.2 S-Side Error Drain (AXI4 slave read)

The `axi4_slave_rd` leaf terminates the incoming AXI4 read channel and presents a clean FUB read interface (`fub_axi_ar*` / `fub_axi_r*`) to the core. The core's slave-read slicer serves each 192-bit error record as three 64-bit beats (timestamp, packet-hi, packet-lo). Because the leaf is full AXI4, a single AR burst can request several records at once. The AXI4-only AR sideband fields (`arlock`/`arcache`/`arqos`/`arregion`/`aruser`) are consumed by the leaf and not forwarded to the core; `s_axi_ruser` is tied to 0.

24.4.3 M-Side Bulk Capture (AXI4 master write)

The core's master-write FUB drives the `axi4_master_wr` leaf, which emits the full AXI4 write channel. The wrapper hard-wires the AXI4 sideband fields the core does not generate to safe defaults at the leaf's FUB inputs: `awlock=0`, `awcache=4'b0010` (normal non-cacheable bufferable), `awprot=0`, `awqos=0`, `awregion=0`, `awuser=0`, `wuser=0`. `MAX_BURST_BEATS` defaults to 64, so a flush typically drains as one large sub-burst.

24.4.4 Shared Egress Configuration

All filtering, FIFO sizing, address-window, watermark, timeout, and compression behavior is set by the config ports and handed straight to `monbus_group_core`. The three per-protocol mask

groups (`cfg_axi_*`, `cfg_axis_*`, `cfg_core_*`) select which packet types drop, which route to the error FIFO, and which event codes are masked — identical semantics across all four group wrappers.

24.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AR`	2 entries
`SKID_DEPTH_R`	4 entries
`SKID_DEPTH_AW`	2 entries
`SKID_DEPTH_W`	4 entries
`SKID_DEPTH_B`	2 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

24.6 Usage Examples

```
monbus_axi4_axi4_group #(
    .ADDR_WIDTH      (32),
    .AXI_ID_WIDTH_S  (8),
    .AXI_ID_WIDTH_M  (8),
    .MAX_BURST_BEATS (64),
    .USE_COMPRESSION (0)
) u_mon_group (
    .axi_aclk    (aclk),
    .axi_aresetn (aresetn),
    .cam_clear   (1'b0),

    .monbus_valid     (agg_valid),   // from monbus_arbiter
    .monbus_ready     (agg_ready),
    .monbus_packet    (agg_packet),
    .monbus_timestamp (agg_timestamp),
```

```
    .mon_time_out     (mon_time),

    // AXI4 slave-read drain -> CPU
    .s_axi_arid
(cpu_arid), .s_axi_araddr(cpu_araddr), .s_axi_arlen(cpu_arlen),
    .s_axi_arvalid(cpu_arvalid), .s_axi_arready(cpu_arready),
    .s_axi_rid
(cpu_rid), .s_axi_rdata(cpu_rdata), .s_axi_rlast(cpu_rlast),
    .s_axi_rvalid(cpu_rvalid), .s_axi_rready(cpu_rready),
    // ... AR sideband ...

    // AXI4 master-write bulk capture -> memory
    .m_axi_awaddr(cap_awaddr), .m_axi_awlen(cap_awlen), .m_axi_awvalid
(cap_awvalid),
    .m_axi_awready(cap_awready), .m_axi_wdata(cap_wdata), .m_axi_wlast
(cap_wlast),
    .m_axi_wvalid(cap_wvalid), .m_axi_wready(cap_wready),
    .m_axi_bvalid(cap_bvalid), .m_axi_bready(cap_bready),
    // ... AW sideband ...

    .irq_out             (mon_irq),
    .cfg_base_addr       (cap_base),
    .cfg_limit_addr      (cap_limit),
    .cfg_flush_watermark (16'd48),
    .cfg_compress_en     (1'b0)
    // ... per-protocol masks ...
);
```

24.7 Design Notes

24.7.1 Both Sides Are Full AXI4

Unlike the AXIL variants, neither leaf forces `MAX_BURST_BEATS=1` and neither injects single-beat defaults into the core — the AXI4 IDs and burst lengths flow through. This is the highest-throughput member of the group family.

24.7.2 AW Sideband Defaults Live in the Wrapper

The core deliberately does not model `awlock`/`awcache`/`awqos`/`awregion`/`awuser`. The wrapper supplies conservative constants at the master-write leaf so downstream interconnect sees a well-formed AXI4 burst.

24.7.3 Skid Depths Are Tunable

The five `SKID_DEPTH_*` parameters size the per-channel skid buffers in the leaves; increase them to absorb more backpressure on congested interconnect.

24.8 Related Modules

Used by:

- SoC monitoring subsystems requiring an AXI4/AXI4 MonBus capture block

Uses:

- **monbus_group_core.sv** — protocol-agnostic capture core (all logic)
- **axi4_slave_rd.sv** — AXI4 slave-read leaf (error drain)
- **axi4_master_wr.sv** — AXI4 master-write leaf (bulk capture)
- **monitor_common_pkg** — packet and timestamp types

See also:

- **monbus_axi4_axil4_group.sv** — AXI4 read + AXIL write variant
- **monbus_axil4_axi4_group.sv** — AXIL read + AXI4 write variant
- **monbus_axil4_axil4_group.sv** — AXIL read + AXIL write variant
- **monbus_arbiter.sv** — upstream aggregation

24.9 Testing

No test coverage. There is no `val/**/test_monbus_axi4_axi4_group.py`, and no module that instantiates this one has a testbench either, so nothing in the repository exercises it.

Treat any behaviour described on this page as unverified by simulation.

24.10 References

24.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. — AXI4 burst read (drain) and burst write (capture) ports.
- Hopkins, A.B.T., McDonald-Maier, K.D. “Debug Support for Complex Systems On-Chip: A Review.” *IEE Proc. Computers and Digital Techniques*, 153(4):197-207, 2006. DOI: 10.1049/ip-cdt:20050194. — related work: on-chip trace buffers versus trace ports.
- Yang, F.-C., Lin, Y., Kao, C.-F., Huang, I.-J. “An On-Chip AHB Bus Tracer With Real-Time Compression and Dynamic Multiresolution Supports for SoC.” *IEEE Trans. VLSI Systems*, 19(4):571-584, Apr. 2011. DOI: 10.1109/TVLSI.2009.2039811. — related work: a bus tracer that records into an on-chip circular buffer.
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

24.10.2 Source Code

- RTL: `rtl/amba/ocelus/monbus_axi4_axi4_group.sv`
- Core: `rtl/amba/ocelus/monbus_group_core.sv`

24.10.3 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Group Core: `docs/markdown/rtl-amba/monitor/monbus_group_core.md`
- Packet Format: `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`

Last Updated: 2026-07-15

24.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

25 MonBus Group — AXI4 / AXIL

Module: monbus_axi4_axil4_group.sv **Location:** rtl/amba/ocelus/ **Status:** Production Ready

Plane: output plane

25.1 Overview

monbus_axi4_axil4_group is the AXI4-slave-read + AXIL-master-write wrapper around monbus_group_core. It pairs a full AXI4 slave-read leaf (the burst-capable error/interrupt drain port) with a single-beat AXI4-Lite master-write leaf (the bulk-capture port). The slave-read FUB passes through to the core's AXI4-shaped read FUB, while the master-write side forces MAX_BURST_BEATS=1 so the core emits one beat per AW — matching AXIL's single-beat semantics.

Some systems read error records over a high-throughput AXI4 port but write the bulk trace to a simple AXI4-Lite peripheral bus — a register-file-style capture sink, a low-cost bridge, or a control-plane fabric. This wrapper mixes the two: an axi4_slave_rd leaf for burst error reads and an axil4_master_wr leaf for single-beat trace writes, both driven by the shared capture core.

Use it for:

- Trace capture to an AXIL-only memory-mapped sink while reading errors over AXI4
- Control-plane monitoring where the write path is a lightweight AXIL bus
- Mixed-fabric SoCs pairing an AXI4 CPU read port with an AXIL capture port

The key benefit: this reuses the exact same capture core as the all-AXI4 variant — only MAX_BURST_BEATS and the master-write leaf differ — so behavior and configuration stay identical across the family, with the write side degrading gracefully to single beats.

Feature summary:

- AXI4 burst reads on the error-drain side (walk multiple records per burst)
- AXIL single-beat writes on the bulk-capture side (MAX_BURST_BEATS=1)
- Thin structural adapter — all logic lives in monbus_group_core
- AXI4-only AR sideband fields dropped at the wrapper boundary
- Master-write ID forced to width 1 (AXIL has no ID)
- Optional runtime compression (cfg_compress_en) with USE_COMPRESSION
- Full per-protocol filter mask set (AXI / AXIS / CORE) passed to the core

---

25.2 Parameters

Parameter	Type	Default	Description
FIFO_DEPTH_ERR	int	64	Error FIFO depth in 192-bit records
FIFO_DEPTH_WRITE	int	96	Write FIFO depth in 64-bit beats
ADDR_WIDTH	int	32	Address width for both ports
AXI_ID_WIDTH	int	8	Slave-read ID width
AXI_USER_WIDTH	int	1	AXI user-signal width (slave-read side)
FLUSH_TIMEOUT_CYCLES	int	1024	Cycles before a timeout-triggered flush
NUM_PROTOCOLS	int	3	Informational only
USE_COMPRESSION	int	0	0 = raw 3-beat records; 1 = elaborate the compressor
SKID_DEPTH_AR	int	2	Slave-read AR channel skid depth
SKID_DEPTH_R	int	4	Slave-read R channel skid depth
SKID_DEPTH_AW	int	2	AXIL master-write AW channel skid depth
SKID_DEPTH_W	int	2	AXIL master-write W channel skid depth

Parameter	Type	Default	Description
SKID_DEPTH_B	int	2	AXIL master-write B channel skid depth

Internally, the core is instantiated with AXI_ID_WIDTH_M=1, AXI_ID_WIDTH_S=AXI_ID_WIDTH, and MAX_BURST_BEATS=1.

25.3 Ports

25.3.1 Clock, Reset, Clear

Port	Direction	Width	Description
axi_aclk	input	1	Clock
axi_aresetn	input	1	Active-low asynchronous reset
cam_clear	input	1	Synchronous compressor CAM + stats clear

25.3.2 MonBus Ingress and Timestamp

Port	Direction	Width	Description
monbus_valid	input	1	Incoming packet valid
monbus_ready	output	1	Ready to accept
monbus_packet	input	monitor_packet_t (128)	Monitor packet
monbus_timestamp	input	monbus_timestamp_t (64)	Side-band timestamp
mon_time_out	output	monbus_timestamp_t (64)	Free-running counter for wrappers' i_mon_time

25.3.3 S-Side — AXI4 Slave Read (error record drain)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH	Read address ID
s_axi_araddr	input	ADDR_WIDTH	Read address
s_axi_arlen	input	8	Burst length (beats − 1)
s_axi_arsize	input	3	Burst size
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock
s_axi_arcache	input	4	Cache attributes
s_axi_arprot	input	3	Protection
s_axi_arqos	input	4	QoS
s_axi_arregion	input	4	Region
s_axi_aruser	input	AXI_USER_WIDTH	User
s_axi_arvalid	input	1	AR valid
s_axi_arready	output	1	AR ready
s_axi_rid	output	AXI_ID_WIDTH	Read data ID
s_axi_rdata	output	64	Read data (sliced error record)
s_axi_rresp	output	2	Read response
s_axi_rlast	output	1	Read last
s_axi_ruser	output	AXI_USER_WIDTH	User (tied to 0)
s_axi_rvalid	output	1	R valid
s_axi_rready	input	1	R ready

25.3.4 M-Side — AXIL Master Write (bulk trace capture)

Port	Direction	Width	Description
m_axil_awvalid	output	1	AW valid
m_axil_awready	input	1	AW ready
m_axil_awaddr	output	ADDR_WIDTH	Write address
m_axil_awprot	output	3	Protection
m_axil_wvalid	output	1	W valid
m_axil_wready	input	1	W ready
m_axil_wdata	output	64	Write data
m_axil_wstrb	output	8	Write strobes
m_axil_bvalid	input	1	B valid
m_axil_bready	output	1	B ready
m_axil_bresp	input	2	Write response

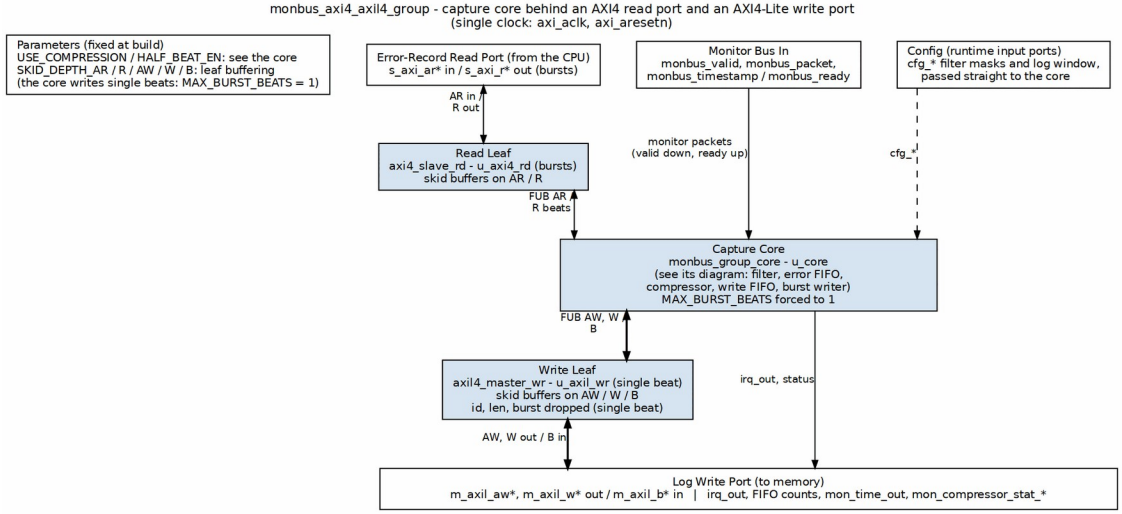
25.3.5 Status and Egress Configuration

Port	Direction	Width	Description
irq_out	output	1	Error FIFO non-empty
cfg_base_addr	input	ADDR_WIDTH	Capture window base
cfg_limit_addr	input	ADDR_WIDTH	Capture window limit
cfg_flush_watermark	input	16	Flush-trigger beat count
cfg_compress_en	input	1	Runtime compression enable
cfg_axi_* / cfg_axis_* / cfg_core_*	input	16 each	Per-protocol filter masks (drop / err-select / per-event-code); see monbus_group_core

Port	Direction	Width	Description
err_fifo_full	output	1	Error FIFO full
write_fifo_full	output	1	Write FIFO full
err_fifo_count	output	16	Error FIFO occupancy (records)
write_fifo_count	output	16	Write FIFO occupancy (beats)
mon_compressor_stat_*	output	32 each	Compressor statistics (0 in raw mode)

25.4 Functional Description

25.4.1 Block diagram



monbus_axi4_axil4_group block diagram

A structural wrapper over monbus_group_core, drawn as one box. Error records are read over AXI4 (axi4_slave_rd, bursts), and the log is written over AXI4-Lite (axil4_master_wr), so the core's MAX_BURST_BEATS is forced to 1 and every log write is a single beat.

25.4.2 S-Side Error Drain (AXI4 slave read)

Identical to the AXI4/AXI4 variant on the read side: the `axi4_slave_rd` leaf terminates the AXI4 read channel and hands a clean read FUB to the core, whose slicer serves each 192-bit error

record as three 64-bit beats. Full AXI4 bursts can request multiple records at once. The AXI4-only AR sideband fields (arlock/arcache/arqos/arregion/aruser) are dropped at the wrapper boundary; s_axi_ruser is tied to 0.

25.4.3 M-Side Bulk Capture (AXIL master write)

The core is parameterized with MAX_BURST_BEATS=1, so its master-write burst writer emits one AW per beat — the single-beat pattern AXIL requires. The core's AXI4-shaped master-write FUB is bridged to the axil4_master_wr leaf: awaddr, awvalid/awready, wdata, wstrb, wvalid/wready, and bresp/bvalid/bready connect through, while the AXI4-only FUB outputs the core still drives (awid, awlen, awsize, awburst, wlast) are captured in unused nets and discarded. The AXIL leaf drives awprot=0, and the core's bid input is tied to 0.

25.4.4 Shared Egress Configuration

Filtering, FIFO sizing, address-window, watermark, timeout, and compression are all handled by monbus_group_core via the same config ports as every other group wrapper. The three per-protocol mask groups (cfg_axi_*, cfg_axis_*, cfg_core_*) behave identically here.

25.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AR	2 entries
SKID_DEPTH_R	4 entries
SKID_DEPTH_AW	2 entries
SKID_DEPTH_W	2 entries
SKID_DEPTH_B	2 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

25.6 Usage Examples

```
monbus_axi4_axil4_group #(
    .ADDR_WIDTH      (32),
    .AXI_ID_WIDTH    (8),
    .USE_COMPRESSION (0)
) u_mon_group (
    .axi_aclk    (aclk),
    .axi_aresetn (aresetn),
    .cam_clear   (1'b0),

    .monbus_valid     (agg_valid),
    .monbus_ready     (agg_ready),
    .monbus_packet    (agg_packet),
    .monbus_timestamp (agg_timestamp),
    .mon_time_out     (mon_time),

    // AXI4 slave-read drain -> CPU
    .s_axi_arid(cpu_arid), .s_axi_araddr(cpu_araddr), .s_axi_arlen(cpu
_arlen),
    .s_axi_arvalid(cpu_arvalid), .s_axi_arready(cpu_arready),
    .s_axi_rdata(cpu_rdata), .s_axi_rlast(cpu_rlast),
    .s_axi_rvalid(cpu_rvalid), .s_axi_rready(cpu_rready),
    // ... AR sideband ...

    // AXIL master-write bulk capture -> peripheral sink
    .m_axil_awaddr(cap_awaddr), .m_axil_awvalid(cap_awvalid), .m_axil_
awready(cap_awready),
    .m_axil_wdata(cap_wdata), .m_axil_wstrb(cap_wstrb),
    .m_axil_wvalid(cap_wvalid), .m_axil_wready(cap_wready),
    .m_axil_bvalid(cap_bvalid), .m_axil_bready(cap_bready), .m_axil_br
esp(cap_bresp),

    .irq_out             (mon_irq),
    .cfg_base_addr       (cap_base),
    .cfg_limit_addr      (cap_limit),
    .cfg_flush_watermark (16'd48),
    .cfg_compress_en     (1'b0)
    // ... per-protocol masks ...
);
```

25.7 Design Notes

25.7.1 AXIL Forces Single-Beat Writes

The only functional difference from the AXI4/AXI4 variant is `MAX_BURST_BEATS=1` and the AXIL master-write leaf. Every flush emits one beat per AW handshake, so throughput on the capture side is lower than the AXI4/AXI4 group — appropriate for a lightweight AXIL sink.

25.7.2 Core Still Drives AXI4 FUB Fields

Because the core is AXI4-shaped internally, it always drives `awlen`/`awsize`/`awburst`/`awid`/`wlast`. On the AXIL side these are simply not routed to the leaf (captured in `_unused` nets). This is intentional and keeps the core single-source.

25.7.3 Read Side Is Unchanged

The error-drain path is byte-for-byte the same as the AXI4/AXI4 group — the record slicer, burst AR handling, and IRQ behavior are all in the shared core.

25.8 Related Modules

Used by:

- SoC monitoring subsystems reading errors over AXI4 but capturing the trace over AXIL

Uses:

- **monbus_group_core.sv** — protocol-agnostic capture core (all logic), `MAX_BURST_BEATS=1`
- **axi4_slave_rd.sv** — AXI4 slave-read leaf (error drain)
- **axil4_master_wr.sv** — AXIL master-write leaf (bulk capture)
- **monitor_common_pkg** — packet and timestamp types

See also:

- **monbus_axi4_axi4_group.sv** — AXI4 read + AXI4 write variant
- **monbus_axil4_axi4_group.sv** — AXIL read + AXI4 write variant
- **monbus_axil4_axil4_group.sv** — AXIL read + AXIL write variant
- **monbus_arbiter.sv** — upstream aggregation

25.9 Testing

`val/amba/test_monbus_axi4_axil4_group.py` exercises this module. It collects 1 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_monbus_axi4_axil4_group.py -v
```

25.10 References

25.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. — AXI4 burst slave-read (drain) port.
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021. developer.arm.com/documentation/ihi0022/h. — Part B ch. B1 (AXI4-Lite) for the master-write capture port.
- Hopkins, A.B.T., McDonald-Maier, K.D. “Debug Support for Complex Systems On-Chip: A Review.” *IEE Proc. Computers and Digital Techniques*, 153(4):197-207, 2006. DOI: 10.1049/ip-cdt:20050194. — related work: on-chip trace buffers versus trace ports.
- Yang, F.-C., Lin, Y., Kao, C.-F., Huang, I.-J. “An On-Chip AHB Bus Tracer With Real-Time Compression and Dynamic Multiresolution Supports for SoC.” *IEEE Trans. VLSI Systems*, 19(4):571-584, Apr. 2011. DOI: 10.1109/TVLSI.2009.2039811. — related work: a bus tracer that records into an on-chip circular buffer.
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

25.10.2 Source Code

- RTL: `rtl/amba/ocelus/monbus_axi4_axil4_group.sv`
- Core: `rtl/amba/ocelus/monbus_group_core.sv`

25.10.3 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Group Core: `docs/markdown/rtl-amba/monitor/monbus_group_core.md`
- Packet Format: `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`

Last Updated: 2026-07-15

25.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

26 Monitor Bus Group — AXIL Slave-Read / AXI4 Master-Write

Module: `monbus_axil4_axi4_group.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: output plane

26.1 Overview

`monbus_axil4_axi4_group` is the mixed-fabric wrapper of the monitor-bus delivery family. It wraps the protocol-agnostic `monbus_group_core` with an **AXIL slave-read err-drain port** (the CPU IRQ handler pops error records) and a **full AXI4 burst master-write port** (streams captured records into a memory ring, bunching multiple records into multi-beat bursts to amortize address-channel overhead). It is the delivery block `stream_char` / `rapids_char` instantiate for MonBus egress when the drain fabric is AXIL but the memory ring lives behind an AXI4 fabric.

Like the AXIL/AXIL variant, this wrapper splits the arbitrated monitor-bus stream into a CPU-facing error drain and a memory-ring bulk dump. The difference is the dump side: a full AXI4 burst master lets the burst writer emit many records in a single AW + N × W + B, which is throughput-optimal when the ring sits behind an AXI4 fabric.

The core's FUBs are AXI4-shaped internally. On the read side the wrapper bridges the AXIL leaf into the core's AXI4-shaped read FUB (`arid=0 / arlen=0 / arsize=3 / arburst=INCR`). On the write side the core drives most AXI4 fields directly; the fields the core does not produce (`awlock / awcache / awqos / awregion / awuser / wuser`) are tied to safe defaults at the AXI4 leaf boundary.

Use it for:

- MonBus egress for `stream_char` / `rapids_char` with an AXIL drain and an AXI4 ring fabric
- CPU IRQ error drain plus high-throughput burst dump into memory
- Trace capture where amortizing AXI4 address-channel overhead matters

The key benefit: a lightweight AXIL CPU drain paired with a full AXI4 burst dump, so trace records land in memory with minimal address-channel overhead.

Feature summary:

- AXIL slave-read err-FIFO drain (CPU-facing IRQ path)
- AXI4 burst master-write bulk-capture (`MAX_BURST_BEATS` up to 256, default 64)

- Selectable err-drain data width: 64-bit (one beat per record slice) or 32-bit (2:1 read serializer for a 32-bit host crossbar)
- Shared per-protocol egress packet filter (AXI / AXIS / CORE)
- `irq_out` asserted whenever the err FIFO is non-empty
- Optional runtime compression (`USE_COMPRESSION` + `cfg_compress_en`)
- Compressor statistics and FIFO status/count outputs

26.2 Parameters

Parameter	Type	Default	Description
FIFO_DEPTH_ERR	int	64	Error FIFO depth, in records
FIFO_DEPTH_WRITE	int	96	Write FIFO depth, in beats (96 beats = 32 raw records)
ADDR_WIDTH	int	32	Address width on the slave-read and master-write ports
S_AXIL_DATA_WIDTH	int	64	Err-drain read data width. 64 = one beat per 64-bit record slice; 32 = a 2:1 read serializer presents each slice as a low then high beat (6 beats/record) for a 32-bit host crossbar
AXI_ID_WIDTH	int	8	Master-write AXI4 ID width
AXI_USER_WIDTH	int	1	Master-write AXI4 USER width
MAX_BURST_BEA	int	64	Maximum beats

Parameter	Type	Default	Description
TS			per master-write AW (AXI4 protocol max is 256)
FLUSH_TIMEOUT_CYCLES	int	1024	Cycles since last accepted W handshake before a timeout-driven flush
NUM_PROTOCOLS	int	3	Informational — AXI / AXIS / CORE filter configs are unconditional
USE_COMPRESSION	int	0	Elaboration: 0 omits the compressor (raw-only); 1 elaborates it for runtime selection via `cfg_compress_en`
SKID_DEPTH_AR	int	2	Slave-read AR skid depth
SKID_DEPTH_R	int	4	Slave-read R skid depth
SKID_DEPTH_AW	int	2	Master-write AW skid depth
SKID_DEPTH_W	int	4	Master-write W skid depth (deeper to absorb burst W)
SKID_DEPTH_B	int	2	Master-write B skid depth

26.3 Ports

26.3.1 Clock, Reset, and CAM Clear

Port	Direction	Width	Description
axi_aclk	input	1	Clock
axi_aresetn	input	1	Active-low asynchronous reset
cam_clear	input	1	Synchronous clear of the compressor CAM + stat counters (tied off in raw-only builds)

26.3.2 Monitor-Bus Input + Timestamp

Port	Direction	Width	Description
monbus_valid	input	1	Monitor-bus packet valid
monbus_ready	output	1	Monitor-bus ready (backpressure to the arbiter)
monbus_packet	input	`monitor_packet_t`	128-bit monitor packet
monbus_timestamp	input	`monbus_timestamp_t`	64-bit source timestamp
mon_time_out	output	`monbus_timestamp_t`	Free-running 64-bit timestamp counter

26.3.3 AXIL Slave Read — Err-FIFO Drain

Port	Direction	Width	Description
s_axil_arvalid	input	1	Read-address valid
s_axil_arready	output	1	Read-address

Port	Direction	Width	Description
			ready
s_axil_araddr	input	ADDR_WIDTH	Read address
s_axil_arprot	input	3	Read protection attributes
s_axil_rvalid	output	1	Read-data valid
s_axil_rready	input	1	Read-data ready
s_axil_rdata	output	S_AXIL_DATA_WIDTH	Read data (record slice, or half-slice in 32-bit mode)
s_axil_rresp	output	2	Read response

26.3.4 AXI4 Master Write — Burst Bulk Capture

Port	Direction	Width	Description
m_axi_awid	output	AXI_ID_WIDTH	Write-address ID
m_axi_awaddr	output	ADDR_WIDTH	Write burst base address (within the ring window)
m_axi_awlen	output	8	Burst length minus 1 (`sub_burst_beats - 1`)
m_axi_awsize	output	3	Beat size (fixed at 3 = 8 bytes)
m_axi_awburst	output	2	Burst type (fixed INCR)
m_axi_awlock	output	1	Lock (tied 0 at the leaf)
m_axi_awcache	output	4	Cache (tied `4'b0010` Normal Non-cacheable Bufferable)
m_axi_awprot	output	3	Protection (tied 0)
m_axi_awqos	output	4	QoS (tied 0)

Port	Direction	Width	Description
m_axi_awregion	output	4	Region (tied 0)
m_axi_awuser	output	AXI_USER_WIDTH	User (tied 0)
m_axi_awvalid	output	1	Write-address valid
m_axi_awready	input	1	Write-address ready
m_axi_wdata	output	64	Write data (one 64-bit beat)
m_axi_wstrb	output	8	Write strobe
m_axi_wlast	output	1	Last beat of the sub-burst
m_axi_wuser	output	AXI_USER_WIDTH	Write-data user (tied 0)
m_axi_wvalid	output	1	Write-data valid
m_axi_wready	input	1	Write-data ready
m_axi_bid	input	AXI_ID_WIDTH	Write-response ID
m_axi_bresp	input	2	Write response
m_axi_buser	input	AXI_USER_WIDTH	Write-response user (unused)
m_axi_bvalid	input	1	Write-response valid
m_axi_bready	output	1	Write-response ready

26.3.5 Interrupt

Port	Direction	Width	Description
irq_out	output	1	Asserted whenever the err FIFO is non-empty

26.3.6 Configuration

Port	Direction	Width	Description
cfg_base_addr	input	ADDR_WIDTH	Master-write ring base address
cfg_limit_addr	input	ADDR_WIDTH	Master-write ring limit address (writer wraps, does not saturate)
cfg_flush_watermark	input	16	Write-FIFO depth (beats) at which a flush fires
cfg_compress_en	input	1	Runtime compression enable (effective only when `USE_COMPRESSION=1`)

26.3.7 Per-Protocol Filter Masks

Three protocols (AXI, AXIS, CORE), each with a packet-type mask, an err-select mask, and per-event-category masks. All are 16-bit inputs — same shape as the AXIL/AXIL variant.

Port	Protocol	Role
cfg_axi_pkt_mask	AXI	Drop by packet type
cfg_axi_err_select	AXI	Route to err FIFO by packet type
cfg_axi_error_mask / cfg_axi_timeout_mask / cfg_axi_compl_mask / cfg_axi_thresh_mask / cfg_axi_perf_mask / cfg_axi_addr_mask / cfg_axi_debug_mask	AXI	Per-event-code drop masks
cfg_axis_pkt_mask	AXIS	Drop by packet type
cfg_axis_err_select	AXIS	Route to err FIFO by packet type

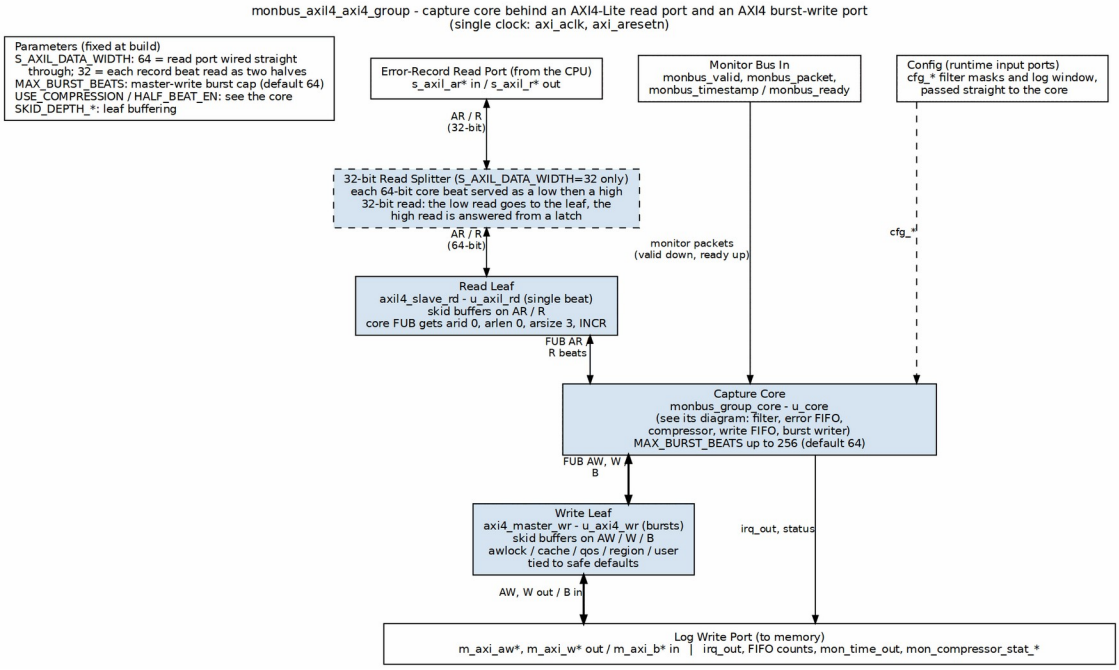
Port	Protocol	Role
cfg_axis_error_mask / cfg_axis_timeout_mask / cfg_axis_compl_mask / cfg_axis_credit_mask / cfg_axis_channel_mask / cfg_axis_stream_mask	AXIS	Per-event-code drop masks
cfg_core_pkt_mask	CORE	Drop by packet type
cfg_core_err_select	CORE	Route to err FIFO by packet type
cfg_core_error_mask / cfg_core_timeout_mask / cfg_core_compl_mask / cfg_core_thresh_mask / cfg_core_perf_mask / cfg_core_debug_mask	CORE	Per-event-code drop masks

26.3.8 Status / Debug

Port	Direction	Width	Description
err_fifo_full	output	1	Err FIFO write port not ready
write_fifo_full	output	1	Write FIFO write port not ready
err_fifo_count	output	16	Err FIFO entry count (records)
write_fifo_count	output	16	Write FIFO entry count (beats)
mon_compressor_stat_tier1_a / _tier1_b / _tier1_c / _tier0 / _cam_miss / _delta_ts_ovf / _event_data_ovf / _ed_delta_ovf	output	32 each	Compressor statistics (live only when `USE_COMPRESSION=1`)

26.4 Functional Description

26.4.1 Block diagram



monbus_axil4_axi4_group block diagram

A structural wrapper over monbus_group_core, drawn as one box. Error records are read over AXI4-Lite (axil4_slave_rd). The wrapper supplies arid 0, arlen 0, arsize 3 and INCR to the core's read FUB, and with S_AXIL_DATA_WIDTH=32 the dashed splitter serves each 64-bit record beat as two 32-bit reads. The log is written in AXI4 bursts through axi4_master_wr.

26.4.2 S-Side Err-Drain Read Protocol

Each monbus err record is three 64-bit slices — `{tag, source_ts}`, `packet[127:64]`, `packet[63:0]`. A 64-bit `axil4_slave_rd` leaf bridges the external AXIL read onto the core's 64-bit read FUB, and the core's slice counter returns the three slices in sequence; the err-FIFO record is popped only after the packet-low slice is read.

- **64-bit drain (`S_AXIL_DATA_WIDTH == 64`):** the leaf is wired straight through — 3 beats/record.
- **32-bit drain (`S_AXIL_DATA_WIDTH == 32`):** a phase bit splits each 64-bit leaf beat into a low then high 32-bit external read — 6 beats/record. The AR is forwarded to the leaf only on the low phase (one core read per pair, no prefetch); the low read consumes the beat and latches its high half; the high read replays the latch, gating R on its own accepted AR (`r_hi_ar`) to

honor AXIL AR-before-R ordering. This is the identical mechanism used in `monbus_axil4_axil4_group`.

26.4.3 M-Side Bulk-Capture Protocol (AXI4 Burst)

Surviving (non-err) packets go to the write FIFO and are streamed out the AXI4 master-write port by the core's burst writer. Because this is a full AXI4 master, one drain cycle typically becomes one large sub-burst — e.g. 24 beats = 8 raw records in a single AW + 24 × W + B — capped per AW by `MAX_BURST_BEATS`. `awsize` is fixed at 3 (8 bytes/beat), `awburst` at INCR, `awlen = sub_burst_beats - 1`, and `wlast` asserts on the final beat of each sub-burst. The address is **circular** within `[cfg_base_addr, cfg_limit_addr]` (wraps, does not saturate). The writer fires on watermark or timeout. See `monbus_group` for the full burst-writer geometry (3-stage pipeline, mod-3 rounding, 4KB-boundary respect, rewind-snap).

The core drives `awid / awaddr / awlen / awsize / awburst` and `wdata / wstrb / wlast / wvalid`. The AXI4 fields the core does not produce are tied to safe defaults at the leaf: `awlock=0`, `awcache=4'b0010` (Normal Non-cacheable Bufferable), `awprot=0`, `awqos=0`, `awregion=0`, `awuser=0`, `wuser=0`.

26.4.4 Interrupt

`irq_out` is asserted by the core whenever the err FIFO is non-empty.

26.4.5 Shared Egress Packet Filter

The per-protocol filter (AXI, AXIS, CORE) decides for each packet: drop, route to the err FIFO (`err_select`), or route to the write FIFO (default). Protocols not in the supported set (APB, ARB) are always dropped. The filter config is identical across all four family wrappers — see `monbus_group` for the per-event category tables and masking rules.

26.4.6 Compression

When `USE_COMPRESSION=1`, `monbus_compressor` is selectable at runtime via `cfg_compress_en`, emitting a 64-bit self-tagged slot per COMPRESSED record instead of three raw beats (an incompressible record escapes to tier-0 and still costs three); `cam_clear` synchronously empties the template CAM and zeroes stats between runs. In raw-only builds these fold away. (Unlike the AXIL/AXIL variant, this wrapper does not expose `HALF_BEAT_EN`.)

26.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AR`	2 entries
`SKID_DEPTH_R`	4 entries
`SKID_DEPTH_AW`	2 entries

Skid parameter	Default depth
SKID_DEPTH_W	4 entries
SKID_DEPTH_B	2 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

26.6 Usage Examples

```
monbus_axil4_axi4_group #(
    .FIFO_DEPTH_ERR       (64),
    .FIFO_DEPTH_WRITE     (96),    // beats (3x for raw mode)
    .ADDR_WIDTH           (32),
    .S_AXIL_DATA_WIDTH    (64),    // or 32 for a narrow host crossbar
    .AXI_ID_WIDTH         (8),
    .MAX_BURST_BEATS      (64),
    .FLUSH_TIMEOUT_CYCLES (1024),
    .USE_COMPRESSION      (0)
) u_monbus_egress (
    .axi_aclk    (aclk),
    .axi_aresetn (aresetn),
    .cam_clear   (1'b0),

    // Arbitrated monitor-bus input
    .monbus_valid     (mon_valid),
    .monbus_ready     (mon_ready),
    .monbus_packet    (mon_packet),
    .monbus_timestamp (mon_timestamp),
    .mon_time_out     (mon_time),

    // CPU-facing AXIL err drain
    .s_axil_arvalid (cpu_arvalid),
    .s_axil_arready (cpu_arready),
    .s_axil_araddr  (cpu_araddr),
    .s_axil_arprot  (cpu_arprot),
    .s_axil_rvalid  (cpu_rvalid),
    .s_axil_rready  (cpu_rready),
```

```
    .s_axil_rdata   (cpu_rdata),
    .s_axil_rresp   (cpu_rresp),

    // AXI4 burst memory-ring dump
    .m_axi_awid    (ring_awid),
    .m_axi_awaddr  (ring_awaddr),
    .m_axi_awlen   (ring_awlen),
    /* ... remaining m_axi_aw*, m_axi_w*, m_axi_b* ... */

    .irq_out             (monbus_irq),
    .cfg_base_addr       (RING_BASE),
    .cfg_limit_addr      (RING_LIMIT),
    .cfg_flush_watermark (16'd24),
    .cfg_compress_en     (1'b0),

    // Per-protocol filter masks (AXI / AXIS / CORE) ...
    .cfg_axi_pkt_mask    (axi_pkt_mask),
    .cfg_axi_err_select  (axi_err_select),
    /* ... remaining masks ... */

    .err_fifo_count   (err_count),
    .write_fifo_count (wr_count)
    /* ... compressor stats ... */
);
```

26.7 Design Notes

26.7.1 When to Pick This Over AXIL/AXIL

Choose this wrapper when the memory ring lives behind an AXI4 fabric and you want to bunch records into multi-beat bursts to amortize address-channel overhead. The memory image is identical to the AXIL-master case; only the address-channel handshake count differs. Pick `MAX_BURST_BEATS` to taste (up to 256).

26.7.2 AXI4 Leaf Default Tie-Offs

The core only produces the AXI4 fields it needs. The remaining AXI4 qualifier fields (`awlock / awcache / awqos / awregion / awuser / wuser`) are tied at the `axi4_master_wr` leaf boundary, with `awcache` set to Normal Non-cacheable Bufferable — appropriate for a trace-ring write that must reach memory.

26.7.3 Why One Core + Four Wrappers

The filter, FIFOs, burst writer, and drain live once in `monbus_group_core`; the AXIL read leaf, the AXI4 write leaf, and the two FUB bridges live here. Each family wrapper carries only the exact port shape its fabric demands.

26.8 Related Modules

Used by:

- `stream_char` / `rapids_char` MonBus egress (AXIL drain + AXI4 ring)
- Trace-capture topologies needing high-throughput burst dump into AXI4 memory

Uses:

- **monbus_group_core.sv** — Protocol-agnostic filter + FIFO + burst-writer + drain
- **axil4_slave_rd.sv** — Slave-read (err-drain) skid leaf
- **axi4_master_wr.sv** — Master-write (burst bulk-capture) skid leaf
- **monbus_compressor.sv** — Optional runtime compressor (`USE_COMPRESSION=1`)
- **monitor_common_pkg** — `monitor_packet_t` / `monbus_timestamp_t` types
- **reset_defs.svh** — Reset macros

See also:

- **monbus_axil4_axil4_group.sv** — Same drain, AXIL single-beat master-write
- **monbus_axi4_axil4_group.sv** / **monbus_axi4_axi4_group.sv** — AXI4-burst slave-read variants
- **monbus_arbiter.sv** — Upstream multi-source merge (instantiate before this wrapper for N>1 sources)
- **axi4_intf_master_observer.sv** — Instantiates this wrapper as its central filter + err FIFO + dump writer (`projects/components/utility-ip/misc/rtl/`)

26.9 Testing

- `val/amba/test_monbus_axil4_axi4_group.py`

26.10 References

26.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021. developer.arm.com/documentation/ihi0022/h. — Part B ch. B1 (AXI4-Lite) for the slave-read drain port.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. — AXI4 burst master-write capture port.
- Hopkins, A.B.T., McDonald-Maier, K.D. “Debug Support for Complex Systems On-Chip: A Review.” *IEE Proc. Computers and Digital Techniques*, 153(4):197-207, 2006. DOI: 10.1049/ip-cdt:20050194. — related work: on-chip trace buffers versus trace ports.
- Yang, F.-C., Lin, Y., Kao, C.-F., Huang, I.-J. “An On-Chip AHB Bus Tracer With Real-Time Compression and Dynamic Multiresolution Supports for SoC.” *IEEE Trans. VLSI Systems*, 19(4):571-584, Apr. 2011. DOI: 10.1109/TVLSI.2009.2039811. — related work: a bus tracer that records into an on-chip circular buffer.
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

26.10.2 Source Code

- RTL: `rtl/amba/ocelus/monbus_axil4_axi4_group.sv`
- Core: `rtl/amba/ocelus/monbus_group_core.sv`

26.10.3 Documentation

- Family spec: `docs/markdown/rtl-amba/monitor/monbus_group.md`
- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Packet format: `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`

Last Updated: 2026-07-15

26.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

27 Monitor Bus Group — AXIL Slave-Read / AXIL Master-Write

Module: `monbus_axil4_axil4_group.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: output plane

27.1 Overview

`monbus_axil4_axil4_group` is the all-AXI4-Lite wrapper of the monitor-bus delivery family. It wraps the protocol-agnostic `monbus_group_core` with an **AXIL slave-read err-drain port** (for the CPU IRQ handler to pop error records) and an **AXIL single-beat master-write port** (to stream captured records into a memory ring). It is the delivery block that `stream_char` / `rapids_char` instantiate for MonBus egress when both the drain and dump fabrics are AXIL.

This wrapper replaces the legacy `monbus_axil_group.sv`, which fused the core and the AXIL skids into one module.

Here's the problem it solves. The monitor bus produces a single arbitrated stream of 128-bit packets plus a 64-bit timestamp. That stream has to be split into two destinations: error/interrupt records the CPU polls, and bulk trace records written into a memory ring for later offline analysis. `monbus_group_core` does the filtering, FIFOing, watermark/timeout burst-writing, and slave-read drain. This wrapper gives the core an exact AXIL port shape on both the CPU-facing (slave-read) and memory-facing (master-write) sides, so the caller ties off no spurious AXI4-only fields.

The core's FUBs are AXI4-shaped internally; the wrapper bridges the AXIL leaves to those FUBs (supplying `arid=0 / arlen=0 / arsize=3 / arburst=INCR` on the read side and forcing `MAX_BURST_BEATS=1` on the write side).

Use it for:

- MonBus egress for `stream_char` / `rapids_char` on an AXIL fabric
- CPU IRQ-driven error drain with a memory-mapped ring dump
- Minimal-footprint monitor delivery where both fabrics are AXIL
- 32-bit host crossbar drain via the built-in 2:1 read serializer

The key benefit: exact AXIL port shape on both sides with no fake AXI4 fields, plus a built-in 32-bit err-drain serializer for narrow host buses.

Feature summary:

- AXIL slave-read err-FIFO drain (CPU-facing IRQ path)
- AXIL single-beat master-write bulk-capture (memory-ring egress)

- Selectable err-drain data width: 64-bit (one beat per record slice) or 32-bit (2:1 read serializer for a 32-bit host crossbar)
- Shared per-protocol egress packet filter (AXI / AXIS / CORE)
- `irq_out` asserted whenever the err FIFO is non-empty
- Optional runtime compression (`USE_COMPRESSION` + `cfg_compress_en`) with half-beat packing (`HALF_BEAT_EN`)
- Compressor statistics and FIFO status/count outputs

27.2 Parameters

Parameter	Type	Default	Description
FIFO_DEPTH_ERR	int	64	Error FIFO depth, in records
FIFO_DEPTH_WRITE	int	96	Write FIFO depth, in beats (96 beats = 32 raw records)
ADDR_WIDTH	int	32	Address width on the slave-read and master-write ports
S_AXIL_DATA_WIDTH	int	64	Err-drain read data width. 64 = one beat per 64-bit record slice (3 beats/record); 32 = a 2:1 read serializer presents each slice as a low then high beat (6 beats/record) for a 32-bit host crossbar
FLUSH_TIMEOUT_CYCLES	int	1024	Cycles since last accepted W handshake before a timeout-driven

Parameter	Type	Default	Description
			flush
NUM_PROTOCOLS	int	3	Informational — AXI / AXIS / CORE filter configs are unconditional
USE_COMPRESSION	int	0	Elaboration: 0 omits the compressor (raw-only); 1 elaborates it for runtime selection via `cfg_compress_en`
HALF_BEAT_EN	int	0	Elaboration: pack two 30-bit half-slots per 64-bit beat downstream of the compressor (requires `USE_COMPRESSION=1`)
SKID_DEPTH_AR	int	2	Slave-read AR skid depth
SKID_DEPTH_R	int	4	Slave-read R skid depth
SKID_DEPTH_AW	int	2	Master-write AW skid depth
SKID_DEPTH_W	int	2	Master-write W skid depth
SKID_DEPTH_B	int	2	Master-write B skid depth

The AXIL master forces `MAX_BURST_BEATS=1` in the core, so this wrapper has no `MAX_BURST_BEATS` parameter (unlike the AXI4-master variant).

27.3 Ports

27.3.1 Clock, Reset, and CAM Clear

Port	Direction	Width	Description
axi_aclk	input	1	Clock
axi_aresetn	input	1	Active-low asynchronous reset
cam_clear	input	1	Synchronous clear of the compressor CAM + stat counters (tied off in raw-only builds)

27.3.2 Monitor-Bus Input + Timestamp

Port	Direction	Width	Description
monbus_valid	input	1	Monitor-bus packet valid
monbus_ready	output	1	Monitor-bus ready (backpressure to the arbiter)
monbus_packet	input	`monitor_packet_t`	128-bit monitor packet
monbus_timestamp	input	`monbus_timestamp_t`	64-bit source timestamp
mon_time_out	output	`monbus_timestamp_t`	Free-running 64-bit timestamp counter

27.3.3 AXIL Slave Read — Err-FIFO Drain

Port	Direction	Width	Description
s_axil_arvalid	input	1	Read-address valid
s_axil_arready	output	1	Read-address

Port	Direction	Width	Description
			ready
s_axil_araddr	input	ADDR_WIDTH	Read address (record-slice index; address value not memory-mapped by the core)
s_axil_arprot	input	3	Read protection attributes
s_axil_rvalid	output	1	Read-data valid
s_axil_rready	input	1	Read-data ready
s_axil_rdata	output	S_AXIL_DATA_WIDTH	Read data (record slice, or half-slice in 32-bit mode)
s_axil_rresp	output	2	Read response

27.3.4 AXIL Master Write — Bulk Capture

Port	Direction	Width	Description
m_axil_awvalid	output	1	Write-address valid
m_axil_awready	input	1	Write-address ready
m_axil_awaddr	output	ADDR_WIDTH	Write address (within `[cfg_base_addr, cfg_limit_addr]`)
m_axil_awprot	output	3	Write protection attributes
m_axil_wvalid	output	1	Write-data valid
m_axil_wready	input	1	Write-data ready
m_axil_wdata	output	64	Write data (one 64-bit beat)
m_axil_wstrb	output	8	Write strobe
m_axil_bvalid	input	1	Write-response

Port	Direction	Width	Description
			valid
m_axil_bready	output	1	Write-response ready
m_axil_bresp	input	2	Write response

27.3.5 Interrupt

Port	Direction	Width	Description
irq_out	output	1	Asserted whenever the err FIFO is non-empty

27.3.6 Configuration

Port	Direction	Width	Description
cfg_base_addr	input	ADDR_WIDTH	Master-write ring base address
cfg_limit_addr	input	ADDR_WIDTH	Master-write ring limit address (writer wraps, does not saturate)
cfg_flush_watermark	input	16	Write-FIFO depth (beats) at which a flush fires
cfg_compress_en	input	1	Runtime compression enable (effective only when `USE_COMPRESSION=1`)

27.3.7 Per-Protocol Filter Masks

Three protocols (AXI, AXIS, CORE), each with a packet-type mask, an err-select mask, and per-event-category masks. All are 16-bit inputs.

Port	Protocol	Role
cfg_axi_pkt_mask	AXI	Drop by packet type

Port	Protocol	Role
cfg_axi_err_select	AXI	Route to err FIFO by packet type
cfg_axi_error_mask / cfg_axi_timeout_mask / cfg_axi_compl_mask / cfg_axi_thresh_mask / cfg_axi_perf_mask / cfg_axi_addr_mask / cfg_axi_debug_mask	AXI	Per-event-code drop masks
cfg_axis_pkt_mask	AXIS	Drop by packet type
cfg_axis_err_select	AXIS	Route to err FIFO by packet type
cfg_axis_error_mask / cfg_axis_timeout_mask / cfg_axis_compl_mask / cfg_axis_credit_mask / cfg_axis_channel_mask / cfg_axis_stream_mask	AXIS	Per-event-code drop masks
cfg_core_pkt_mask	CORE	Drop by packet type
cfg_core_err_select	CORE	Route to err FIFO by packet type
cfg_core_error_mask / cfg_core_timeout_mask / cfg_core_compl_mask / cfg_core_thresh_mask / cfg_core_perf_mask / cfg_core_debug_mask	CORE	Per-event-code drop masks

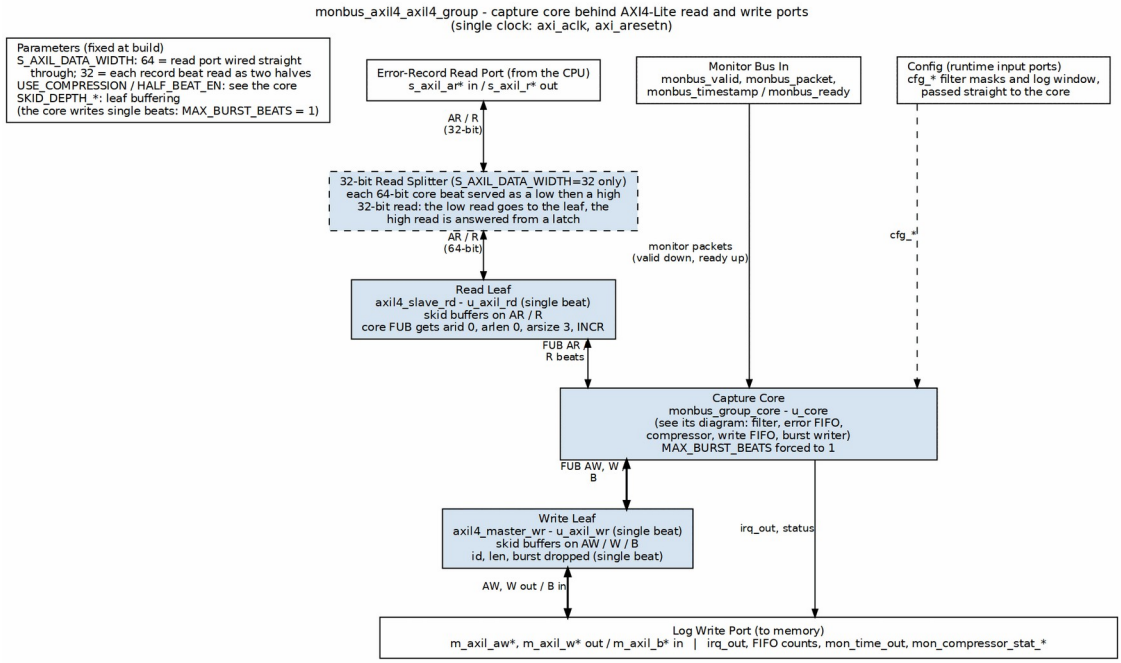
27.3.8 Status / Debug

Port	Direction	Width	Description
err_fifo_full	output	1	Err FIFO write port not ready
write_fifo_full	output	1	Write FIFO write port not ready

Port	Direction	Width	Description
err_fifo_count	output	16	Err FIFO entry count (records)
write_fifo_count	output	16	Write FIFO entry count (beats)
mon_compressor_stat_tier1_a / _tier1_b / _tier1_c / _tier0 / _cam_miss / _delta_ts_ovf / _event_data_ovf / _ed_delta_ovf	output	32 each	Compressor statistics (live only when `USE_COMPRESSION=1`)

27.4 Functional Description

27.4.1 Block diagram



monbus_axil4_axil4_group block diagram

A structural wrapper over monbus_group_core, drawn as one box, with AXI4-Lite on both sides. Error records are read through axil4_slave_rd; with S_AXIL_DATA_WIDTH=32 the dashed splitter

serves each 64-bit beat as two 32-bit reads. The log is written through axil4_master_wr one beat at a time (MAX_BURST_BEATS forced to 1).

27.4.2 S-Side Err-Drain Read Protocol

Each monbus err record is three 64-bit slices — `{tag, source_ts}`, `packet[127:64]`, `packet[63:0]`. A 64-bit `axil4_slave_rd` leaf bridges the external AXIL read onto the core's 64-bit read FUB, and the core's slice counter returns the three slices in sequence; the err-FIFO record is popped only after slice 2 (packet low half) is read.

- **64-bit drain (`S_AXIL_DATA_WIDTH == 64`, `g_drain_direct`):** the leaf is wired straight through, one beat per slice — 3 beats/record.
- **32-bit drain (`S_AXIL_DATA_WIDTH == 32`, `g_drain_2to1`):** a phase bit splits each 64-bit leaf beat into a low then high 32-bit external read — 6 beats/record. The external AR is forwarded to the leaf only on the low phase, so the leaf issues exactly one core read per pair and never prefetches. The low read streams and consumes the beat while latching its high half; the high read replays the latch and gates R on its own accepted AR (`r_hi_ar`) to honor AXIL AR-before-R ordering (a master still holding `rready` from the low beat cannot consume the high beat early, which would desync the pair and double the core reads).

27.4.3 M-Side Bulk-Capture Protocol

Surviving (non-err) packets go to the write FIFO and are streamed out the AXIL master-write port by the core's burst writer. Because this is an AXIL master, `MAX_BURST_BEATS=1` — each record is emitted as single-beat AW/W/B handshakes at consecutive addresses (three per raw record). The writer fires on watermark (`write_fifo_count >= cfg_flush_watermark`) or timeout (`FLUSH_TIMEOUT_CYCLES`), and the address is **circular** within `[cfg_base_addr, cfg_limit_addr]` — it wraps rather than saturating. Host-side ring pointers must match this wrap behavior. See `monbus_group` for the full burst-writer geometry.

27.4.4 Interrupt

`irq_out` is asserted by the core whenever the err FIFO is non-empty, so a CPU can take an interrupt and drain records over the slave-read port.

27.4.5 Shared Egress Packet Filter

The per-protocol filter (AXI, AXIS, CORE) decides for each packet: drop (via `pkt_mask` or a per-event mask), route to the err FIFO (via `err_select`), or route to the write FIFO (default). Protocols not in the supported set (APB, ARB) are always dropped. The filter config is identical across all four family wrappers — see `monbus_group` for the per-event category tables and masking rules.

27.4.6 Compression

When `USE_COMPRESSION=1`, `monbus_compressor` can be selected at runtime via `cfg_compress_en`, emitting a 64-bit self-tagged slot per COMPRESSED record into the write FIFO instead of three raw beats (an incompressible record escapes to tier-0 and still costs three); `HALF_BEAT_EN=1` further packs two 30-bit half-slots per beat. `cam_clear` synchronously empties the compressor template CAM and zeroes its stat counters between runs. In raw-only builds these fold away.

27.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AR`	2 entries
`SKID_DEPTH_R`	4 entries
`SKID_DEPTH_AW`	2 entries
`SKID_DEPTH_W`	2 entries
`SKID_DEPTH_B`	2 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

27.6 Usage Examples

```
monbus_axil4_axil4_group #(
    .FIFO_DEPTH_ERR       (64),
    .FIFO_DEPTH_WRITE     (96),    // beats (3x for raw mode)
    .ADDR_WIDTH           (32),
    .S_AXIL_DATA_WIDTH    (64),    // or 32 for a narrow host crossbar
    .FLUSH_TIMEOUT_CYCLES (1024),
    .USE_COMPRESSION      (0)
) u_monbus_egress (
    .axi_aclk    (aclk),
    .axi_aresetn (aresetn),
```

```
    .cam_clear   (1'b0),

    // Arbitrated monitor-bus input
    .monbus_valid     (mon_valid),
    .monbus_ready     (mon_ready),
    .monbus_packet    (mon_packet),
    .monbus_timestamp (mon_timestamp),
    .mon_time_out     (mon_time),

    // CPU-facing err drain
    .s_axil_arvalid (cpu_arvalid),
    .s_axil_arready (cpu_arready),
    .s_axil_araddr  (cpu_araddr),
    .s_axil_arprot  (cpu_arprot),
    .s_axil_rvalid  (cpu_rvalid),
    .s_axil_rready  (cpu_rready),
    .s_axil_rdata   (cpu_rdata),
    .s_axil_rresp   (cpu_rresp),

    // Memory-ring dump
    .m_axil_awvalid (ring_awvalid),
    .m_axil_awready (ring_awready),
    .m_axil_awaddr  (ring_awaddr),
    /* ... m_axil_w*, m_axil_b* ... */

    .irq_out             (monbus_irq),
    .cfg_base_addr       (RING_BASE),
    .cfg_limit_addr      (RING_LIMIT),
    .cfg_flush_watermark (16'd24),
    .cfg_compress_en     (1'b0),

    // Per-protocol filter masks (AXI / AXIS / CORE) ...
    .cfg_axi_pkt_mask    (axi_pkt_mask),
    .cfg_axi_err_select  (axi_err_select),
    /* ... remaining masks ... */

    .err_fifo_count   (err_count),
    .write_fifo_count (wr_count)
    /* ... compressor stats ... */
);
```

27.7 Design Notes

27.7.1 Legacy Replacement

This wrapper (plus its three siblings) replaces the fused `monbus_axil_group.sv`. Key surface changes from the legacy module: `cfg_flush_watermark` is a new required input, `err_fifo_count` / `write_fifo_count` are 16 bits, and `FIFO_DEPTH_WRITE` is in beats (not records). See the migration section in `monbus_group`.

27.7.2 Why One Core + Four Wrappers

SystemVerilog cannot conditionally include ports in a single module's port list, so each protocol combination gets its own thin wrapper with the exact port shape. The filter, FIFOs, burst writer, and drain live once in `monbus_group_core`; the AXIL leaves and the FUB bridge live here.

27.7.3 32-Bit Drain Serializer

The `g_drain_2to1` path exists for 32-bit host crossbars. It is a careful 2:1 serializer: consuming the leaf beat on the low read (rather than holding it across both) makes `s_axil_rvalid` pulse once per external beat, so a master holding `rready` cannot mis-toggle the phase. See the shared mechanism note in the AXIL/AXI4 wrapper — the two wrappers use identical code.

27.8 Related Modules

Used by:

- `stream_char` / `rapids_char` MonBus egress (AXIL fabric)
- Any monitor-delivery topology with AXIL drain and AXIL dump fabrics

Uses:

- **monbus_group_core.sv** — Protocol-agnostic filter + FIFO + burst-writer + drain
- **axil4_slave_rd.sv** — Slave-read (err-drain) skid leaf
- **axil4_master_wr.sv** — Master-write (bulk-capture) skid leaf
- **monbus_compressor.sv** — Optional runtime compressor (`USE_COMPRESSION=1`)
- **monitor_common_pkg** — `monitor_packet_t` / `monbus_timestamp_t` types
- **reset_defs.svh** — Reset macros

See also:

- **monbus_axil4_axi4_group.sv** — Same drain, AXI4-burst master-write

- **monbus_axi4_axil4_group.sv** / **monbus_axi4_axi4_group.sv** — AXI4-burst slave-read variants
- **monbus_arbiter.sv** — Upstream multi-source merge (instantiate before this wrapper for N>1 sources)
- **sdpram_slave_axil_axil.sv** — Canonical memory-ring backend for the master-write port

27.9 Testing

- `val/amba/test_monbus_axil4_axil4_group.py`
- `val/amba/test_monbus_axil4_axil4_group_compressed.py`
- `val/amba/test_monbus_axil4_axil4_group_master_write.py`

27.10 References

27.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021. developer.arm.com/documentation/ihi0022/h. — Part B ch. B1 (AXI4-Lite) for both ports.
- Hopkins, A.B.T., McDonald-Maier, K.D. “Debug Support for Complex Systems On-Chip: A Review.” *IEE Proc. Computers and Digital Techniques*, 153(4):197-207, 2006. DOI: 10.1049/ip-cdt:20050194. — related work: on-chip trace buffers versus trace ports.
- Yang, F.-C., Lin, Y., Kao, C.-F., Huang, I.-J. “An On-Chip AHB Bus Tracer With Real-Time Compression and Dynamic Multiresolution Supports for SoC.” *IEEE Trans. VLSI Systems*, 19(4):571-584, Apr. 2011. DOI: 10.1109/TVLSI.2009.2039811. — related work: a bus tracer that records into an on-chip circular buffer.
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

27.10.2 Source Code

- RTL: `rtl/amba/ocelus/monbus_axil4_axil4_group.sv`

- Core: `rtl/amba/ocelus/monbus_group_core.sv`

27.10.3 Documentation

- Family spec: `docs/markdown/rtl-amba/monitor/monbus_group.md`
- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Packet format: `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`

Last Updated: 2026-07-15

27.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

28 Monitor Bus LRU CAM

Module: monbus_cam.sv **Location:** rtl/amba/ocelus/ **Category:** Bulk-Trace Compression Infrastructure **Status:** Reference design — superseded in-production by monbus_cam_pipe

Plane: output plane

28.1 Overview

> **Deprecation note.** monbus_cam.sv is the single-cycle reference design. The in-production CAM inside monbus_compressor is now monbus_cam_pipe.sv, a 2-cycle pipelined variant that splits the 49-bit compare → priority-encode → move-to-front chain into two stages so the path closes at 100 MHz on Nexys A7 (f909f01f / 0d1c0a1a). The two share LRU semantics and per-entry storage, but they are **not** interchangeable in interface: monbus_cam_pipe adds a cycle of latency and has **no ACTION_NONE** — every access_en cycle is a commit, with TOUCH/INSTALL derived from hit/miss, because the commit lands before the tier decision exists. It also has **no skid and no credit logic**; the credit-gated result skid (u_res_skid / r_credit) lives in monbus_compressor, and it is what sets sustained tier-1 throughput. At SKID_DEPTH = 3 (the current value) that is 1 record/cycle measured; at the earlier depth of 2 it measured 0.67. See monbus_compressor. Both files are kept in tree: this single-cycle module serves as the executable spec for the LRU semantics and the compressor's CAM behavior, and is what the algorithmic tests (test_monbus_cam.py) target. New code should instantiate monbus_cam_pipe.

monbus_cam is a **true-LRU caching content-addressable memory** that defines the template-index semantics the monbus_compressor depends on (the compressor instantiates the pipelined monbus_cam_pipe; this module is the executable spec). It assigns 5-bit template indices (tmpl_idx) to the 49-bit template keys it extracts from monitor packets. "True LRU" means the eviction victim is the least recently *accessed* entry (matched, touched, or installed) — not the least recently *inserted*. This matters because the bulk-trace compression format relies on both encoder and decoder maintaining identical CAM state from the slot stream alone; if the two sides diverge on eviction order, the decoder produces garbage.

The module is a **bit-exact mirror** of the Python Cam class in bin/TBClasses/monbus/monbus_compressor.py. Any divergence between the two implementations is a regression.

At a glance:

- 32-entry capacity (locked by the bulk-trace format spec)
- 49-bit key, 64-bit payload (both parameterizable)
- **Per-entry timestamp storage** (`TS_WIDTH=24`) for per-template `delta_ts` — see the dedicated section below
- Single combinational access port: lookup + commit in one cycle
- True LRU eviction via **position-indexed storage** — the slot index IS the recency rank (slot 0 = MRU, slot `DEPTH-1` = LRU)
- 3 caller-driven actions: `NONE`, `TOUCH`, `INSTALL`
- Eviction pulse output for stats / instrumentation
- Simulation-only protocol assertions on the caller

28.2 Parameters

The default values (`KEY_WIDTH=49`, `DATA_WIDTH=64`, `DEPTH=32`) are what the compressor instantiates and what the locked format spec mandates. The parameters exist so the module can be reused in other contexts (e.g. a smaller on-chip event cache) but `monbus_compressor` itself doesn't override them.

Parameter	Type	Default	Description
`KEY_WIDTH`	int	49	Template key width (locked by the format spec)
`DATA_WIDTH`	int	64	Payload width
`TS_WIDTH`	int	24	Per-entry `last_ts` width (per-template `delta_ts`)
`DEPTH`	int	32	Entry capacity (locked at 32 by the format spec)
`IDX_WIDTH`	int	`(DEPTH > 1) ? $clog2(DEPTH) : 1`	Derived index width
`CNT_WIDTH`	int	`$clog2(DEPTH + 1)`	Derived count width

If you change DEPTH, the corresponding change in the Python golden's DEFAULT_CAM_SIZE constant must happen in lockstep — otherwise the encoded slot stream will diverge.

28.3 Ports

```
module monbus_cam #(
    parameter int KEY_WIDTH  = 49,
    parameter int DATA_WIDTH = 64,
    parameter int TS_WIDTH   = 24,   // per-entry last_ts width (per-
template delta_ts)
    parameter int DEPTH      = 32,
    parameter int IDX_WIDTH  = (DEPTH > 1) ? $clog2(DEPTH) : 1,
    parameter int CNT_WIDTH  = $clog2(DEPTH + 1)
) (
    input  logic                    clk,
    input  logic                    rst_n,

    // Access port (one combinational lookup + one commit per cycle)
    input  logic [KEY_WIDTH-1:0]  access_key,
    output logic                    access_hit,
    output logic [IDX_WIDTH-1:0]  access_idx,       // position rank
(only valid on hit)
    output logic [DATA_WIDTH-1:0] access_old_data,  // pre-commit
payload at access_idx
    output logic [TS_WIDTH-1:0]   access_old_ts,    // pre-commit
timestamp at access_idx

    input  logic [1:0]            access_action,
    input  logic [DATA_WIDTH-1:0] access_new_data,
    input  logic [TS_WIDTH-1:0]   access_new_ts,    // timestamp to
write on TOUCH / INSTALL

    // Status
    output logic                    cam_full,
    output logic [CNT_WIDTH-1:0]  cam_count,
    output logic                    evicted,          // pulses on full-
CAM INSTALL

    // Counting-consumer ports (additive; the compressor ties/ignores
these)
    output logic [KEY_WIDTH-1:0]  evict_key,        // victim key,
valid when evicted
    output logic [DATA_WIDTH-1:0] evict_data,       // victim data,
valid when evicted
    input  logic [IDX_WIDTH-1:0]  dump_idx,         // position to
```

```
observe
    output logic                  dump_valid,       // entry at
dump_idx is occupied
    output logic [KEY_WIDTH-1:0]  dump_key,
    output logic [DATA_WIDTH-1:0] dump_data,
    input  logic                  soft_clear        // synchronous
invalidate-all
);
```

28.3.1 Counting-Consumer Ports

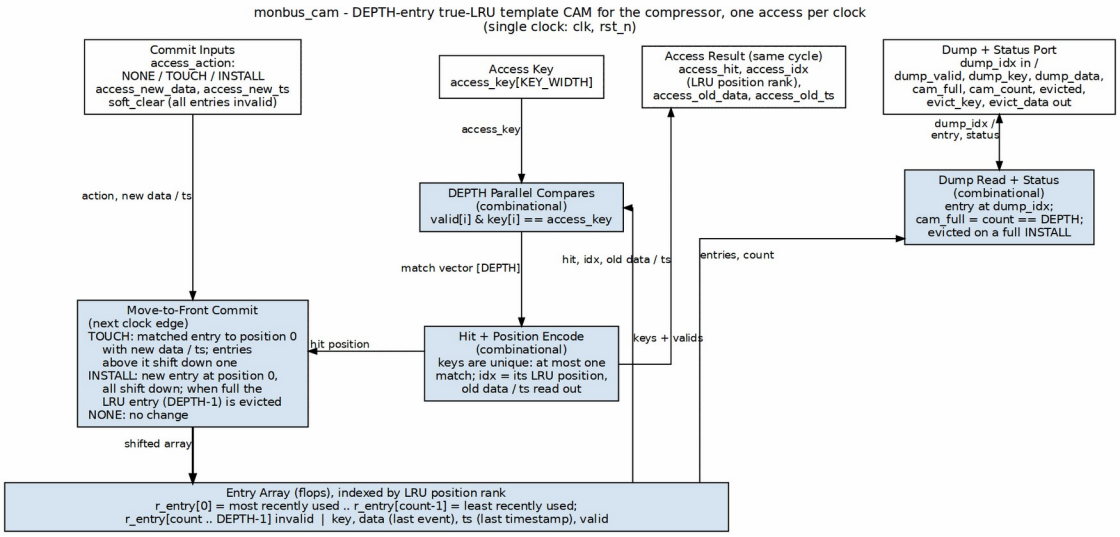
The compressor uses only the access port + `evicted`. A second class of consumer — a **counting** cache such as `monbus_pkt_tally`, where the payload is a partial count that must survive eviction — needs to see the victim and to walk live entries. These ports were added **additively** for that use; they do not change the LRU behaviour the compressor golden depends on, and any instantiation that does not need them may tie `dump_idx`/`soft_clear` low and leave the new outputs open. (The compressor no longer instantiates *this* module – it uses `monbus_cam_pipe`, which has neither port.)

Port	Direction	Width	Description
`evict_key` / `evict_data`	output	KEY_WIDTH / DATA_WIDTH	The LRU victim (position `DEPTH`-1), combinational, valid the cycle `evicted` is high. A counting consumer folds `evict_data` back into its backing store before the entry is lost.
`dump_idx`	input	IDX_WIDTH	Position to observe (for a freeze/flush walk over all entries).
`dump_valid` / `dump_key` / `dump_data`	output	1 / KEY_WIDTH / DATA_WIDTH	Occupancy + contents of `dump_idx`, purely observational (no state change).
`soft_clear`	input	1	Synchronous

Port	Direction	Width	Description
			invalidate-all (`cam_count → 0`) without an async reset pulse; re-arms the cache between capture windows. Takes priority over any concurrent `access_action`.

28.4 Functional Description

28.4.1 Block diagram



monbus_cam block diagram

The access key enters the center column, is compared against every valid entry at once, and the single match is encoded into its LRU position. Hit, position and the entry's old data and timestamp return to the caller in the same cycle. On the left, the commit applies the access on the next clock edge. A TOUCH moves the matched entry to position 0, and an INSTALL puts a new entry at position 0 and evicts the least recently used one when the CAM is full. The entry array is the bottom bar: the compares, the commit and the dump/status read on the right all work on it directly.

28.4.2 Storage Model: Position-Indexed LRU

The fundamental design choice is that **the slot index IS the position rank**:

```
r_entry[0]              = most-recently-used (MRU)
r_entry[1] .. r_entry[count-1]  = ordered, newer first
r_entry[count..DEPTH-1] = invalid (empty slots)
```

This means:

- The `access_idx` output on a hit is the entry's current position rank, which is exactly what `tmpl_idx` needs to be in the compressed slot stream.
- On `TOUCH` or `INSTALL`, the matched/new entry moves to slot 0 and older entries shift down by one position. This is one structural operation that updates *both* the storage AND the rank simultaneously.
- The LRU victim is always whoever sits at slot `DEPTH-1`. No tag pointers, no doubly-linked list, no per-entry counter — pure structural.

The trade-off: every `TOUCH`/`INSTALL` performs a shift of up to `DEPTH-1` entries on a single clock edge. For `DEPTH=32` this is well within timing budget (parallel per-slot updates in a generate loop) but it's the reason the format spec locks the size at 32 and not 64 or 128.

28.4.3 Actions

The caller drives exactly one action per cycle on the `access_action` port:

Action	Encoding	Caller protocol	State change on commit
`ACTION_NONE`	`2'b00`	Pure lookup — anytime	None. Just samples hit/idx/old_data.
`ACTION_TOUCH`	`2'b01`	Must coincide with a hit	Matched entry moves to slot 0, payload updated with `access_new_data`.
`ACTION_INSTALL`	`2'b10`	Must coincide with a miss	New entry installed at slot 0. If `cam_full`, the entry at slot `DEPTH-1` is evicted and `evicted`

Action	Encoding	Caller protocol	State change on commit
			pulses high that cycle.
2'b11	reserved	—	Treated as NONE (no state change).

Caller protocol enforcement (simulation-only, via `$error`):

- `TOUCH` without `access_hit` is illegal — the caller saw a miss but is claiming to be touching an existing entry.
- `INSTALL` while `access_hit` is illegal — the key is already present and reinstalling would create a duplicate.
- The internal match vector must be at most one-hot.

The natural compressor pattern is `TOUCH-on-hit / INSTALL-on-miss`, which satisfies all three constraints by construction.

28.4.4 Per-Slot Update Mechanics

On every clock edge:

Action	What happens to slot `i`
`NONE` or reserved	Unchanged.
`TOUCH` matching slot `P`	Slots `1..P` each take their predecessor's contents (slot `i` <= slot `i-1`), so the entries formerly at `0..P-1` shift down by one and the matched entry's old position `P` is overwritten. Slot 0 becomes the (matched key, new_data). Slots **above** `P` are untouched.
`INSTALL` when `!cam_full`	Insertion position is `cam_count`. Slots `1..cam_count` shift down from `0..cam_count-1`. Slot 0 becomes the new entry. `cam_count++`.
`INSTALL` when `cam_full`	Insertion position is `DEPTH-1` (overwriting the LRU). Slots `1..DEPTH-1` shift down from `0..DEPTH-2`. Slot 0 becomes the new entry. `evicted`

Action	What happens to slot `i`
	pulses high. `cam_count` stays at `DEPTH`.

A single `ALWAYS_FF_RST` block contains a `for` loop over slots 1..DEPTH-1, each iteration gated by `do_shift && (CNT_WIDTH'(i) <= shift_to)` (slot 0 is handled separately). Synthesis infers the same per-slot enables. This compiles to ~one LUT level per slot on the per-bit datapath — Vivado synthesises the whole shift as `DEPTH` parallel small update cones.

28.4.5 Per-Entry Timestamp Storage

Earlier revisions of the CAM stored only `(key, data)` per entry; the compressor measured `delta_ts` against a single global `r_last_ts`. That worked for single-source streams but collapsed compression to raw whenever multiple sources interleaved templates with non-monotonic absolute timestamps (the 4-channel STREAM characterization case).

The current CAM adds a **per-entry `r_ts[TS_WIDTH=24]` array** that shifts in lockstep with the key and data on every `TOUCH` / `INSTALL`:

```
r_key[i], r_data[i], r_ts[i]   shift together when slot i moves
```

`access_old_ts` outputs the *pre-commit* timestamp at the matched slot (valid only when `access_hit`) — i.e. the timestamp of the previous record that used this template. The compressor uses it to compute `delta_ts = src_ts_lo - cam_access_old_ts`, then writes the current record's `source_ts[23:0]` back into the slot via `access_new_ts` in the same cycle.

> **TS_WIDTH = 24 bits.** Format-B (the 23-bit-delta Tier-1 format) needs 24 bits to *detect* its delta overflow. 16 bits silently aliases large gaps to wrong encodes.

The CAM is therefore no longer pure opaque-payload — its caller needs to drive `access_new_ts` along with `access_new_data` on every `TOUCH` / `INSTALL`. The compressor wires this directly from the incoming record's low 24 timestamp bits.

28.4.6 Match Logic

The match vector is a parallel one-hot:

```
always_comb begin
    for (int i = 0; i < DEPTH; i++) begin
        w_match_oh[i] = r_valid[i] && (r_key[i] == access_key);
    end
end
```

That's 1 LUT per bit × 32 bits × 49-bit equality. Vivado fuses each equality into ~3 LUT levels; the whole match is independent of the rest of the cycle's logic (no chained dependencies on shift/count signals). The priority encoder feeding `access_idx` is then `DEPTH-1`-input.

28.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

28.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
monbus_cam #(
    .KEY_WIDTH              (49),
    .DATA_WIDTH             (64),
    .TS_WIDTH               (24),
    .DEPTH                  (32),
    .IDX_WIDTH              ((DEPTH > 1)),
    .CNT_WIDTH              ($clog2(DEPTH + 1))
) u_monbus_cam (
    .clk                    (clk),
    .rst_n                  (rst_n),
    .access_key             (access_key),
    .access_hit             (access_hit),
    .access_idx             (access_idx),
    .access_old_data        (access_old_data),
    .access_old_ts          (access_old_ts),
    .access_action          (access_action),
    .access_new_data        (access_new_data),
    .access_new_ts          (access_new_ts),
    .cam_full               (cam_full),
    .cam_count              (cam_count),
    .evicted                (evicted),
    .evict_key              (evict_key),
    .evict_data             (evict_data),
    .dump_idx               (dump_idx),
    .dump_valid             (dump_valid),
    .dump_key               (dump_key),
    .dump_data              (dump_data),
    .soft_clear             (soft_clear)
);
```

28.7 Design Notes

`cam_clear` is level-sensitive and must be driven. Sample it for one cycle while the CAM is idle; it empties the template store and zeroes the compressor statistics so the next run starts from a clean hit/miss profile, without asserting `axi_aresetn`. Leaving it unconnected means the profile never resets between runs.

Tied off in raw-only builds. With `USE_COMPRESSION=0` there is no CAM and the port folds away.

28.8 Testing

`val/amba/test_monbus_cam.py` runs 10 sub-tests covering:

1. Reset state (all empty, `cam_full=0`, no matches)
2. Install + lookup basic round-trip
3. Fill to `DEPTH-1` (no overflow path exercised)
4. `TOUCH` updates payload, idx becomes 0
5. `cam_full` asserts on the Nth install
6. LRU eviction on `INSTALL` when full
7. `evicted` pulses **only** on full-CAM install (not on lookup, not on touch)
8. Miss on absent key (no state change)
9. `TOUCH` moves the entry to MRU (the LRU-specific invariant)
10. Random stress (~500 ops at FUNC, 5000 at FULL) cross-checked against a Python LRU model

The Python model in the test is a bit-exact mirror of the RTL — same storage semantics, same shift rules. Random stress is constrained to the caller protocol (no `INSTALL` on hit, no `TOUCH` on miss) so the protocol assertions never fire spuriously.

```
pytest val/amba/test_monbus_cam.py -v
```

REG_LEVEL parameter sweep (`gate` / `func` / `full`):

- **GATE:** 1 config (default 49/64/32)
- **FUNC:** 2 configs (+ small DEPTH=8)
- **FULL:** 6 configs (DEPTH 4/8/16/32, key 16/32/49/64, data 16/32/64)

28.9 Related Modules

Module	Role
`monbus_compressor`	Consumer of the CAM *semantics* this module specifies — but it instantiates `monbus_cam_pipe`, not this module
`bin/TBClasses/monbus/monbus_compressor.py` (`Cam` class)	Python golden mirror
`monitor_trans_cam`	Sister CAM, different use case (AXI ID matching, multi-port, no LRU)

28.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Bentley, J.L., Sleator, D.D., Tarjan, R.E., Wei, V.K. "A Locally Adaptive Data Compression Scheme." *Communications of the ACM*, 29(4):320-330, Apr. 1986. DOI: 10.1145/5684.5688. — move-to-front list coding; the true-LRU order with the most recent entry at slot 0 is a move-to-front list kept in a CAM (comparison, not the source).
- Uzelac, V., Milenković, A. "A Real-Time Program Trace Compressor Utilizing Double Move-to-Front Method." *Proc. 46th Design Automation Conf. (DAC 2009)*, pp. 738-743, July 2009. DOI: 10.1145/1629911.1630102. — related work: move-to-front in a hardware trace compressor.
- Pagiamtzis, K., Sheikholeslami, A. "Content-Addressable Memory (CAM) Circuits and Architectures: A Tutorial and Survey." *IEEE J. Solid-State Circuits*, 41(3):712-727, Mar. 2006. DOI: 10.1109/JSSC.2005.864128. — for the CAM structure.
- Indexed in `rtl/amba/references/README.md` (local PDF: `MoveToFrontCoding__Bentley1986.pdf`, `TraceCompressionMoveToFront__Uzelac2009.pdf`, `rtl/common/references/ContentAddressableMemory__Pagiamtzis2006.pdf`)

28.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

29 Monitor Bus LRU CAM (Pipelined)

Module: monbus_cam_pipe.sv **Location:** rtl/amba/ocelus/ **Category:** Bulk-Trace Compression Infrastructure **Status:** Production — replaces monbus_cam inside the compressor

Plane: output plane

29.1 Overview

monbus_cam_pipe is a **2-cycle pipelined variant** of monbus_cam. It implements the same true-LRU CAM semantics — same key/data/timestamp storage, same MRU-at-slot-0 move-to-front, same hit/idx/old_* result — but splits the combinational match → priority-encode → commit chain into two registered stages so the path closes at 100 MHz on Nexys A7.

It is the in-production CAM inside monbus_compressor. The single-cycle monbus_cam.sv is retained as a reference design and as the algorithmic oracle for unit tests.

```
single-cycle monbus_cam :  one cycle: compare → encode → commit +
result
pipelined  monbus_cam_pipe : cycle T   : 32-way compare
                             cycle T+1 : priority-encode + commit +
result reg
```

The latency cost is +**1 cycle**. Throughput is unchanged: a forwarding network corrects each new lookup for the in-flight commit, so back-to-back accesses see bit-exact (hit, idx, old_data, old_ts) sequences identical to monbus_cam. Validated by val/amba/test_monbus_cam_pipe.py.

29.2 Parameters

Same defaults as the single-cycle reference — and the same lock: these are what the compressor instantiates and what the format spec mandates.

Parameter	Type	Default	Description
KEY_WIDTH	int	49	Template key width
DATA_WIDTH	int	64	Payload width
TS_WIDTH	int	24	Per-entry timestamp width

Parameter	Type	Default	Description
DEPTH	int	32	Entry capacity

29.3 Ports

```
module monbus_cam_pipe #(
    parameter int KEY_WIDTH  = 49,
    parameter int DATA_WIDTH = 64,
    parameter int TS_WIDTH   = 24,
    parameter int DEPTH      = 32
) (
    input  logic                    clk,
    input  logic                    rst_n,

    // Synchronous clear: invalidate all entries and flush the
    // lookup pipeline on the next edge. Lets the host rebase the
    // template table without asserting rst_n. Takes priority over
    // access_en.
    input  logic                    clear,

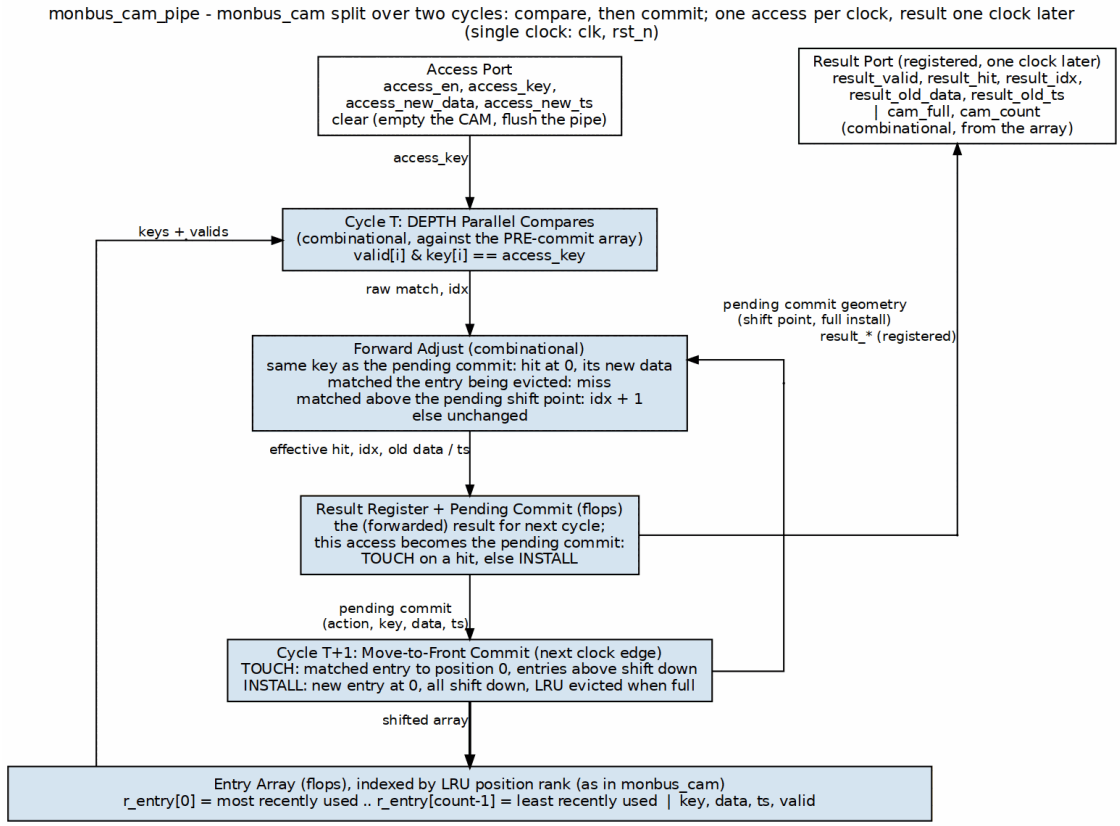
    // Access presented this cycle (one per cycle when access_en=1).
    input  logic                    access_en,
    input  logic [KEY_WIDTH-1:0]    access_key,
    input  logic [DATA_WIDTH-1:0]   access_new_data,
    input  logic [TS_WIDTH-1:0]     access_new_ts,

    // Result of the access presented LAST cycle.
    output logic                    result_valid,
    output logic                    result_hit,
    output logic [IDX_WIDTH-1:0]    result_idx,
    output logic [DATA_WIDTH-1:0]   result_old_data,
    output logic [TS_WIDTH-1:0]     result_old_ts,

    output logic                    cam_full,
    output logic [CNT_WIDTH-1:0]    cam_count
);
```

29.4 Functional Description

29.4.1 Block diagram



monbus_cam_pipe block diagram

The compare and the commit run in different cycles. In cycle T the key is compared against the array as it stands before the pending commit. The forward-adjust block then corrects that raw result for the commit landing this same cycle: a repeat of the same key, the evicted entry, or a shift. The result is registered and leaves on the right one clock later, and the same access becomes the pending commit that writes the array in cycle T+1. Throughput stays one access per clock, and the result sequence matches monbus_cam.

The two registered stages are:

Stage	Cycle	Work
1 (compare)	T	32-way 49-bit key compare against the **pre-commit** register array. Combinational priority encode picks the **lowest-**

Stage	Cycle	Work
		numbered match: the descending loop's last write wins, so slot 0 (the MRU end) takes priority – identical to the single-cycle module, whose comment states the same rule. Functionally moot, since the match vector is at most one-hot by construction.
2 (commit)	T+1	Apply the *pending* commit from the previous access (move-to-front shift). Register the **forward-corrected** result of cycle T's access. Capture cycle T+1's access as the next pending commit.

The cycle T+1 commit and the cycle T+1 result registration both reference the same `r_*` snapshot, so the result reflects the array state *after* the in-flight move-to-front.

29.4.2 Forwarding (depth-1)

When access **A** is compared at cycle T+1, the previous access **P** (captured at cycle T) is committing this same cycle. A's combinational match sees the *pre-P* array, so the forward-correct adjustment is needed:

Case	Effect on A
`A.key == P.key`	P just wrote slot 0 with its new data/ts → A hits with `idx=0`, `old_data = P.new_data`, `old_ts = P.new_ts`
A matched the LRU entry that P evicted (full-CAM install)	A is now a miss (the evicted key is gone)
`A.raw_idx < P.shift_to`	A's entry shifted one slot down → `eff_idx = raw_idx + 1`

Case	Effect on A
Otherwise	A's match position is unchanged

`P.shift_to` and the actual commit shift both read the **same** pre-commit `r_*` this cycle, so the forwarding and the commit always agree. Bubbles (`access_en=0`) still let P commit and clear the pending slot, so a later access sees a fully-committed array and no forwarding is needed.

The end result: the `(hit, idx, old_data, old_ts)` output sequence for any access trace is **byte-exact** with the single-cycle `monbus_cam` for the same input trace (modulo the +1-cycle latency). This is the property the equivalence test guards.

29.4.3 Self-Derived Action

The two-bit `access_action` port from `monbus_cam` is gone. Instead:

```
s1_action = eff_hit ? ACTION_TOUCH : ACTION_INSTALL  (registered)
```

On a hit, the matched entry moves to slot 0 with the new data/ts written through (TOUCH). On a miss, a new entry is installed at slot 0 (INSTALL), evicting the LRU if the CAM is full. The caller cannot suppress the commit — every `access_en=1` cycle is a commit.

For the compressor this is exactly the desired behavior: every record TOUCHes on hit and INSTALLs on a CAM miss; the previous CAM's `ACTION_NONE` lookups were not used in production. If a future caller needs lookups without commits, it would need a separate input port gating the commit (not present today).

29.4.4 Synchronous Clear

`clear` is a single-cycle synchronous reset:

- All `r_valid[i]` cleared (so the CAM appears empty next cycle).
- `r_count` zeroed.
- `s1_valid` cleared (any pending commit is dropped).
- `result_valid` cleared (any in-flight result is dropped).
- Storage (`r_key`, `r_data`, `r_ts`) is **not** cleared — it's ignored while `r_valid[i]=0` so leaving it stale is harmless.

`clear` takes priority over `access_en`: an access pulse arriving in the same cycle as a clear is dropped. The host should hold `clear` high for one cycle while the upstream compression source is idle, which is the contract for the Stream `CTRL[4]` bit driving this through the monbus group wrapper.

29.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

29.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
monbus_cam_pipe #(
    .KEY_WIDTH              (49),
    .DATA_WIDTH             (64),
    .TS_WIDTH               (24),
    .DEPTH                  (32),
    .IDX_WIDTH              ((DEPTH > 1)),
    .CNT_WIDTH              ($clog2(DEPTH + 1))
) u_monbus_cam_pipe (
    .clk                    (clk),
    .rst_n                  (rst_n),
    .clear                  (clear),
    .access_en              (access_en),
    .access_key             (access_key),
    .access_new_data        (access_new_data),
    .access_new_ts          (access_new_ts),
    .result_valid           (result_valid),
    .result_hit             (result_hit),
    .result_idx             (result_idx),
    .result_old_data        (result_old_data),
    .result_old_ts          (result_old_ts),
    .cam_full               (cam_full),
    .cam_count              (cam_count)
);
```

29.7 Design Notes

29.7.1 How It Differs from `monbus_cam`

Aspect	`monbus_cam`	`monbus_cam_pipe`
Latency	0 (combinational lookup + commit)	1 cycle
Throughput	1 access/cycle	1 access/cycle
Action selection	Caller drives `action[1:0]`	**Self-derived** from forwarded hit (TOUCH on hit, INSTALL on miss)
`clear` input	(added in same commit)	yes — sync invalidate + pipe flush

The self-derived action drop matters: a pipelined caller cannot know the hit at present-cycle time (exposing a combinational hit would defeat the pipeline). The compressor always TOUCHes on hit and INSTALLs on miss, so the CAM derives this internally and removes the two action bits from its own port surface.

29.8 Testing

```
pytest val/amba/test_monbus_cam_pipe.py -v
```

The test drives random access traces through both `monbus_cam` and `monbus_cam_pipe` in parallel and asserts the `(hit, idx, old_data, old_ts)` sequence matches cycle-for-cycle (modulo the +1-cycle latency on the pipelined side). Both modules are fed from the same random key/data/ts stream so any divergence shows up immediately.

Additional coverage:

- `test_monbus_compressor.py` — uses `monbus_cam_pipe` end-to-end inside the compressor and cross-checks the slot stream against the Python golden encoder.
- `test_monbus_axil4_axil4_group_compressed.py` — same coverage at the group-wrapper level, including window wrap.

29.9 Related Modules

Module	Role
`monbus_cam`	Single-cycle reference design — same LRU semantics, used by the unit-test oracle
`monbus_compressor`	Production caller of this CAM
`monbus_group` family	Host of the compressor + master writer

29.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Bentley, J.L., Sleator, D.D., Tarjan, R.E., Wei, V.K. "A Locally Adaptive Data Compression Scheme." *Communications of the ACM*, 29(4):320-330, Apr. 1986. DOI: 10.1145/5684.5688. — move-to-front list coding, comparable to the MRU-at-slot-0 order.
- Uzelac, V., Milenković, A. "A Real-Time Program Trace Compressor Utilizing Double Move-to-Front Method." *Proc. 46th Design Automation Conf. (DAC 2009)*, pp. 738-743, July 2009. DOI: 10.1145/1629911.1630102. — related work: move-to-front in a hardware trace compressor.
- Pagiamtzis, K., Sheikholeslami, A. "Content-Addressable Memory (CAM) Circuits and Architectures: A Tutorial and Survey." *IEEE J. Solid-State Circuits*, 41(3):712-727, Mar. 2006. DOI: 10.1109/JSSC.2005.864128. — for the CAM structure.
- Indexed in `rtl/amba/references/README.md` (local PDF: `MoveToFrontCoding__Bentley1986.pdf`, `TraceCompressionMoveToFront__Uzelac2009.pdf`, `rtl/common/references/ContentAddressableMemory__Pagiamtzis2006.pdf`)

29.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

30 Monitor Bus Compressor

Module: `monbus_compressor.sv` **Location:** `rtl/amba/ocelus/` **Category:** Bulk-Trace Compression **Status:** Production Ready

Plane: output plane

30.1 Overview

`monbus_compressor` is a **hardware bulk-trace encoder** for the monitor bus. It consumes streamed `(monitor_packet_t, monbus_timestamp_t)` records, exploits the heavy templating that real workloads exhibit, and emits a 64-bit slot stream that compresses by roughly **2.6×** with **byte-identical** equivalence to a Python golden encoder.

The compressor sits in front of the master writer inside the `monbus_group` family when a wrapper opts in via the `USE_COMPRESSION=1` parameter. The slot stream is what eventually lands in the SRAM dump ring on the FPGA, so smaller slots → longer recording windows in the same memory budget.

The format was **locked** at commit `5bbb83d1` after a sweep across CAM sizes and a real-silicon validation run on a Nexys A7. Once locked, the slot stream is *the* wire format — any RTL change here that diverges from the Python golden constitutes a regression.

30.1.1 Why Compress Monitor Traces?

A 128-bit monitor packet plus a 64-bit timestamp is 24 bytes per event. At even modest event rates (a few million events/sec), an AXI4 monitor running all packet types enabled can saturate a 32-bit AXIL writer's bandwidth and fill a typical SRAM dump region in milliseconds.

The key observation behind this compressor: real workloads are *extremely* repetitive at the monitor-packet level. In the locked validation dataset (`desc_axi_16desc_8ch_1MB`, 682 records of descriptor-fetch traffic):

Metric	Value
Distinct `(pkt_type, protocol, event_code, channel_id, agent_id, unit_id)` tuples	**32** (4.7 % unique)
Records emitted	682
Compression ratio	**2.66×**
Tier-1 hit rate	**93.5 %**
Tier-0 escapes	6.5 % (44 records)

The 32 distinct “templates” fit *exactly* in a 32-entry CAM. Going wider buys zero incremental hits on this workload. That’s the design point.

30.2 Parameters

The 64-bit slot format and CAM size (32 entries × 49-bit key) are locked to match the Python golden. Only `HALF_BEAT_EN` is selectable: it gates an *additional* output port pair that feeds an optional downstream packer; the main `out_slot` stream is unchanged.

Parameter	Type	Default	Description
`HALF_BEAT_EN`	int	0	0 = default; 64-bit-slot codec only (the committed, timing-closed path). 1 = also expose a 30-bit half-slot sideband (`out_half_*`) for the downstream `monbus_halfbeat_packer`. Folds away when 0.

30.3 Ports

```
module monbus_compressor
    import monitor_common_pkg::*;
#(
    // 0 = default; 64-bit-slot codec only (the committed, timing-
closed path).
    // 1 = also expose a 30-bit half-slot sideband (out_half_*) for
the
    //     downstream `monbus_halfbeat_packer`. Folds away when 0.
    parameter int HALF_BEAT_EN = 0
) (
    input  logic                    clk,
    input  logic                    rst_n,
    // Synchronous clear: empty the template CAM (and zero the stat
counters)
    // without asserting rst_n. Pulse one cycle while idle between
runs.
```

```
    input  logic                    clear,

    // Records in (one valid/ready handshake)
    input  logic                    in_valid,
    output logic                    in_ready,
    input  monitor_packet_t         in_packet,      // 128 bits
    input  monbus_timestamp_t       in_source_ts,   // 64 bits

    // Slots out (one valid/ready handshake)
    output logic                    out_valid,
    input  logic                    out_ready,
    output logic [63:0]             out_slot,

    // Half-slot sideband (valid alongside out_valid on a tier-1 beat
that
    // also fits a 30-bit half-slot). Used by monbus_halfbeat_packer
when
    // HALF_BEAT_EN=1; tied 0 when HALF_BEAT_EN=0.
    output logic                    out_half_valid,
    output logic [29:0]             out_half_slot,

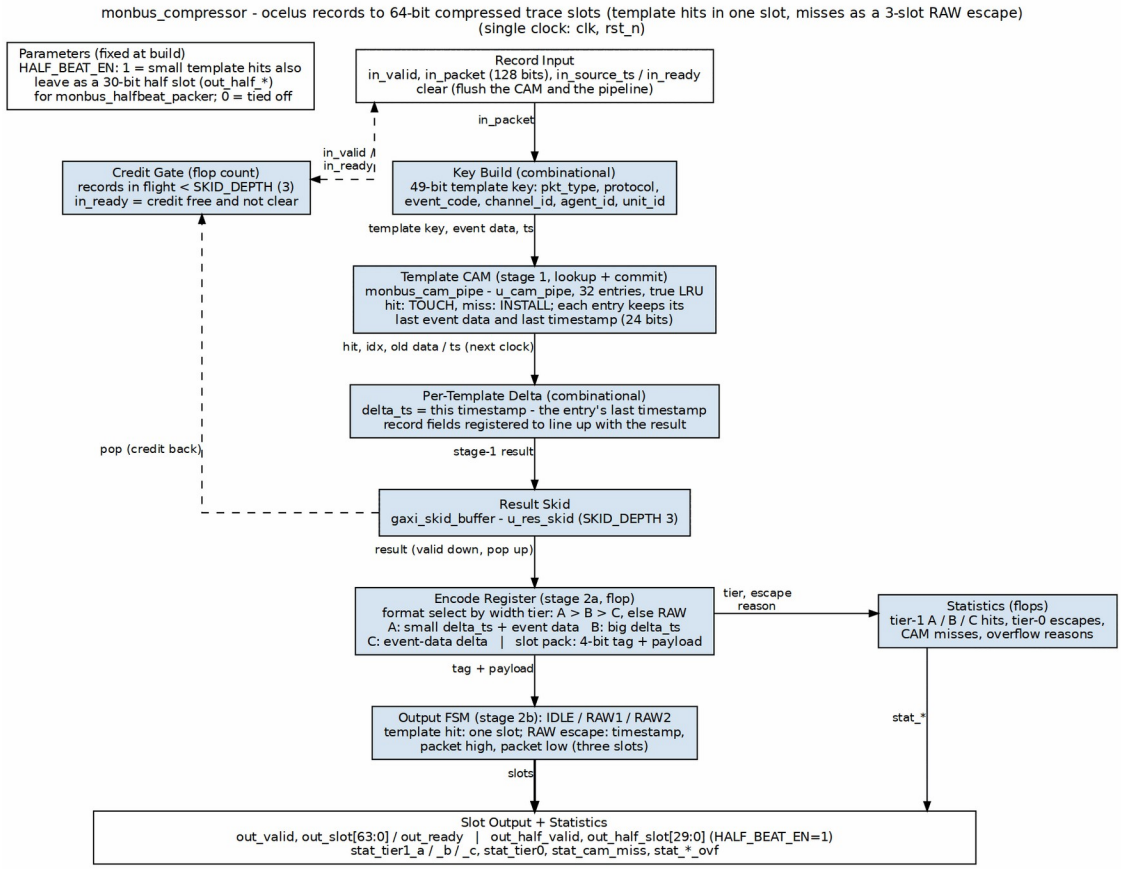
    // Statistics counters (combinational from registers)
    output logic [31:0]             stat_tier1_a,
    output logic [31:0]             stat_tier1_b,
    output logic [31:0]             stat_tier1_c,
    output logic [31:0]             stat_tier0,
    output logic [31:0]             stat_cam_miss,
    output logic [31:0]             stat_delta_ts_ovf,
    output logic [31:0]             stat_event_data_ovf,
    output logic [31:0]             stat_ed_delta_ovf
);
```

30.3.1 Synchronous CAM Clear

`clear` is a synchronous, level-sensitive input. Holding it for one cycle (while idle) invalidates every CAM entry — `r_valid`, `last_event_data`, and `last_ts` — *and* zeroes the stat counters. This is the path the Stream characterization block uses (CTRL[4]): the host can rebase compression statistics between runs without toggling `rst_n` (which would also reset the surrounding monbus pipeline, FIFOs, and AXI skids). Drive it low at all other times.

30.4 Functional Description

30.4.1 Block diagram



monbus_compressor block diagram

Records enter at the top and run down one column. A 49-bit template key is built from the packet's identifying fields. The template CAM (monbus_cam_pipe) looks the key up and commits it, and the per-template delta timestamp is formed against the entry's last timestamp. A credit-gated skid decouples that stage from the encode register, which picks the narrowest format that fits (A, B, C, else a RAW escape) and packs the slot. The output FSM sends one slot for a template hit and three for a RAW record. In-flight credits gate in_ready on the left, and the per-tier statistics run down the right.

30.4.2 Compression Techniques

The compressor combines four ideas, applied in priority order per record:

1. **Template extraction with CAM-backed indexing**
2. **Delta-encoding of the timestamp**
3. **Width-tiered payload encoding** (Tier-1 formats A/B/C)

4. **Differential encoding of the payload** (Tier-1 format C for monotonic counters / sequential addresses)
5. **Tier-0 RAW escape** when none of the above apply

Each successful Tier-1 encoding squeezes 24 bytes (raw record) into 8 bytes (one slot) — that's the 3× upper bound. The 2.66× achieved ratio reflects the 6.5 % escape rate, plus the framing overhead in Tier-0.

30.4.2.1 1. Template extraction

The 128-bit monitor packet has six "low-entropy" fields that repeat heavily across events generated by the same logical agent:

Field	Bits	Description
`packet_type`	4	error / timeout / completion / threshold / perf / debug
`protocol`	4	AXI / AXIS / APB / CORE
`event_code`	8	sub-category within packet type
`channel_id`	9	per-engine channel identifier
`agent_id`	16	per-IP agent identifier
`unit_id`	8	sub-unit within an IP
template key total	**49**	

Two events from the same agent reporting the same condition share all 49 bits. The only differences cycle-to-cycle are the 64-bit `event_data` field (addresses, counters, byte counts) and the timestamp. The compressor identifies each template once, assigns it a **5-bit `tmpl_idx`** (0..31), and from then on sends only `(tmpl_idx, event_data, delta_timestamp)` instead of the full 49-bit key.

The mapping `(key) → tmpl_idx` lives in a 32-entry **caching CAM** (`monbus_cam`). Both the encoder and the decoder maintain the same CAM state, driven by identical (action, key, data) sequences derived from the slot stream. No out-of-band table sync is required — the slot stream is self-describing.

30.4.2.2 2. Delta-encoded timestamp

Absolute timestamps are 60 bits, which covers any practical recording window (2^{60} cycles at 100 MHz is roughly 365 years). Deltas between consecutive events on the same *template* rarely exceed a few thousand cycles in the steady state, so the delta needs only 15 to 23 bits.

The delta is measured **per template**, against the matched CAM entry's stored 24-bit timestamp — not against a single global last-timestamp:

```
delta_ts = source_ts[23:0] - cam_entry.last_ts     // mod 2^24
```

Each CAM entry carries its own `last_ts`, updated when that entry is touched, so interleaved templates from different sources cannot corrupt each other's deltas. A decoder must therefore chain deltas **per template**, with 24-bit wrap. See Per-template `delta_ts` for the full rationale and the history of the global-delta scheme this replaced.

30.4.2.3 3. Width-tiered Tier-1 formats

Three Tier-1 slot formats each cover a different `(delta_ts_width, event_data_width)` trade-off. The encoder picks the **first one that fits** in this priority order:

Format	Tag	`tmpl_idx`	`delta_ts`	`event_data`	Notes
A	`0x1`	5b @ [59:55]	15b @ [54:40]	40b @ [39:0]	Common case
B	`0x2`	5b @ [59:55]	23b @ [54:32]	32b @ [31:0]	Big delta_ts
C	`0x3`	5b @ [59:55]	15b @ [54:40]	40b @ [39:0] **signed delta**	Monotonic counters

The choice is governed by:

```
delta_ts ≤ 2¹⁵−1  AND  event_data ≤ 2⁴⁰−1   → Format A (most common)
delta_ts ≤ 2²³−1  AND  event_data ≤ 2³²−1   → Format B
delta_ts ≤ 2¹⁵−1  AND  ed_delta in [-2³⁹, 2³⁹−1]  → Format C
otherwise                                    → Tier-0 escape
```

Format A assumes the agent has been active recently (delta_ts < ~328 µs @ 100 MHz) and reports an `event_data` value that fits in 40 bits. This covers most steady-state traffic where the same template keeps firing.

Format B handles longer idle periods — up to ~84 ms between events on the same template. The `event_data` budget shrinks from 40 to 32 bits, which still covers most addresses on 32-bit address spaces.

Format C is the differential payload encoder, described next.

30.4.2.4 4. Differential payload encoding (Format C)

For monotonically increasing counters (transaction counts, accumulated byte totals, sequential addresses), absolute `event_data` may exceed 40 bits but the *delta* between consecutive events on the same template fits in 40 signed bits. Format C exploits this:

```
ed_delta = event_data - CAM[tmpl_idx].last_event_data       # signed
if (-2³⁹) ≤ ed_delta < 2³⁹:
    emit Format C slot containing (tmpl_idx, delta_ts, ed_delta)
    update CAM[tmpl_idx].last_event_data = event_data
```

The encoder records the previous `event_data` per template (stored as the CAM's **payload** field — 64 bits per slot). The decoder maintains the same CAM, so when it sees a Format C slot it reconstructs:

```
event_data = CAM[tmpl_idx].last_event_data + ed_delta
```

The locked validation dataset has 0 Format C hits — descriptor-fetch traffic isn't monotonic enough. But for stream-data or descriptor-write traces, this format kicks in heavily.

30.4.2.5 5. Tier-0 RAW escape

If none of the Tier-1 formats fit (CAM miss, or delta_ts > 8M cycles, or event_data >= 2^{40}), the compressor falls back to a 3-beat RAW record:

```
beat 0 (tag = 0x0):  [63:60] = 0x0  | [59:0] = source_ts[59:0]
beat 1            :  packet[127:64]
beat 2            :  packet[63:0]
```

On a Tier-0 escape **caused by a CAM miss**, the encoder also **installs** the new template at the MRU position of the CAM (evicting the LRU if the CAM is full). The decoder does the identical install on its side, so subsequent Tier-1 references to the new `tmpl_idx` resolve correctly.

On a Tier-0 escape **caused by an overflow** (delta_ts or event_data too big), the encoder does NOT re-install — the key was already in the CAM with the right `tmpl_idx`, and the escape is just a 3-beat fallback for the one unusually-large record. Statistics counters record which overflow reason caused each escape, so the host can tell the difference.

30.4.3 Slot Bit Layouts

The 64-bit slot is always self-describing through the 4-bit tag in `[63:60]`:

```
Tag 0x0 — RAW (Tier-0 escape, 3 beats total)
  beat 0:   [63:60] tag=0x0
            [59:0]  source_ts[59:0]
  beat 1:   packet[127:64]
  beat 2:   packet[63:0]

Tag 0x1 — Tier-1 Format A (1 beat)
  beat 0:   [63:60] tag=0x1
            [59:55] tmpl_idx[4:0]
            [54:40] delta_ts[14:0]
            [39:0]  event_data[39:0]

Tag 0x2 — Tier-1 Format B (1 beat)
```

```
  beat 0:   [63:60] tag=0x2
            [59:55] tmpl_idx[4:0]
            [54:32] delta_ts[22:0]
            [31:0]  event_data[31:0]

Tag 0x3 — Tier-1 Format C (1 beat)
  beat 0:   [63:60] tag=0x3
            [59:55] tmpl_idx[4:0]
            [54:40] delta_ts[14:0]
            [39:0]  ed_delta[39:0]  (signed)

Tags 0x4-0xF — Reserved (decoder must error on these)
```

The decoder reads one 64-bit slot, inspects bits `[63:60]`, and knows immediately whether to read 0 more beats (Tier-1) or 2 more beats (Tier-0 RAW). No lookahead is required.

30.4.4 CAM Design

The 32-entry caching CAM lives in `rtl/amba/ocelus/monbus_cam_pipe.sv`. It is a **true LRU** CAM with position-indexed storage — the position rank IS the `tmpl_idx`. This matters because both encoder and decoder must agree on the rank assignment when they install or touch a template, and position-indexed storage makes the rank update trivially atomic with the storage update.

Production CAM is pipelined. Earlier builds used the single-cycle `monbus_cam` directly in the encoder's combinational critical path. To close 100 MHz on Nexys A7, the in-production CAM is now `monbus_cam_pipe` — a 2-cycle variant that registers the 32-way compare → priority-encode → move-to-front sequence into two stages, and feeds the encoder through a credit-gated result skid (`SKID_DEPTH=2`). A `r_credit` counter tracks records in flight + skid occupancy so `cam_en` only fires when there is a slot to land the result, even under back-to-back compression bursts. Sustained throughput is **1 record/cycle** (measured): the pipelined CAM's +1 cycle is absorbed, and `SKID_DEPTH=3` gives three credits against the 3-cycle credit round trip (present → CAM result → registered skid pop → credit returned), which is exactly what the input handshake can consume. At the previous depth of 2 the measured rate was 0.67/cycle. The single-cycle `monbus_cam.sv` is kept in-tree as the reference design (see its doc for the deprecation note).

Per-entry size:

Field	Bits
valid	1
key (concatenated 6-tuple)	49
last_event_data	64
last_ts (`r_ts`, 24-bit, per-template delta base)	24
total per entry	**138**

Field	Bits

No rank is stored: the slot index **is** the recency rank (entries shift on move-to-front), so recency costs no bits.

Total CAM storage: 32 entries × 138 bits = **4416 bits ≈ 4.3 Kb (552 bytes)**.

The production CAM (`monbus_cam_pipe`) has **no action input**. It derives the action itself, on every access, from whether the lookup hit:

Derived action	When	Effect
`ACTION_TOUCH`	any hit	Matched entry moves to MRU; `last_event_data` and `last_ts` updated
`ACTION_INSTALL`	any miss	New entry installed at MRU (evicts LRU if full)

```
s1_action <= eff_hit ? ACTION_TOUCH : ACTION_INSTALL;   // on every
access_en
```

This is a consequence of pipelining, not an oversight: the CAM commits at **presentation** time, one cycle before the tier/format decision exists, so the commit *cannot* depend on it. Every presented record commits — including a Tier-0 overflow escape, which still touches its matched entry. There is no `ACTION_NONE` path and no FSM-issued action; RAW beat expansion stalls the input through credit/skid backpressure instead.

30.4.5 Encoder Decision Tree

Per input record (one cycle):

```
1. Build the 49-bit template key from in_packet.
   Compute event_data = in_packet[63:0] (the lower 64 bits).
   (delta_ts is computed in step 2, against the matched entry's
last_ts.)

2. CAM lookup (pipelined, `monbus_cam_pipe`) — the CAM commits here,
deriving
   its own action from hit/miss, one cycle BEFORE the tier decision
below:
   - MISS  → CAM self-installs (key, event_data). Tier-0 escape.
             FSM enters S_RAW1 → S_RAW2; emits 3 slots over 3 cycles.
             Bump stat_cam_miss + stat_tier0.

   - HIT(idx) → CAM self-touches (moves to MRU, updates
last_event_data and
             last_ts). delta_ts = source_ts[23:0] - entry.last_ts
```

```
(mod 2^24).
             Continue to step 3.

3. Try Tier-1 formats A → B → C in order:

   A. if delta_ts < 2¹⁵  AND  event_data < 2⁴⁰:
        emit Format A slot. Bump stat_tier1_a. Done.
        (The CAM already touched the entry in step 2.)

   B. else if delta_ts < 2²³  AND  event_data < 2³²:
        emit Format B slot. Bump stat_tier1_b. Done.

   C. else if delta_ts < 2¹⁵:
        ed_delta = event_data - CAM[idx].last_event_data    (signed)
        if -2³⁹ ≤ ed_delta < 2³⁹:
            emit Format C slot. Bump stat_tier1_c. Done.

   else:
        Tier-0 escape (CAM hit but record didn't fit any Tier-1).
        Do NOT install (key already in CAM).
        Bump stat_delta_ts_ovf / stat_event_data_ovf /
stat_ed_delta_ovf
        depending on which overflow was the trigger, + stat_tier0.
```

These three things happen on **three different clock edges**, not one: the CAM commits at presentation (inside `monbus_cam_pipe`), the format select is registered into `q_*` at `enc_commit` the next cycle, and `out_valid = q_valid` drives the slot the cycle after that.

30.4.6 Statistics Counters

Each output stat is a 32-bit registered counter with no saturation guard, so they **wrap** at 2^{32}. A capture long enough to overflow one (2^{32} records of a single format) silently rolls that counter back through 0 – read them as free-running counters, not as protected totals.

Counter	Increments when
`stat_tier1_a`	Format A slot emitted
`stat_tier1_b`	Format B slot emitted
`stat_tier1_c`	Format C slot emitted
`stat_tier0`	Tier-0 RAW escape emitted (3 slots)
`stat_cam_miss`	Tier-0 escape caused by CAM miss
`stat_delta_ts_ovf`	Tier-0 escape caused by delta_ts >= 2^{23} (the fit test is < 2^{23}, so 2^{23} exactly does *not* fit)

Counter	Increments when
`stat_event_data_ovf`	Tier-0 escape caused by event_data >= 2^{40} (delta_ts fit)
`stat_ed_delta_ovf`	Tier-0 escape caused by ed_delta out of $\pm 2^{39}$

The host firmware reads these via a configurable register block at the end of a capture run to characterize compression effectiveness per workload. The ratio `(stat_tier1_a + stat_tier1_b + stat_tier1_c) / total_records` is the hit rate; on the validation dataset it's 93.5 %.

30.5 Timing Characteristics

The encoder is split into **three stages** (1, 2a, 2b in the table below), with a tier-1 record ~3 cycles in flight. Per-record slot counts are unchanged from the original single-cycle design, and so is the sustained input rate, at 1 record/cycle measured (SKID_DEPTH=3).

Stage	Logic
1 — lookup / commit	key build → CAM lookup → CAM commit (including per-template `last_ts` update via `access_new_ts`)
2a — encode register (`q_*`)	delta_ts (per-template) / fits / format-select / slot pack — REGISTERED
2b — output	drive the slot(s); RAW (tier-0) 3-beat expansion

Path	Latency	Throughput
Tier-1 record (CAM hit, Tier-1 fits)	1 record → 1 slot, ~3 cycles in flight	**1 record / cycle** (measured)
Tier-0 record (CAM miss, or all Tier-1 overflow)	1 record → 3 slots, 2 cycles + 2 RAW beats	1 record / 3 cycles

There is **no CAM read-after-write hazard**: the commit action depends only on hit/miss (the stage-1 lookup), not on the stage-2 format. The next record's lookup one cycle later sees the committed state — exactly as in the single-cycle design. Bit-exact slot stream against the Python golden either way.

30.5.1 Per-template `delta_ts`

Earlier revisions measured `delta_ts` against a single global `r_last_ts`. With interleaved templates from multiple sources (the 4-channel STREAM characterization case), that global delta could go apparently-negative between two interleaved templates and escape to raw — collapsing compression.

The current encoder measures `delta_ts` against **the matched CAM entry's stored `last_ts`** (`cam_access_old_ts`), and writes back the current record's `source_ts[23:0]` to that slot via `access_new_ts` in the same cycle. The CAM's per-entry `r_ts[TS_WIDTH=24]` shifts in lockstep with the LRU move-to-front. The Python encoder/decoder were updated in lockstep — 24 / 24 golden + 1 / 1 RTL compressor + 3 / 3 group cosim tests pass, bit-exact.

`TS_STORE_BITS = 24`. Format-B (the 23-bit-delta format) needs 24 to *detect* its overflow — 16 silently aliases large gaps to wrong encodes.

30.5.2 Input skid

A 2-deep `gaxi_skid_buffer` registers the `(source_ts, packet)` feed into the compressor. The monbus aggregator's output skid sits far from this compressor's CAM in the Nexys A7 floorplan, and aggregator → in_key → 32-way CAM match/commit was the route-dominated 100 MHz worst path (WNS swung ±0.25 ns on placement alone). The input skid ends that long hop at a local flop.

30.5.3 pblock (Nexys A7 build only)

The expanded per-template CAM (32 entries × `TS_STORE_BITS=24`) crowded the descriptor monitor's column on Nexys A7 and pushed its internal `trans_mgr/u_cam → r_alloc_cnt` path route-bound. The characterization build adds a `pblock_monbus` constraint pinning the monbus group to `CLOCKREGION_X0Y2:X0Y3` (out of column X1), so the monitor can recompact. The monitor → group crossing is registered through the input skid above, so the longer inter-region hop is fine. 100 MHz closes (WNS +0.012 ns, 0 failing endpoints). See `projects/NexysA7/stream_characterization/` for the constraint file and floorplan details.

30.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
monbus_compressor #(
    .HALF_BEAT_EN          (0)
) u_monbus_compressor (
    .clk                   (clk),
    .rst_n                 (rst_n),
    .clear                 (clear),
    .in_valid              (in_valid),
    .in_ready              (in_ready),
    .in_packet             (in_packet),
```

```
    .in_source_ts          (in_source_ts),
    .out_valid             (out_valid),
    .out_ready             (out_ready),
    .out_slot              (out_slot),
    .out_half_valid        (out_half_valid),
    .out_half_slot         (out_half_slot),
    .stat_tier1_a          (stat_tier1_a),
    .stat_tier1_b          (stat_tier1_b),
    .stat_tier1_c          (stat_tier1_c),
    .stat_tier0            (stat_tier0),
    .stat_cam_miss         (stat_cam_miss),
    .stat_delta_ts_ovf     (stat_delta_ts_ovf),
    .stat_event_data_ovf   (stat_event_data_ovf),
    .stat_ed_delta_ovf     (stat_ed_delta_ovf)
);
```

30.7 Design Notes

Compression is a runtime choice, not a build one. With `USE_COMPRESSION=1` the compressor is selectable via `cfg_compress_en`, emitting one 64-bit self-tagged slot per compressed record instead of three raw beats. An incompressible record escapes to tier 0 and still costs three, so the worst case is no worse than raw.

The template CAM is the timing-critical path in this family. On the STREAM characterisation build this route is the chronic critical path; the fix there was floorplanning, not pipelining.

`cam_clear` zeroes the statistics as well as the templates, which is what makes per-run hit-rate figures comparable.

30.8 Testing

The acceptance criterion is **byte-identical equivalence** between the RTL slot stream and the Python golden encoder (`bin/TBClasses/monbus/monbus_compressor.py`) for the same input record sequence. There is no other golden — the Python class IS the spec.

Two layers of validation:

30.8.1 Unit-level (the compressor in isolation)

```
pytest val/amba/test_monbus_compressor.py -v
```

Two phases:

- **Phase 1:** small synthesized stream that exercises all 4 slot tags and CAM eviction. ~9 slots, easy to debug.
- **Phase 2:** the real-silicon dataset `desc_axi_16desc_8ch_1MB.json` (682 records → 770 golden slots). Cross-checks each slot bit-exact, plus asserts the per-tier statistics counters match the Python encoder's per-tier counts exactly.

```
$ pytest val/amba/test_monbus_compressor.py -v
...
INFO  records=682, golden_slots=770
INFO  rtl_a=628, rtl_b=10, rtl_c=0, tier0=44
INFO  === Phase 2: PASS ===
INFO  === ALL PHASES PASSED ===
```

30.8.2 Integration-level (compressor + AXIL writer + base/limit wrap)

```
pytest val/amba/test_monbus_axil4_axil4_group_compressed.py -v
```

Three phases:

- Generous window (no wrap): 9 slots from the synthesized stream
- Tight 8-slot window: forces a mid-stream wrap back to `cfg_base_addr`
- Real-silicon dataset: 770 slots through the full `monbus_axil4_axil4_group` with `USE_COMPRESSION=1`

Both layers run as part of the standard amba regression and must pass before any compressor change merges.

30.8.3 Validation Numbers (locked dataset)

Workload: `desc_axi_16desc_8ch_1MB` — 8 channels × 16 descriptors × 1 MB DMA, descriptor-fetch monitor on a Nexys A7.

Metric	Value
Input records	682
Output slots	770
Compression ratio	**2.66×**
Tier-1 A hits	628 (92.1 %)
Tier-1 B hits	10 (1.5 %)
Tier-1 C hits	0 (0 %)
Tier-0 escapes	44 (6.5 %)
caused by CAM miss	32

Metric	Value
caused by delta_ts overflow	12
caused by event_data overflow	0
caused by ed_delta overflow	0
Distinct templates seen	32 (perfectly fits CAM=32)

The dataset and acceptance recipe live at `projects/NexysA7/stream_characterization/reports/compression_dataset/`.

30.8.4 Test Suite Summary

- **Acceptance (byte-identical vs golden):** `val/amba/test_monbus_compressor.py`
- **End-to-end (with AXIL writer + wrap):** `val/amba/test_monbus_axil4_axil4_group_compressed.py`
- **CAM sub-module:** `val/amba/test_monbus_cam.py`

Run all three:

```
pytest val/amba/test_monbus_cam.py \
       val/amba/test_monbus_compressor.py \
       val/amba/test_monbus_axil4_axil4_group_compressed.py -v
```

30.9 Related Modules

Module	Role
`monbus_cam_pipe`	Pipelined 32-entry LRU CAM — production CAM in the compressor
`monbus_cam`	Single-cycle reference CAM (superseded; kept for comparison)
`monbus_halfbeat_packer.sv`	Pairs two 30-bit half-slots into one 64-bit beat downstream of the compressor (enabled by `HALF_BEAT_EN=1`)
`monbus_group` family	Host of the compressor, plus the master writer that drains slots to memory
`sdpram_slave_axil_axil`	Typical SRAM-ring backend for the compressed slot stream

Module	Role
`bin/TBClasses/monbus/monbus_compressor.py`	Python golden encoder/decoder — the format spec

30.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Bentley, J.L., Sleator, D.D., Tarjan, R.E., Wei, V.K. “A Locally Adaptive Data Compression Scheme.” *Communications of the ACM*, 29(4):320-330, Apr. 1986. DOI: 10.1145/5684.5688. — move-to-front list coding, comparable to sending a template’s LRU index instead of the template.
- Uzelac, V., Milenković, A. “A Real-Time Program Trace Compressor Utilizing Double Move-to-Front Method.” *Proc. 46th Design Automation Conf. (DAC 2009)*, pp. 738-743, July 2009. DOI: 10.1145/1629911.1630102. — related work: a real-time hardware trace compressor built on move-to-front.
- Kao, C.-F., Huang, S.-M., Huang, I.-J. “A Hardware Approach to Real-Time Program Trace Compression for Embedded Processors.” *IEEE Trans. Circuits and Systems I*, 54(3):530-543, Mar. 2007. DOI: 10.1109/TCSI.2006.887613. — related work: real-time hardware trace compression (LZ-based) for processors.
- Anis, E., Nicolici, N. “On Using Lossless Compression of Debug Data in Embedded Logic Analysis.” *Proc. IEEE Int. Test Conf. (ITC 2007)*, pp. 1-10, Oct. 2007. DOI: 10.1109/TEST.2007.4437613. — related work: lossless compression to extend an on-chip trace buffer’s observation window.
- Yang, F.-C., Lin, Y., Kao, C.-F., Huang, I.-J. “An On-Chip AHB Bus Tracer With Real-Time Compression and Dynamic Multiresolution Supports for SoC.” *IEEE Trans. VLSI Systems*, 19(4):571-584, Apr. 2011. DOI: 10.1109/TVLSI.2009.2039811. — related work: real-time compression in an on-chip bus tracer.
- IEEE-ISTO 5001-2003. *The Nexus 5001 Forum Standard for a Global Embedded Processor Debug Interface*, Version 2.0, IEEE Industry Standards and Technology Organization, Dec. 2003 (defining non-peer standard). — §5.2 “address threads”: addresses coded relative to the last one of the same kind, comparable to the per-template delta timestamp.

- Indexed in `rtl/amba/references/README.md` (local PDF: `MoveToFrontCoding__Bentley1986.pdf`, `TraceCompressionMoveToFront__Uzelac2009.pdf`, `DebugTraceInterfaceStandard__IeeeIsto2003.pdf`)

30.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

31 monbus_group_core

Module: `monbus_group_core.sv` **Location:** `rtl/amba/ocelus/` **Status:** Production Ready

Plane: output plane

31.1 Overview

The `monbus_group_core` module is the protocol-agnostic heart of the `monbus_*_*_group` family. It receives a single monitor-bus (MonBus) stream plus a side-band timestamp, applies per-protocol filter masks, and routes each accepted packet into one of two destinations: a record-granular error/interrupt FIFO drained over an AXI4-shaped slave-read port, or a beat-granular write FIFO drained over an AXI4-shaped master-write port with watermark and timeout flushing.

This core is the single source of truth for filtering, FIFO management, optional compression, and the burst writer/slicer state machines. The `monbus_<p1>_<p2>_group.sv` wrappers are pure structural adapters — they bridge its two AXI4-shaped FUB ports into protocol-specific leaf skids, and nothing more.

31.1.1 Key Features

- Single MonBus ingress with per-protocol filter masks (AXI, AXIS, CORE); CXS shares the AXIS masks
- Free-running timestamp generator distributed to every monitor (`mon_time_out`)
- Error/interrupt FIFO: record-granular (192-bit records = 128-bit packet + 64-bit timestamp)
- Write FIFO: beat-granular (one entry = one 64-bit beat)
- AXI4-shaped slave-read drain with burst AR support and a 3-slice record slicer
- AXI4-shaped master-write burst writer with address-window, 4KB, and watermark controls
- Runtime-selectable compression (`cfg_compress_en`) with optional half-beat packing
- Raw mode emits complete 24-byte (3-beat) records; compressed mode emits self-tagged 8-byte slots
- Timing-closed 3-stage geometry pipeline for burst planning at 100 MHz

Aggregated monitor packets have to serve two consumers with very different needs: a CPU interrupt handler that walks recent error records, and a bulk-capture memory buffer that stores a long trace. This core splits the filtered stream between an error FIFO (read on demand by an IRQ

handler) and a write FIFO (flushed to memory as AXI bursts), and handles all the address arithmetic, record framing, and optional compression in one place.

Use Cases: - Central MonBus capture block behind `monbus_arbiter` in an SoC monitoring subsystem - Streaming error records to a CPU IRQ handler while bulk-capturing a full trace to DRAM - Compressed trace capture to fit more monitor history in a fixed buffer - Building protocol-specific group wrappers (AXI4/AXI4, AXI4/AXIL, AXIL/AXI4, AXIL/AXIL) on a shared core

Key Benefit: All the hard logic — filtering, dual-FIFO routing, burst geometry, compression — lives once in this core. The four protocol-pair wrappers reduce to thin structural adapters, so a bug fix or feature lands in one file for every group.

31.2 Parameters

Parameter	Type	Default	Description
`FIFO_DEPTH_ERR`	int	64	Error FIFO depth in 192-bit records
`FIFO_DEPTH_WRITE`	int	96	Write FIFO depth in 64-bit beats (beat-granular)
`ADDR_WIDTH`	int	32	Address width for both FUB ports
`AXI_ID_WIDTH_M`	int	1	Master-write ID width (1 in AXIL builds)
`AXI_ID_WIDTH_S`	int	1	Slave-read ID width (1 in AXIL builds)
`MAX_BURST_BEATS`	int	1	Max beats per master-write sub-burst (1 for AXIL, up to 256 for AXI4)
`FLUSH_TIMEOUT_CYCLES`	int	1024	Cycles since last accepted W beat before forcing a flush
`NUM_PROTOCOLS`	int	3	Informational only
`USE_COMPRESSION`	int	0	0 = raw 3-beat records only; 1 = elaborate the compressor hardware
`HALF_BEAT_EN`	int	0	1 = pack two 30-bit half-slots per beat (requires

Parameter	Type	Default	Description
			USE_COMPRESSION==1)

31.3 Ports

31.3.1 Clock, Reset, and Clear

Port	Direction	Width	Description
axi_aclk	input	1	Core clock
axi_aresetn	input	1	Active-low asynchronous reset
cam_clear	input	1	Synchronous clear of the compressor template CAM + stats (no effect when USE_COMPRESSION==0); pulse when idle

31.3.2 MonBus Ingress and Timestamp

Port	Direction	Width	Description
monbus_valid	input	1	Incoming packet valid
monbus_ready	output	1	Core ready to accept the packet
monbus_packet	input	monitor_packet_t (128)	Monitor packet
monbus_timestamp	input	monbus_timestamp_t (64)	Side-band timestamp for the packet
mon_time_out	output	monbus_timestamp_t (64)	Free-running counter; drive to every wrapper's i_mon_time

31.3.3 Status / IRQ / Debug

Port	Direction	Width	Description
irq_out	output	1	Asserted while the error FIFO is non-empty

Port	Direction	Width	Description
err_fifo_full	output	1	Error FIFO cannot accept a write
write_fifo_full	output	1	Write FIFO cannot accept a beat
err_fifo_count	output	16	Error FIFO occupancy (records)
write_fifo_count	output	16	Write FIFO occupancy (beats)

31.3.4 Address Window and Flush Configuration

Port	Direction	Width	Description
cfg_base_addr	input	ADDR_WIDTH	Base address of the master-write capture window
cfg_limit_addr	input	ADDR_WIDTH	Inclusive limit address of the capture window
cfg_flush_watermark	input	16	Beat count in the write FIFO that triggers a flush burst
cfg_compress_en	input	1	Runtime compression select (meaningful only when USE_COMPRESSION==1); hold stable while the write path is active

31.3.5 Per-Protocol Filter Masks

Three protocol groups (AXI = protocol 0, AXIS = protocol 1, CORE = protocol 4), each with a packet-drop mask, an error-select mask, and per-packet-type event masks. All are 16 bits. CXS (protocol 6) packets go through the AXIS masks: CXS is a stream protocol with the same packet types as AXIS (Error, Timeout, Completion, Credit, Channel, Stream), so it uses the AXIS slot instead of adding another eight mask ports to every group wrapper. Each event-mask bit indexes the low nibble of

the packet's own protocol's event code, so one AXIS bit masks both the AXIS event and the CXS event with the same code number.

Port	Direction	Width	Description
cfg_axi_pkt_mask	input	16	AXI: per-packet-type drop mask (bit = packet type)
cfg_axi_err_select	input	16	AXI: per-packet-type route-to-error-FIFO select
cfg_axi_error_mask	input	16	AXI: per-event-code mask for Error packets
cfg_axi_timeout_mask	input	16	AXI: per-event-code mask for Timeout packets
cfg_axi_compl_mask	input	16	AXI: per-event-code mask for Completion packets
cfg_axi_thresh_mask	input	16	AXI: per-event-code mask for Threshold packets
cfg_axi_perf_mask	input	16	AXI: per-event-code mask for Perf packets
cfg_axi_addr_mask	input	16	AXI: per-event-code mask for AddrMatch packets
cfg_axi_debug_mask	input	16	AXI: per-event-code mask for Debug packets
cfg_axis_pkt_mask	input	16	AXIS: per-packet-type drop mask
cfg_axis_err_select	input	16	AXIS: per-packet-type route-to-error-FIFO select
cfg_axis_error_mask	input	16	AXIS: per-event-code mask for Error packets
cfg_axis_timeout_mask	input	16	AXIS: per-event-code mask for Timeout packets
cfg_axis_compl_mask	input	16	AXIS: per-event-code mask for Completion packets
cfg_axis_credit_ma	input	16	AXIS: per-event-code mask

Port	Direction	Width	Description
sk			for Credit packets
cfg_axis_channel_mask	input	16	AXIS: per-event-code mask for Channel packets
cfg_axis_stream_mask	input	16	AXIS: per-event-code mask for Stream packets
cfg_core_pkt_mask	input	16	CORE: per-packet-type drop mask
cfg_core_err_select	input	16	CORE: per-packet-type route-to-error-FIFO select
cfg_core_error_mask	input	16	CORE: per-event-code mask for Error packets
cfg_core_timeout_mask	input	16	CORE: per-event-code mask for Timeout packets
cfg_core_compl_mask	input	16	CORE: per-event-code mask for Completion packets
cfg_core_thresh_mask	input	16	CORE: per-event-code mask for Threshold packets
cfg_core_perf_mask	input	16	CORE: per-event-code mask for Perf packets
cfg_core_debug_mask	input	16	CORE: per-event-code mask for Debug packets

31.3.6 Compressor Statistics

Port	Direction	Width	Description
mon_compressor_stat_tier1_a	output	32	Tier-1 compression counter A (0 in raw mode)
mon_compressor_stat_tier1_b	output	32	Tier-1 compression counter B
mon_compr	output	32	Tier-1 compression

Port	Direction	Width	Description
essor_stat_tier1_c			counter C
mon_compressor_stat_tier0	output	32	Tier-0 (raw escape) count
mon_compressor_stat_cam_miss	output	32	Template CAM miss count
mon_compressor_stat_delta_ts_ovf	output	32	Delta-timestamp overflow count
mon_compressor_stat_event_data_ovf	output	32	Event-data overflow count
mon_compressor_stat_ed_delta_ovf	output	32	Event-data-delta overflow count

31.3.7 AXI4-Shaped Master-Write FUB (bulk capture)

Port	Direction	Width	Description
fub_m_awid	output	AXI_ID_WIDTH_M	Write address ID (driven to 0)
fub_m_awaddr	output	ADDR_WIDTH	Write burst start address
fub_m_awlen	output	8	Sub-burst length (beats − 1)
fub_m_awsize	output	3	Fixed 3 (8 bytes/beat)
fub_m_awburst	output	2	Fixed INCR (2’b01)
fub_m_awvalid	output	1	AW valid
fub_m_awready	input	1	AW ready
fub_m_wdata	output	64	Write beat data (from write FIFO)

Port	Direction	Width	Description
fub_m_wstrb	output	8	Write strobes (fixed 8'hFF)
fub_m_wlast	output	1	Last beat of the sub-burst
fub_m_wvalid	output	1	W valid
fub_m_wready	input	1	W ready
fub_m_bid	input	AXI_ID_WIDTH_M	Write response ID (ignored)
fub_m_bresp	input	2	Write response (ignored)
fub_m_bvalid	input	1	B valid
fub_m_bready	output	1	B ready

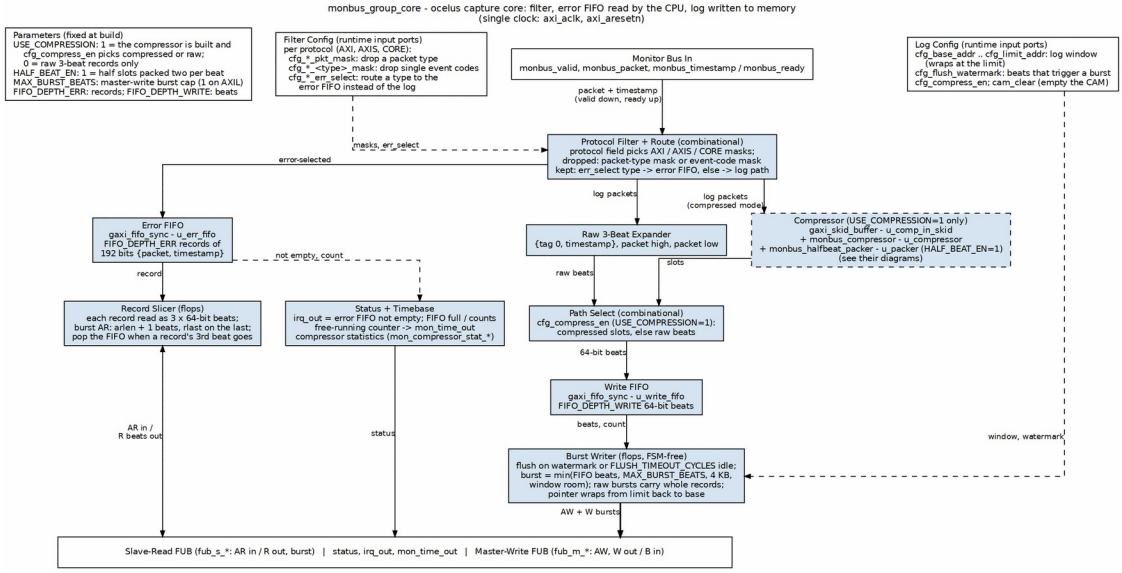
31.3.8 AXI4-Shaped Slave-Read FUB (error drain)

Port	Direction	Width	Description
fub_s_arid	input	AXI_ID_WIDTH_S	Read address ID
fub_s_araddr	input	ADDR_WIDTH	Read address (unused; drain is a stream)
fub_s_arlen	input	8	Burst length (beats − 1)
fub_s_arsize	input	3	Burst size (unused; always 64-bit)
fub_s_arburst	input	2	Burst type (unused; always INCR)
fub_s_arvalid	input	1	AR valid
fub_s_arready	output	1	AR ready (accepts only when idle)
fub_s_rid	output	AXI_ID_WIDTH_S	Read data ID (echoes burst ID)
fub_s_rdata	output	64	Sliced record beat

Port	Direction	Width	Description
fub_s_rresp	output	2	Read response (fixed OKAY)
fub_s_rlast	output	1	Last beat of the burst
fub_s_rvalid	output	1	R valid
fub_s_rready	input	1	R ready

31.4 Functional Description

31.4.1 Block diagram



monbus_group_core block diagram

The monitor bus enters at the top and the protocol filter drops or routes each packet using the per-protocol masks. Packets selected as errors go down the left into the error FIFO, as 192-bit records. The record slicer serves them to the CPU as three 64-bit beats each over the slave-read FUB, and irq_out stays high while the FIFO is not empty. Everything else goes down the right to be logged: as raw 3-beat records, or through the compressor when it is built and enabled. Those beats go into the write FIFO, and the FSM-free burst writer flushes them to memory over the master-write FUB on a watermark or a timeout, wrapping inside the configured window.

31.4.2 Free-Running Timestamp

A counter (`r_ts_counter`) increments every clock and is exposed as `mon_time_out`. Every monitor wrapper drives its `i_mon_time` from this output, so all packets across the group share one time base.

31.4.3 Packet Filtering

Each incoming packet is decomposed into `pkt_type`, `pkt_protocol` (bits [108:105]), and `pkt_event_code`. Filtering runs in three tiers per protocol:

1. **Drop mask** (`cfg_*_pkt_mask[pkt_type]`) — if set, the packet is dropped outright.
2. **Error-select** (`cfg_*_err_select[pkt_type]`) — if set (and not dropped), the packet routes to the error FIFO; otherwise it routes to the write path.
3. **Event mask** — when the event code's high nibble is 0 (`ec_in_mask_range`), the low nibble indexes a per-packet-type event mask; a set bit forces a drop and clears the error-FIFO routing.

An unrecognized protocol drops the packet. Before 2026-10-15 that included CXS, which meant every `PROTOCOL_CXS` packet was dropped without any sign that it had been. The final routing decision is `pkt_to_write_path = !pkt_drop && !pkt_to_err_fifo`.

31.4.4 Error FIFO and Slave-Read Drain

Accepted error packets are stored as 192-bit records `{timestamp, packet}` in `u_err_fifo`, and `irq_out` asserts whenever the FIFO is non-empty. The slave-read drain presents each record as three 64-bit slices over a burst read:

- slice 0 = `{tag=4'h0, source_ts[59:0]}`
- slice 1 = `packet[127:64]`
- slice 2 = `packet[63:0]` (record is popped here)

AR is accepted whenever no burst is in flight (`fub_s_aready = !r_rd_in_burst`) — there is no slice-position or FIFO-occupancy condition. That has two consequences worth knowing before you write the driver: an AR on an empty FIFO is accepted and `rvalid` simply stalls until a record arrives, and `rvalid` drops mid-burst if the FIFO underruns, resuming when a new record shows up. `rlast` asserts on the `(arlen+1)`-th beat. Size `arlen` as a multiple of 3 minus one so the burst lands cleanly on record boundaries.

31.4.5 Write Path — Raw Expander vs Compressor

Two producers feed the beat-granular write FIFO, selected at runtime by `w_use_comp = (USE_COMPRESSION != 0) && cfg_compress_en`:

- **Raw expander** (always elaborated): a 3-state FSM (EXP_TS/EXP_HI/EXP_LO) pushes {ts, pkt_hi, pkt_lo} beats atomically — a record is never split across backpressure. It uses per-beat wr_ready rather than a "3 slots free" precheck, because the precheck would build a count → valid → count combinational loop.
- **Compressor** (elaborated only when USE_COMPRESSION==1): monbus_compressor sits between the input and the FIFO, emitting self-tagged 64-bit slots. A compressed (tier-1) record is ONE slot; an incompressible record escapes to tier-0 and emits THREE, the same as raw mode. Size the write FIFO and cfg_flush_watermark for the traffic you expect: 1 beat/record only holds while records compress, and escape-heavy traffic reaches 3 beats/record. A 2-deep input skid registers (source_ts, packet) right at the compressor boundary so the route-dominated aggregator → CAM path ends at a local flop (+1 cycle latency, throughput preserved, slot stream bit-exact). When HALF_BEAT_EN==1, monbus_halfbeat_packer packs two 30-bit half-slots per beat downstream of the compressor.

Only one path is active at a time; the two FIFO-write outputs are muxed by w_use_comp. monbus_ready is driven by whichever path accepted the packet (drop, error-FIFO write ready, or write-path term).

31.4.6 Master-Write Burst Writer

The write FIFO is flushed to memory as AXI4 bursts. A flush fires when either the FIFO occupancy reaches cfg_flush_watermark or FLUSH_TIMEOUT_CYCLES elapse since the last accepted W beat — and at least one whole record (BEATS_PER_UNIT, 3 in raw mode, 1 compressed) is available.

Burst length is bounded by the minimum of FIFO occupancy, MAX_BURST_BEATS, distance to cfg_limit_addr, and distance to the next 4KB boundary, then rounded down to whole records. The address arithmetic is pipelined over 3 registered stages — the plan trails r_wr_addr, which moves in WR_IDLE (at each drain commit and on the rewind-snap and base-step-over branches) — keeping the wide window/4KB/round-to-record chain off the 100 MHz critical path. The fresh FIFO-occupancy cap is applied combinationally at commit, so a stale count cannot short the burst. The mod-3 whole-record rounding uses math_mod_3_compress carry-save instances rather than a wide divider.

The writer FSM (WR_IDLE → WR_AW → WR_W → WR_B) emits as many AW + N×W + B sub-bursts as needed to drain the planned beat count, advancing the address one 8-byte beat per accepted W. When the current window/4KB region cannot fit a whole record, the writer rewinds r_wr_addr to cfg_base_addr and re-settles the geometry pipeline.

31.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

31.6 Usage Examples

You don't instantiate `monbus_group_core` directly — the group wrappers do. A wrapper connects the two AXI4-shaped FUBs to protocol leaves:

```
monbus_group_core #(
    .FIFO_DEPTH_ERR       (64),
    .FIFO_DEPTH_WRITE     (96),
    .ADDR_WIDTH           (32),
    .AXI_ID_WIDTH_M       (8),
    .AXI_ID_WIDTH_S       (8),
    .MAX_BURST_BEATS      (64),      // AXI4 build; use 1 for AXIL
master-write
    .FLUSH_TIMEOUT_CYCLES (1024),
    .USE_COMPRESSION      (0)
) u_core (
    .axi_aclk    (axi_aclk),
    .axi_aresetn (axi_aresetn),
    .cam_clear   (cam_clear),

    .monbus_valid     (monbus_valid),
    .monbus_ready     (monbus_ready),
    .monbus_packet    (monbus_packet),
    .monbus_timestamp (monbus_timestamp),
    .mon_time_out     (mon_time_out),

    .cfg_base_addr       (cfg_base_addr),
    .cfg_limit_addr      (cfg_limit_addr),
    .cfg_flush_watermark (cfg_flush_watermark),
    .cfg_compress_en     (cfg_compress_en),
    // ... per-protocol masks ...
```

```
    // AXI4-shaped master-write FUB -> axi4_master_wr leaf
    .fub_m_awaddr (wr_fub_awaddr), .fub_m_awlen (wr_fub_awlen),
    .fub_m_awvalid(wr_fub_awvalid), .fub_m_awready(wr_fub_awready),
    .fub_m_wdata  (wr_fub_wdata),  .fub_m_wlast (wr_fub_wlast),
    // ...

    // AXI4-shaped slave-read FUB <- axi4_slave_rd leaf
    .fub_s_arlen  (rd_fub_arlen),  .fub_s_arvalid(rd_fub_arvalid),
    .fub_s_rdata  (rd_fub_rdata),  .fub_s_rlast (rd_fub_rlast)
    // ...
);
```

31.7 Design Notes

31.7.1 Hold `cfg_compress_en` Stable

`cfg_compress_en` changes both the record framing (raw 3-beat vs compressed 1-beat) and `BEATS_PER_UNIT`. Flip it mid-stream and you mix formats in the write FIFO, which corrupts burst sizing. Program it once before monitoring starts.

31.7.2 AXIL Builds Use the Same Core

AXIL group wrappers instantiate this same AXI4-shaped core with `MAX_BURST_BEATS=1` and ID widths of 1, supplying single-beat defaults (len=0, size=$clog2(8), INCR, id=0) at the leaf. There is no AXIL-specific variant of the core, and there doesn't need to be.

31.7.3 FIFO Granularity Difference

The error FIFO is record-granular (192-bit); the write FIFO is beat-granular (64-bit). `err_fifo_count` reports records while `write_fifo_count` reports beats — don't compare them directly.

31.7.4 Timing-Motivated Structure

Several structural choices exist purely to close 100 MHz timing: the 3-stage geometry pipeline, local registering of `cfg_base_addr`/`cfg_limit_addr` with a `max_fanout` cap, the registered raw FIFO count shared by both the flush trigger and the burst cap, and the compressor input skid. A prior split (fresh trigger against a lagged cap) shorted bursts to 21/24 beats — both now derive from the same registered count.

31.7.5 4KB and Window Compliance

The burst is always sized so its last byte stays at or below `cfg_limit_addr` and never crosses a 4KB boundary, satisfying AXI4 addressing rules without mid-burst wrapping.

31.8 Related Modules

31.8.1 Used By

- **monbus_axi4_axi4_group.sv** — AXI4 slave-read + AXI4 master-write wrapper
- **monbus_axi4_axil4_group.sv** — AXI4 slave-read + AXIL master-write wrapper
- **monbus_axil4_axi4_group.sv** — AXIL slave-read + AXI4 master-write wrapper
- **monbus_axil4_axil4_group.sv** — AXIL slave-read + AXIL master-write wrapper

31.8.2 Uses

- **gaxi_fifo_sync.sv** — error FIFO and write FIFO
- **gaxi_skid_buffer.sv** — compressor input skid
- **monbus_compressor.sv** — optional packet compressor (present when `USE_COMPRESSION==1`)
- **monbus_halfbeat_packer.sv** — optional half-beat packer (`HALF_BEAT_EN==1`)
- **math_mod_3_compress.sv** (`rtl/math/`) — carry-save mod-3 for whole-record rounding
- **monitor_common_pkg** — packet types, protocols, `monitor_packet_t`, `monbus_timestamp_t`

31.8.3 See Also

- **monbus_arbiter.sv** — aggregates multiple monitor streams upstream of this core
- **monbus_group.md** — group-level overview

31.9 Testing

No dedicated testbench for this module. It has no `val/**/test_monbus_group_core.py`; it is exercised indirectly, through the tests of the modules that instantiate it:

- `monbus_axi4_axil4_group` – `val/amba/test_monbus_axi4_axil4_group.py`
- `monbus_axil4_axi4_group` – `val/amba/test_monbus_axil4_axi4_group.py`

- `monbus_axil4_axil4_group` – `val/amba/test_monbus_axil4_axil4_group.py`

Indirect coverage exercises this module only in the configurations its parents elaborate. A parameter or mode no parent uses is untested.

31.10 References

31.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Hopkins, A.B.T., McDonald-Maier, K.D. “Debug Support for Complex Systems On-Chip: A Review.” *IEE Proc. Computers and Digital Techniques*, 153(4):197-207, 2006. DOI: 10.1049/ip-cdt:20050194. — related work: on-chip trace buffers versus trace ports.
- Yang, F.-C., Lin, Y., Kao, C.-F., Huang, I.-J. “An On-Chip AHB Bus Tracer With Real-Time Compression and Dynamic Multiresolution Supports for SoC.” *IEEE Trans. VLSI Systems*, 19(4):571-584, Apr. 2011. DOI: 10.1109/TVLSI.2009.2039811. — related work: a bus tracer that records into an on-chip circular buffer.
- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. “An Event-Based Monitoring Service for Networks on Chip.” *ACM Trans. Design Automation of Electronic Systems*, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126. — related work: run-time configurable selection of monitoring events.
- IEEE-ISTO 5001-2003. *The Nexus 5001 Forum Standard for a Global Embedded Processor Debug Interface*, Version 2.0, IEEE Industry Standards and Technology Organization, Dec. 2003 (defining non-peer standard). — §4.11: absolute or relative timestamps, comparable to the group’s free-running counter.
- Lamport, L. “Time, Clocks, and the Ordering of Events in a Distributed System.” *Communications of the ACM*, 21(7):558-565, July 1978. DOI: 10.1145/359545.359563. — for ordering events from several groups without a shared clock.

- Indexed in `rtl/amba/references/README.md` (local PDF: `DebugTraceInterfaceStandard__IeeeIsto2003.pdf`, `LogicalClocksEventOrdering__Lamport1978.pdf`)

31.10.2 Source Code

- RTL: `rtl/amba/ocelus/monbus_group_core.sv`
- Package: `rtl/amba/includes/ocelus_common_pkg.sv`

31.10.3 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Compressor: `docs/markdown/rtl-amba/monitor/monbus_compressor.md`
- Packet Format: `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`

Last Updated: 2026-07-15

31.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

32 monbus_group

Modules: - `monbus_group_core.sv` — protocol-agnostic backend (filter, FIFOs, watermark/timeout burst writer, slice-counter slave-read drain) - `monbus_axil4_axil4_group.sv` — wrapper: AXIL slave-read + AXIL master-write - `monbus_axil4_axi4_group.sv` — wrapper: AXIL slave-read + AXI4-burst master-write - `monbus_axi4_axil4_group.sv` — wrapper: AXI4-burst slave-read + AXIL master-write - `monbus_axi4_axi4_group.sv` — wrapper: AXI4-burst slave-read + AXI4-burst master-write

Location: `rtl/amba/ocelus/` **Category:** Monitor Bus → AXI / AXIL Aggregation **Status:** Production Ready

Plane: output plane (family overview)

32.1 Overview

The `monbus_group` family is the **delivery layer** for the monitor bus. It takes a single monbus stream — already arbitrated upstream if multiple sources need to be merged — and:

1. **Filters** packets per protocol (drop / route-to-err-FIFO / route-to-write-FIFO)
2. **Drains** the error FIFO over a slave-read port for CPU IRQ handlers
3. **Streams** the write FIFO over a master-write port into a memory ring
4. **Optionally compresses** the write-path traffic before it lands in memory (gated by `USE_COMPRESSION=1`)

The family replaces the previous single `monbus_axil_group.sv` module. The new design exposes the same behavior under four protocol-permutation wrappers, so you pick the exact slave/master shape your fabric demands without tying off “fake AXI4” fields on AXIL sides.

32.2 Parameters

All wrappers expose the same set of behavioral knobs (per-wrapper parameters like `AXI_ID_WIDTH` / `AXI_USER_WIDTH` are only present where at least one side is AXI4):

Parameter	Default	Notes
`FIFO_DEPTH_ERR`	64	Error FIFO depth, in **records** (192-bit entries).
`FIFO_DEPTH_WRITE`	96	Write FIFO depth, in **beats** (64-bit entries). 96

Parameter	Default	Notes
		beats = 32 raw records.
ADDR_WIDTH	32	Address width on both slave and master ports.
FLUSH_TIMEOUT_CYCLES	1024	Cycles since the last accepted W handshake before the writer fires a timeout-driven flush.
MAX_BURST_BEATS	1 (AXIL master) / 64 (AXI4 master)	Maximum beats per master-write burst. AXI4 protocol max is 256.
NUM_PROTOCOLS	3	Informational — AXI, AXIS, CORE filter configs are unconditional.
USE_COMPRESSION	0	**Elaboration**: 0 omits the compressor entirely (raw-only build, minimum area); 1 elaborates the compressor so it can be selected at runtime via `cfg_compress_en`. The raw 3-beat-per-record path is always elaborated.
HALF_BEAT_EN	0	**Elaboration**: pack two 30-bit half-slots per 64-bit beat downstream of the compressor (`monbus_halfbeat_packer`), pushing tier-1 bandwidth from 1 beat/record toward 0.5 beat/record. Requires `USE_COMPRESSION=1`; folds away when 0.

Parameter	Default	Notes
SKID_DEPTH_AR/R/AW/W/B	2/4/2/2/2 (AXIL master) / 2/4/2/**4**/2 (AXI4 master)	Skid buffer depths per channel. monbus_axi4_axi4_group defaults SKID_DEPTH_W=4 to absorb burst W traffic; the AXIL-master wrappers keep 2, since MAX_BURST_BEATS=1 leaves no burst to absorb.

32.2.1 Runtime Config

In addition to the elaboration parameters, the group exposes runtime inputs that pick mode and shape on the fly:

Input	Width	Notes
cfg_compress_en	1	Runtime compression enable. Only effective when USE_COMPRESSION=1 (otherwise the compressor isn't elaborated and the bit folds away). Must be stable while the write path is active.
cam_clear	1	**Synchronous CAM clear** (level-sensitive, sample one cycle while idle). Empties the compressor template CAM and zeroes the compressor stat counters so the next run starts with a fresh hit/miss profile — without asserting

Input	Width	Notes
		axi_aresetn. Tied off in raw-only builds (USE_COMPRESSION=0) where there is no CAM. Stream uses this on CTRL[4].
cfg_base_addr / cfg_limit_addr	ADDR_WIDTH	Master-write address window.
cfg_flush_watermark	16	Master-write FIFO depth (in beats) at which a flush is triggered.
cfg_<proto>_*_mask / cfg_<proto>_err_select	16 each	Per-protocol filter and err-FIFO routing masks; see the dedicated section below.

32.3 Ports

The status and debug outputs below are common to all wrappers:

Port	Direction	Width	Description
irq_out	output	1	High whenever the err FIFO is non-empty
err_fifo_full	output	1	Err FIFO write port not ready
write_fifo_full	output	1	Write FIFO write port not ready
err_fifo_count	output	16	Err FIFO entry count (records)
write_fifo_count	output	16	Write FIFO entry count (beats)
mon_time_out	output	64	Free-running timestamp counter
mon_compressor_sta	output	32 each	Compressor statistics, only

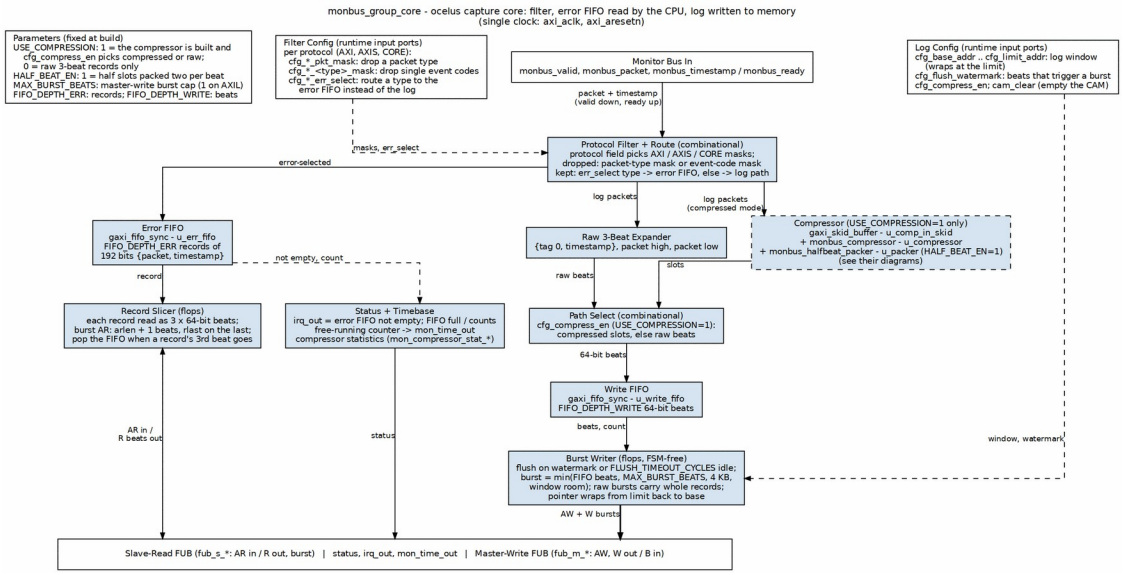
Port	Direction	Width	Description
t_*			live when USE_COMPRESSION=1; tied to 0 in raw mode

The compressor stat ports stay in the port list regardless of elaboration mode, so wrapper layers see a consistent port surface; only their semantics differ.

32.4 Functional Description

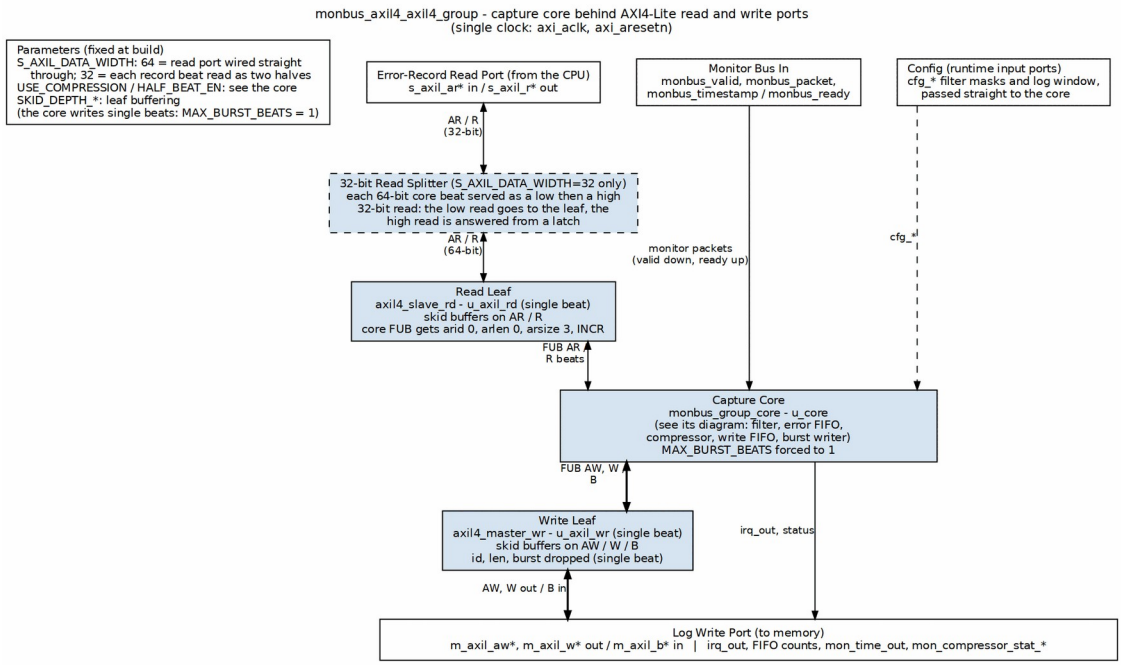
32.4.1 Architecture

The core is shared by all four wrappers:



monbus_group_core block diagram

Packets enter at the top and the protocol filter routes each one: error-selected types go down the left into the error FIFO, which the CPU drains three 64-bit beats per record over the slave-read port; everything else goes down the center through the raw expander (or the compressor when USE_COMPRESSION=1) into the write FIFO, and the burst writer flushes it to the memory log on watermark or timeout. Each wrapper adds only the leaf protocol modules on the two FUB ports, for example the AXIL/AXIL variant:



monbus_axil4_axil4_group block diagram

The other three wrappers differ only in those leaves (`axi4_slave_rd` instead of `axil4_slave_rd`, and so on); each has its own page and diagram.

The module has **no built-in arbitration** — it is a single-input block. Callers with N upstream sources (e.g., RAPIDS source+sink) instantiate a separate `monbus_arbiter` upstream to merge their streams before feeding the group. Keeping arbitration orthogonal to capture means the family never has to duplicate the arbiter for each consumer's input count.

32.4.2 Beat Layout

The write-FIFO and slave-read drain are **beat-granular** — one queue entry is one 64-bit beat. There are two emit modes.

32.4.2.1 Raw mode (`cfg_compress_en = 0`, or `USE_COMPRESSION = 0` build)

Each accepted monbus record becomes three sequential 64-bit beats in the write FIFO:

```
beat 0 = {tag[3:0]=4'h0, source_ts[59:0]}
beat 1 = packet[127:64]
beat 2 = packet[63:0]
```

The expander is **atomic per record**: once it commits to driving the ts beat, it drives beats 1 and 2 before accepting a new record. Partial records never appear in the FIFO, so the master-write burst writer can safely round burst lengths down to a multiple of 3.

The `tag` nibble in beat 0 is reserved for future on-the-wire format variants. The raw writer hardwires it to `4'h0`.

32.4.2.2 Compressed mode (`cfg_compress_en = 1`, requires `USE_COMPRESSION = 1` build)

`monbus_compressor` consumes `(packet, source_ts)` records via its in-handshake and emits 64-bit self-tagged slots via its out-handshake. Each slot becomes one beat directly in the write FIFO. Slot tags (bits `[63:60]`):

```
4'h0 = raw expansion beat (Tier-0 fallback)
4'h1 = Tier-1 format A
4'h2 = Tier-1 format B
4'h3 = Tier-1 format C
4'h4 = half-beat pair (TAG_HALF_PAIR, emitted when HALF_BEAT_EN=1)
     4'h5..4'hF = reserved
```

See `monbus_compressor.md` for the encoder's internal state and the byte-exact Python golden.

The raw expander is **always elaborated**; the compressor is elaborated only when `USE_COMPRESSION=1`. The two paths' FIFO-write outputs and `monbus_ready` are muxed on `w_use_comp = (USE_COMPRESSION != 0) && cfg_compress_en`. Only one path is ever active; the inactive one is gated off at its input. `BEATS_PER_UNIT` (3 in raw, 1 in compressed) is therefore a runtime quantity (`w_beats_per_unit`), feeding the burst-writer geometry described below.

> **Note on switching modes:** `cfg_compress_en` must be stable while there is unflushed data in the write FIFO. Toggling it mid-stream would mix raw 3-beat records with compressed 1-beat slots at the same memory addresses — the host decoder couldn't reassemble them. Software should drain the FIFO (poll `write_fifo_count` to zero) before flipping the bit.

32.4.2.3 Slave-read drain (both modes)

The slave-read port returns the same 3-beat layout per err-FIFO record, sliced by an internal counter. In AXI4 mode, `arlen+1` beats are emitted across the burst with `rlast` asserting on the final beat; the slicer advances through records, and the FIFO entry is popped only when slice 2 (`SLICE_PKT_LO`) fires. In AXIL mode each AR returns one beat.

32.4.3 Master-Write Behavior

The burst writer fires when **either**:

- `flush_trigger_watermark`: `r_fifo_beats >= cfg_flush_watermark`
- `flush_trigger_timeout`: `r_timeout_cnt >= FLUSH_TIMEOUT_CYCLES`

…and at least `BEATS_PER_UNIT` beats are available (3 in raw mode, 1 in compressed mode).

32.4.3.1 Burst geometry: 4-stage pipeline + fresh-FIFO cap at commit

The drain-plan math was originally a single combinational chain off `r_wr_addr` feeding straight back into `r_wr_addr` — the 100 MHz critical path on Nexys A7 (-1) (WNS −7.06 ns post-route).

Since r_wr_addr is stable while the writer sits in WR_IDLE (only WR_W advances it) and the write FIFO only grows there, the **address geometry** is now a **4-stage registered pipeline**, one carry chain per stage:

```
stage 0 (window):          s0_in_window (r_wr_addr inside [base,
limit]) and
                           s0_gaddr, the address the geometry is
measured from
                           (r_wr_addr in window, else base). Two
compares + mux.
stage 1 (caps):            s1_beats_to_limit = (limit + 1 - s0_gaddr)
>> 3,
                           saturated (one 33-bit subtract; limit + 1 is
                           registered with the config), and
s1_beats_to_4kb.
stage 2 (planned):         min-cap tree ONLY (min of window / 4KB).
                           u_mod3_geo is fed combinationally from this
                           register and consumed by stage 3, so
                           s2_beats_planned is NOT yet record-rounded.
stage 3 (rounded + addr): r_plan_geo_units (address-feasible whole-
                           record count) + r_plan_addr (eff_addr).
```

Stage 0 was split out of stage 1 on 2026-09-28 (amba ISSUE-001): the window compare gating the limit subtract in one cycle was 11 CARRY4 and 16 logic levels, 10.1 ns on an Artix-7 100T -1 – the worst register-to-register path of every monitored bridge. The plan now trails r_wr_addr by four cycles instead of three, which only the settle counter sees.

A geom_valid settle counter (now four) holds the plan invalid for the first few cycles after the writer **leaves WR_IDLE**, so the pipeline reflects the settled address before the FSM commits. The counter resets on that state exit only – **not** when r_wr_addr moves *inside* WR_IDLE, which the rewind-snap and base-step-over branches below both do. On those paths geom_valid stays asserted and the FSM can act on a stale plan for a few cycles. That is tolerable rather than hazardous: a commit always consumes r_plan_addr, which travels through the pipeline with its own plan, so a stale plan can never be paired with a fresh address.

32.4.3.1.1 Rewind-snap: in-window with no record-room left

If a flush fires while r_wr_addr is in-window but the remaining bytes to the next 4KB boundary cannot fit a whole record, the pipeline produces r_plan_ok=false (units rounded down to zero) and the writer would otherwise wedge in WR_IDLE. The FSM detects this case (do_flush && geom_valid && !r_plan_ok && r_wr_addr != cfg_base_addr) and **snaps r_wr_addr to cfg_base_addr**, staying in WR_IDLE so the pipeline re-settles with fresh geometry.

A **second branch** covers the case where cfg_base_addr *itself* cannot host a whole record: do_flush && geom_valid && !r_plan_ok && (r_wr_addr == cfg_base_addr) steps r_wr_addr over the short stub to the next 4KB boundary and drains there. So the host does **not** have to place cfg_base_addr with a record's worth of room before the next 4KB line — the

hardware handles a base that lands in a stub. (The `!=`-only guard, without this second branch, wedged the writer; that is why it exists.)

This case is hit by the AXIL/AXIL master_write stress in Phase 5, where `cfg_base_addr` is intentionally placed only a few beats below a 4KB boundary so the very first drain has to wrap-snap before it can emit anything.

The **FRESH FIFO-occupancy cap** is applied combinationally at the `WR_IDLE` commit (not inside the pipeline):

```
// Pipeline produces r_plan_geo_units = address-feasible whole-record
beats.
// At commit time, cap by the CURRENT FIFO contents:
units_commit = min(r_plan_geo_units, w_fifo_units)
```

where `w_fifo_units = r_fifo_beats - (r_fifo_beats mod 3)` (or `r_fifo_beats` itself in compressed mode where the unit is 1 beat). This matters because the FIFO keeps filling while the writer sits in `WR_IDLE`; a stale (3-cycle-old) FIFO count would short the burst — for example, capping a watermark-triggered 24-beat plan to 21. The fresh path starts from a single counter (no address subtract/shift), so it doesn't recreate the critical path.

Per-AW sub-burst sizing happens **inside the FSM**, capping each AW by `MAX_BURST_BEATS`:

```
sub_burst_beats = min(remaining_in_cycle, MAX_BURST_BEATS)
awlen           = sub_burst_beats - 1
```

The split between drain-cycle plan (whole records) and per-AW sub-burst (`MAX_BURST_BEATS`) is what makes the family work for both AXIL (`MAX_BURST_BEATS = 1`) and AXI4 masters in raw mode (`BEATS_PER_UNIT = 3`):

- **AXI4 master, raw mode:** one drain cycle ≈ one large sub-burst. e.g. 24 beats = 8 records in a single AW + 24 × W + B. Throughput-optimal.
- **AXIL master, raw mode:** one drain cycle = N single-beat sub-bursts. With `MAX_BURST_BEATS=1` the FSM runs one single-beat sub-burst per beat, so a 3-beat raw record costs three AW + three W + three B handshakes – the W and B counts scale too, not just the address channel – at consecutive addresses. The memory image is identical to the AXI4 case; the handshake counts on all three channels differ.

32.4.3.2 Address-window behavior: wrap, not saturate

The master-write address is **circular** within `[cfg_base_addr, cfg_limit_addr]`. When the writer rolls past the window's end (or starts out of window, e.g. `r_wr_addr = 0` immediately after reset), the geometry pipeline finds `r_wr_addr` outside `[cfg_base_addr, cfg_limit_addr]` and computes against a pre-rewound `geom_addr = cfg_base_addr`. The final `r_plan_addr` reflects that rewind — so the next burst starts at `cfg_base_addr` and the dump ring keeps moving forever.

This is **not** a saturation behavior. The writer never stalls at `cfg_limit_addr`; it always wraps. Any host-side bookkeeping (SRAM-dump write pointer, ring-buffer position cursor, etc.) **must match this wrap behavior** — saturating a host-visible pointer at the SRAM cap will desynchronize it from the actual ring state. The `stream_char_harness.sv` debug pointer was fixed for exactly this reason on 2026-06-14 (`d82b1e04`); see that commit message for the hardware-confirmed symptom.

32.4.3.3 mod-3 rounding (no /3 operator)

Rounding a beat count down to a whole record (`X - (X mod 3)`) is done by two `math_mod_3_compress` instances (`u_mod3_geo`, `u_mod3_fifo`) — a carry-save-compressor implementation of `X mod 3` via base-4 digit sum (each 2-bit group has weight `4^k ≡ 1 mod 3`). No `*` or `/` operators, so Vivado neither infers a DSP48 (a combinational path through a far-placed DSP was catastrophic) nor a CARRY4 iterative divider — both used to blow timing. Exhaustively verified for all 65 536 inputs (`val/math/test_math_mod_3_compress.py`).

32.4.3.4 Final port behavior

`awsize` is fixed at 3 (8 bytes per beat) and `awburst` at INCR (`2'b01`). `awlen` is `sub_burst_beats - 1`. `wlast` asserts on the final beat of each sub-burst.

32.4.4 Per-Protocol Filter Configuration

The filter is configured per protocol (AXI, AXIS, CORE). For each protocol the caller provides:

- `cfg_<proto>_pkt_mask[15:0]` — drop the packet if the bit at `pkt_type` is set.
- `cfg_<proto>_err_select[15:0]` — route to the err FIFO if the bit at `pkt_type` is set (and not dropped). All other surviving packets go to the write FIFO.
- `cfg_<proto>_<event_type>_mask[15:0]` — per-event-code mask. If the bit at `event_code[3:0]` is set, the packet is dropped regardless of the above. Per-protocol event categories differ:
  - **AXI:** error, timeout, compl, thresh, perf, addr, debug
  - **AXIS:** error, timeout, compl, credit, channel, stream
  - **CORE:** error, timeout, compl, thresh, perf, debug

Per-event masks are 16 bits but only `event_code[3:0]` indexes; codes ≥ 16 are treated as “not in mask range” (no masking applied). Protocols not in the supported set (APB=2, ARB=3) are always dropped.

32.5 Usage Examples

32.5.1 Migration from `monbus_axil_group`

The legacy `monbus_axil_group.sv` module is gone. Migrate to the wrapper matching your fabric. The port-surface changes that matter:

1. **`M_AXIL_DATA_WIDTH` dropped; `S_AXIL_DATA_WIDTH` retained** — the master-write data path is locked at 64 bits, so `M_AXIL_DATA_WIDTH` is gone. `S_AXIL_DATA_WIDTH` survives on the two AXIL-slave-read wrappers (`monbus_axil4_axil4_group`, `monbus_axil4_axi4_group`) with a default of 64 (one beat per record slice); set it to 32 to enable the built-in 2:1 read serializer for a 32-bit host crossbar (6 beats per record). The AXI4-slave-read wrappers have no such parameter.
2. **`cfg_flush_watermark[15:0]` is a new required input.** Set to 1 for the legacy "fire immediately" behavior; set higher to batch.
3. **`err_fifo_count` and `write_fifo_count` are now 16 bits** (were 8).
4. **`FIFO_DEPTH_WRITE` is in beats**, not records. 32 records of the legacy module = 96 beats in raw mode.

Example (AXIL/AXIL — the closest analog to the legacy module):

```
// Old
monbus_axil_group #(
    .FIFO_DEPTH_ERR    (64),
    .FIFO_DEPTH_WRITE  (32),   // records
    .S_AXIL_DATA_WIDTH (64),
    .M_AXIL_DATA_WIDTH (64),
    .USE_COMPRESSION   (0)
) u_group (...);

// New
monbus_axil4_axil4_group #(
    .FIFO_DEPTH_ERR       (64),
    .FIFO_DEPTH_WRITE     (96),   // beats (3x for raw mode)
    .FLUSH_TIMEOUT_CYCLES (1024),
    .USE_COMPRESSION      (0)
) u_group (
    .cfg_flush_watermark  (16'd24),  // 8 records worth, eager flush
    ...
);
```

For AXI4 burst capture — say the memory ring lives behind an AXI4 fabric and you want to bunch records into multi-beat bursts to amortize address-channel overhead — switch the wrapper to `monbus_axil4_axi4_group` and pick `MAX_BURST_BEATS` to taste.

32.6 Design Notes

32.6.1 Why One Core Plus Four Wrappers

SystemVerilog cannot conditionally include or exclude ports from a single module's port list. The port list is a static syntactic construct, fixed at module declaration. So to give each protocol combination an exact port shape — no spurious AXI4-only fields for the caller to tie off — the family is split:

- **One common backend** (`monbus_group_core.sv`) carries the filter, FIFOs, watermark/timeout burst-writer FSM, slice-counter drain, and optional compressor. Its FUBs are AXI4-shaped on both sides (id / awlen / awsize / awburst / wlast / arlen / rlast / etc.). AXIL wrappers feed single-beat defaults (`len=0`, `size=3`, `burst=INCR`, `id=0`) and force `MAX_BURST_BEATS=1`.
- **Four thin wrappers**, each with the right protocol-shaped port list, that directly instantiate the matching leaf skids (`axi4_slave_rd` / `axil4_slave_rd` and `axi4_master_wr` / `axil4_master_wr`) and bridge their FUBs to the core's FUB.

Wrapper	Slave-read shape	Master-write shape	`MAX_BURST_BEATS` (default)
`monbus_axil4_axil4_group`	AXIL `s_axil_ar*/r*`	AXIL `m_axil_aw*/w*/b*`	1
`monbus_axil4_axi4_group`	AXIL `s_axil_ar*/r*`	AXI4 `m_axi_aw*/w*/b*` (full)	64
`monbus_axi4_axil4_group`	AXI4 `s_axi_ar*/r*` (full)	AXIL `m_axil_aw*/w*/b*`	1
`monbus_axi4_axi4_group`	AXI4 `s_axi_ar*/r*` (full)	AXI4 `m_axi_aw*/w*/b*` (full)	64

Callers pick the wrapper module name matching their fabric. The bare `monbus_group_core` is not intended to be instantiated directly — its AXI4-shaped FUBs are an internal contract.

32.7 Related Modules

32.7.1 Sub-modules Instantiated

Sub-module	Role
`axi4_slave_rd` / `axil4_slave_rd`	Slave-read skid (one per wrapper)
`axi4_master_wr` / `axil4_master_wr`	Master-write skid (one per wrapper)
`gaxi_fifo_sync` × 2	Err FIFO + write FIFO inside the core
`monbus_compressor` (conditional)	Compressed-mode encoder; only when `USE_COMPRESSION=1`
`monbus_halfbeat_packer` (conditional)	Pairs two 30-bit half-slots into one 64-bit beat downstream of the compressor; only when `HALF_BEAT_EN=1` (which itself requires `USE_COMPRESSION=1`)
`monbus_cam_pipe` (transitive)	**Pipelined** 49-bit-key LRU CAM with per-template `last_ts`/`last_data`; replaces the unpipelined `monbus_cam` as the in-production CAM inside the compressor. See `monbus_cam` for the deprecation note.
`math_mod_3_compress` × 2 (`rtl/math/`)	Geometry / FIFO whole-record rounding (X − (X mod 3))
`math_adder_carry_save_nbit` (transitive)	3:2 compressor primitive used by `math_mod_3_compress`
`gaxi_skid_buffer` (`u_comp_in_skid`, in `monbus_group_core`)	Input skid on the `(source_ts, packet)` feed into the compressor; breaks the aggregator → CAM long route. The compressor has a separate internal skid (`u_res_skid`) on the CAM-result path

A single canonical filelist enumerates the group core's dependency tree, so a new core dep lands in **one place**:

`rtl/amba/filelists/monbus_group.f`

It lists `math_adder_carry_save_nbit` + `math_mod_3_compress` + `monbus_cam` + `monbus_cam_pipe` + `monbus_compressor` + `monbus_halfbeat_packer` + `monbus_group_core`.

All consumers (`val/amba` tests, RAPIDS / STREAM macro filelists) `-f`-include it rather than listing those sources inline.

32.7.2 See Also

Module	Role
`monbus_compressor`	Bulk-trace encoder used when `USE_COMPRESSION=1`
`monbus_cam_pipe`	Pipelined LRU CAM backing the compressor (production)
`monbus_cam`	Unpipelined reference CAM (superseded by `monbus_cam_pipe`)
`monbus_arbiter`	Upstream multi-source merge (instantiate before this family if you have N>1 sources)
`sdpram_slave_axil_axil`	Canonical memory-ring backend for the master-write port (AXIL/AXIL variant pairs with `sdpram_slave_axil_axil`)
`axi_monitor_base`	Source of the monitor packets this family captures
`axi_monitor_reporter`	Per-protocol packet-emission frontend

32.8 Testing

Verification lives in `val/amba/`:

- `test_monbus_axil4_axil4_group.py` — basic flow + err-FIFO drain on the AXIL/AXIL wrapper (slave-read coverage; master-write side is driven into a synthetic sink in this test).
- `test_monbus_axil4_axil4_group_compressed.py` — byte-exact compressed slot stream comparison against the Python `Encoder` golden across three phases (small synth stream, real-silicon 682-record dataset, wrap-window).
- `test_monbus_axil4_axil4_group_master_write.py` — raw-mode master-write coverage on the AXIL/AXIL wrapper. Three phases: watermark-driven flush (asserts `3*N` beats at 8-byte stride starting at `cfg_base_addr`),

timeout-driven flush, window wrap-back. The test that would have caught the AXIL-master raw-mode flush deadlock fixed on 2026-06-11.

- `test_monbus_axil4_axi4_group.py` — AXI4 burst master-write coverage on the AXIL/AXI4 wrapper. Three phases: watermark-driven multi-beat burst, timeout-triggered flush, 4KB-boundary respect.
- `test_monbus_axi4_axil4_group.py` — dedicated AXI4-slave + AXIL-master coverage. Phase 1 covers the master-write raw-mode flush; Phase 2 covers the AXI4 burst slave-read drain (asserts `rlast` timing across a 6-beat AR with custom `arid`).

```
pytest val/amba/test_monbus_axil4_axil4_group.py \
       val/amba/test_monbus_axil4_axil4_group_compressed.py \
       val/amba/test_monbus_axil4_axil4_group_master_write.py \
       val/amba/test_monbus_axil4_axi4_group.py \
       val/amba/test_monbus_axi4_axil4_group.py -v
```

`monbus_axi4_axi4_group` does not yet have a dedicated test; its master-write path is covered by `test_monbus_axil4_axi4_group.py` (same master leaf) and its AXI4-burst slave-read drain is covered by `test_monbus_axi4_axil4_group.py` Phase 2 (same slave leaf). Adding a direct test for the pure-AXI4 wrapper is a future-work item.

32.9 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Hopkins, A.B.T., McDonald-Maier, K.D. “Debug Support for Complex Systems On-Chip: A Review.” *IEE Proc. Computers and Digital Techniques*, 153(4):197-207, 2006. DOI: 10.1049/ip-cdt:20050194. — related work: on-chip trace buffers versus trace ports.
- Yang, F.-C., Lin, Y., Kao, C.-F., Huang, I.-J. “An On-Chip AHB Bus Tracer With Real-Time Compression and Dynamic Multiresolution Supports for SoC.” *IEEE Trans. VLSI Systems*, 19(4):571-584, Apr. 2011. DOI: 10.1109/TVLSI.2009.2039811. — related work: a bus tracer that records into an on-chip circular buffer.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. — AXI4 burst ports of the `axi4` wrappers.
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021. developer.arm.com/documentation/ihi0022/h. — Part B ch. B1 (AXI4-Lite) for the `axil4` ports.

- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

33 monbus_halfbeat_packer

Module: monbus_halfbeat_packer.sv **Location:** rtl/amba/ocelus/ **Status:** Production Ready

Plane: output plane

33.1 Overview

The monbus_halfbeat_packer module packs two 30-bit half-slots into a single 64-bit beat, sitting downstream of monbus_compressor. The compressor emits one 64-bit beat per tier-1 record — a 66.7% reduction ceiling, 1 beat per 3-beat raw record. Pair two compatible records into one beat and this packer pushes the reduction to as much as 83.3% (0.5 beat per record). It is bit-exact to the Python golden model Encoder(half_beat=True).

33.1.1 Key Features

- Pairs two 30-bit half-slots into one TAG_HALF_PAIR (0x4) beat
- Buffers one half-slot until a partner arrives
- Preserves record order: flushes a buffered half before forwarding a non-half beat
- Flushes a lone trailing half when the input goes idle (last record never stranded)
- Forwards full 64-bit slots and raw-escape beats verbatim
- Bit-exact to the compressor Python golden model for gap-free record streams
- Simple valid/ready handshake on both ports

Trace-capture bandwidth is precious. The compressor already reduces most records to a single 64-bit beat, but records that also fit in 30 bits can be paired two-to-a-beat. This packer performs that pairing with a one-slot holding buffer, emitting a combined beat when a second eligible half arrives while preserving strict record order for everything that cannot be paired.

Use Cases: - Maximizing trace history stored in a fixed capture buffer - Reducing MonBus write bandwidth to memory in compressed-capture builds - Matching the software decoder's half-beat encoding exactly (cosim parity)

Key Benefit: Breaks the compressor's 1-beat-per-record floor without changing record semantics — pairing is opportunistic and always order-preserving, so it is never wrong, only occasionally less-packed under input bubbles.

33.2 Parameters

This module has no parameters.

33.3 Ports

33.3.1 Clock and Reset

Port	Direction	Width	Description
`clk`	input	1	Clock
`rst_n`	input	1	Active-low asynchronous reset

33.3.2 Input — Beats from the Compressor

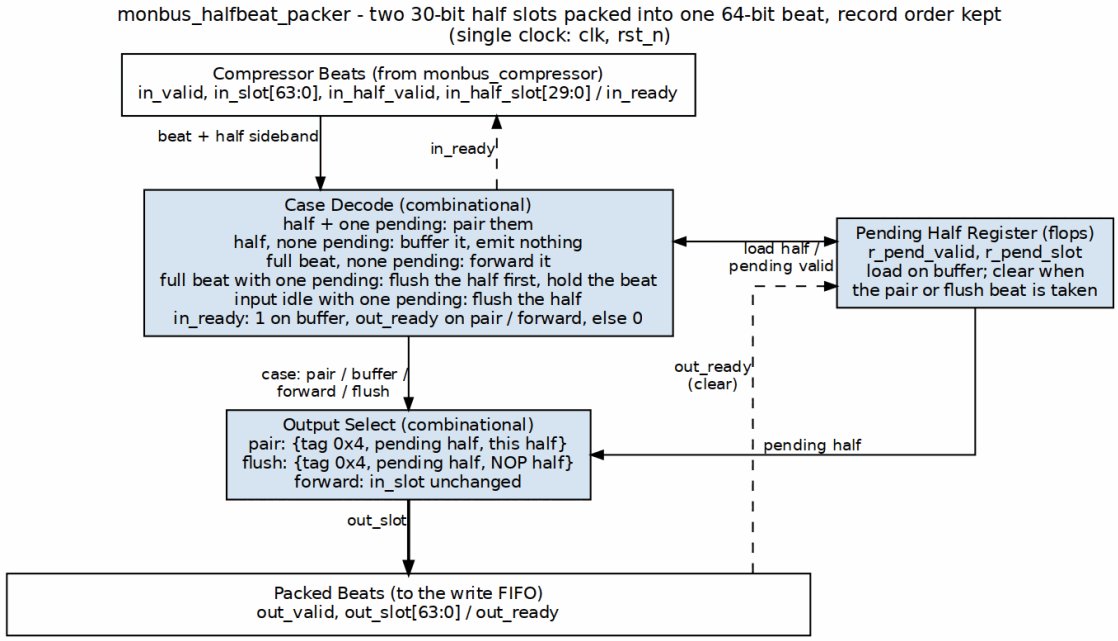
Port	Direction	Width	Description
`in_valid`	input	1	Input beat valid
`in_ready`	output	1	Packer ready to accept the input beat
`in_slot`	input	64	Full 64-bit slot (tier-1 slot or one of a raw record's 3 beats)
`in_half_valid`	input	1	Set when `in_slot`'s record also fits a 30-bit half-slot
`in_half_slot`	input	30	The 30-bit half-slot for the current record

33.3.3 Output — Packed Beats to the Write FIFO

Port	Direction	Width	Description
`out_valid`	output	1	Output beat valid
`out_ready`	input	1	Downstream ready
`out_slot`	output	64	Packed beat (paired half-slots, forwarded full slot, or half + NOP flush)

33.4 Functional Description

33.4.1 Block diagram



monbus_halfbeat_packer block diagram

Beats from monbus_compressor enter at the top with their half-slot sideband. The case decode chooses one of five cases each cycle: pair, buffer, forward, flush-then-forward, or idle flush. The output select builds the 64-bit beat. A first half-slot waits in the pending register on the right until a second arrives to pair with it. Any full beat, or an idle input, flushes it first with a NOP partner, so record order is never broken.

33.4.2 Beat Layout

The paired beat matches `monbus_compressor.py`:

```
TAG_HALF_PAIR beat = {tag[63:60]=0x4, slotA[59:30], slotB[29:0]}
half-slot          = {sub[29:28], idx[27:23], delta_ts[22:13],
data[12:0]}
NOP slot           = 30'd0   (sub == HSUB_NOP)
```

33.4.3 Case Decode

Five mutually-considered cases drive the packer combinationally, based on `in_valid`, `in_half_valid`, and whether a half is buffered (`r_pend_valid`):

Signal	Condition	Action
`pair_now`	half arrives, one already	Emit the pair `{TAG,`

Signal	Condition	Action
	buffered	`pend, in_half}`, consume input on accept
`buffer_now`	half arrives, none buffered	Latch it, emit nothing, always consume input
`fwd_now`	non-half, none buffered	Forward `in_slot` verbatim, consume input on accept
`flush_fwd`	non-half, one buffered	Emit the lone buffered half (paired with NOP) first, **hold** the input
`idle_flush`	input idle, one buffered	Emit the lone trailing half (paired with NOP)

`out_valid` asserts for any of `pair_now`, `fwd_now`, `flush_fwd`, or `idle_flush`.

33.4.4 Handshake and Ordering

`in_ready` is asserted unconditionally for `buffer_now` (no output is produced that cycle), and follows `out_ready` for `pair_now`/`fwd_now`. For `flush_fwd`, `in_ready` is held low: the buffered half must be flushed first, so the non-half input is consumed the next cycle as `fwd_now`. This is what preserves record order — a full slot can never jump ahead of a still-buffered earlier half.

33.4.5 Pending-Slot Register

`r_pend_valid` / `r_pend_slot` hold the first-of-pair half. The register sets on `buffer_now` and clears when the buffered half is consumed — either paired (`pair_now`) or flushed (`flush_fwd` / `idle_flush`) — gated by `out_ready`.

33.4.6 Bit-Exactness

The golden model flushes a lone trailing half exactly once at end-of-stream. This packer flushes whenever its input is idle with a half buffered, which is identical for a gap-free record stream (the cosim drives records back-to-back, then idles once). A mid-stream input bubble would flush early — harmless (slightly less packing), never wrong.

33.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

33.6 Usage Examples

The packer is instantiated inside `monbus_group_core` under the `HALF_BEAT_EN==1` generate branch:

```
monbus_halfbeat_packer u_packer (
    .clk           (axi_aclk),
    .rst_n         (axi_aresetn),
    .in_valid      (comp_out_valid),
    .in_ready      (comp_out_ready),
    .in_slot       (comp_out_slot),
    .in_half_valid (comp_out_half_valid),
    .in_half_slot  (comp_out_half_slot),
    .out_valid     (comp_wr_valid),
    .out_ready     (write_fifo_wr_ready),
    .out_slot      (comp_wr_data)
);
```

When `HALF_BEAT_EN==0`, the compressor drives the write FIFO directly (the committed, timing-closed path) and this packer is not elaborated.

33.7 Design Notes

33.7.1 Requires the Compressor

Half-beat packing only makes sense downstream of the compressor, so `HALF_BEAT_EN==1` requires `USE_COMPRESSION==1`. In raw-only builds neither block exists.

33.7.2 Order Preservation Is Non-Negotiable

The `flush_fwd` hold is the crux of correctness: without it, a full slot could be emitted while an earlier half still sits buffered, reordering the record stream. The single-cycle hold guarantees the buffered half is written first.

33.7.3 One-Slot Buffer Only

The packer holds at most one pending half. It never accumulates more than two records' worth of state, keeping it cheap and its latency bounded to a single beat.

33.8 Related Modules

33.8.1 Used By

- **monbus_group_core.sv** — instantiates the packer in the `HALF_BEAT_EN==1` branch

33.8.2 Uses

- **reset_defs.svh** — reset macros (`ALWAYS_FF_RST`, `RST_ASSERTED`)

33.8.3 See Also

- **monbus_compressor.sv** — upstream compressor supplying the half sideband
- **monbus_group_core.sv** — capture core that owns the compressed write path

---

33.9 Testing

No dedicated testbench for this module. It has no `val/**/test_monbus_halfbeat_packer.py`. It is exercised indirectly, through the tests of modules that instantiate it (directly or further up):

- `monbus_axi4_axil4_group` – `val/**/test_monbus_axi4_axil4_group.py`
- `monbus_axil4_axi4_group` – `val/**/test_monbus_axil4_axi4_group.py`
- `monbus_axil4_axil4_group` – `val/**/test_monbus_axil4_axil4_group.py`

Indirect coverage exercises this module only in the configurations those parents elaborate. A parameter or mode no parent uses is untested.

Treat any behaviour described on this page as unverified by simulation.

33.10 References

33.10.1 Related work and specifications

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Anis, E., Nicolici, N. "On Using Lossless Compression of Debug Data in Embedded Logic Analysis." *Proc. IEEE Int. Test Conf. (ITC 2007)*, pp. 1-10, Oct. 2007. DOI: 10.1109/TEST.2007.4437613. — related work: compressing debug data to extend an on-chip trace buffer. The pairing of two half-slots in one beat is standard bit packing with no paper.
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

33.10.2 Source Code

- RTL: `rtl/amba/ocelus/monbus_halfbeat_packer.sv`
- Golden model: `bin/TBClasses/monbus/monbus_compressor.py` (`Encoder(half_beat=True)`)

33.10.3 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Compressor: `docs/markdown/rtl-amba/monitor/monbus_compressor.md`
- Group Core: `docs/markdown/rtl-amba/monitor/monbus_group_core.md`

Last Updated: 2026-07-15

33.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

34 Monitor Bus Legal-Set CAM

Module: monbus_legal_cam.sv **Location:** rtl/amba/ocelus/ **Category:** Coverage / Observability **Status:** Production Ready

Plane: output plane

34.1 Overview

monbus_legal_cam is the legal-set match CAM used by monbus_pkt_tally in profile mode. A CSR-loaded set of legal message identities (agent / protocol / packet-type / event-code keys) is matched combinationally against an incoming key. A **hit** returns the entry's **dense index** — used directly as the tally bin, so per-agent coverage is a first-class count. A **miss** is reported so the caller can route the packet to a single UNEXPECTED bin.

The point is density: the legal message space is ~1.2% of the full protocol × type × event grid, and crossing it with agent_id would be ~99% empty. Rather than a sparse direct-mapped SRAM, the legal set is enumerated into N_ENTRIES dense bins, and everything else collapses to one UNEXPECTED bin — which doubles as a spec-violation detector.

34.2 Parameters

Parameter	Type	Default	Description
N_ENTRIES	int	64	Legal-set capacity (dense bins 0..N-1)
KEY_WIDTH	int	32	Message-identity key width
IDX_WIDTH	int	derived	clog2(N_ENTRIES)

The tally builds the key as {agent_id[15:0], protocol[3:0], pkt_type[3:0], event_code[7:0]} (32 bits).

34.3 Ports

Port	Direction	Width	Description
clk	input	1	Clock
rst_n	input	1	Active-low asynchronous reset
load_clear	input	1	Pulse: invalidate all entries
load_we	input	1	Pulse: write one entry
load_addr	input	IDX_WIDTH	Entry index to write
load_valid	input	1	Entry valid bit written with the key
load_key	input	KEY_WIDTH	Legal message-identity key
lookup_key	input	KEY_WIDTH	Incoming identity to match
lookup_hit	output	1	1 when lookup_key matches a valid entry
lookup_idx	output	IDX_WIDTH	Dense index of the match (valid on hit)

34.4 Functional Description

- **Valid** is a packed N_ENTRIES-bit vector, so its reset is a single-shot assign — no per-element delayed-array loop (avoids Verilator BLKLOOPINIT). Keys are gated by valid, so they need no reset.
- **Lookup** is a combinational parallel exact-match against every valid entry, priority-encoded to the low matching index. The host loads unique tuples,

so at most one entry matches; the low index wins if a duplicate is ever loaded.

- A hit drives `lookup_idx` = the entry index; a miss (`lookup_hit = 0`) is the caller's cue to use its UNEXPECTED bin.

34.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

34.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
monbus_legal_cam #(
    .N_ENTRIES              (64),
    .KEY_WIDTH              (32),
    .IDX_WIDTH              ((N_ENTRIES > 1))
) u_monbus_legal_cam (
    .clk                    (clk),
    .rst_n                  (rst_n),
    .load_clear             (load_clear),
    .load_we                (load_we),
    .load_addr              (load_addr),
    .load_valid             (load_valid),
    .load_key               (load_key),
    .lookup_key             (lookup_key),
    .lookup_hit             (lookup_hit),
    .lookup_idx             (lookup_idx)
);
```

34.7 Design Notes

Area scales with `N_ENTRIES × KEY_WIDTH` comparators. When the full legal set exceeds `N_ENTRIES`, the host loads a slice per run and reprograms for the next — the CAM is CSR-loadable at runtime.

34.8 Related Modules

Module	Relationship
`monbus_pkt_tally`	The sole consumer; instantiates this in `PROFILE_MODE`.
`monbus_cam`	The tally's separate LRU write-combining cache (unrelated role).

Coverage design and the on-board scenario flow: `vault/handbook/fpga/Genesys2/stream-mon/monitor-board-coverage.md`.

34.9 Testing

Exercised through the tally's fub test (`val/amba/test_monbus_pkt_tally.py`, profile-mode config): a legal set is loaded, matching and deliberately-illegal packets are driven, and the dense bins + UNEXPECTED are cross-checked against a Python golden.

34.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Pagiamtzis, K., Sheikholeslami, A. "Content-Addressable Memory (CAM) Circuits and Architectures: A Tutorial and Survey." *IEEE J. Solid-State Circuits*, 41(3):712-727, Mar. 2006. DOI: 10.1109/JSSC.2005.864128. — for the CAM structure.
- Sprunt, B. "The Basics of Performance-Monitoring Hardware." *IEEE Micro*, 22(4):64-71, July/Aug. 2002. DOI: 10.1109/MM.2002.1028477. — related work: counting events in hardware (the tally this CAM feeds).

- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/common/references/ContentAddressableMemory__Pagiamtzis2006.pdf`)

34.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

35 monbus_pkt_tally

Module: monbus_pkt_tally.sv **Location:** rtl/amba/ocelus/ **Category:** Coverage / Observability **Status:** Production Ready

Plane: output plane

35.1 Overview

monbus_pkt_tally is an **on-chip packet-coverage histogram**. Every accepted monitor-bus packet's identity key {agent[15:0], protocol[3:0], pkt_type[3:0], event_code[7:0]} is looked up in a host-loaded **legal set** (a monbus_legal_cam); the CAM returns a **dense bin index** — the key's position in the legal set — and the count SRAM at that index is incremented. A key that is *not* in the legal set increments a single **UNEXPECTED** bin (index N_PROFILE), so anything unforeseen is caught rather than silently mis-binned. The CAM is **always** in the path — there is no direct-mapped bypass and no write-combining cache.

It's the silicon twin of the sim-side packet-type coverage matrix (bin/monbus_coverage_report.py + TBClasses.monbus.parse): a bin count > 0 means "this message happened on hardware", and one readback sweep dumps the whole coverage matrix. And because a counter absorbs any arrival rate, a coverage run can span **millions of cycles** with no capture-bandwidth limit — unlike the compressor+log path, which bounds capture to the log SRAM depth.

The intended home is the Genesys 2 monitor board-validation build; see vault/handbook/fpga/Genesys2/stream-mon/monitor-board-coverage.md.

35.2 Parameters

Parameter	Default	Description	Constraints
PKT_WIDTH	128	Monitor packet width	Locked by monitor_common_pkg
TS_WIDTH	64	Side-band timestamp width	Locked
COUNT_WIDTH	32	Saturating bin-count width	≥ 1; sizes the SRAM word
NUM_LATCH	4	First-event capture slots	≥ 1
ADDR_BITS	7	Bin address width	≥

Parameter	Default	Description	Constraints
			clog2(N_PROFILE+1) — sizes the dense count SRAM
N_PROFILE	64	Legal-set entries (dense bins)	Dense bins 0..N-1; N = UNEXPECTED
SRAM_DEPTH	1<<ADDR_BITS	Derived count-SRAM depth	Do not override
PROF_KEY_W	int	32	{agent[15:0],proto[3:0],type[3:0],event[7:0]}

35.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
PROF_IDX_W	(N_PROFILE > 1) ? $clog2(N_PROFILE) : 1
LSEL_WIDTH	(NUM_LATCH > 1) ? $clog2(NUM_LATCH) : 1
LFILL_WIDTH	$clog2(NUM_LATCH + 1)

35.3 Ports

The full interface, straight from the RTL:

```
module monbus_pkt_tally #(
    parameter int PKT_WIDTH   = 128,   // monitor_packet_t width
(locked)
    parameter int TS_WIDTH    = 64,    // side-band timestamp width
(locked)
    parameter int COUNT_WIDTH = 32,    // saturating bin count width
    parameter int NUM_LATCH   = 4,     // first-event capture slots
    parameter int ADDR_BITS   = 7,     // bin address width (sizes the
dense SRAM)
    parameter int N_PROFILE   = 64,    // legal-set entries; dense
bins 0..N-1, UNEXPECTED = N
    parameter int SRAM_DEPTH  = (1 << ADDR_BITS)
```

```
    // ... plus the legal-set load port
(profile_clear/we/waddr/wvalid/wkey)
) (
    input  logic                    clk,
    input  logic                    rst_n,

    // Accepted-packet input (valid/ready; one packet per handshake)
    input  logic                    in_valid,
    output logic                    in_ready,
    input  logic [PKT_WIDTH-1:0]    in_packet,
    input  logic [TS_WIDTH-1:0]     in_ts,

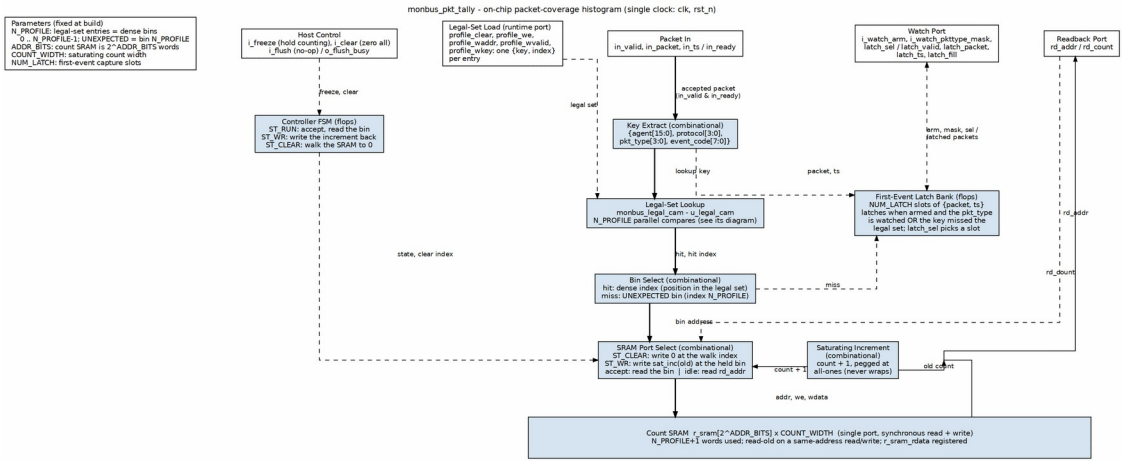
    // Window / snapshot control
    input  logic                    i_freeze,      // level: hold
counting
    input  logic                    i_flush,       // no-op (no cache
to drain)
    output logic                    o_flush_busy,  // high while the
clear walk runs
    input  logic                    i_clear,       // pulse: zero
everything

    // Count readback (registered; valid one cycle after rd_addr, idle
only)
    input  logic [ADDR_BITS-1:0]    rd_addr,
    output logic [COUNT_WIDTH-1:0]  rd_count,

    // First-event latch
    input  logic                    i_watch_arm,          // level:
capture enable
    input  logic [15:0]             i_watch_pkttype_mask, // bit p =
watch pkt_type p
    input  logic [$clog2(NUM_LATCH)-1:0] latch_sel,
    output logic                    latch_valid,
    output logic [PKT_WIDTH-1:0]    latch_packet,
    output logic [TS_WIDTH-1:0]     latch_ts,
    output logic [$clog2(NUM_LATCH+1)-1:0] latch_fill
);
```

35.4 Functional Description

35.4.1 Architecture



monbus_pkt_tally block diagram

An accepted packet's key `{agent, protocol, pkt_type, event_code}` is looked up in the host-loaded legal set (`monbus_legal_cam`); a hit gives the dense bin, a miss the `UNEXPECTED` bin. The count SRAM is single-port: the bin is read in `ST_RUN`, and the saturating increment of that old count climbs the right edge back into the port select and is written in `ST_WR`. When the controller is idle the same port serves host readback at `rd_addr`. The first-event latch bank captures `{packet, ts}` when armed and the `pkt_type` is in the watch mask **or** the key missed the legal set, so the first unexpected packets are kept for inspection.

A tiny FSM runs the accept: `ST_RUN` reads the CAM-resolved bin, `ST_WR` writes the saturating-incremented count back; a walk-counter (`ST_CLEAR`) zeroes the SRAM on clear. `i_flush` is accepted for interface compatibility but is a **no-op** — there is no cache to drain.

35.4.2 Data Model

```
total(bin) = SRAM[bin]            # always live; no cache term
bin        = legal_cam.lookup(key) ? dense_index : UNEXPECTED   (index
N_PROFILE)
```

- **Legal-set hit** — increment `SRAM[dense_index]` (the key's slot in the loaded legal set).
- **Legal-set miss** — increment `SRAM[UNEXPECTED]`. A non-zero `UNEXPECTED` count means a packet arrived with a tuple the host did not load, and should be flagged loudly.

All counts **saturate**: a pegged bin never wraps.

The **legal set** is loaded by the host over the config port (clear, then one `{key, index}` pair per entry). The 32-bit key is `{agent[15:0], protocol[3:0], pkt_type[3:0],`

event_code[7:0]} — note it includes **agent**, so per-instance identity is resolved, not just the message class. Dense bins run 0..N_PROFILE-1 with UNEXPECTED = N_PROFILE, so only N_PROFILE+1 words are ever *used*, regardless of how sparse the underlying tuple space is. The SRAM itself is SRAM_DEPTH = 1 << ADDR_BITS words (128 at the default ADDR_BITS = 7), and the clear walk covers all of them.

35.4.3 Legal-Set (Dense) Binning

A direct-mapped {protocol, pkt_type, event_code} matrix cannot answer *"did every agent fire?"* (agent_id is not in the key) and is ~99% empty (only ~245 of the 20,480 protocol×type×event cells are ever legal). So the tally does **not** use one; the CAM-resolved dense binning is the **only** mode (the earlier PROFILE_MODE = 0 direct-mapped path was removed).

A monbus_legal_cam holds up to N_PROFILE legal {agent, protocol, pkt_type, event_code} keys; an incoming packet is matched to the entry's **dense bin index** (a hit) or routed to the single **UNEXPECTED bin** at index N_PROFILE (a miss, also captured by the first-event latch). This makes per-agent coverage a first-class count, and turns any out-of-profile message — a wrong event code, an untracked agent, a protocol a unit should not speak — into a swept spec-violation signal.

The legal set is loaded/cleared over the config port. Two host encodings exist: the raw profile-load pins (profile_clear / profile_we / profile_waddr / profile_wvalid / profile_wkey), and — in the STREAM-monitor AXIL wrapper — a **register model** (CAM_CLEAR, then per entry CAM_KEY followed by CAM_LOAD = (valid<<31)|index) that carries the index in the write *data* so it is immune to bus-width/stride hazards. Either way the host can reprogram slices per run when the full legal set exceeds N_PROFILE.

Coverage gate: every expected dense bin > 0 (all agents/types fired) **and** UNEXPECTED == 0 (nothing rogue slipped through). Note that UNEXPECTED == 0 is also the *fault-free* signal — the fault classes (error/timeout/threshold) only appear here when a fault was deliberately injected.

35.4.4 Snapshot Protocol

The host reads a coherent coverage snapshot over the CSR/AXIL window as follows:

1. i_freeze = 1 — stop counting at a coherent boundary.
2. Sweep rd_addr over every bin and read rd_count (one-cycle registered read, valid only while idle). No flush is needed — counts are live in SRAM.
3. Pulse i_clear — zero the SRAM + the first-event latches for the next window. o_flush_busy is high during the SRAM walk.

rd_count is **always** the coherent count (= SRAM[rd_addr]); there is no cache to fold in. i_flush is retained on the interface but is a no-op.

35.4.5 First-Event Latch

NUM_LATCH slots capture the full 128-bit packet + timestamp of the first accepted packets whose pkt_type bit is set in i_watch_pkttype_mask (with i_watch_arm = 1). This yields the offending packet behind a nonzero error bin, not just a count. latch_fill reports how many slots are populated; read a slot by driving latch_sel and sampling latch_valid / latch_packet / latch_ts. The bank is cleared by i_clear.

35.5 Timing Characteristics

- **Accept path:** a combinational legal-set CAM lookup resolves the bin, then a 2-cycle read-then-saturating-write (ST_RUN → ST_WR) commits it. in_ready is (r_st == ST_RUN) && !i_freeze && !i_clear && !r_clear_pend – so a clear pulse also drops it, and it stays low until the SRAM clear walk finishes. The documented protocol freezes before clearing, which masks this, but a checker written from the "running and unfrozen" summary alone will false-flag a clear that arrives unfrozen. The clear terms are required for valid/ready correctness, not tidiness.
- **Read:** rd_count is a registered read of SRAM[rd_addr], always live.
- **Clear:** walks SRAM_DEPTH entries writing zero (ST_CLEAR).
- **Flush:** no-op (no cache).

35.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
monbus_pkt_tally #(
    .PKT_WIDTH              (128),
    .TS_WIDTH               (64),
    .COUNT_WIDTH            (32),
    .NUM_LATCH              (4),
    .ADDR_BITS              (7),
    .N_PROFILE              (64),
    .SRAM_DEPTH             ((1 << ADDR_BITS)),
    .PROF_IDX_W             ((N_PROFILE > 1)),
    .PROF_KEY_W             (32),
    .LSEL_WIDTH             ((NUM_LATCH > 1))
) u_monbus_pkt_tally (
    .clk                    (clk),
    .rst_n                  (rst_n),
    .in_valid               (in_valid),
```

```
    .in_ready               (in_ready),
    .in_packet              (in_packet),
    .in_ts                  (in_ts),
    .i_freeze               (i_freeze),
    .i_flush                (i_flush),
    .o_flush_busy           (o_flush_busy),
    .i_clear                (i_clear),
    .rd_addr                (rd_addr),
    .rd_count               (rd_count),
    .i_watch_arm            (i_watch_arm),
    .i_watch_pkttype_mask   (i_watch_pkttype_mask),
    .latch_sel              (latch_sel),
    .latch_valid            (latch_valid),
    .latch_packet           (latch_packet),
    .latch_ts               (latch_ts),
    .latch_fill             (latch_fill),
    .profile_clear          (profile_clear),
    .profile_we             (profile_we),
    .profile_waddr          (profile_waddr),
    .profile_wvalid         (profile_wvalid),
    .profile_wkey           (profile_wkey)
);
```

35.7 Design Notes

35.7.1 Why Count Instead of Log?

A 128-bit monitor packet plus a 64-bit timestamp is 24 bytes per event. Log every one and a run tops out at `SRAM_depth / 24` events — a millisecond or two at realistic rates, then you're full. But the board-validation goal is *coverage* ("did every packet type happen, and how often?"), not a trace. For that, a **saturating counter per message bin** is the right structure: it absorbs any rate, drops the compressor (and its worst-case CAM timing path) from the board build, and *is* the coverage matrix in hardware.

Each accepted packet is a read-modify-write on the count SRAM (read the bin, saturating-increment, write it back). A short two-cycle accept sequence services that RMW directly; there is no cache in front. An earlier design added an LRU write-combining cache to collapse the RMWs, but it stranded low-volume counts in the cache (a readback returned SRAM only), so it was removed in favour of the simple, always-coherent direct RMW. `rd_count` is therefore always live — it is `SRAM[rd_addr]` with nothing to drain first.

35.8 Related Modules

35.8.1 Building Blocks Reused

Block	Role
monbus_legal_cam	The CSR-loaded legal-set match CAM that maps {agent, protocol, pkt_type, event_code} to a dense bin index (hit) or a miss (UNEXPECTED). Always in the path.
monitor_common_pkg	The locked 128-bit packet field map (pkt_type[127:124], protocol[108:105], event_code[104:97], agent_id[87:72]).
synchronous single-port SRAM	The backing dense count matrix (N_PROFILE+1 deep).

35.8.2 See Also

- monbus_legal_cam.md — the legal-set CAM that resolves each packet to its dense bin.
- monbus_compressor.md — the alternative capture path (bounded compressed log) this histogram replaces on the board build.
- axi_perf_latency_hist.md — a companion histogram that bins latency magnitude (not message type) with the same freeze/clear window semantics.
- monbus_group_core.md — the filter/route front whose accepted-packet stream feeds this block.

35.9 Testing

Location: val/amba/test_monbus_pkt_tally.py **Run:** pytest val/amba/test_monbus_pkt_tally.py -v

The acceptance criterion is an **exact** cross-check: after a freeze/flush, the hardware bin counts must equal a pure-Python golden count of the same accepted (protocol, pkt_type, event_code) stream. A lost increment shows up as a per-bin mismatch. Phases: random count + readback, back-to-back same-bin accepts (the RMW hazard the two-cycle accept sequence exists to cover), saturation (a bin pegs and never wraps), first-event latch, and clear. (Earlier revisions of

this section described eviction-stress phases against an LRU write-combining cache; that cache was removed – every accept is now a direct SRAM read-modify-write.)

35.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Sprunt, B. “The Basics of Performance-Monitoring Hardware.” *IEEE Micro*, 22(4):64-71, July/Aug. 2002. DOI: 10.1109/MM.2002.1028477. — related work: on-chip event counters; a packet-coverage histogram has no peer source.
- Pagiamtzis, K., Sheikholeslami, A. “Content-Addressable Memory (CAM) Circuits and Architectures: A Tutorial and Survey.” *IEEE J. Solid-State Circuits*, 41(3):712-727, Mar. 2006. DOI: 10.1109/JSSC.2005.864128. — for the legal-set CAM lookup.
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/common/references/ContentAddressableMemory__Pagiamtzis2006.pdf`)

35.11 Navigation

- **← Back to monitor index**
- **← Back to rtl-amba index**

36 monbus_wb4_axil4_group / monbus_wb4_axi4_group

Plane: output plane

36.1 Overview

The monitor-bus groups with their read-only CSR port presented as a **Wishbone B4 slave** instead of an AXI4-Lite one, so a Wishbone host can own a monitor group without an AXI bridge in front of it.

Module	Host read port	Record-write master
`monbus_wb4_axil4_group`	Wishbone B4	AXI4-Lite
`monbus_wb4_axi4_group`	Wishbone B4	AXI4 (burst)

Both are **thin wrappers**. The drain path, compressor, FIFOs, interrupt and statistics are the existing `monbus_axil4_axil4_group` and `monbus_axil4_axi4_group` bodies, untouched; only the host read port changes, through `monbus_wb4_rd_shim`. Duplicating the group body would have created a second copy to keep in step, and the copy nobody edits is the one that rots.

36.2 monbus_wb4_rd_shim

Wishbone B4 slave in, AXI4-Lite **read** master out.

Parameter	Default	Description
ADDR_WIDTH	32	Address width
DATA_WIDTH	64	The group's read beat; 32 for a 32-bit host
CMD_DEPTH / RSP_DEPTH	2	The internal `wb4_slave`'s queues
CLASSIC	0	0 = B4 pipelined, 1 = B4 standard
AXIL_PROT	3'b000	`ARPROT` driven on every read

Writes are refused, not ignored. The CSR port is read-only. A Wishbone write terminates ERR in the shim and never reaches the group, so a host that writes here learns it was wrong instead of believing it worked.

One transfer at a time. This port is a host reading a drain register; pipelining it would add a reorder buffer to save nothing. The internal slave is built `MAX_OUTSTANDING = 1`.

This is deliberately not `wb4_to_axil4`, which carries write channels and an outstanding queue that a read-only CSR port has no use for.

36.3 Reading a record

Identical to the AXI4-Lite groups: each read pops one beat of the error FIFO, and a record is three 64-bit slices. With `S_AXIL_DATA_WIDTH = 32` the group's 2:1 serializer presents each slice as a low then a high read, so a record is six beats. The Wishbone side sees ordinary single reads either way.

The drain register is **address-insensitive**: any read in the region pops the next beat. That is the group's existing behaviour, not something the shim adds, and it means a test cannot detect a corrupted read address on this port. The data path is checked instead.

36.4 Notes

- Under `ifdef FORMAL` the shim asserts that a write never reaches the group, that only one read is outstanding, that `R` is consumed only while a read is open, and that a write is always refused rather than acknowledged.
- The wrappers forward every other group port straight through, so the configuration, statistics and interrupt interfaces are unchanged.

36.5 Related

- monbus_axil4_axil4_group, monbus_axil4_axi4_group - the bodies these wrap
- wb4_slave - the Wishbone end of the shim

36.6 Test

`val/amba/test_monbus_wb4_axil4_group.py` is the AXI4-Lite group's test with its drain switched to Wishbone through the shared `MonbusGroupHarness`, which gained a `drain_proto="wb4"` path for it. Records decode correctly at both the 64-bit and 32-bit drain widths. A mutation rotating the shim's read data fails the test.

36.7 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Herveille, R. (ed.). *Wishbone B4: WISHBONE System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores*, OpenCores, 2010. – ch. 3 classic and pipelined bus cycles, the slave port these wrappers present.
- Hopkins, A.B.T., McDonald-Maier, K.D. “Debug Support for Complex Systems On-Chip: A Review.” *IEE Proc. Computers and Digital Techniques*, 153(4):197-207, 2006. DOI: 10.1049/ip-cdt:20050194. – related work: on-chip trace buffers versus trace ports.
- Indexed in `rtl/amba/references/README.md` (local PDF: `WishboneB4Spec__OpenCores2010.pdf`)

37 Monitor Characterization

What this is: the measured cost of every monitor variant in `rtl/amba`, on the two parts the repo's boards carry, from one repeatable flow. It answers "what does adding a monitor to this port cost, in LUTs, flops, timing and (estimated) power, and how do the variants compare". Written for amba TASK-034 on 2026-09-28; re-run it rather than trusting it once the RTL moves.

How to reproduce:

```
rtl/amba/fpga/bin/monitor_synth_sweep.sh                  # Kintex-7
325T -2 at 6.667 ns (Genesys 2)
TARGETS="xc7a100tcsg324-1:10.0"
rtl/amba/fpga/bin/monitor_synth_sweep.sh <modules...>
rtl/amba/fpga/bin/summary_table.py                         # the tables
below, from reports/summary.csv
```

Each module is synthesized, placed and routed **out of context** by `rtl/amba/fpga/tcl/monitor_synth.tcl` – the same recipe as the bridge fixture (`projects/components/fabric-gen-ip/bridge/fpga`): Vivado 2025.1, one clock at the stated period, 30 % input/output delays, reset false-pathed. Utilization is post-route. Power is Vivado's vectorless estimate (`report_power`, confidence "Medium"): it ranks variants against each other and says nothing absolute.

Plane: cross-plane (measured cost of protocol-plane monitors and the output-plane group)

37.1 What was measured

Every module at its **default parameters** (the wrapper's own defaults, not the bridge's), so the numbers here are larger than the bridge fixture's per-instance figures in axi_monitor_lite.md: the lite there has the bridge's parameters; here it has its own defaults.

Family	Modules	Defaults that set the size
AXI4 read master	`axi4_master_rd` (plain), `_cg`, `_mon`, `_mon_cg`, `_monlite`, `_monlite_cg`	ID 8, addr 32, data 32; full monitor 16 slots; lite 8 slots; skids AR 2 / R 4
AXI4 write master	`axi4_master_wr` (plain), `_mon`, `_monlite`	as above
AXI4 read slave	`axi4_slave_rd` (plain), `_mon`, `_monlite`	as above
AXI5 read master	`axi5_master_rd` (plain), `_mon`, `_monlite`	as above
AXI-Lite read/write master	`axil4_master_rd`, `axil4_master_wr` (plain),	addr 32, data 32

Family	Modules	Defaults that set the size
	`_mon`, `_monlite`; `axil5_master_rd_monlite`	
APB	`apb4_monitor`, `apb5_monitor`	4 slots, FIFO 8
AXIS	`axis4_master_monlite`, `_cg`	data 32, ID 8, skid 4
Wishbone	`wb4_monitor`	8 slots, FIFO 8
Aggregation	`monbus_arbiter` (4 clients), `monbus_axil4_axil4_group`, `monbus_axi4_axi4_group`	group: error FIFO 64 records, write FIFO 96 beats, raw records

The `_mon` wrappers embed the full monitor (`axi_monitor_base`), the `_monlite` wrappers the lite (`axi_monitor_lite`); the plain block is the same master or slave without a monitor, so **wrapper minus plain is the monitor**. `_cg` adds the clock-gating controller.

37.1.1 Kintex-7 325T -2 (Genesys 2), clock constrained at 6.667 ns, out of context

Module	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	WNS incl. I/O (ns)	Worst logic levels	Dynamic power (est.)
`apb4_monitor`	572	653	0	+0.939	+0.939	12	18 mW
`apb5_monitor`	594	676	0	+0.596	+0.596	15	19 mW
`axi4_master_rd`	285	328	0	+3.861	+1.515	2	7 mW
`axi4_master_rd_cg`	308	342	0	+1.711	+1.029	3	5 mW
`axi4_master_rd_mon`	7,291	5,547	0	+0.477	-0.166	12	76 mW

Module	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	WNS incl. I/O (ns)	Worst logic levels	Dynamic power (est.)
axi4_master_rd_mon_cg	7,372	5,561	0	-0.608	-0.608	10	83 mW
axi4_master_rd_monlite	1,400	1,355	0	+2.145	+1.005	5	17 mW
axi4_master_rd_monlite_cg	1,421	1,369	0	+1.619	+0.854	12	20 mW
axi4_master_wr	297	332	0	+4.306	+1.515	2	7 mW
axi4_master_wr_mon	8,492	6,160	0	-1.053	-1.546	13	77 mW
axi4_master_wr_monlite	1,511	1,414	0	+2.046	+0.910	13	18 mW
axi4_slave_rd	285	328	0	+3.861	+1.515	2	7 mW
axi4_slave_rd_mon	7,269	5,552	0	+0.566	+0.067	9	72 mW
axi4_slave_rd_monlite	1,400	1,355	0	+2.139	+1.179	6	16 mW
axi5_master_rd	380	444	0	+3.981	+1.503	2	10 mW

Module	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	WNS incl. I/O (ns)	Worst logic levels	Dynamic power (est.)
axi5_master_rd_mon	7,387	5,663	0	+0.501	+0.023	12	77 mW
axi5_master_rd_monlite	1,495	1,471	0	+1.948	+0.876	6	18 mW
axil4_master_rd	214	218	0	+4.378	+1.505	2	4 mW
axil4_master_rd_mon	3,105	2,854	0	+1.432	+0.408	7	42 mW
axil4_master_rd_monlite	1,143	1,156	0	+1.889	+1.011	9	14 mW
axil4_master_wr	179	164	0	+4.392	+1.515	2	3 mW
axil4_master_wr_mon	3,481	2,955	0	+0.722	+0.163	6	44 mW
axil4_master_wr_monlite	1,209	1,156	0	+2.055	+1.515	9	15 mW
axil5_master_rd_monlite	1,221	1,272	0	+1.936	+1.044	6	15 mW
axis4_master_monlite	1,550	935	0	+1.867	+0.363	6	29 mW

Module	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	WNS incl. I/O (ns)	Worst logic levels	Dynamic power (est.)
axis4_master_monlite_cg	1,570	949	0	+1.695	+0.308	3	22 mW
monbus_arbiter	1,821	1,972	0	+2.937	+2.295	2	17 mW
monbus_axi4_axi4_group	1,967	1,386	0	+1.366	-1.728	13	35 mW
monbus_axil4_axil4_group	1,715	1,154	0	+1.152	-2.004	11	31 mW
wb4_monitor	622	909	0	+0.731	+0.731	16	27 mW

37.1.1.1 The monitor's own cost (Kintex-7 325T -2 (Genesys 2)): wrapper minus the plain block

Monitored wrapper	Wraps	Monitor LUTs	Monitor FFs	Wrapper WNS reg-to-reg (ns)	Plain WNS reg-to-reg (ns)
axi4_master_rd_mon	axi4_master_rd	7,006	5,219	+0.477	+3.861
axi4_master_rd_monlite	axi4_master_rd	1,115	1,027	+2.145	+3.861
axi4_master_rd_mon_cg	axi4_master_rd_cg	7,064	5,219	-0.608	+1.711
axi4_master_rd_monlite_cg	axi4_master_rd_cg	1,113	1,027	+1.619	+1.711
axi4_ma	axi4_mas	8,195	5,828	-1.053	+4.306

Monitored wrapper	Wraps	Monitor LUTs	Monitor FFs	Wrapper WNS reg-to-reg (ns)	Plain WNS reg-to-reg (ns)
ster_wr_mon	ter_wr				
axi4_master_wr_monlite	axi4_master_wr	1,214	1,082	+2.046	+4.306
axi4_slave_rd_mon	axi4_slave_rd	6,984	5,224	+0.566	+3.861
axi4_slave_rd_monlite	axi4_slave_rd	1,115	1,027	+2.139	+3.861
axi5_master_rd_mon	axi5_master_rd	7,007	5,219	+0.501	+3.981
axi5_master_rd_monlite	axi5_master_rd	1,115	1,027	+1.948	+3.981
axil4_master_rd_mon	axil4_master_rd	2,891	2,636	+1.432	+4.378
axil4_master_rd_monlite	axil4_master_rd	929	938	+1.889	+4.378
axil4_master_wr_mon	axil4_master_wr	3,302	2,791	+0.722	+4.392
axil4_master_wr_monlite	axil4_master_wr	1,030	992	+2.055	+4.392

37.1.2 Artix-7 100T -1 (Nexys A7), clock constrained at 10.000 ns, out of context

Module	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	WNS incl. I/O (ns)	Worst logic levels	Dynamic power (est.)
apb4_monitor	580	653	0	+0.152	+0.152	13	12 mW
axi4_	285	328	0	+5.969	+1.895	2	5 mW

Module	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	WNS incl. I/O (ns)	Worst logic levels	Dynamic power (est.)
master_rd							
axi4_master_rd_mon	7,636	5,547	0	-0.868	-1.451	12	49 mW
axi4_master_rd_monlite	1,403	1,355	0	+2.097	+0.617	14	10 mW
axi4_master_rd_monlite_cg	1,421	1,369	0	+1.095	+0.341	13	15 mW
axil4_master_rd_monlite	1,144	1,156	0	+1.478	+0.393	11	9 mW
axis4_master_monlite	1,550	935	0	+1.646	+0.599	6	20 mW
axis4_master_monlite_cg	1,571	949	0	+1.652	+0.303	6	15 mW
monbus_axil4_axil4_group	1,725	1,154	0	+0.111	-4.920	8	21 mW
wb4_monitor	622	909	0	+0.045	+0.045	16	18 mW

37.1.2.1 The monitor's own cost (Artix-7 100T -1 (Nexys A7)): wrapper minus the plain block

Monitored wrapper	Wraps	Monitor LUTs	Monitor FFs	Wrapper WNS reg-to-reg (ns)	Plain WNS reg-to-reg (ns)
axi4_master_rd_mon	axi4_master_rd	7,351	5,219	-0.868	+5.969
axi4_master_rd_monlite	axi4_master_rd	1,118	1,027	+2.097	+5.969

The "WNS incl. I/O" column carries the out-of-context I/O budget: every data pin is given 30 % of the period, so a block whose configuration inputs feed logic directly (the groups' `cfg_*_mask` pins, the AXIS lite's `cfg_timeout_cycles`) shows a primary-input miss that a real design, where those pins come from CSR flops nearby, does not have. The register-to-register column is the block's own timing and is the one to read.

37.2 What the numbers say

Full monitor against lite. At their defaults the full AXI monitor is about 7,000 LUTs and 5,200 flops (16 slots, the CAM-based transaction manager, six reporter cones); the lite is about 1,100 LUTs and 1,030 flops (8 slots, linked lists, one event stage): 6.3x in LUTs, 5x in flops, on the same read, write, slave and AXI5 wrappers alike, because the monitor is the same core behind every one. On AXI-Lite the ratio is 3x (2,900 against 930): the ID-less path removes most of the full monitor's matching logic and leaves the lite's fixed costs. The lite in the bridge fixture measured 677 LUTs (axi_monitor_lite.md) because the bridge sets narrower IDs and fewer slots; both figures are right for their parameters, and the ratio to the full monitor is the number that carries.

Timing. Every lite variant meets 6.667 ns on the Kintex-7 with 0.8 to 1.5 ns to spare, register to register. The full AXI4 monitors at 16 slots do not: the read master misses by 0.17 ns, the write master by 1.55 ns, and the worst path in each is inside `monitor_trans_cam` (`g_slot[n]` compare and select). The bridge fixture met on this part with the full monitor because its `error_only` preset compiles fewer cones; at the wrapper's defaults the CAM is the limit of the full monitor near 150 MHz. On the Artix-7 at 10 ns the full read monitor misses by 1.45 ns and the lite meets by 0.6 ns.

Clock gating. `_cg` adds 20 to 80 LUTs and about 14 flops for the gating controller and costs 0.15 to 0.45 ns of register-to-register slack (the enable now sits on the clock enable of every register). Its purpose is dynamic power in an idle design; the vectorless estimate cannot show that (it assumes default toggle rates), so the power column reads a few milliwatts higher for `_cg`, which is the controller's own activity, not a measurement of savings. Measure clock gating on a board with real traffic or not at all.

Across protocols. Per monitor, at defaults: AXI4/AXI5 lite about 1,100 LUTs; AXI-Lite lite about 930 to 1,030; AXIS lite about 1,550 (it is the whole wrapper here, there is no plain `axis4_master` to subtract, and it carries a skid of 4); APB4/APB5 monitors 570 to 600; Wishbone 620. The APB and Wishbone monitors are older, table-based designs at 4 and 8 slots and sit between the two AXI variants in cost per slot.

Aggregation. A four-client `monbus_arbiter` with skids is 1,800 LUTs and 2,000 flops (the flops are four 192-bit input skids and one output skid); a group at its defaults (64-record error FIFO, 96-beat write FIFO) is 1,700 to 2,000 LUTs and about 1,200 flops, meeting 6.667 ns register to register with 1.2 to 1.4 ns to spare after the planner split of amba ISSUE-001.

Power. The estimates scale with LUT count as expected (full monitor 75 to 80 mW dynamic against the lite's 15 to 20 mW at default activity); treat them only as a ratio between variants on the same part.

37.3 Recommendations

1. **Default to the lite.** It is one sixth of the full monitor in LUTs, one fifth in flops, meets 150 MHz on the Kintex-7 with margin and 100 MHz on the Artix-7, and reports every event it cannot deliver. Reach for the full monitor only for a class it alone has (performance windows, debug trace, address filtering with the ID filter).
2. **If the full monitor is needed at 150 MHz, cut its slots.** The CAM is the critical path and scales with `MAX_TRANSACTIONS`; the bridge fixture met with fewer cones. Measure the chosen configuration with this sweep rather than the defaults.
3. **Budget the group once per subsystem, not per port.** Its 1,700 to 2,000 LUTs are the fixed cost of the drain; three or thirty ports share it.
4. **Register configuration inputs next to the block.** Every out-of-context miss in the tables is a `cfg_*` pin driving arithmetic directly. In a real design those come from CSR flops; keep them adjacent (the group already re-registers its window config internally for this reason).
5. **Leave `_cg` off unless idle power is the goal**, and measure that on silicon. It buys nothing in area or timing.

37.4 Open findings from this sweep

- `axis4_master_monlite` missed 10 ns on the Artix-7 register to register by 1.18 ns in the first sweep (16 to 19 levels into `r_dropped`): the AXIS lite decided its events and counted drops in one cycle. Fixed the next day as rtl/amba/monitors ISSUE-003 with an event stage; the tables above carry

the re-run: +1.65 ns on the Artix-7 and +1.87 ns on the Kintex-7 register to register, 6 logic levels, for 44 LUTs and 199 flops more.

- The full AXI4 monitors at 16 slots miss 6.667 ns on the Kintex-7 in the CAM; recorded above, no item filed: the lite is the recommended answer and the bridge's smaller preset meets.

37.5 Files

`rtl/amba/fpga/tcl/monitor_synth.tcl` (the recipe), `bin/monitor_synth_sweep.sh` (the matrix), `bin/summary_table.py` (these tables), `reports/<module>__<part>/` (utilization, timing, power per run) and `reports/summary.csv` (one row per run).

37.6 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. "An Event-Based Monitoring Service for Networks on Chip." *ACM Trans. Design Automation of Electronic Systems*, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126. – for comparison: probe area reported at 17-24% of NoC area.
- Goossens, K., Vermeulen, B., van Steeden, R., Bennebroek, M. "Transaction-Based Communication-Centric Debug." *Proc. 1st Int. Symp. Networks-on-Chip (NOCS 2007)*, pp. 95-106, May 2007. DOI: 10.1109/NOCS.2007.46. – for comparison: monitors plus event interconnect at 4.5% of NoC area.
- Ciordas, C., Goossens, K., Basten, T., Rădulescu, A., Boon, A. "Transaction Monitoring in Networks on Chip: The On-Chip Run-Time Perspective." *Proc. IEEE Int. Symp. Industrial Embedded Systems (IES 2006)*, pp. 1-10, Oct. 2006. DOI: 10.1109/IES.2006.357464. – for comparison: transaction monitoring at about 5% of NoC area.
- Fiorin, L., Palermo, G., Silvano, C. "MPSoCs Run-Time Monitoring through Networks-on-Chip." *Proc. Design, Automation & Test in Europe (DATE 2009)*, pp. 558-561, Apr. 2009. DOI: 10.1109/DATE.2009.5090726. – for comparison: ASIC overhead and added traffic of a run-time monitoring system.
- Indexed in `rtl/amba/references/README.md` (Monitor subsystem family; citation only)

38 Ocelus plane map

Scope: the five purpose-sliced layers that make up the Ocelus AMBA monitor, their membership, and how they fit together. For the repo-wide definition of a "plane" see the handbook note what-is-a-plane.

38.1 The five planes

Ocelus is sliced into five planes. Each plane is a unit you can decide to keep or drop; dropping one never reroutes the others.

Plane	Job	Members
Protocol plane	Observe the bus and emit protocol-event packets	`axi_monitor_base`, `axi_monitor_lite`, `axis_monitor_lite`, `cxs_monitor_lite`, `axi4ace_snoop_monitor_lite`, `axi_monitor_trans_mgr`, `monitor_trans_cam`, `axi_monitor_addr_check`, `apb_monitor_addr_check`, `axi_monitor_timeout`, `axi_monitor_timer`, `axi_monitor_filtered`
Error plane	Detect and packetize errors, timeouts and threshold violations	`axi_monitor_reporter_error`, `axi_monitor_reporter_timeout`, `axi_monitor_reporter_threshold`
Debug plane	Emit completion and debug-trace packets	`axi_monitor_reporter_debug`, `axi_monitor_reporter_compl`
Perf plane	Roll up performance counters into perf packets	`axi_monitor_reporter_perf`

Plane	Job	Members
Output plane	Arbitrate, filter, compress and drain packets off-chip	`arbiter_monbus_common`, `arbiter_rr_pwm_monbus`, `arbiter_wrr_pwm_monbus`, `monbus_arbiter`, `monbus_group_core`, `monbus_axi4_axi4_group`, `monbus_axi4_axil4_group`, `monbus_axil4_axi4_group`, `monbus_axil4_axil4_group`, `monbus_wb4_axi4_group`, `monbus_wb4_axil4_group`, `monbus_wb4_rd_shim`, `monbus_compressor`, `monbus_halfbeat_packer`, `monbus_cam`, `monbus_cam_pipe`, `monbus_legal_cam`, `monbus_pkt_tally`

38.1.1 Shared reporter infrastructure

`axi_monitor_reporter.sv` (no suffix) is **not a plane** and **not a protocol-plane member**. It is the thin top-level dispatcher that multiplexes the six per-packet-type reporter cones onto the shared MonBus output and reports events back to `axi_monitor_trans_mgr`. The error, debug and perf planes feed through it; dropping one of those planes simply disables its cone, leaving the dispatcher and the others untouched.

38.2 Build personas (examples, not prescriptions)

These are illustrative inclusion matrices. The exact parameters live on each module page.

Persona	Planes kept	Output-plane options	Typical use
bring-up	protocol + error	AXI4 group drain, no compressor, no	catch SLVERR, DECERR, timeouts

Persona	Planes kept	Output-plane options	Typical use
		CAM capture	and bus hangs
silicon-perf	protocol + perf	compressor + halfbeat_packer enabled	long runs measuring utilization and error roll-ups with compressed bulk trace
field-debug	protocol + error + debug	CAM capture enabled (`monbus_cam` / `monbus_legal_cam`), Wishbone drain	capture trace around a rare failure and drain through a low-pin-count WB4 port
full	all five	all options enabled	maximum observability; pay the area and bandwidth cost

Omitting a plane at build time is a parameter change (`ENABLE_*_LOGIC` on the reporter cones, `USE_COMPRESSION` on the group wrappers, `USE_CAM` on capture) not a rewrite of the data path.

38.3 Data-flow contract

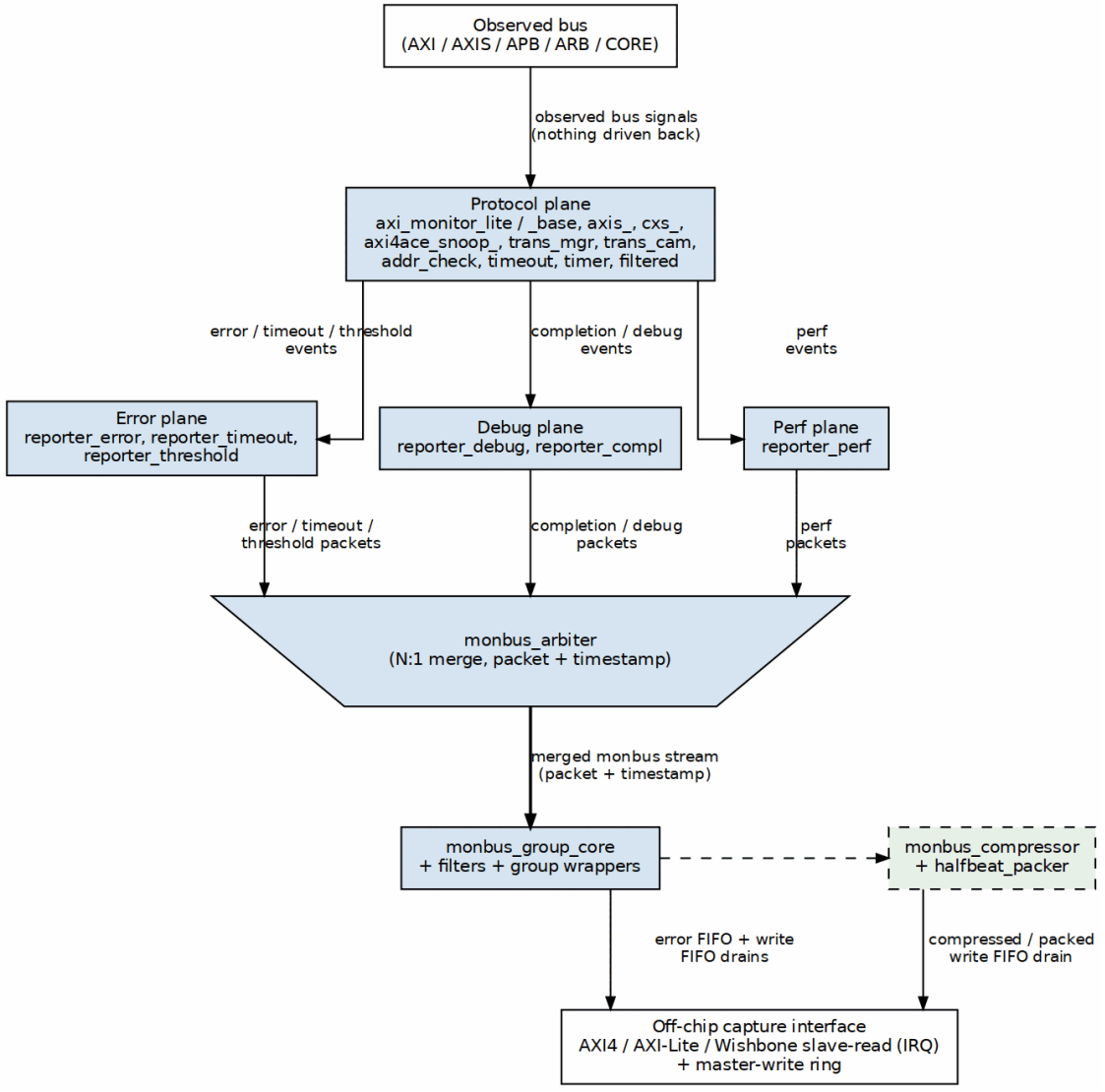
The contract is the same in every build:

1. **Observed bus → protocol plane.** The protocol plane taps the bus as a passive observer. It never drives the bus back.
2. **Protocol plane → reporter planes.** The protocol plane produces transaction state and event triggers. Each reporter plane consumes only the events it owns.
3. **Reporter planes → output plane.** Each reporter plane emits typed `monitor_packet_t` records with a side-band 64-bit timestamp. They are independent sources.
4. **Output plane → off-chip capture.** `monbus_arbiter` merges N sources into one MonBus stream. `monbus_group_core` filters and steers records to the

interrupt/read FIFO or the master-write ring; optional compression/packing sits on the write path. The final drain leaves through one capture interface, regardless of which reporter planes produced the packets.

Dropping a plane removes its source from the arbiter; the remaining sources still merge the same way.

38.4 Plane architecture



Ocelus five-plane architecture

The observed bus enters at the top. The protocol plane sits on it. The error, debug and perf planes consume protocol events independently, feed the MonBus arbiter inside the output plane, and the merged stream is filtered, optionally compressed, and drained off-chip through the single capture interface at the bottom.

38.5 Related

- Handbook: what is a plane
- Ocelus architecture overview
- Monitor package / packet format

38.6 Navigation

- **← Back to Shared Infrastructure Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

39 The Ocelus Architecture

What it is: Ocelus – from the Latin ocellus, “little eye”; a bee’s simple always-on eye – on-chip observability that emits a fixed 128-bit packet whenever something worth knowing about happens – an error, a timeout, a completion, a performance sample, a debug trace point – and a set of transports that get those packets off the chip.

What it is not: protocol-specific. The packet format, the transports, the filtering and the capture back ends know nothing about AXI. A protocol wrapper supplies detection; everything downstream is shared. That is why an arbiter – which is not a bus protocol at all – rides the same infrastructure as AXI4.

Scope of this document. The architecture and what it can do. Per-module detail lives in the module pages; the packet field map and event-code assignments live in monitor_package_spec.

39.1 Planes

Ocelus is purpose-sliced into five planes: the **protocol plane** observes the bus and emits event packets; the **error**, **debug** and **perf** planes each consume those events independently and turn them into typed reporter packets; and the **output plane** arbitrates, filters, optionally compresses and drains whatever is kept off-chip. Each plane can be included or excluded at build time without rerouting the others. See the Ocelus plane map for the full membership table, example build personas and the data-flow contract.

39.2 The one thing to understand first

Everything is a `monitor_packet_t`: 128 bits, laid out identically no matter who produced it.

Bits	Field	Width	Meaning
[127:124]	packet_type	4	error, timeout, completion, perf, debug, …
[123:109]	reserved	15	forward-compat slack
[108:105]	protocol	4	AXI / AXIS / APB / ARB / CORE
[104:97]	event_code	8	which error,

Bits	Field	Width	Meaning
			which event, which metric
[96:88]	channel_id	9	channel, or AXI transaction ID
[87:72]	agent_id	16	which instance
[71:64]	unit_id	8	which subsystem
[63:0]	event_data	64	payload – a full 64-bit address fits

A 64-bit timestamp travels beside the packet as side-band, not inside it. The pair `(packet, timestamp)` is carried atomically from the point of emission all the way to the capture back end – `monbus_arbiter` moves both through a 192-bit skid precisely so a consumer can never observe a packet against the wrong timestamp.

The consequence worth internalising: **`{protocol, packet_type, event_code}` is a message identity.** Filtering keys on it. The coverage histogram bins on it. The compressor templates on it. Adding a new event to a new kind of block means choosing a `protocol` and allocating `event_code` values – and the rest of the system already handles it.

39.3 Two coordinate systems and a topology

This is the property that makes one 128-bit word serve a whole SoC, and none of it is obvious from the field list.

Read the first two as coordinate systems, not as a tree with a fixed root. The packet carries a classification (*what happened*) and an identity (*who it happened to*), and the two are orthogonal. Neither is the base. It is entirely reasonable to organise by `protocol` and treat identity as an attribute of the event – and equally reasonable to organise by `unit_id` and treat the classification as an attribute of the block. The packet is indifferent.

That choice is not cosmetic. Putting `unit_id` first scopes `protocol` inside it and turns a hard 16-protocol ceiling into an effectively unbounded space – see Precedence decides how many protocols exist below, which is the most consequential decision on this page.

Today's *hardware* consumers happen to be classification-major: the filter masks are per-protocol then per-type (`cfg_axi_pkt_mask`, `cfg_axis_error_mask`, …), and `monbus_pkt_tally` addresses its histogram with `{protocol, pkt_type, event_code}` directly. That is an implementation choice of those two blocks, not a property of the format – **nothing in the monitor hardware filters or indexes on `unit_id` or `agent_id` at all.** A unit-major consumer ("show me everything subsystem 9 did, whatever protocol") is just as legitimate, needs no change to the

packet, and is what a software decoder typically wants; `get_unit_id()` and `get_agent_id()` exist for exactly that.

If a future block wants to filter or bin by identity, the packet already carries what it needs. Only the consumer is missing.

39.3.1 Precedence decides how many protocols exist

This is not a reporting preference. **Precedence determines whether `protocol` is a global namespace or a per-unit one, and that changes the ceiling by a factor of 256** – 16 protocol identities against 256 x 16 = 4096, with the full message space scaling by the same 256x (see the table below).

`protocol` is 4 bits. Read protocol-major – `protocol / packet_type / unit` – and those 4 bits are a **global** namespace: 16 protocols for the entire SoC, for all time, 5 already spent. Every new one must be centrally allocated and must never collide with anything, anywhere. It is a scarce, coordinated resource and it runs out.

Read unit-major – `unit / protocol / type` – and `protocol` is **scoped by `unit_id`**. Unit 9's protocol 3 and unit 12's protocol 3 are unrelated. Nothing central allocates them; each unit owns its own 16.

	protocol-major	unit-major
`protocol` namespace	global	per `unit_id`
distinct protocol identities	**16**	256 x 16 = **4096**
full message space	16 x 16 x 256 = 65,536	256 x 16 x 16 x 256 = **16,777,216**
allocating a new protocol	central registry, collides SoC-wide	local to the unit, collides with nobody

For any practical purpose the unit-major space is unbounded – you exhaust `unit_id` long before you exhaust protocols, and `unit_id` is the field you were going to assign per-block anyway.

This is the same mechanism as `event_code`, applied one level up. `event_code` is already scoped per `{protocol, packet_type}` rather than globally, which is why adding events never competes with existing assignments. Scoping `protocol` under `unit_id` extends that property to protocols themselves. The packet already works this way one level down; the only question is how far up you take it.

What it costs. A protocol-major packet is self-describing: any decoder can read `protocol` and know what it is holding, with no context. A unit-major packet is not – `protocol = 3` means nothing until you know the unit, so every consumer needs the unit-to-protocol map, and that map becomes a real artifact the project must maintain and version. You also give up the free global

query: “every AXI error in the SoC” is one mask protocol-major, and a map lookup per unit otherwise.

So it is a genuine trade: **a hard 16-protocol ceiling with zero bookkeeping, against an effectively unbounded space that requires a maintained map.** Small or single-protocol projects should take the ceiling. Large multi-team SoCs, where blocks arrive with their own event vocabularies and no one wants a central protocol registry, should take the map.

39.3.2 Fix the choice per project

Whichever you pick, **pick once and hold it for the life of the project.** A different project may reasonably pick the other; two consumers inside the same project may not – with unit-major especially, since a consumer that assumes global `protocol` will silently mis-decode every packet from a unit that reused a value.

Precedence also leaks into everything downstream: decoder grouping, coverage report row order, the register map a filter is programmed through, dashboards, and the shorthand people use in bug reports. Half a project sorting one way and half the other makes all of those incomparable.

Nothing in the RTL enforces either choice – the packet is a flat tuple and no hardware indexes on identity – so this is a convention, and conventions need a home. Record it where the project records its other integration decisions, alongside the `UNIT_ID` and `AGENT_ID` assignments, since those are the same conversation. If the project is unit-major, the unit-to-protocol map lives there too.

39.3.3 Coordinate A – classification: what happened

```
protocol  (4b, 16 slots)   AXI / AXIS / APB / ARB / CORE
   packet_type (4b)      error / timeout / completion / threshold /
perf / debug / ...
        event_code (8b)  ARB_ERR_STARVATION,
AXI_PERFWIN_BP_CYCLES, ...
```

Each level narrows the one above, and **each level is independently useful**. A consumer that only understands `packet_type` can still sort errors from performance samples in a stream containing protocols it has never heard of. One that understands `protocol` too can route AXI traffic to an AXI decoder and pass the rest through untouched. Only the innermost level requires knowing the block.

This is what lets the filter work in stages (drop a whole type, then individual codes within a type you are keeping, then restrict to an address window) and what lets the histogram bin on the whole tuple without knowing what any of it means.

39.3.4 Healthy classes vs fault classes

The packet types split into two groups, and the split changes how you cover them.

Healthy classes – **`completion`**, **`addr_match`**, **`perf`**/**`perfwin`**/**`perfhist`**. These arise from *correct* operation. Every transaction completes; every access matches a configured watch range; every

window closes with utilization buckets. Drive normal traffic and they appear. A capture that tallies them is measuring a working system.

Fault classes – `error`, `timeout`, `threshold`. These arise only from a *misbehaving* slave or an *illegal* access, and in a correct system they **never occur** – the fault tally reads **zero**, and that zero is the pass condition, the "nothing went wrong" signal. You cannot exercise them with healthy traffic; you must **explicitly inject the fault**:

Fault packet	Inject by
`timeout`	a slave that does not respond (hold R/B past the timeout window)
`threshold`	a slave that is slow (latency past `LATENCY_THRESH`, under the timeout)
`error`	a slave that returns `SLVERR`/`DECERR`, or an access outside an enabled address-range allowlist (see axi_monitor_addr_check)

An error, by definition, hangs the system. A transaction that errors or is never answered does not retire, so injecting an error *wedges* the traffic that provoked it – that stall is the fault, not a test artifact to engineer around. The monitor's job, and its whole value, is to **emit the fault packet as the hang happens** so a downstream capture records what stalled and why, instead of the system simply going dark. Consequently, fault coverage belongs in a dedicated fault-injection test that deliberately misbehaves the slaves – kept separate from the healthy-traffic capture, whose job is to assert the fault tally stays zero.

39.3.5 Coordinate B – identity: who it happened to

```
unit_id    (8b)   which subsystem
   agent_id  (16b)   which instance within it
        channel_id (9b)   which channel, or which AXI transaction ID
```

Note this nests the same way the classification does, and to the same depth – three levels, each narrowing the one above. That symmetry is why either can serve as the major key without the other becoming awkward.

`UNIT_ID` and `AGENT_ID` are elaboration parameters on the monitor, so identity is assigned structurally at integration time, not discovered at runtime. Two instances of the same wrapper differ only by parameter. A consumer can aggregate at any level – all errors in a subsystem, or one channel of one instance – without the producers knowing which grouping anyone intends.

39.3.6 Topology – how packets reach the capture point

`monbus_arbiter` takes `CLIENTS` monbus inputs and produces **one monbus output of exactly the same shape**. That is the whole trick: an arbiter's output is a valid arbiter input, so aggregation nests to any depth without a special "root" or "leaf" module.

STREAM builds a real three-level tree this way – one arbiter per level, each merging the level below:

Level	Module	CLIENTS
leaf	scheduler_group	2
middle	scheduler_group_array	NUM_CHANNELS + 1
root	stream_core	3

Each level collapses its children onto one stream, and the packet is unchanged by the trip – agent_id still says which leaf produced it. Only the root attaches to a monbus_*_group and off-chip transport. Adding a channel changes one CLIENTS parameter; it does not change the packet, the transport, or any consumer.

Note that the timestamp rides beside the packet through every level, and monbus_arbiter carries the 192-bit (packet, timestamp) pair atomically at each hop – so depth in the tree does not risk pairing a packet with the wrong time.

The shape of the tree is arbitrary, and it is expected to change. Nothing in the packet, the transport or any consumer encodes how many levels there are, which leaf hangs off which branch, or how wide each merge is. It is an integration choice: re-parent a block, add a level, collapse two levels into one wider CLIENTS, and every downstream stage behaves identically. That freedom is the point of the input/output symmetry – the tree can be re-drawn to match floorplan or timing without touching a line of monitor logic.

Two consequences follow, and both are contracts rather than advice:

- **Arrival order across producers means nothing.** Each level merges with arbiter_round_robin in ACK mode, so the interleaving you observe is a product of arbitration phase, backpressure and tree shape – all three of which can change. **Order packets by their timestamp, never by the order they came out of the pipe.** A consumer that infers causality from arrival order is reading an artifact of the topology it was captured on, and will silently disagree with itself when the tree is re-drawn.
- **Per-producer order IS preserved.** A single producer's packets stay in the order it emitted them, at every level, because arbitration reorders *between* clients and never *within* one. So "this agent's events, in sequence" is reliable; "these two agents' events, interleaved" is not.

What the tree does guarantee is fairness and no loss: round-robin prevents a chatty leaf from starving a quiet one, and the grant-hold contract (the grant retires only on grant && valid && ready) means backpressure delays packets rather than dropping them. Depth costs latency, never data.

39.4 Adaptability: what is deliberately left open

The format is locked, which is what makes bit-exact tooling possible. It is not, however, full:

Space	Used	Free
`protocol`	5 (AXI, AXIS, APB, ARB, CORE)	11 slots
`packet_type`	13	`0xA`, `0xB`, `0xC` reserved
`event_code`	per `{protocol, packet_type}`	8 bits *per pair*
reserved bits	none	`[123:109]`, 15 bits of forward-compat slack
`event_data`	payload	64 bits, fits a full address

The `event_code` row is the important one. Because the code is scoped by the `{protocol, packet_type}` pair rather than global, a new protocol gets a fresh 256-value space for each packet type it uses. Extension does not compete with existing assignments, so nobody has to coordinate a global registry.

`PROTOCOL_CORE` exists as the general-purpose slot for blocks that are not a bus protocol – reach for it before spending one of the 11 free `protocol` values. Spend a protocol slot only when the block is a *family* worth separating in filters and coverage reports.

The 15 reserved bits are genuine slack, not padding to a round number: they sit between `packet_type` and `protocol`, so a future field can be added without moving either the type tag at the top or the 64-bit payload at the bottom, and without changing the width. Existing decoders that mask on the documented fields keep working.

39.5 Functional Description

```
   DETECT                    SHAPE                     TRANSPORT
CAPTURE
   ------                    -----                     ---------
-------

 protocol wrapper      axi_monitor_base          monbus_arbiter
```

```
monbus_*_group
 axi4_master_rd_mon      trans_mgr (CAM)            (N:1 merge,
error FIFO
 axi5_slave_wr_mon       timer / timeout            packet+ts
-> AXI read
 axil4_*_mon             reporter_error            atomic)
(IRQ)
 apb4_monitor             reporter_timeout
write FIFO
 apb5_monitor            reporter_compl
-> AXI write
                         reporter_threshold
(bulk trace)
 custom producer         reporter_perf
 arbiter_rr_pwm_monbus   reporter_debug
monbus_compressor
 arbiter_wrr_pwm_monbus
(optional, in-line)
 your block

monbus_pkt_tally
       > axi_monitor_filtered             (counting only)
                      (staged drop filter)
```

Four stages, and you can stop after any of them.

39.5.1 1. Detect

A **protocol wrapper** pairs one protocol block with the shared monitor core. `axi4_master_rd_mon` instantiates `axi4_master_rd` and `axi_monitor_filtered`; the wrapper's whole job is to present protocol signals to a core that does not know what AXI is. These live with their protocols (`rtl/amba/axi4/`, `axi5/`, `axil4/`, `apb/`, `apb5/`), not in `rtl/amba/ocelus/`.

For AXI-shaped traffic the core tracks outstanding transactions in `monitor_trans_cam`, a multi-port ID CAM with payload storage, managed by `axi_monitor_trans_mgr`. That table is what makes completion, timeout and latency reporting possible: an event is attributed to the transaction that caused it.

A custom block skips all of this. See "Instrumenting something that is not a bus protocol" below.

39.5.2 2. Shape

`axi_monitor_base` decides what is worth reporting, through six reporters that each own one packet type:

Reporter	Emits	Answers
`axi_monitor_reporter_error`	`PktTypeError`	a protocol or response

Reporter	Emits	Answers
		error occurred
`axi_monitor_reporter_timeout`	`PktTypeTimeout`	a transaction outlived its budget
`axi_monitor_reporter_compl`	`PktTypeCompletion`	a transaction finished (and how long it took)
`axi_monitor_reporter_threshold`	`PktTypeThreshold`	a watermark was crossed
`axi_monitor_reporter_perf`	`PktTypePerf` only	lifetime completion/error count rollups. **Not** PerfWin/PerfHist – nothing packetizes the window buckets onto the MonBus yet (see Performance Monitoring below); they are readable as counters only
`axi_monitor_reporter_debug`	`PktTypeDebug`	trace points

Each has an `ENABLE_*_LOGIC` parameter that removes its detection cone at elaboration. This matters: an unused reporter is not gated at runtime, it is **not built**. `ENABLE_TIMEOUT_LOGIC=0` also drops the `axi_monitor_timeout` instance outright. Size the monitor to what you will actually read.

39.5.33. Filter

Three mechanisms drop packets *at the source*, before one ever consumes transport bandwidth. Note that these are not the Level 1/2/3 numbers used inside `axi_monitor_filtered.sv` – that module numbers its own two active stages 1 and 3, with 2 reserved. Here they are grouped by what they do:

1. **`cfg_axi_pkt_mask`** – drop whole packet types (16 bits, one per type).
2. **per-type event masks** – `cfg_axi_error_mask`, `cfg_axi_timeout_mask`, `cfg_axi_compl_mask`, `cfg_axi_thresh_mask`, `cfg_axi_perf_mask`, `cfg_axi_addr_mask`, `cfg_axi_debug_mask`. Drop individual event codes within a type you are otherwise keeping.

3. **address-range selection** – `axi_monitor_addr_check` (and `apb_monitor_addr_check` for APB) restrict reporting to address windows.

Filtering at the source rather than at the consumer is deliberate. The monitor bus is a shared resource and the failure mode is congestion: enable everything at once and the interesting packets queue behind the boring ones.

39.5.4 4. Transport and capture

`monbus_arbiter` merges N producer streams into one, carrying packet and timestamp atomically. The `monbus_group` family then provides two independent drains from one `monbus_group_core`:

- **Error / interrupt path** – a FIFO drained over an AXI4-shaped *slave read* interface, 192 bits per record. An IRQ handler walks records as 3 x 64-bit beats; `arlen` may fetch several records per burst. This is the "wake the CPU, something is wrong" path.
- **Master-write path** – a beat-granular FIFO drained over an AXI4-shaped *master write* interface with watermark and timeout flush, bursting as far as FIFO contents, `MAX_BURST_BEATS`, the 4 KB boundary and the address-window wrap allow. This is the bulk-trace path.

Four wrappers exist for the four combinations of interface width on those two ports: `monbus_axi4_axi4_group`, `monbus_axi4_axil4_group`, `monbus_axil4_axi4_group`, `monbus_axil4_axil4_group`. They are wrappers only – the logic is in `monbus_group_core`.

39.6 Choosing a capture strategy

This is the decision that actually shapes a deployment, and there are three answers.

	Bulk trace	Compressed trace	Counting
Module	`monbus_group_core` write path	`+ monbus_compressor`	`monbus_pkt_tally`
Off-chip cost	24 B per record	~8 B per record on template hits	zero until readback
Run length	bounded by log SRAM depth	bounded, but several times longer	**unbounded**
Keeps	every packet,	every packet,	counts per

	Bulk trace	Compressed trace	Counting
	exactly	exactly	message identity
Loses	nothing	nothing	ordering, timestamps, payloads
Use when	debugging a specific failure	long capture, still need every packet	“did this ever happen?”

39.6.1 Bulk trace

Raw mode emits complete 24-byte (3-beat) records. Simple, exact, and the run length is whatever your log SRAM holds.

39.6.2 Compressed trace – `monbus_compressor`

Opt in per wrapper via `USE_COMPRESSION`. It sits in front of the master writer and turns `(packet, timestamp)` records into 64-bit self-tagged slots:

Tag	Format	Beats	When
`0x0`	RAW escape	3	no template match
`0x1`	T1-A	1	template hit, small payload
`0x2`	T1-B	1	template hit, big `delta_ts`
`0x3`	T1-C	1	template hit, `event_data` delta

A 32-entry true-LRU CAM (`monbus_cam`) holds the templates, keyed on message identity. Each entry stores **its own** last timestamp, so `delta_ts` is measured per template – interleaved producers do not force each other into the raw escape. Throughput is one record per cycle on template hits, one per three cycles on escapes.

The format is a locked spec, and the acceptance criterion is unusually strong: the hardware slot stream is **bit-exact** to what the Python `Encoder` in `bin/TBClasses/monbus/monbus_compressor.py` produces from the same record sequence. Per-tier hit counters (`tier1_a`, `tier1_b`, `tier1_c`, `tier0_escape`) let you measure the compression you are actually getting rather than assuming it.

`monbus_halfbeat_packer` sits downstream and packs two 30-bit half-slots into one 64-bit beat where the format allows.

39.6.3 Counting – `monbus_pkt_tally`

The one that changes what is *possible* rather than what is efficient.

It counts accepted packets into an SRAM histogram addressed by `{protocol, packet_type, event_code}`, fronted by a 32-entry LRU write-combining cache so back-to-back hits on hot bins never reach the SRAM. One readback sweep dumps the matrix.

A counter absorbs any arrival rate. The trace paths are bounded by log SRAM depth; the tally is not bounded at all. A coverage run can span millions of cycles. This is the silicon twin of the simulation-side packet-type coverage matrix (`bin/monbus_coverage_report`, `TBClasses.monbus.parse`) – a bin count greater than zero means *this message was observed on hardware*.

You give up ordering, timestamps and payloads. If the question is "which of these 200 events ever fired in a week of running", that is the right trade.

39.7 Instrumenting something that is not a bus protocol

The system was built for AXI and then generalised, and the arbiters are the proof that the generalisation is real. `PROTOCOL_ARB` is a first-class protocol with its own event codes:

Code	Event
`0x0`	`ARB_ERR_STARVATION` – a client never got service
`0x1`	`ARB_ERR_ACK_TIMEOUT`
`0x2`	`ARB_ERR_PROTOCOL_VIOLATION`
`0x3`	`ARB_ERR_CREDIT_VIOLATION`
`0x4`	`ARB_ERR_FAIRNESS_VIOLATION` – weighted round-robin is not honouring weights
`0x5`	`ARB_ERR_WEIGHT_UNDERFLOW`
`0x6`	`ARB_ERR_CONCURRENT_GRANTS`
`0x7`	`ARB_ERR_INVALID_GRANT_ID`
`0x8`	`ARB_ERR_ORPHAN_ACK`
`0x9`	`ARB_ERR_GRANT_OVERLAP`
`0xA`	`ARB_ERR_MASK_ERROR`
`0xB`	`ARB_ERR_STATE_MACHINE`
`0xC`	`ARB_ERR_CONFIGURATION`

None of these mean anything to AXI. They ride the identical packet, the identical arbiter, the identical filter, and land in the identical histogram bin. `arbiter_monbus_common` carries the shared production logic; `arbiter_rr_pwm_monbus` and `arbiter_wrr_pwm_monbus` are the instrumented round-robin and weighted round-robin arbiters.

To instrument your own block:

1. Pick a protocol value. `PROTOCOL_CORE` (`4'h4`) exists for exactly this – blocks that are not a bus. Add a new `protocol_type_t` only if your block is a family worth separating; there are 16 slots and 5 are used.
2. Allocate `event_code` values in a package next to `monitor_arbiter_pkg.sv`, and document them. The code is 8 bits per `{protocol, packet_type}` pair, so the space is not tight.
3. Emit a `monitor_packet_t`. Fill `unit_id` / `agent_id` so the packet identifies your instance, and use the 64 bits of `event_data` for whatever the event needs.
4. Feed `monbus_arbiter`. From that point on you inherit filtering, both transports, compression and the coverage histogram without writing any of it.

Step 4 is the payoff. The reason to use this system rather than adding your own debug registers is that everything after emission already exists.

39.8 Performance monitoring

Two distinct perf paths exist, and only one currently reaches the bus:

- **`PktTypePerf` rollup (implemented).** `ENABLE_PERF_LOGIC` (legacy alias `ENABLE_PERF_PACKETS`) builds `axi_monitor_reporter_perf`, which rolls up lifetime completion/error counts and emits `PktTypePerf` (`AXI_PERF_COMPLETED_COUNT = 0x7`, `AXI_PERF_ERROR_COUNT = 0x8`). This is the perf packet you see on the MonBus today. It advances only while the output is idle and loses the reporter mux to completion/threshold, so it surfaces in the gaps between traffic rather than under sustained load.
- **`PktTypePerfWin` / `PktTypePerfHist` window+histogram (RFC Stage B/F – not yet emitted).** The window state machine below exists (perfmon Stage A in `axi_monitor_base`) and maintains its bucket counters, but the code that *packetizes* those counters onto the MonBus is not wired yet. **No module currently emits `PktTypePerfWin` or `PktTypePerfHist`** – the

buckets are readable only as counters, so a MonBus tally can never bin them. The description below is the intended shape once Stage B/F lands.

The window state machine buckets every cycle into one of four states and (once Stage B/F lands) reports at window close:

Code	AXI_PERFWIN_*	Meaning
0x1	PROD_CYCLES	valid && ready – productive
0x2	BP_CYCLES	valid && !ready – backpressured
0x3	STARV_CYCLES	!valid && ready – starved
0x4	IDLE_CYCLES	!valid && !ready – idle

Those four partition the window, which is what makes them useful: a link at 40% utilization is a different problem depending on whether the other 60% is backpressure (the consumer is slow) or starvation (the producer is). Alongside them: BEAT_COUNT, BYTE_COUNT (beats x 1<<axsize, masked by strb), BURST_COUNT, window start and end timestamps, and per-channel splits (CHAN_PROD, CHAN_STARV, CHAN_BP).

PktTypePerfHist reports histogram buckets instead of totals: event_code[7:4] selects the histogram, event_code[3:0] the bucket (0..15, log2 cycle thresholds). Use it for latency distributions, where a mean hides the tail that actually matters.

39.9 Configuration cautions

cam_clear is legal only while the monitor is idle. Idle means no outstanding transactions in the table and no packet in flight through the reporter or the output queue. The clear empties the table synchronously and does not flush what the reporter is still assembling, so a clear issued a cycle or two after a completion can leave a packet behind it; on a clock-gated wrapper at idle-count 0 that packet is stranded, because the table going empty was the last thing holding the clock (amba ISSUE-002, closed as this rule). Software sequence: stop traffic, wait for busy low and monbus_valid low, then clear. The clock-gating test does exactly that – it waits for the completion packet to be delivered before its housekeeping clear.

Two that have bitten before:

- **Do not enable every packet type at once.** cfg_compl_enable together with cfg_perf_enable is the documented congestion case – completion

packets are frequent and perf packets are bursty, and together they crowd out errors. Use separate test configurations.

- **The transaction table has a saturation-recovery contract.** Command-originated entries are capped below `MAX_TRANSACTIONS` so a saturated table can always drain; `cmd_entry_reserve()` in `monitor_common_pkg` is the single source of truth, and both `axi_monitor_trans_mgr` and `axi_monitor_base` derive from it. Tables smaller than 16 take reserve 0 and trade the guarantee for capacity. This was a comment-encoded invariant once, and that is how a saturation wedge shipped.

39.10 Related Modules

For	Read
Packet fields, event-code assignments	monitor_package_spec
The monitor core	axi_monitor_base
Filtering	axi_monitor_filtered
Capture back end	monbus_group_core
Compression	monbus_compressor
Counting	monbus_pkt_tally
Arbiter instrumentation	arbiter_monbus_common
A protocol wrapper	axi4_master_rd_mon

39.11 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. “An Event-Based Monitoring Service for Networks on Chip.” *ACM Trans. Design Automation of Electronic Systems*, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126. – related work: an event-based monitoring service built from hardware probes.
- Goossens, K., Vermeulen, B., van Steeden, R., Bennebroek, M. “Transaction-Based Communication-Centric Debug.” *Proc. 1st Int. Symp. Networks-on-Chip (NOCS 2007)*, pp. 95-106, May 2007. DOI: 10.1109/NOCS.2007.46. – related work: monitors plus a dedicated event-distribution interconnect.

- IEEE-ISTO 5001-2003. *The Nexus 5001 Forum Standard for a Global Embedded Processor Debug Interface*, Version 2.0, IEEE Industry Standards and Technology Organization, Dec. 2003 (defining non-peer standard). – a defining non-peer standard for message-based trace: typed messages, source IDs, overrun reports, timestamps (§5.2, §4.4.3, §4.11). The monitor bus is not Nexus-compatible.
- Abramovici, M., Bradley, P., Dwarakanath, K., Levin, P., Memmi, G., Miller, D. “A Reconfigurable Design-for-Debug Infrastructure for SoCs.” *Proc. 43rd Design Automation Conf. (DAC 2006)*, pp. 7-12, July 2006. DOI: 10.1145/1146909.1146916. – related work: checkers, triggers and event counters in a debug fabric.
- Hopkins, A.B.T., McDonald-Maier, K.D. “Debug Support for Complex Systems On-Chip: A Review.” *IEE Proc. Computers and Digital Techniques*, 153(4):197-207, 2006. DOI: 10.1049/ip-cdt:20050194. – survey of on-chip debug support.
- Vermeulen, B. “Functional Debug Techniques for Embedded Systems.” *IEEE Design & Test of Computers*, 25(3):208-215, May/June 2008. DOI: 10.1109/MDT.2008.66. – survey of functional silicon debug.
- Lamport, L. “Time, Clocks, and the Ordering of Events in a Distributed System.” *Communications of the ACM*, 21(7):558-565, July 1978. DOI: 10.1145/359545.359563. – for ordering events without a shared clock.
- Indexed in `rtl/amba/references/README.md` (local PDF: `DebugTraceInterfaceStandard__IeeeIsto2003.pdf`, `LogicalClocksEventOrdering__Lamport1978.pdf`)

39.12 Navigation

- **← Back to Shared Infrastructure Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

40 monitor_trans_cam

Module: `monitor_trans_cam.sv` **Location:** `rtl/amba/ocelus/` **Category:** AXI Monitor Infrastructure **Status:** Production Ready

Plane: protocol plane (transaction CAM for the full monitor)

40.1 Overview

`monitor_trans_cam` is a **multi-port ID CAM with opaque payload storage** used by the `axi_monitor_trans_mgr` to track outstanding AXI transactions by ID. It replaces the inline parallel-match logic that previously lived in `axi_monitor_trans_mgr` (2026-04-23 revision), moving the keying state and payload storage into a reusable shared module without changing the synthesis shape that closes 100 MHz on the xc7a100t-1 Artix-7.

Don't confuse it with `monbus_cam` — the two solve different problems:

Feature	`monbus_cam`	`monitor_trans_cam`
Use case	Caching (compressor template index)	Associative storage (AXI ID tracking)
Eviction	True LRU	None (explicit alloc / free)
Ports	1 lookup	3 lookups + alloc priority encoder
Payload	64 b	Parameterizable (typically `$bits(bus_transaction_t)`)
Default depth	32	16
Match output	hit + idx + old_data	Three one-hot vectors

40.1.1 Key Features

- 3 independent ID lookup ports per cycle (`addr` / `data` / `resp`)
- One-hot match vectors (`*_match_oh`) plus a lowest-index "first" variant (`data_match_first_oh`; provided by the CAM but left unconnected by the current `axi_monitor_trans_mgr`, which resolves multi-match cases itself, oldest-first)

- Free-slot vector (`free_oh`) and priority-encoded **3-way mutex alloc** (`addr_alloc_oh` / `data_alloc_oh` / `resp_alloc_oh`)
- Per-slot write port (one-hot enable, separate next-state inputs per slot)
- Per-slot read port (registered current state) exposed for the caller
- `(* keep = "true" *)` attributes on match/free vectors so synth can't fuse them into downstream update cones (preserves the trans_mgr WNS fix)
- Simulation-only protocol assertions on the caller

40.2 Parameters

Parameter	Default	Notes
`DEPTH`	16	Maximum outstanding transactions. Asserts in TBs require ≥ 4.
`ID_WIDTH`	8	AXI ID width. Typically 4–8 for AXI4.
`PAYLOAD_WIDTH`	128	Caller's opaque payload. trans_mgr uses `$bits(bus_transaction_t)` (~285 bits).

The trans_mgr instantiates with `PAYLOAD_WIDTH = $bits(bus_transaction_t)`, storing the full struct (including the redundant `id` field — the CAM's `entry_id` and `payload.id` are written atomically so they always agree).

40.3 Ports

The full interface, straight from the RTL:

```
module monitor_trans_cam #(
    parameter int DEPTH         = 16,
    parameter int ID_WIDTH      = 8,
    parameter int PAYLOAD_WIDTH = 128
) (
    input  logic                            clk,
    input  logic                            rst_n,

    // Synchronous clear: on the next edge, invalidate every entry
    // (empty the CAM) so the host can rebase the transaction table
```

```
    // without a full rst_n. Takes priority over per-slot entry_we.
    input  logic                            clear,

    // ---- Lookup ports (combinational) ----
    input  logic [ID_WIDTH-1:0]             lookup_addr_id,
    input  logic [ID_WIDTH-1:0]             lookup_data_id,
    input  logic [ID_WIDTH-1:0]             lookup_resp_id,
    output logic [DEPTH-1:0]                addr_match_oh,
    output logic [DEPTH-1:0]                data_match_oh,
    output logic [DEPTH-1:0]                resp_match_oh,
    output logic [DEPTH-1:0]                data_match_first_oh,
                                            // lowest-index match for
data port

    // ---- Free + 3-way alloc priority encoder ----
    output logic [DEPTH-1:0]                free_oh,
    input  logic                            addr_wants_alloc,
    input  logic                            data_wants_alloc,
    input  logic                            resp_wants_alloc,
    // Per-phase allocation masks -- TIE ALL ONES for unrestricted
behaviour.
    // Every alloc loop gates on these; leaving them unconnected
floats them
    // (X in simulation) or ties them to 0 (synthesis), and allocation
then
    // never fires at all.
    input  logic [DEPTH-1:0]                addr_alloc_mask,
    input  logic [DEPTH-1:0]                data_alloc_mask,
    input  logic [DEPTH-1:0]                resp_alloc_mask,
    output logic [DEPTH-1:0]                addr_alloc_oh,
    output logic [DEPTH-1:0]                data_alloc_oh,
    output logic [DEPTH-1:0]                resp_alloc_oh,

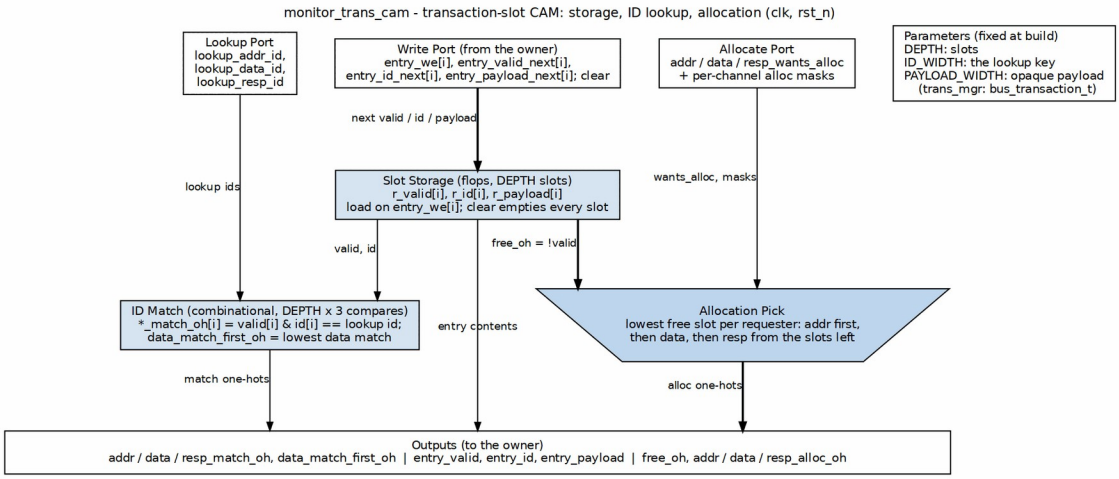
    // ---- Per-slot write port ----
    input  logic [DEPTH-1:0]                entry_we,
    input  logic [DEPTH-1:0]                entry_valid_next,
    input  logic [ID_WIDTH-1:0]             entry_id_next
[DEPTH],
    input  logic [PAYLOAD_WIDTH-1:0]        entry_payload_next
[DEPTH],

    // ---- Per-slot read port (registered) ----
    output logic [DEPTH-1:0]                entry_valid,
    output logic [ID_WIDTH-1:0]             entry_id
[DEPTH],
    output logic [PAYLOAD_WIDTH-1:0]        entry_payload
```

```
[DEPTH]
);
```

40.4 Functional Description

40.4.1 Block diagram



monitor_trans_cam block diagram

The slot flops sit under the write port, through which the owner (`axi_monitor_trans_mgr`) loads each slot's next contents. Their valid and ID state drops into the three ID compares (address, data and response lookups), and as the free vector onto the allocation pick, which grants the lowest free slot to address, then data, then response requests. All results, and the raw slot contents, go back to the owner.

40.4.2 Architecture

40.4.3 Why Three Lookup Ports?

The AXI monitor needs to match an arriving transaction beat against the outstanding transaction table along three independent channels in the same cycle:

Lookup port	Purpose
`lookup_addr_id`	Matches an arriving AW/AR beat to an in-flight transaction (or detects "this is a brand-new transaction")
`lookup_data_id`	Matches an arriving R beat (read) or detects orphan-data-without-AW (write, AXI-Lite only)

Lookup port	Purpose
`lookup_resp_id`	Matches an arriving B beat (write) to an in-flight transaction

All three lookups go in parallel. The CAM's internal storage feeds 3 one-hot match vectors, each a 1-LUT-per-bit `valid && (id == lookup_id)` test. With DEPTH=16 that's 16 small independent cones per port, no chained dependencies.

The `(* keep = "true" *)` attribute on the match vectors prevents Vivado from fusing them into the downstream `axi_monitor_trans_mgr` update logic — without that, the 16 entries' update cones would fold together into one shared wide function, producing ~12 LUT levels and failing 100 MHz on the -1 part. With the attribute, each entry's update stays an independent small cone.

40.4.4 The "First-Match" Variant

For **AXI4 writes**, the data channel has no WID. The trans_mgr matches an arriving W beat to an outstanding write transaction by state predicate (`valid && state ∈ {ADDR_PHASE, DATA_PHASE} && cmd_received && !data_completed`), NOT by id. That state predicate is computed *outside* the CAM (it requires fields the CAM doesn't see, like `state` and `cmd_received`), and — because several entries can satisfy it — is resolved to the **oldest** candidate via the trans_mgr's rank-based `pick_oldest()` selector, matching AXI4's write-data ordering rule.

The CAM also provides `data_match_first_oh`, the lowest-index bit of `data_match_oh` (the id-based match). The current `axi_monitor_trans_mgr` leaves this output **unconnected**: lowest-index is not issue order once slots are recycled, so for reads the trans_mgr feeds the raw `data_match_oh` candidates through `pick_oldest()` instead, and for writes it uses its own state-predicate vector as above. The port remains for callers that need a deterministic lowest-index pick. The two paths share the rest of the CAM machinery (free, alloc, write port) so the cost is minimal.

40.4.5 3-Way Mutex Alloc Priority Encoder

When a new transaction arrives that doesn't match an existing entry, the trans_mgr needs to allocate a free slot for it. Up to three independent "want to alloc" requests can fire in the same cycle (addr, data, resp), and each needs a *different* slot — the priority encoder ensures no two phases ever claim the same free slot.

The encoder is purely combinational:

```
remaining_free = free_oh

if addr_wants_alloc:
    addr_alloc_oh = lowest-set-bit(remaining_free & addr_alloc_mask)
    remaining_free &= ~addr_alloc_oh

if data_wants_alloc:
```

```
    data_alloc_oh = lowest-set-bit(remaining_free & data_alloc_mask)
    remaining_free &= ~data_alloc_oh

if resp_wants_alloc:
    resp_alloc_oh = lowest-set-bit(remaining_free & resp_alloc_mask)
    remaining_free &= ~resp_alloc_oh
```

The three outputs are guaranteed to be disjoint (mutex assertion enforces this in simulation). If `wants_alloc` is asserted with no free slot, the corresponding `*_alloc_oh` is `'0` — the caller need not gate its wants_alloc requests on free-slot availability (the trans_mgr gates them on match suppression and its command-entry cap instead).

Priority order is `addr → data → resp`. AXI4 writes can have all three firing in the same cycle on a tight pipeline, and the order matches the phase order so the slot indices in the table grow in a reasonable chronological order.

40.4.6 Per-Slot Storage and Write Port

The CAM owns the registered storage:

```
logic                       r_valid   [DEPTH];
logic [ID_WIDTH-1:0]        r_id      [DEPTH];
logic [PAYLOAD_WIDTH-1:0]   r_payload [DEPTH];
```

The write port is one-hot per slot:

```
// Per slot i:  if (entry_we[i]) then on next clock edge,
//                r_valid[i]   <= entry_valid_next[i];
//                r_id[i]      <= entry_id_next[i];
//                r_payload[i] <= entry_payload_next[i];
```

To clear a slot, drive `entry_we[i]=1` with `entry_valid_next[i]=0`. The payload and id get written too (with whatever the caller drives) but the valid bit determines whether subsequent lookups see the entry.

The per-slot updates live in a `generate` loop of independent `always_ff` blocks, one per slot. Each block depends only on local one-hot bits and shared inputs — synth cannot fuse them across slots. The result is `DEPTH` parallel small update cones rather than one giant shared one.

40.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

40.6 Usage Examples

40.6.1 Caller Pattern (`axi_monitor_trans_mgr`)

The trans_mgr uses the CAM's outputs to compute per-slot next-state in a generate-loop `always_comb`, then drives the write port:

```
For each slot i (combinational):
    next_payload = current_payload  (default: hold)
    next_we      = 0

    if addr_alloc_oh[i]:
        # New transaction starting in addr phase
        next_payload = initialize from cmd_*
        next_we = 1
        next_id = cmd_id

    elif addr_update_oh[i] && cmd_handshake:
        # Existing entry getting addr fields updated
        next_payload.cmd_received = 1
        next_payload.addr_timestamp = timestamp
        next_we = 1

    ... data and resp phases similar ...

    if can_cleanup[i]:
        next_payload.valid = 0
        next_we = 1

    if event_reported_flag[i] && !current_payload.event_reported:
        next_payload.event_reported = 1
        next_we = 1

    Drive entry_we[i], entry_valid_next[i], entry_payload_next[i] for
the CAM.
```

The CAM's `entry_valid` output mirrors the payload's `.valid` field — the trans_mgr writes them in sync atomically. Lookups use `entry_valid` (the CAM's view) so the match logic stays tight.

40.7 Design Notes

40.7.1 Synthesis Notes

Key design choices to preserve the trans_mgr's 2026-04-23 WNS fix:

Construct	Rationale
`(* keep = "true" *)` on `*_match_oh`, `free_oh`	Anchors the match vectors so Vivado doesn't fuse them into downstream update cones
Per-slot `generate`-loop `always_ff`	`DEPTH` independent small update cones instead of one shared cone
Combinational lookup + commit-in-one-cycle write	Caller can drive lookup → update without an extra pipeline stage
Position-IS-rank NOT used	This CAM has no LRU — slots are positionally assigned by the caller; the index has no recency semantic

The combinational depth from input port to CAM output is:

```
lookup_id  →  match_oh  (per-slot compare; the module exports the
              one-hot vector, and the consumer ORs it if it needs a hit
flag)

                          first_oh (priority encoder)
```

That's ~3–4 LUT levels for DEPTH=16 — comfortably under the 100 MHz budget on the -1 part.

40.8 Related Modules

Module	Role
`axi_monitor_trans_mgr`	Sole consumer — uses the CAM for ID tracking and per-slot storage
`monbus_cam`	Sister CAM for the bulk-trace compressor (LRU, single port, different design intent)
`bin/TBClasses/shared/`	Test framework utilities

40.9 Testing

`val/amba/test_monitor_trans_cam.py` runs 17 sub-tests covering:

#	Sub-test
1	Reset state — all entries invalid, `free_oh = all-1s`
2	Single write + lookup — write slot 0 with id=K, lookup hits
3	Independent writes — each slot writable in isolation
4	Three-port lookup — distinct addr/data/resp ids resolve independently
5	`data_match_first_oh` — duplicate ids: lowest-index wins
6	`free_oh` tracks valid bit per slot
7	Clear via `we=1 + valid_next=0`
8	Alloc one phase — `addr_wants_alloc` → lowest-index free slot
9	Alloc mutex — all three phases want → three distinct slots
10	Alloc partial wants — only some phases want, others zero
11	Alloc when full — no free slots → all `*_alloc_oh = 0`
12	Payload preserved across idle cycles
13	Concurrent writes — `entry_we` to multiple slots in one cycle
14	Lookup miss — id absent from CAM → match_oh = 0
15	Alloc priority cascade — pre-occupy slots, watch alloc skip them
16	ID collision tolerance — two valid slots

#	Sub-test
	with same id
17	Random stress — random write/lookup/alloc mix vs Python model

The Python golden model mirrors the RTL state and combinational outputs exactly. At FULL `REG_LEVEL` the random stress runs 5000 ops per config, and every cycle cross-checks all 9 combinational outputs (3 `*_match_oh`, `first_oh`, `free_oh`, 3 `*_alloc_oh`, `entry_valid`) against the model.

```
pytest val/amba/test_monitor_trans_cam.py -v
```

REG_LEVEL parameter sweep: - **GATE:** 1 config (8/64/16) - **FUNC:** 2 configs (+ 4/32/8) - **FULL:** 7 configs (depth 4/8/16/32, id 4/6/8, payload 32/64/128/256)

40.10 References

The Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitor observes.

- Pagiamtzis, K., Sheikholeslami, A. “Content-Addressable Memory (CAM) Circuits and Architectures: A Tutorial and Survey.” *IEEE J. Solid-State Circuits*, 41(3):712-727, Mar. 2006. DOI: 10.1109/JSSC.2005.864128. — for the CAM structure.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l. — §A5.1 transaction identifiers (the key this CAM matches).
- Goossens, K., Vermeulen, B., van Steeden, R., Bennebroek, M. “Transaction-Based Communication-Centric Debug.” *Proc. 1st Int. Symp. Networks-on-Chip (NOCS 2007)*, pp. 95-106, May 2007. DOI: 10.1109/NOCS.2007.46. — related work: transaction-level monitoring.
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/common/references/ContentAddressableMemory__Pagiamtzis2006.pdf`)

40.11 Navigation

- **← Back to monitor index**

- **← Back to rtl-amba index**